Universidad Mariano Gálvez de Guatemala.

Facultad de Ingeniería en Sistemas de Información y

Ciencias de la Computación.

Nombre del Curso: Análisis de Sistemas II

Nombre del Catedrático: Edgar Raúl Molina Rey

Sección A



Documento de pruebas unitarias del sistema SIGAC

Jose Manuel Maquin Leal

4090-22-3506

24/10/2026

Índice

Introducción 9
Objetivos 10
Alcance 11
Elementos excluidos 11
Entorno y herramientas 12
Uso de inteligencia artificial como herramienta de apoyo 14
1 Pruebas unitarias 16
1.1 Hash del token (WF-01) 16
1.1.1 Código base del nodo Hash del token 16
1.1.2 Código de test del nodo Hash del token 16
1.1.3 Tabla de pruebas unitarias realizadas al nodo Hash del token (WF-01) 19
1.2 Valida datos (WF-01) 21
1.2.1 Código base del nodo Valida datos 21
1.2.2 Código de test del nodo Valida datos 25
1.2.3 Tabla de pruebas unitarias realizadas al nodo Valida datos (WF-01) 36
1.2.4 Nuevo código base del nodo Valida datos 42
1.3 Cifra datos sensibles (WF-01) 49
1.3.1 Código base del nodo Cifra datos sensibles 49
1.3.2 Código de test del nodo Cifra datos sensibles 49
1.3.3 Tabla de pruebas unitarias realizadas al nodo Cifra datos sensibles (WF-01) 54
1.4 Registro exitoso (WF-01) 57
1.4.1 Código base del nodo Registro exitoso 57
1.4.2 Código de test del nodo Registro exitoso 57
1.4.3 Tabla de pruebas unitarias realizadas al nodo Registro exitoso (WF-01) 59
1.5 Rechazo 409 (WF-01) 61
1.5.1 Código base del nodo Rechazo 409 61
1.5.2 Código de test del nodo Rechazo 409 61
1.5.3 Tabla de pruebas unitarias realizadas al nodo Rechazo 409 (WF-01) 63
1.6 Rechazo 400 (WF-01) 65
1.6.1 Código base del nodo Rechazo 400 65

1.6.2 Código de test del nodo Rechazo 400 65
1.6.3 Tabla de pruebas unitarias realizadas al nodo Rechazo 400 (WF-01) 67
1.7 Rechazo 401 (WF-01) 69
1.7.1 Código base del nodo Rechazo 401 69
1.7.2 Código de test del nodo Rechazo 401 69
1.7.3 Tabla de pruebas unitarias realizadas al nodo Rechazo 401 (WF-01) 71
1.8 Rechazo 403 (WF-01) 73
1.8.1 Código base del nodo Rechazo 403 73
1.8.2 Código de test del nodo Rechazo 403 73
1.8.3 Tabla de pruebas unitarias realizadas al nodo Rechazo 403 (WF-01) 75
1.9 Calcula prioridad (WF-02) 77
1.9.1 Código base del nodo Calcula prioridad 77
1.9.2 Código de test del nodo Calcula prioridad 77
1.9.3 Tabla de pruebas unitarias realizadas al nodo Calcula prioridad (WF-02) 81
1.10 Reparte cupos (WF-02) 83
1.10.1 Código base del nodo Reparte cupos 83
1.10.2 Código de test del nodo Reparte cupos 84
1.10.3 Nuevo código base de Reparte Cupos 92
1.10.4 Se corrió el mismo código test pero ya con la corrección del código base. 95
1.11 Hash del token (WF-04) 96
1.11.1 Código base del nodo Hash del token 96
1.11.2 Código de test del nodo Hash del token 96
1.12 Rechazo 401 (WF-04) 100
1.12.1 Código base del nodo Rechazo 401 100
1.12.2 Código de test del nodo Rechazo 401 100
1.13 Rechazo 404 (WF-04) 103
1.13.1 Código base del nodo Rechazo 404 103
1.13.2 Código de test del nodo Rechazo 404 103
1.14 Reasignación exitosa (WF-04) 106
1.14.1 Código base del nodo Reasignación exitosa 106
1.14.2 Código de test del nodo Reasignación exitosa 106

1.15 Cancelado sin reasignar (WF-04) 109
1.15.1 Código base del nodo Cancelado sin reasignar 109
1.15.2 Código de test del nodo Cancelado sin reasignar 109
1.16 Hash del token (WF-05) 111
1.16.1 Código base del nodo Hash del token 111
1.16.2 Código de test del nodo Hash del token 111
1.17 Rechazo 401 (WF-05) 115
1.17.1 Código base del nodo Rechazo 401 115
1.17.2 Código de test del nodo Rechazo 401 115
1.18 Rechazo 404 (WF-05) 118
1.18.1 Código base del nodo Rechazo 404 118
1.18.2 Código de test del nodo Rechazo 404 118
1.19 Cierre exitoso (WF-05) 121
1.19.1 Código base del nodo Cierre exitoso 121
1.19.2 Código de test del nodo Cierre exitoso 121
1.20 Compone el reporte (WF-06) 123
1.20.1 Código base del nodo Compone el reporte 123
1.20.2 Código de test del nodo Compone el reporte 125
1.21 Nombra el archivo (WF-06) 133
1.21.1 Código base del nodo Nombra el archivo 133
1.21.2 Código de test del nodo Nombra el archivo 133
1.22 Hash del token (WF-07) 136
1.22.1 Código base del nodo Hash del token 136
1.22.2 Código de test del nodo Hash del token 136
1.23 Valida datos (WF-07) 140
1.23.1 Código base del nodo Valida datos 140
1.23.2 Código de test del nodo Valida datos 141
1.23.3 Nuevo código base de Valida Datos 146
1.23.4 Se corrió el mismo código test pero ya con la corrección del código base. 147
1.24 Cifra la nota (WF-07) 148
1.24.1 Código base del nodo Cifra la nota 148

1.24.2 Código de test del nodo Cifra la nota 148
1.25 Nota registrada (WF-07) 153
1.25.1 Código base del nodo Nota registrada 153
1.25.2 Código de test del nodo Nota registrada 153
1.26 Rechazo 400 (WF-07) 156
1.26.1 Código base del nodo Rechazo 400 156
1.26.2 Código de test del nodo Rechazo 400 156
1.27 Rechazo 401 (WF-07) 159
1.27.1 Código base del nodo Rechazo 401 159
1.27.2 Código de test del nodo Rechazo 401 159
1.28 Rechazo 403 (WF-07) 162
1.28.1 Código base del nodo Rechazo 403 162
1.28.2 Código de test del nodo Rechazo 403 162
1.29 Rechazo 404 (WF-07) 165
1.29.1 Código base del nodo Rechazo 404 165
1.29.2 Código de test del nodo Rechazo 404 165
1.30 Hash del token (WF-08) 168
1.30.1 Código base del nodo Hash del token 168
1.30.2 Código de test del nodo Hash del token 168
1.31 Valida datos (WF-08) 172
1.31.1 Código base del nodo Valida datos 172
1.31.2 Código de test del nodo Valida datos 172
1.31.3 Nuevo código base de Valida Datos. 175
1.31.4 Se corrió el mismo código test pero ya con la corrección del código base. 175
1.32 Descifra (WF-08) 176
1.32.1 Código base del nodo Descifra 176
1.32.2 Código de test del nodo Descifra 177
1.33 Historial listo (WF-08) 183
1.33.1 Código base del nodo Historial listo 183
1.33.2 Código de test del nodo Historial listo 183
1.34 Rechazo 400 (WF-08) 186

1.34.1 Código base del nodo Rechazo 400 186
1.34.2 Código de test del nodo Rechazo 400 186
1.35 Rechazo 401 (WF-08) 189
1.35.1 Código base del nodo Rechazo 401 189
1.35.2 Código de test del nodo Rechazo 401 189
1.36 Rechazo 403 (WF-08) 192
1.36.1 Código base del nodo Rechazo 403 192
1.36.2 Código de test del nodo Rechazo 403 192
1.37 Rechazo 404 (WF-08) 195
1.37.1 Código base del nodo Rechazo 404 195
1.37.2 Código de test del nodo Rechazo 404 195
1.38 Hash del token (WF-09) 198
1.38.1 Código base del nodo Hash del token 198
1.38.2 Código de test del nodo Hash del token 198
1.39 Valida datos (WF-09) 202
1.39.1 Código base del nodo Valida datos 202
1.39.2 Código de test del nodo Valida datos 204
1.39.3 Nuevo código base de Valida Datos. 210
1.39.4 Se corrió el mismo código test pero ya con la corrección del código base. 213
1.40 Rechazo 400 (WF-09) 214
1.40.1 Código base del nodo Rechazo 400 214
1.40.2 Código de test del nodo Rechazo 400 214
1.41 Rechazo 401 (WF-09) 217
1.41.1 Código base del nodo Rechazo 401 217
1.41.2 Código de test del nodo Rechazo 401 217
1.42 Rechazo 403 (WF-09) 220
1.42.1 Código base del nodo Rechazo 403 220
1.42.2 Código de test del nodo Rechazo 403 220
1.43 Genera token (WF-09) 223
1.43.1 Código base del nodo Genera token 223
1.43.2 Código de test del nodo Genera token 223

1.44 Recepcionista agregada (WF-09) 226
1.44.1 Código base del nodo Recepcionista agregada 226
1.44.2 Código de test del nodo Recepcionista agregada 226
1.45 Estado recepcionista (WF-09) 229
1.45.1 Código base del nodo Estado recepcionista 229
1.45.2 Código de test del nodo Estado recepcionista 229
1.46 Genera token nuevo (WF-09) 232
1.46.1 Código base del nodo Genera token nuevo 232
1.46.2 Código de test del nodo Genera token nuevo 232
1.47 Recepcionista reactivada (WF-09) 235
1.47.1 Código base del nodo Recepcionista reactivada 235
1.47.2 Código de test del nodo Recepcionista reactivada 235
1.48 Estado paciente (WF-09) 238
1.48.1 Código base del nodo Estado paciente 238
1.48.2 Código de test del nodo Estado paciente 239
1.49 Paciente reactivado (WF-09) 242
1.49.1 Código base del nodo Paciente reactivado 242
1.49.2 Código de test del nodo Paciente reactivado 242
1.50 Genera token medico (WF-09) 245
1.50.1 Código base del nodo Genera token medico 245
1.50.2 Código de test del nodo Genera token medico 245
1.51 Medico agregado (WF-09) 248
1.51.1 Código base del nodo Medico agregado 248
1.51.2 Código de test del nodo Medico agregado 248
1.52 Genera token medico nuevo (WF-09) 251
1.52.1 Código base del nodo Genera token medico nuevo 251
1.52.2 Código de test del nodo Genera token medico nuevo 251
1.53 Medico reactivado (WF-09) 254
1.53.1 Código base del nodo Medico reactivado 254
1.53.2 Código de test del nodo Medico reactivado 254
1.54 Reparte reemplazos (WF-09) 257

1.54.1 Código base del nodo Reparte reemplazos 257
1.54.2 Código de test del nodo Reparte reemplazos 259
1.54.3 Nuevo código de Reparte Reemplazos. 268
1.54.4 Se corrió el mismo código test pero ya con la corrección del código base. 271
1.55 Rechazo estado medico (WF-09) 272
1.55.1 Código base del nodo Rechazo estado medico 272
1.55.2 Código de test del nodo Rechazo estado medico 272
1.56 Medico desactivado (WF-09) 274
1.56.1 Código base del nodo Medico desactivado 274
1.56.2 Código de test del nodo Medico desactivado 274
1.57 Prepara correos (WF-09) 277
1.57.1 Código base del nodo Prepara correos 277
1.57.2 Código de test del nodo Prepara correos 278
2 Ejecución de todos los test y todos pasaron y ya con todos los nodos corregidos. 282
3 Pruebas con simulación (Mockito) 283
3.1 Organización 283
3.2 Pruebas con simulación (Mockito) por nodo Code 283
3.3 ¿Por qué 52 nodos y no 57? 283
3.4 ¿Por qué 53 pruebas en 52 nodos? 283
4 Qué hace JaCoCo en las pruebas 291
4.1 Las cuatro métricas son: 291
Conclusiones 297

Introducción

SIGAC (Sistema de Gestión de Atención Clínica) automatiza el ciclo de las citas médicas de una clínica: recibe las solicitudes de los pacientes, las clasifica por prioridad, asigna médico y horario, notifica a los pacientes, gestiona cancelaciones y reasignaciones, registra notas clínicas cifradas y administra al personal. El sistema está construido sobre n8n, una plataforma de automatización por flujos, con PostgreSQL como base de datos, todo orquestado con Docker Compose en un entorno local.

La lógica del sistema se reparte entre diez flujos de trabajo, identificados de WF-00 a WF-09. Dentro de ellos, 57 nodos de tipo Code contienen el código JavaScript que valida los datos de entrada, cifra la información sensible, decide la asignación de citas y arma las respuestas que recibe cada usuario. Son los nodos donde se concentran las reglas del negocio y, por lo tanto, los puntos donde un error tiene mayor impacto: una validación incompleta puede dejar pasar datos inválidos a la base, y un error en el reparto de cupos puede asignar dos pacientes al mismo médico a la misma hora.

Este documento describe las pruebas unitarias aplicadas a esos 57 nodos. Las pruebas siguen los principios vistos en el curso con JUnit, Mockito y JaCoCo, trasladados a sus equivalentes en el ecosistema de JavaScript: Jest para las pruebas y la simulación de dependencias, y su informe de cobertura integrado. Cada nodo cuenta con su propio archivo de pruebas, diseñado a partir de técnicas formales de diseño de casos, y cada prueba se identifica con un código único que permite rastrearla desde la terminal hasta las tablas de este documento.

Las pruebas no solo verificaron el comportamiento esperado: detectaron tres defectos reales en seis nodos, que fueron corregidos en n8n y cuya corrección quedó confirmada por las mismas pruebas que los encontraron.

Objetivos

Objetivo general

Verificar, mediante pruebas unitarias automatizadas, que los 57 nodos Code del sistema SIGAC cumplen con las reglas de negocio, de validación y de seguridad para las que fueron diseñados, y detectar los defectos antes de que afecten la operación de la clínica.

Objetivos específicos

- Diseñar los casos de prueba de cada nodo con técnicas formales: partición de equivalencia, análisis de valores límite, cobertura de ramas, manejo de errores y verificación del contrato de salida.
- Ejecutar cada nodo de forma aislada, fuera de n8n, simulando las dependencias que recibe de la plataforma y de los nodos anteriores, a la manera de Mockito.
- Verificar las medidas de seguridad implementadas en el código: el cálculo de la huella de los tokens, el cifrado AES-256-GCM de los datos sensibles y la ausencia de información confidencial en las respuestas.
- Detectar defectos, documentar su causa y confirmar su corrección con la misma prueba que los detectó.
- Medir la cobertura de código alcanzada por las pruebas, con un umbral mínimo del 80 % de líneas y de ramas.
- Dejar un conjunto de pruebas reproducible, que pueda volver a ejecutarse cada vez que un flujo cambie en n8n.

Alcance

Elementos incluidos

Las pruebas cubren los 57 nodos Code de los flujos que contienen lógica en JavaScript:

Flujo	Propósito	Nodos Code
WF-01	Ingesta de solicitudes de cita	8
WF-02	Clasificación y asignación de citas	2
WF-04	Cancelación y reasignación de citas	5
WF-05	Cierre de citas	4
WF-06	Reporte semanal en PDF	2
WF-07	Registro de nota clínica	8
WF-08	Consulta de historial clínico	8
WF-09	Administración de personas	20
Total		**57**

Para cada nodo se verifican:

Las validaciones de datos de entrada: formatos, longitudes, rangos de fechas y horas, y campos obligatorios.

Las reglas de negocio: la prioridad de las consultas, el reparto de cupos, la reasignación de citas y los cambios de estado de las personas.

Las medidas de seguridad: la huella de los tokens, el cifrado y descifrado, y la minimización de los datos devueltos.

El contrato de salida: la forma exacta de lo que cada nodo entrega al siguiente.

La interacción con los nodos anteriores: qué nodos consulta y cuáles no, verificada con objetos simulados.

Elementos excluidos

Quedan fuera de las pruebas unitarias:

WF-00 y WF-03, que no contienen nodos Code: el primero registra errores y el segundo envía notificaciones, ambos con nodos propios de n8n.

Las consultas SQL de los nodos PostgreSQL: sus condiciones, inserciones y transacciones se ejecutan en la base de datos y corresponden a las pruebas de integración.

Los nodos propios de n8n, como Webhook, If, Switch y Send Email, cuyo funcionamiento es responsabilidad de la plataforma.

El funcionamiento de los flujos completos de extremo a extremo, que también corresponde a las pruebas de integración.

Entorno y herramientas

Entorno de ejecución

Las pruebas se ejecutan en el mismo equipo donde se desarrolla el sistema:

Componente	Detalle
Sistema operativo	Windows con WSL (Subsistema de Windows para Linux), distribución Ubuntu
Entorno de ejecución	Node.js v24.22.0, instalado con nvm
Ubicación de las pruebas	Carpeta pruebas/ del repositorio del proyecto
Sistema bajo prueba	n8n 2.x y PostgreSQL 16, en contenedores de Docker Compose
Control de versiones	Git, con repositorio en GitHub

Las pruebas unitarias no necesitan que los contenedores estén en ejecución: los nodos se ejecutan directamente en Node.js, con sus dependencias simuladas.

Herramientas

Herramienta	Versión	Uso en el proyecto
Jest	30.5.2	Marco de pruebas: define, ejecuta y reporta las pruebas; simula dependencias y mide la cobertura
Luxon	3.x	Librería de fechas que n8n pone a disposición de los nodos Code como DateTime; se instala para que los nodos que la usan funcionen fuera de n8n
npm	Incluido con Node.js	Gestión de dependencias y ejecución de los scripts de prueba
extraer_nodos.js	Propio	Generador que lee los flujos exportados de n8n y escribe cada nodo Code como un archivo independiente en nodos/, con su código exacto
simulacion.js	Propio	Herramientas compartidas que simulan lo que n8n entrega a cada nodo: la entrada, los nodos anteriores y las variables de entorno
Visual Studio Code	—	Edición de los archivos de prueba

Equivalencia con las herramientas vistas en el curso

El curso trabajó las pruebas con herramientas de Java. Jest cumple las mismas funciones en JavaScript:

Herramienta del curso	**Función**	**Equivalente en SIGAC**
JUnit	Definir y ejecutar pruebas unitarias	Jest: test(), describe()
assertEquals, assertThrows	Verificar resultados y excepciones	expect().toBe(), toEqual(), toThrow()
@ParameterizedTest	Repetir una prueba con varios valores	test.each()
Mockito	Simular dependencias y verificar su uso	jest.fn(), toHaveBeenCalledWith(), not.toHaveBeenCalled()
JaCoCo	Medir la cobertura de código	jest --coverage
pom.xml (Maven)	Declarar dependencias y tareas	package.json (npm)

Scripts de ejecución

Comando	**Qué ejecuta**	**Equivalente en Maven**
npm run extraer	Regenera nodos/ a partir de los flujos exportados	—
npm run test:unitarias	Las 57 suites de pruebas unitarias	mvn clean test
npm run test:cobertura	Las pruebas unitarias con el informe de cobertura	mvn clean verify con JaCoCo

Uso de inteligencia artificial como herramienta de apoyo

Para la elaboración de las pruebas conté con el apoyo de Claude Opus 5.5, un modelo de inteligencia artificial desarrollado por Anthropic. Su uso se limitó a acelerar el diseño y la escritura de las pruebas; las decisiones, la revisión y la corrección del sistema fueron trabajo propio.

El proceso se desarrolló en cuatro etapas:

1. Recolección del código base. El primer paso fue reunir el código de los 57 nodos Code del sistema. Para no copiarlo a mano, con el riesgo de que la copia quedara desfasada, se construyó el generador extraer_nodos.js, que lee los flujos exportados de n8n y escribe cada nodo como un archivo independiente. Con el comando npm run extraer, la carpeta nodos/ queda siempre idéntica al código publicado en n8n.

2. Análisis y propuesta de casos de prueba. Escribir a mano las pruebas de 57 nodos habría requerido mucho más tiempo del disponible. Por eso le entregué a Claude el código base de cada nodo y le pedí que lo leyera, lo analizara y me sugiriera cuántas pruebas y cuáles correspondían a cada uno. Según lo visto en clase, la cantidad de combinaciones posibles de un fragmento de código crece como 2^n, donde n es el número de condiciones que contiene. Como probar todas las combinaciones no es práctico en los nodos con muchas reglas, la selección de casos se apoyó en técnicas formales de diseño: partición de equivalencia, valores límite, cobertura de ramas, manejo de errores y contrato de salida. Así se obtiene un conjunto de pruebas que recorre cada condición en sus dos sentidos sin necesidad de ejecutar todas las combinaciones.

3. Revisión y generación de los archivos. Revisé cada propuesta para confirmar que los casos correspondían al comportamiento esperado de cada nodo. Una vez validadas, le pedí que generara el archivo de pruebas de cada nodo Code, con un archivo por nodo y un identificador único por prueba. Esto me permitió avanzar con mayor rapidez sin renunciar a una cobertura completa.

4. Ejecución y corrección de defectos. Ejecuté cada archivo de pruebas en mi entorno, uno por uno, y documenté los resultados. Las pruebas revelaron defectos reales en el código que tenía publicado en n8n, que habían pasado inadvertidos durante el desarrollo y las pruebas manuales:

Defecto	Nodos afectados	Qué ocurría
12.23	WF-01 · Valida datos	Un valor true se aceptaba como el médico 1
12.24	WF-07, WF-08 y WF-09 · Valida datos	El mismo error con los identificadores de cita y de persona; en WF-09, un true podía dar de baja a la persona 1
12.25	WF-02 · Reparte cupos y WF-09 · Reparte reemplazos	Se podían asignar citas que se cruzaban en el horario de un mismo médico

Corregí los seis nodos en n8n, exporté los flujos, regeneré el código base y volví a ejecutar las pruebas. Las mismas pruebas que habían detectado los defectos confirmaron su corrección, y la suite completa terminó con las 857 pruebas superadas.

La experiencia mostró el valor de las pruebas unitarias más allá de la verificación: los tres defectos encontrados no se habían manifestado en las pruebas manuales, porque solo aparecían con entradas que nadie habría enviado a propósito, como un valor booleano en lugar de un número, o una hora fuera de los intervalos habituales de media hora. Las pruebas automatizadas las encontraron porque se diseñaron para probar sistemáticamente los límites de cada regla, no solo los casos esperados.

1 Pruebas unitarias

1.1 Hash del token (WF-01)

El nodo Hash del token recibe la petición del webhook, lee el token de la cabecera x-sigac-token y calcula su huella SHA-256, que es la que se compara contra la base de datos. El token nunca se compara ni se guarda en claro. Si la petición no trae token, el nodo usa el valor centinela sin-token, que nunca puede coincidir con una huella real, porque una huella SHA-256 solo contiene caracteres hexadecimales.

El nodo tiene tres ramas: la petición con o sin cabeceras (headers || {}), con o sin cuerpo (body || {}), y con o sin token (token ? hash : 'sin-token'). Las pruebas recorren las tres, verifican las propiedades de la huella y comprueban que el token en claro nunca salga del nodo.

1.1.1 Código base del nodo Hash del token

```
const crypto = require('crypto');

const cabeceras = $input.first().json.headers || {};
const cuerpo    = $input.first().json.body    || {};

const token = cabeceras['x-sigac-token'] || '';

const tokenHash = token
  ? crypto.createHash('sha256').update(token).digest('hex')
  : 'sin-token';

return [{
  json: {
    token_hash: tokenHash,
    tiene_token: token.length > 0,
    datos: cuerpo
  }
}];
```

1.1.2 Código de test del nodo Hash del token

```
// sigac.wf01.hashdeltoken.test.js
// pruebas unitarias del nodo Hash del token de WF-01. lee el token de
// la cabecera x-sigac-token y calcula su huella sha-256, que es la que
// se compara contra la base. el token nunca se compara ni se guarda en
// claro. sin token, usa el centinela sin-token, que nunca puede
// coincidir con una huella real.
// tecnicas: particion de equivalencia, valores limite, cobertura de
// ramas (headers || {}, body || {}, token ? hash : centinela),
// propiedades del hash y contrato de salida.
// cada prueba lleva su ID de la tabla de pruebas: UT-HASH01-NN


'use strict';
```

```
const hashDelToken = require('../nodos/wf01_hash_del_token.js');
const { ejecutar, sha256 } = require('../simulacion.js');

// arma la peticion que entrega el Webhook y ejecuta el nodo
const hash = peticion => ejecutar(hashDelToken, { entrada: [peticion] }).salida;

describe('WF-01 · Hash del token', () => {
  test('UT-HASH01-01 · con token: calcula su huella sha-256', () => {
    const salida = hash({ headers: { 'x-sigac-token': 'abc' }, body: {} });
    expect(salida[0].json.token_hash).toBe(sha256('abc'));
    expect(salida[0].json.tiene_token).toBe(true);
  });

  test('UT-HASH01-02 · sin cabecera de token: centinela sin-token', () => {
    const salida = hash({ headers: {}, body: {} });
    expect(salida[0].json.token_hash).toBe('sin-token');
    expect(salida[0].json.tiene_token).toBe(false);
  });

  test('UT-HASH01-03 · token vacio: centinela sin-token', () => {
    const salida = hash({ headers: { 'x-sigac-token': '' }, body: {} });
    expect(salida[0].json.token_hash).toBe('sin-token');
    expect(salida[0].json.tiene_token).toBe(false);
  });

  test('UT-HASH01-04 · peticion sin headers: no falla y usa el centinela', () => {
    const salida = hash({ body: {} });
    expect(salida[0].json.token_hash).toBe('sin-token');
  });

  test('UT-HASH01-05 · peticion sin body: datos queda como objeto vacio', () => {
    const salida = hash({ headers: { 'x-sigac-token': 'abc' } });
    expect(salida[0].json.datos).toEqual({});
  });

  test('UT-HASH01-06 · el cuerpo pasa intacto en datos', () => {
    const cuerpo = { dpi: '1234567890123', nombre: 'Ana', anidado: { a: 1 } };
    const salida = hash({ headers: {}, body: cuerpo });
    expect(salida[0].json.datos).toEqual(cuerpo);
  });

  test('UT-HASH01-07 · cabecera en mayusculas: no se lee (n8n las entrega en minusculas)', ()
=> {
    const salida = hash({ headers: { 'X-SIGAC-TOKEN': 'abc' }, body: {} });
```

```
  expect(salida[0].json.token_hash).toBe('sin-token');
});

test('UT-HASH01-08 · token con espacios: se usa tal cual, sin recortar', () => {
  const salida = hash({ headers: { 'x-sigac-token': ' abc ' }, body: {} });
  expect(salida[0].json.token_hash).toBe(sha256(' abc '));
  expect(salida[0].json.token_hash).not.toBe(sha256('abc'));
});

test('UT-HASH01-09 · el mismo token da siempre la misma huella', () => {
  const a = hash({ headers: { 'x-sigac-token': 'abc' }, body: {} });
  const b = hash({ headers: { 'x-sigac-token': 'abc' }, body: {} });
  expect(a[0].json.token_hash).toBe(b[0].json.token_hash);
});

test('UT-HASH01-10 · tokens distintos dan huellas distintas', () => {
  const a = hash({ headers: { 'x-sigac-token': 'abc' }, body: {} });
  const b = hash({ headers: { 'x-sigac-token': 'abd' }, body: {} });
  expect(a[0].json.token_hash).not.toBe(b[0].json.token_hash);
});

test('UT-HASH01-11 · la huella tiene 64 caracteres hexadecimales en minuscula', () => {
  const salida = hash({ headers: { 'x-sigac-token': 'abc' }, body: {} });
  expect(salida[0].json.token_hash).toMatch(/^[0-9a-f]{64}$/);
});

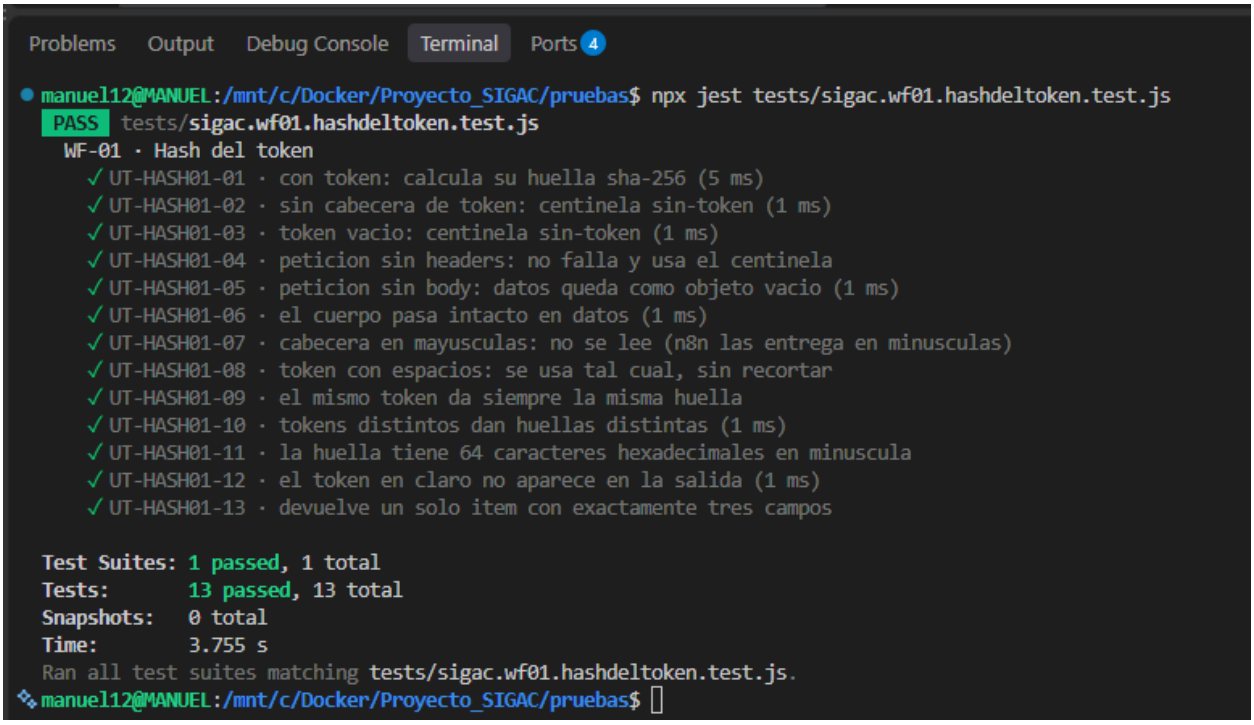
test('UT-HASH01-12 · el token en claro no aparece en la salida', () => {
  // el token lleva guiones, que nunca aparecen en un hexadecimal:
  // si se encuentra en la salida, es porque el token salio en claro
  const token = 'token-secreto-de-prueba';
  const salida = hash({ headers: { 'x-sigac-token': token }, body: {} });
  expect(JSON.stringify(salida)).not.toContain(token);
});

test('UT-HASH01-13 · devuelve un solo item con exactamente tres campos', () => {
  const salida = hash({ headers: { 'x-sigac-token': 'abc' }, body: {} });
  expect(salida).toHaveLength(1);
  expect(Object.keys(salida[0].json).sort()).toEqual(['datos', 'tiene_token', 'token_hash']);
});
});
```

1.1.3 Tabla de pruebas unitarias realizadas al nodo Hash del token (WF-01)

ID	Función	Caso de prueba	Entrada	Resultado esperado	Estado
UT-HASH01-01	Hash del token	Token presente (Happy Path)	Cabecera x-sigac-token: "abc"	token_hash = SHA-256 de "abc"; tiene_token: true	Pass
UT-HASH01-02	Hash del token	Sin cabecera de token	headers: {}	token_hash: "sin-token"; tiene_token: false	Pass
UT-HASH01-03	Hash del token	Token vacío	Cabecera x-sigac-token: ""	token_hash: "sin-token"; tiene_token: false	Pass
UT-HASH01-04	Hash del token	Petición sin headers	Petición solo con body	token_hash: "sin-token", sin lanzar error	Pass
UT-HASH01-05	Hash del token	Petición sin body	Petición solo con headers	datos: {}	Pass
UT-HASH01-06	Hash del token	El cuerpo pasa intacto	body con DPI, nombre y un objeto anidado	datos idéntico al cuerpo recibido	Pass
UT-HASH01-07	Hash del token	Cabecera en mayúsculas	Cabecera X-SIGAC-TOKEN: "abc"	token_hash: "sin-token": el nodo lee la cabecera en minúsculas, como la entrega n8n	Pass
UT-HASH01-08	Hash del token	Token con espacios	Cabecera x-sigac-token: " abc "	Huella de " abc ", distinta de la de "abc": el nodo no recorta	Pass
UT-HASH01-09	Hash del token	Huella determinista	El token "abc" dos veces	Las dos huellas son iguales	Pass
UT-HASH01-10	Hash del token	Tokens distintos	"abc" y "abd"	Huellas distintas	Pass
UT-HASH01-11	Hash del token	Formato de la huella	Cabecera x-sigac-token: "abc"	64 caracteres hexadecimales en minúscula	Pass

UT-HASH01-12	Hash del token	El token en claro no sale del nodo	Cabecera con "token-secreto-de-prueba"	Ningún campo de la salida contiene el token	Pass
UT-HASH01-13	Hash del token	Contrato de salida	Cabecera x-sigac-token: "abc"	Un solo item con exactamente token_hash, tiene_token y datos	Pass



1.2 Valida datos (WF-01)

El nodo Valida datos de WF-01 revisa la solicitud de cita antes de que toque la base de datos: DPI, teléfono, correo, fechas, tipo de consulta, campos obligatorios, motivo y las preferencias del paciente (franja, hora, especialidad y médico). Contrasta el médico y la especialidad pedidos contra el catálogo de médicos activos, acumula todos los errores en una sola respuesta y entrega al siguiente nodo los datos ya recortados.

Es el nodo con más reglas del sistema: 31 ramas. Las pruebas se ejecutan con el reloj fijo en el 23 de octubre de 2026, de modo que «ayer», «hoy» y «mañana» no cambian con el tiempo. El catálogo simulado tiene tres médicos activos: 1 Medicina General, 2 Pediatría y 5 Ginecología.

1.2.1 Código base del nodo Valida datos

```
// valida los datos de la solicitud antes de tocar la base.
// en guatemala el dpi son 13 digitos y el telefono 8,
// empezando entre 2 y 8.
const datos = $('Hash del token').first().json.datos || {};
const errores = [];

let medicosActivos = $('Lee medicos activos').first().json.medicos || [];
if (typeof medicosActivos === 'string') {
  medicosActivos = JSON.parse(medicosActivos);
}

function normalizar(texto) {
  return String(texto || '')
    .normalize('NFD')
    .replace(/[\u0300-\u036f]/g, '')
    .trim()
    .toLowerCase();
}

function fechaReal(texto) {
  const valor = String(texto || '').trim();
  const partes = /^(\d{4})-(\d{2})-(\d{2})$/.exec(valor);
  if (!partes) return null;
  const [, anio, mes, dia] = partes.map(Number);
  const fecha = new Date(anio, mes - 1, dia);
  if (fecha.getFullYear() !== anio || fecha.getMonth() !== mes - 1 || fecha.getDate() !== dia) {
    return null;
  }
  return fecha;
}

const dpi = String(datos.dpi || '').trim();
if (!/^\d{13}$/.test(dpi)) {
  errores.push('El DPI debe tener exactamente 13 digitos');
```

```
}

const telefono = String(datos.telefono || '').trim();
if (telefono && !/^[2-8]\d{7}$/.test(telefono)) {
  errores.push('El telefono debe tener 8 digitos y empezar entre 2 y 8');
}

const correo = String(datos.correo || '').trim();
if (!/^[^@\s]+@[^@\s]+\.[^@\s]+$/.test(correo))  {
  errores.push('El correo no tiene un formato valido');
}

const hoy = new Date();
hoy.setHours(0, 0, 0, 0);
const fechaPref = fechaReal(datos.fecha  preferida);
if (!fechaPref) {
  errores.push('La  fecha preferida no es una fecha valida');
} else if (fechaPref < hoy) {
  errores.push('La  fecha preferida no puede estar en el pasado');
}

const tipos = ['urgencia', 'control', 'general'];
if (!tipos.includes(datos.tipo_consulta)) {
  errores.push('El tipo de consulta debe ser urgencia, control o general');
}

for (const campo of ['nombre', 'apellido', 'fecha_nacimiento']) {
  if (!String(datos[campo] || '').trim()) {
    errores.push('Falta  el campo ' + campo);
  }
}

if (String(datos.fecha_nacimiento || '').trim()) {
  const fechaNac = fechaReal(datos.fecha_nacimiento);
  if (!fechaNac) {
    errores.push('La  fecha de nacimiento debe ser una fecha valida con formato AAAA-MM-
DD');
  } else if (fechaNac > hoy) {
    errores.push('La  fecha de nacimiento no puede estar en el futuro');
  } else if (fechaNac.getFullYear() < 1900) {
    errores.push('La  fecha de nacimiento no puede ser anterior a 1900');
  }
}

if (String(datos.motivo || '').length > 255) {
```

```
  errores.push('El motivo no puede exceder 255 caracteres');
}

const franjas = ['manana', 'tarde', 'indiferente'];
const franja = String(datos.franja_preferida || 'indiferente').trim();
if (!franjas.includes(franja))  {
  errores.push('La franja debe ser manana, tarde o indiferente');
}

let hora = String(datos.hora_solicitada || '').trim() || null;
if (hora) {
  if (!/^([01]\d|2[0-3]):[0-5]\d$/.test(hora))  {
    errores.push('La hora solicitada debe tener el formato HH:MM');
  } else {
    const minutos = Number(hora.slice(0, 2)) * 60 + Number(hora.slice(3, 5));
    const enManana = minutos >= 480 && minutos < 720;
    const enTarde  = minutos >= 810 && minutos < 1020;
    if (!enManana && !enTarde) {
      errores.push('La hora solicitada debe estar entre 08:00 y 12:00 o entre 13:30 y 17:00');
    }
    if (franja === 'manana' && !enManana) {
      errores.push('La hora solicitada no corresponde a la franja de la manana');
    }
    if (franja === 'tarde' && !enTarde) {
      errores.push('La hora solicitada no corresponde a la franja de la tarde');
    }
  }
}

let especialidad = String(datos.especialidad_solicitada || '').trim() || null;
if (especialidad) {
  const encontrada = medicosActivos.find(m => normalizar(m.especialidad) ===
normalizar(especialidad));
  if (encontrada) {
    especialidad = encontrada.especialidad;
  } else {
    errores.push('No hay medicos activos con la especialidad ' + especialidad);
    especialidad = null;
  }
}

let idMedico = datos.id_medico_solicitado;
let medicoPedido = null;
if (idMedico !== undefined && idMedico !== null && idMedico !== '') {
  const valorMedico = (typeof idMedico === 'number' || typeof idMedico === 'string')
```

```
    ? String(idMedico).trim()
    : '';
  if (!/^[1-9]\d*$/.test(valorMedico)) {
    errores.push('El medico solicitado debe ser un identificador valido');
    idMedico = null;
  } else {
    idMedico = Number(valorMedico);
    medicoPedido = medicosActivos.find(m => m.id === idMedico) || null;
    if (!medicoPedido) {
      errores.push('El medico solicitado no existe o no esta activo');
      idMedico = null;
    }
  }
} else {
  idMedico = null;
}

if (medicoPedido && especialidad && medicoPedido.especialidad !== especialidad) {
  errores.push('El medico solicitado no pertenece a la especialidad ' + especialidad);
}

const dpiEnmascarado = dpi.length >= 4 ? '*'.repeat(9) + dpi.slice(-4) : '****';

const datosLimpios = {
  ...datos,
  dpi: dpi,
  telefono: telefono,
  correo: correo,
  nombre: String(datos.nombre || '').trim(),
  apellido: String(datos.apellido || '').trim(),
  fecha_nacimiento: String(datos.fecha_nacimiento || '').trim(),
  fecha_preferida: String(datos.fecha_preferida || '').trim()
};

return [{
  json: {
    valido: errores.length === 0,
    errores: errores,
    rol: $('Verifica token').first().json.rol,
    dpi_enmascarado: dpiEnmascarado,
    franja_preferida: franja,
    hora_solicitada: hora,
    especialidad_solicitada: especialidad,
    id_medico_solicitado: idMedico,
    datos: datosLimpios
```

```
  }
}];
```

1.2.2 Código de test del nodo Valida datos

```
// sigac.wf01.validadatos.test.js

// pruebas unitarias del nodo Valida datos de WF-01. es el nodo con
// mas reglas del sistema: 31 ramas. la matriz agrupa los casos por
// regla, de la A a la N.
// el reloj se fija en el 23 de octubre de 2026 a mediodia en Guatemala,
// para que "ayer", "hoy" y "manana" no cambien con el tiempo.
// el catalogo simulado de medicos activos es:
//   1 Medicina General · 2 Pediatria · 5 Ginecologia
// cada prueba lleva su ID de la tabla de pruebas: UT-VAL01-NN

'use strict';

const validaDatos = require('../nodos/wf01_valida_datos.js');
const { ejecutar } = require('../simulacion.js');

const CATALOGO = [
  { id: 1, especialidad: 'Medicina General' },
  { id: 2, especialidad: 'Pediatria' },
  { id: 5, especialidad: 'Ginecologia' }
];

// una solicitud que cumple todas las reglas. cada prueba cambia
// solo el campo que quiere verificar
const valida = () => ({
  dpi: '1234567890123',
  nombre: 'Ana',
  apellido: 'Coc',
  telefono: '55551234',
  correo: 'ana@correo.com',
  fecha_nacimiento: '1990-05-10',
  tipo_consulta: 'general',
  motivo: 'control',
  fecha_preferida: '2026-10-24'
});

// ejecuta el nodo con los datos y el catalogo indicados
function validar(datos, medicos = CATALOGO, hash = { datos }) {
  return ejecutar(validaDatos, {
    anteriores: {
      'Hash del token': hash,
```

```
    'Lee medicos activos': { medicos },
    'Verifica token': { rol: 'recepcion' }
  }
 });
}

// atajo: solo la salida del nodo
const salidaDe = (cambios, medicos) => validar({ ...valida(), ...cambios },
medicos).salida[0].json;

beforeAll(() => jest.useFakeTimers({ now: new Date('2026-10-23T18:00:00Z') }));
afterAll(() => jest.useRealTimers());

// ---------------------------------------------------------
// A. DPI
describe('WF-01 · Valida datos · A. DPI', () => {
 test.each([
  ['UT-VAL01-01', '12 digitos', '123456789012', false],
  ['UT-VAL01-02', '13 digitos', '1234567890123', true],
  ['UT-VAL01-03', '14 digitos', '12345678901234', false],
  ['UT-VAL01-04', 'con una letra', '123456789012A', false]
 ])('%s · DPI de %s', (_, __, dpi, esperado) => {
  expect(salidaDe({ dpi }).valido).toBe(esperado);
 });

 test('UT-VAL01-05 · sin DPI: error', () => {
  const datos = valida();
  delete datos.dpi;
  expect(validar(datos).salida[0].json.errores).toContain('El DPI debe tener exactamente 13
digitos');
 });

 test('UT-VAL01-06 · DPI con espacios: valido y sale recortado (defecto 12.11)', () => {
  const salida = salidaDe({ dpi: ' 1234567890123 ' });
  expect(salida.valido).toBe(true);
  expect(salida.datos.dpi).toBe('1234567890123');
 });

 test('UT-VAL01-07 · el DPI se enmascara dejando los ultimos 4 digitos', () => {
  expect(salidaDe({}).dpi_enmascarado).toBe('*********0123');
 });

 test('UT-VAL01-08 · un DPI de menos de 4 caracteres se enmascara completo', () => {
  expect(salidaDe({ dpi: '12' }).dpi_enmascarado).toBe('****');
 });
```

```
});

// ---------------------------------------------------------
// B. Telefono (opcional)
describe('WF-01 · Valida datos · B. Telefono', () => {
  test('UT-VAL01-09 · sin telefono: valido, porque es opcional', () => {
    const datos = valida();
    delete datos.telefono;
    expect(validar(datos).salida[0].json.valido).toBe(true);
  });

  test.each([
    ['UT-VAL01-10', 'empieza en 1', '12345678', false],
    ['UT-VAL01-11', 'empieza en 2', '22345678', true],
    ['UT-VAL01-12', 'empieza en 8', '82345678', true],
    ['UT-VAL01-13', 'empieza en 9', '92345678', false],
    ['UT-VAL01-14', 'tiene 7 digitos', '5555123', false],
    ['UT-VAL01-15', 'tiene 9 digitos', '555512345', false]
  ])('%s · telefono que %s', ( ,     , telefono, esperado) => {
    expect(salidaDe({ telefono }).valido).toBe(esperado);
  });

  test('UT-VAL01-16 · telefono con espacios: valido y sale recortado', () => {
    const salida = salidaDe({ telefono: ' 55551234 ' });
    expect(salida.valido).toBe(true);
    expect(salida.datos.telefono).toBe('55551234');
  });
});

// ---------------------------------------------------------
// C. Correo
describe('WF-01 · Valida datos · C. Correo', () => {
  test.each([
    ['UT-VAL01-17', 'valido', 'ana@correo.com', true],
    ['UT-VAL01-18', 'sin arroba', 'ana.correo.com', false],
    ['UT-VAL01-19', 'sin punto en el dominio', 'ana@correo', false],
    ['UT-VAL01-20', 'con un espacio', 'ana @correo.com', false]
  ])('%s · correo %s', (_, __, correo, esperado) => {
    expect(salidaDe({ correo }).valido).toBe(esperado);
  });

  test('UT-VAL01-21 · sin correo: error', () => {
    const datos = valida();
    delete datos.correo;
    expect(validar(datos).salida[0].json.errores).toContain('El correo no tiene un formato valido');
```

```
  });
});

// ---------------------------------------------------------
// D. Fecha preferida
describe('WF-01 · Valida datos · D. Fecha preferida', () => {
  test('UT-VAL01-22 · ayer: error de fecha en el pasado', () => {
    expect(salidaDe({ fecha_preferida: '2026-10-22' }).errores)
      .toContain('La fecha preferida no puede estar en el pasado');
  });

  test.each([
    ['UT-VAL01-23', 'hoy', '2026-10-23'],
    ['UT-VAL01-24', 'manana', '2026-10-24']
  ])('%s · %s: valido', (  ,     , fecha  preferida) => {
    expect(salidaDe({ fecha_preferida }).valido).toBe(true);
  });

  test.each([
    ['UT-VAL01-25', '30 de febrero (defecto 12.14)', '2027-02-30'],
    ['UT-VAL01-26', 'en formato dd/mm/aaaa', '24/10/2026']
  ])('%s · %s: fecha invalida', (_, __, fecha_preferida) => {
    expect(salidaDe({ fecha_preferida }).errores).toContain('La fecha preferida no es una fecha
valida');
  });

  test('UT-VAL01-27 · sin fecha preferida: fecha invalida', () => {
    const datos = valida();
    delete datos.fecha_preferida;
    expect(validar(datos).salida[0].json.errores).toContain('La  fecha preferida no es una fecha
valida');
  });
});

// ---------------------------------------------------------
// E. Tipo de consulta
describe('WF-01 · Valida datos · E. Tipo de consulta', () => {
  test.each(['urgencia', 'control', 'general'])('UT-VAL01-28 · %s: valido', tipo_consulta => {
    expect(salidaDe({ tipo  consulta }).valido).toBe(true);
  });

  test('UT-VAL01-29 · un tipo que no existe: error', () => {
    expect(salidaDe({ tipo  consulta: 'revision' }).errores)
      .toContain('El tipo de consulta debe ser urgencia, control o general');
  });
```

```
  test('UT-VAL01-30 · General con mayuscula: error, la lista es exacta', () => {
    expect(salidaDe({ tipo_consulta: 'General' }).valido).toBe(false);
  });
});

// ---------------------------------------------------------
// F. Campos obligatorios
describe('WF-01 · Valida datos · F. Campos obligatorios', () => {
  test.each(['nombre', 'apellido', 'fecha  nacimiento'])('UT-VAL01-31  · sin %s: error', campo => {
    const datos = valida();
    delete datos[campo];
    expect(validar(datos).salida[0].json.errores).toContain('Falta  el campo ' + campo);
  });

  test('UT-VAL01-32 · nombre con solo espacios: cuenta como ausente', () => {
    expect(salidaDe({ nombre: '   ' }).errores).toContain('Falta el campo nombre');
  });

  test('UT-VAL01-33 · nombre y apellido con espacios: salen recortados', () => {
    const salida = salidaDe({ nombre: '  Ana  ', apellido: ' Coc ' });
    expect(salida.valido).toBe(true);
    expect(salida.datos.nombre).toBe('Ana');
    expect(salida.datos.apellido).toBe('Coc');
  });
});

// ---------------------------------------------------------
// G. Fecha de nacimiento
describe('WF-01 · Valida datos · G. Fecha de nacimiento', () => {
  const FORMATO = 'La fecha de nacimiento debe ser una fecha valida con formato AAAA-
MM-DD';

  test.each([
    ['UT-VAL01-34', 'texto (defecto 12.14)', 'abc'],
    ['UT-VAL01-35', '30 de febrero', '1990-02-30']
  ])('%s · %s: error de formato', ( ,     , fecha  nacimiento) => {
    expect(salidaDe({ fecha_nacimiento }).errores).toContain(FORMATO);
  });

  test('UT-VAL01-36 · manana: error, no puede ser futura', () => {
    expect(salidaDe({ fecha_nacimiento: '2026-10-24' }).errores)
      .toContain('La fecha de nacimiento no puede estar en el futuro');
  });
```

```
  test('UT-VAL01-37 · hoy: valido', () => {
    expect(salidaDe({ fecha_nacimiento: '2026-10-23' }).valido).toBe(true);
  });

  test('UT-VAL01-38 · 31/12/1899: error, anterior a 1900', () => {
    expect(salidaDe({ fecha_nacimiento: '1899-12-31' }).errores)
      .toContain('La fecha de nacimiento no puede ser anterior a 1900');
  });

  test('UT-VAL01-39 · 01/01/1900: valido', () => {
    expect(salidaDe({ fecha_nacimiento: '1900-01-01' }).valido).toBe(true);
  });

  test('UT-VAL01-40 · vacia: solo el error de campo faltante, sin el de formato', () => {
    const errores = salidaDe({ fecha  nacimiento: '' }).errores;
    expect(errores).toContain('Falta el campo fecha_nacimiento');
    expect(errores).not.toContain(FORMATO);
  });
});

// ---------------------------------------------------------
// H. Motivo
describe('WF-01 · Valida datos · H. Motivo', () => {
  test('UT-VAL01-41 · sin motivo: valido', () => {
    const datos = valida();
    delete datos.motivo;
    expect(validar(datos).salida[0].json.valido).toBe(true);
  });

  test.each([
    ['UT-VAL01-42', 255, true],
    ['UT-VAL01-43', 256, false]
  ])('%s · motivo de %i caracteres', (_, largo, esperado) => {
    expect(salidaDe({ motivo: 'x'.repeat(largo) }).valido).toBe(esperado);
  });
});

// ---------------------------------------------------------
// I. Franja y hora
describe('WF-01 · Valida datos · I. Franja y hora', () => {
  test('UT-VAL01-44 · sin franja: queda indiferente', () => {
    expect(salidaDe({}).franja_preferida).toBe('indiferente');
  });
```

```
  test.each(['manana', 'tarde', 'indiferente'])('UT-VAL01-45 · franja %s: valida', franja_preferida
=> {
    expect(salidaDe({ franja_preferida }).valido).toBe(true);
  });

  test('UT-VAL01-46 · franja que no existe: error', () => {
    expect(salidaDe({ franja_preferida: 'noche' }).errores)
      .toContain('La franja debe ser manana, tarde o indiferente');
  });

  test('UT-VAL01-47 · franja con espacios: valida y recortada', () => {
    const salida = salidaDe({ franja_preferida: ' tarde ' });
    expect(salida.valido).toBe(true);
    expect(salida.franja_preferida).toBe('tarde');
  });

  test('UT-VAL01-48 · sin hora: hora_solicitada queda nula', () => {
    expect(salidaDe({}).hora_solicitada).toBeNull();
  });

  test.each([
    ['07:59', false], ['08:00', true], ['11:59', true], ['12:00', false]
  ])('UT-VAL01-49 · limite de la manana: %s', (hora_solicitada, esperado) => {
    expect(salidaDe({ hora_solicitada }).valido).toBe(esperado);
  });

  test.each([
    ['13:29', false], ['13:30', true], ['16:59', true], ['17:00', false]
  ])('UT-VAL01-50 · limite de la tarde: %s', (hora_solicitada, esperado) => {
    expect(salidaDe({ hora_solicitada }).valido).toBe(esperado);
  });

  test('UT-VAL01-51 · hora del almuerzo (12:45): error', () => {
    expect(salidaDe({ hora_solicitada: '12:45' }).errores)
      .toContain('La hora solicitada debe estar entre 08:00 y 12:00 o entre 13:30 y 17:00');
  });

  test.each(['8:00', '24:00', '08:60', 'ocho'])('UT-VAL01-52 · formato invalido: %s',
hora_solicitada => {
    expect(salidaDe({ hora_solicitada }).errores).toContain('La hora solicitada debe tener el
formato HH:MM');
  });

  test('UT-VAL01-53 · franja tarde con 09:00: error', () => {
    expect(salidaDe({ franja_preferida: 'tarde', hora_solicitada: '09:00' }).errores)
```

```
      .toContain('La hora solicitada no corresponde a la franja de la tarde');
  });

  test('UT-VAL01-54 · franja manana con 15:00: error', () => {
    expect(salidaDe({ franja_preferida: 'manana', hora_solicitada: '15:00' }).errores)
      .toContain('La hora solicitada no corresponde a la franja de la manana');
  });

  test('UT-VAL01-55 · franja indiferente con 15:00: valido', () => {
    expect(salidaDe({ franja  preferida: 'indiferente', hora  solicitada: '15:00' }).valido).toBe(true);
  });
});

// ---------------------------------------------------------
// J. Especialidad (defecto 12.20)
describe('WF-01 · Valida datos · J. Especialidad', () => {
  test('UT-VAL01-56 · sin especialidad: queda nula', () => {
    expect(salidaDe({}).especialidad_solicitada).toBeNull();
  });

  test('UT-VAL01-57 · especialidad con solo espacios: nula y sin error', () => {
    const salida = salidaDe({ especialidad_solicitada: '   ' });
    expect(salida.valido).toBe(true);
    expect(salida.especialidad_solicitada).toBeNull();
  });

  test('UT-VAL01-58 · pediatría en minusculas  y con tilde: se normaliza  a Pediatria', () => {
    const salida = salidaDe({ especialidad_solicitada: 'pediatría' });
    expect(salida.valido).toBe(true);
    expect(salida.especialidad_solicitada).toBe('Pediatria');
  });

  test('UT-VAL01-59 · especialidad sin medicos activos: error y queda nula', () => {
    const salida = salidaDe({ especialidad_solicitada: 'Cardiologia' });
    expect(salida.errores).toContain('No hay medicos activos con la especialidad Cardiologia');
    expect(salida.especialidad_solicitada).toBeNull();
  });
});

// ---------------------------------------------------------
// K. Medico solicitado (defectos 12.18 y 12.19)
describe('WF-01 · Valida datos · K. Medico solicitado', () => {
  test.each([
    ['ausente', undefined],
    ['nulo', null],
```

```
    ['texto vacio', '']
  ])('UT-VAL01-60 · medico %s: queda nulo, sin error', (_, id_medico_solicitado) => {
    const salida = salidaDe({ id_medico_solicitado });
    expect(salida.valido).toBe(true);
    expect(salida.id_medico_solicitado).toBeNull();
  });

  test('UT-VAL01-61 · medico 2 como numero: valido', () => {
    expect(salidaDe({ id_medico_solicitado: 2 }).id_medico_solicitado).toBe(2);
  });

  test('UT-VAL01-62 · medico "2" como texto: valido y convertido a numero', () => {
    const salida = salidaDe({ id_medico_solicitado: '2' });
    expect(salida.valido).toBe(true);
    expect(salida.id_medico_solicitado).toBe(2);
  });

  test.each([0, -1])('UT-VAL01-63 · medico %p: identificador invalido', id_medico_solicitado =>
{
    expect(salidaDe({ id_medico_solicitado }).errores)
      .toContain('El medico solicitado debe ser un identificador valido');
  });

  test.each([2.5, 'abc'])('UT-VAL01-64 · medico %p: identificador invalido',
id_medico_solicitado => {
    expect(salidaDe({ id_medico_solicitado }).errores)
      .toContain('El medico solicitado debe ser un identificador valido');
  });

  test('UT-VAL01-65 · medico 99, que no esta en el catalogo: error', () => {
    expect(salidaDe({ id_medico_solicitado: 99 }).errores)
      .toContain('El medico solicitado no existe o no esta activo');
  });

  // hipotesis A: Number(true) es 1, asi que true podria colarse como
  // el medico 1. un medico se pide con su numero, nunca con true.
  test('UT-VAL01-66 · medico true: debe rechazarse (hipotesis A)', () => {
    expect(salidaDe({ id_medico_solicitado: true }).valido).toBe(false);
  });
});
```

```
// ---------------------------------------------------------
// L. Combinacion medico y especialidad
describe('WF-01 · Valida datos · L. Combinacion medico y especialidad', () => {
  test('UT-VAL01-67 · medico 2 con Pediatria: valido', () => {
    expect(salidaDe({ id_medico_solicitado: 2, especialidad_solicitada: 'Pediatria'
}).valido).toBe(true);
  });

  test('UT-VAL01-68 · medico 2 con Ginecologia: error', () => {
    expect(salidaDe({ id_medico_solicitado: 2, especialidad_solicitada: 'Ginecologia' }).errores)
      .toContain('El medico solicitado no pertenece a la especialidad Ginecologia');
  });
});

// ---------------------------------------------------------
// M. Catalogo de medicos
// ---------------------------------------------------------
describe('WF-01 · Valida datos · M. Catalogo de medicos', () => {
  test('UT-VAL01-69 · catalogo como texto json: funciona igual que como arreglo', () => {
    const salida = salidaDe({ id_medico_solicitado: 5 }, JSON.stringify(CATALOGO));
    expect(salida.valido).toBe(true);
    expect(salida.id_medico_solicitado).toBe(5);
  });

  test('UT-VAL01-70 · catalogo nulo: cualquier medico o especialidad pedidos dan error', () => {
    expect(salidaDe({ id_medico_solicitado: 1 }, null).errores)
      .toContain('El medico solicitado no existe o no esta activo');
    expect(salidaDe({ especialidad_solicitada: 'Pediatria' }, null).errores)
      .toContain('No hay medicos activos con la especialidad Pediatria');
  });
});

// ---------------------------------------------------------
// N. Conjunto y contrato de salida
describe('WF-01 · Valida datos · N. Conjunto y contrato', () => {
  const ERRORES_CUERPO_VACIO = [
    'El DPI debe tener exactamente 13 digitos',
    'El correo no tiene un formato valido',
    'La fecha preferida no es una fecha valida',
    'El tipo de consulta debe ser urgencia, control o general',
    'Falta el campo nombre',
    'Falta el campo apellido',
    'Falta el campo fecha_nacimiento'
  ];
```

```
  test('UT-VAL01-71 · cuerpo vacio: todos los errores juntos en una sola respuesta', () => {
    const salida = validar({}).salida[0].json;
    expect(salida.valido).toBe(false);
    expect(salida.errores).toEqual(ERRORES_CUERPO_VACIO);
  });

  test('UT-VAL01-72 · Hash del token sin datos: no falla y valida como cuerpo vacio', () => {
    const salida = validar(undefined, CATALOGO, {}).salida[0].json;
    expect(salida.errores).toEqual(ERRORES_CUERPO_VACIO);
  });

  test('UT-VAL01-73 · solicitud valida: devuelve un item con los nueve campos', () => {
    const { salida } = validar(valida());
    expect(salida).toHaveLength(1);
    expect(Object.keys(salida[0].json).sort()).toEqual([
      'datos', 'dpi_enmascarado', 'errores', 'especialidad_solicitada', 'franja_preferida',
      'hora_solicitada', 'id_medico_solicitado', 'rol', 'valido'
    ]);
  });

  test('UT-VAL01-74 · el rol sale de Verifica token', () => {
    expect(salidaDe({}).rol).toBe('recepcion');
  });

  test('UT-VAL01-75 · consulta los tres nodos anteriores (verify)', () => {
    const { $ } = validar(valida());
    expect($).toHaveBeenCalledWith('Hash del token');
    expect($).toHaveBeenCalledWith('Lee medicos activos');
    expect($).toHaveBeenCalledWith('Verifica token');
  });

  test('UT-VAL01-76 · un campo no previsto se conserva en datos', () => {
    expect(salidaDe({ campo_extra: 'x' }).datos.campo_extra).toBe('x');
  });
});
```

1.2.3 Tabla de pruebas unitarias realizadas al nodo Valida datos (WF-01)

ID	Función	Caso de prueba	Entrada	Resultado esperado	Estado
UT-VAL01-01	Valida datos	DPI de 12 dígitos	dpi: "123456789012"	Error	Pass
UT-VAL01-02	Valida datos	DPI de 13 dígitos (Happy Path)	dpi: "1234567890123"	Válido	Pass
UT-VAL01-03	Valida datos	DPI de 14 dígitos	dpi: "12345678901234"	Error	Pass
UT-VAL01-04	Valida datos	DPI con una letra	dpi: "123456789012A"	Error	Pass
UT-VAL01-05	Valida datos	Sin DPI	Solicitud sin dpi	Error de DPI	Pass
UT-VAL01-06	Valida datos	DPI con espacios (defecto 12.11)	dpi: " 1234567890123 "	Válido y recortado	Pass
UT-VAL01-07	Valida datos	Enmascarado del DPI	DPI válido	*********0123	Pass
UT-VAL01-08	Valida datos	DPI de menos de 4 caracteres	dpi: "12"	****	Pass
UT-VAL01-09	Valida datos	Sin teléfono (opcional)	Solicitud sin telefono	Válido	Pass
UT-VAL01-10	Valida datos	Teléfono que empieza en 1	12345678	Error	Pass
UT-VAL01-11	Valida datos	Teléfono que empieza en 2	22345678	Válido	Pass
UT-VAL01-12	Valida datos	Teléfono que empieza en 8	82345678	Válido	Pass

UT-VAL01-13	Valida datos	Teléfono que empieza en 9	92345678	Error	Pass
UT-VAL01-14	Valida datos	Teléfono de 7 dígitos	5555123	Error	Pass
UT-VAL01-15	Valida datos	Teléfono de 9 dígitos	555512345	Error	Pass
UT-VAL01-16	Valida datos	Teléfono con espacios	" 55551234 "	Válido y recortado	Pass
UT-VAL01-17	Valida datos	Correo válido	ana@correo.com	Válido	Pass
UT-VAL01-18	Valida datos	Correo sin arroba	ana.correo.com	Error	Pass
UT-VAL01-19	Valida datos	Correo sin punto en el dominio	ana@correo	Error	Pass
UT-VAL01-20	Valida datos	Correo con un espacio	ana @correo.com	Error	Pass
UT-VAL01-21	Valida datos	Sin correo	Solicitud sin correo	Error de correo	Pass
UT-VAL01-22	Valida datos	Fecha preferida de ayer	2026-10-22	Error: en el pasado	Pass
UT-VAL01-23	Valida datos	Fecha preferida de hoy	2026-10-23	Válido	Pass
UT-VAL01-24	Valida datos	Fecha preferida de mañana	2026-10-24	Válido	Pass
UT-VAL01-25	Valida datos	30 de febrero (defecto 12.14)	2027-02-30	Error: fecha inválida	Pass

UT-VAL01-26	Valida datos	Fecha en formato dd/mm/aaaa	24/10/2026	Error: fecha inválida	Pass
UT-VAL01-27	Valida datos	Sin fecha preferida	Solicitud sin la fecha	Error: fecha inválida	Pass
UT-VAL01-28	Valida datos	Tipos de consulta válidos	urgencia, control, general	Válido (los tres)	Pass
UT-VAL01-29	Valida datos	Tipo de consulta inexistente	revision	Error	Pass
UT-VAL01-30	Valida datos	Tipo con mayúscula	General	Error: la lista es exacta	Pass
UT-VAL01-31	Valida datos	Campos obligatorios ausentes	Sin nombre, sin apellido, sin fecha_nacimiento	«Falta el campo …» para cada uno	Pass
UT-VAL01-32	Valida datos	Nombre con solo espacios	nombre: " "	Error: cuenta como ausente	Pass
UT-VAL01-33	Valida datos	Nombre y apellido con espacios	" Ana ", " Coc "	Válido y recortados	Pass
UT-VAL01-34	Valida datos	Fecha de nacimiento en texto (defecto 12.14)	abc	Error de formato	Pass
UT-VAL01-35	Valida datos	Fecha de nacimiento 30 de febrero	1990-02-30	Error de formato	Pass
UT-VAL01-36	Valida datos	Fecha de nacimiento futura	2026-10-24	Error: futura	Pass
UT-VAL01-37	Valida datos	Fecha de nacimiento de hoy	2026-10-23	Válido	Pass
UT-VAL01-38	Valida datos	Fecha de nacimiento antes de 1900	1899-12-31	Error	Pass

UT-VAL01-39	Valida datos	Fecha de nacimiento en 1900	1900-01-01	Válido	Pass
UT-VAL01-40	Valida datos	Fecha de nacimiento vacía	""	Solo «Falta el campo», sin error de formato	Pass
UT-VAL01-41	Valida datos	Sin motivo	Solicitud sin motivo	Válido	Pass
UT-VAL01-42	Valida datos	Motivo de 255 caracteres	255 caracteres	Válido	Pass
UT-VAL01-43	Valida datos	Motivo de 256 caracteres	256 caracteres	Error	Pass
UT-VAL01-44	Valida datos	Sin franja	Solicitud sin franja	franja_preferida: "indiferente"	Pass
UT-VAL01-45	Valida datos	Franjas válidas	manana, tarde, indiferente	Válido (las tres)	Pass
UT-VAL01-46	Valida datos	Franja inexistente	noche	Error	Pass
UT-VAL01-47	Valida datos	Franja con espacios	" tarde "	Válido, como tarde	Pass
UT-VAL01-48	Valida datos	Sin hora	Solicitud sin hora	hora_solicitada: null	Pass
UT-VAL01-49	Valida datos	Límites de la mañana	07:59 / 08:00 / 11:59 / 12:00	Error / Válido / Válido / Error	Pass
UT-VAL01-50	Valida datos	Límites de la tarde	13:29 / 13:30 / 16:59 / 17:00	Error / Válido / Válido / Error	Pass
UT-VAL01-51	Valida datos	Hora del almuerzo	12:45	Error: fuera de franja	Pass

UT-VAL01-52	Valida datos	Formatos de hora inválidos	8:00, 24:00, 08:60, ocho	Error de formato (los cuatro)	Pass
UT-VAL01-53	Valida datos	Tarde con hora de mañana	Franja tarde, 09:00	Error	Pass
UT-VAL01-54	Valida datos	Mañana con hora de tarde	Franja manana, 15:00	Error	Pass
UT-VAL01-55	Valida datos	Indiferente con cualquier hora	Franja indiferente, 15:00	Válido	Pass
UT-VAL01-56	Valida datos	Sin especialidad	Solicitud sin especialidad	especialidad_solicitada: null	Pass
UT-VAL01-57	Valida datos	Especialidad con solo espacios	" "	null, sin error	Pass
UT-VAL01-58	Valida datos	Especialidad normalizada (defecto 12.20)	pediatría	Válido, como Pediatria	Pass
UT-VAL01-59	Valida datos	Especialidad sin médicos activos (defecto 12.20)	Cardiologia	Error, y especialidad null	Pass
UT-VAL01-60	Valida datos	Médico ausente, nulo o vacío	undefined, null, ""	id_medico_solicitado: null, sin error	Pass
UT-VAL01-61	Valida datos	Médico como número	2	Válido	Pass
UT-VAL01-62	Valida datos	Médico como texto numérico	"2"	Válido, convertido a 2	Pass
UT-VAL01-63	Valida datos	Médico 0 o negativo	0, -1	Error: identificador inválido	Pass
UT-VAL01-64	Valida datos	Médico decimal o texto	2.5, "abc"	Error: identificador inválido	Pass

UT-VAL01-65	Valida datos	Médico inexistente (defecto 12.18)	99	Error: no existe o no está activo	Pass
UT-VAL01-66	Valida datos	Médico como booleano (defecto 12.23)	true	Error	**Fail**
UT-VAL01-67	Valida datos	Médico y especialidad coherentes	Médico 2 con Pediatria	Válido	Pass
UT-VAL01-68	Valida datos	Médico y especialidad contradictorios	Médico 2 con Ginecologia	Error	Pass
UT-VAL01-69	Valida datos	Catálogo como texto JSON	Catálogo en texto	Funciona igual que como arreglo	Pass
UT-VAL01-70	Valida datos	Catálogo nulo	medicos: null	Médico y especialidad pedidos dan error	Pass
UT-VAL01-71	Valida datos	Cuerpo vacío	{}	Los 7 errores juntos, en una sola respuesta	Pass
UT-VAL01-72	Valida datos	Hash del token sin datos	Salida sin datos	No falla; los mismos 7 errores	Pass
UT-VAL01-73	Valida datos	Contrato de salida	Solicitud válida	Un item con los 9 campos	Pass
UT-VAL01-74	Valida datos	Origen del rol	Solicitud válida	rol tomado de Verifica token	Pass
UT-VAL01-75	Valida datos	Nodos consultados (verify)	Solicitud válida	Consulta Hash del token, Lee medicos activos y Verifica token	Pass
UT-VAL01-76	Valida datos	Campo no previsto	campo_extra: "x"	Se conserva en datos	Pass

1.2.3.1 ***95 pruebas: 94 superadas y 1 fallida, que detectó el defecto 12.23.***

Defecto detectado por las pruebas

La prueba UT-VAL01-66 falló: el nodo aceptaba "id_medico_solicitado": true como si fuera el médico 1, porque Number(true) da 1. La corrección valida el identificador con una expresión regular sobre su texto, igual que id_cita en WF-04 y WF-05 (defecto 12.12).

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf01.validadatos.test.js
    ✓ UT-VAL01-73 · solicitud valida: devuelve un item con los nueve campos (1 ms)
    ✓ UT-VAL01-74 · el rol sale de Verifica token
    ✓ UT-VAL01-75 · consulta los tres nodos anteriores (verify) (1 ms)
    ✓ UT-VAL01-76 · un campo no previsto se conserva en datos (1 ms)

  ● WF-01 · Valida datos · K. Medico solicitado › UT-VAL01-66 · medico true: debe rechazarse (hipotesis A)

    expect(received).toBe(expected) // Object.is equality

    Expected: false
    Received: true

      395 |   // el medico 1. un medico se pide con su numero, nunca con true.
      396 |   test('UT-VAL01-66 · medico true: debe rechazarse (hipotesis A)', () => {
    > 397 |     expect(salidaDe({ id_medico_solicitado: true }).valido).toBe(false);
          |                                                            ^
      398 |   });
      399 | });
      400 |

      at Object.toBe (tests/sigac.wf01.validadatos.test.js:397:61)

Test Suites: 1 failed, 1 total
Tests:       1 failed, 94 passed, 95 total
Snapshots:   0 total
Time:        3.736 s
Ran all test suites matching tests/sigac.wf01.validadatos.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.2.4 Nuevo código base del nodo Valida datos

```
const datos = $('Hash del token').first().json.datos || {};
const errores = [];

let medicosActivos = $('Lee medicos activos').first().json.medicos || [];
if (typeof medicosActivos === 'string') {
  medicosActivos = JSON.parse(medicosActivos);
}

function normalizar(texto) {
  return String(texto || '')
    .normalize('NFD')
    .replace(/[\u0300-\u036f]/g, '')
    .trim()
    .toLowerCase();
}

function fechaReal(texto) {
  const valor = String(texto || '').trim();
  const partes = /^(\d{4})-(\d{2})-(\d{2})$/.exec(valor);
  if (!partes) return null;
```

```
  const [, anio, mes, dia] = partes.map(Number);
  const fecha = new Date(anio, mes - 1, dia);
  if (fecha.getFullYear() !== anio || fecha.getMonth() !== mes - 1 || fecha.getDate() !== dia) {
    return null;
  }
  return fecha;
}

const dpi = String(datos.dpi || '').trim();
if (!/^\d{13}$/.test(dpi)) {
  errores.push('El DPI debe tener exactamente 13 digitos');
}

const telefono = String(datos.telefono || '').trim();
if (telefono && !/^[2-8]\d{7}$/.test(telefono)) {
  errores.push('El telefono debe tener 8 digitos y empezar entre 2 y 8');
}

const correo = String(datos.correo || '').trim();
if (!/^[^@\s]+@[^@\s]+\.[^@\s]+$/.test(correo))  {
  errores.push('El correo no tiene un formato valido');
}

const hoy = new Date();
hoy.setHours(0, 0, 0, 0);
const fechaPref = fechaReal(datos.fecha_preferida);
if (!fechaPref) {
  errores.push('La  fecha preferida no es una fecha valida');
} else if (fechaPref < hoy) {
  errores.push('La  fecha preferida no puede estar en el pasado');
}

const tipos = ['urgencia', 'control', 'general'];
if (!tipos.includes(datos.tipo_consulta)) {
  errores.push('El tipo de consulta debe ser urgencia, control o general');
}

for (const campo of ['nombre', 'apellido', 'fecha_nacimiento']) {
  if (!String(datos[campo] || '').trim()) {
    errores.push('Falta  el campo ' + campo);
  }
}

if (String(datos.fecha  nacimiento || '').trim()) {
  const fechaNac = fechaReal(datos.fecha_nacimiento);
```

```
  if (!fechaNac) {
    errores.push('La fecha de nacimiento debe ser una fecha valida con formato AAAA-MM-
DD');
  } else if (fechaNac > hoy) {
    errores.push('La fecha de nacimiento no puede estar en el futuro');
  } else if (fechaNac.getFullYear() < 1900) {
    errores.push('La fecha de nacimiento no puede ser anterior a 1900');
  }
}

if (String(datos.motivo || '').length > 255) {
  errores.push('El motivo no puede exceder 255 caracteres');
}

const franjas = ['manana', 'tarde', 'indiferente'];
const franja = String(datos.franja_preferida || 'indiferente').trim();
if (!franjas.includes(franja)) {
  errores.push('La franja debe ser manana, tarde o indiferente');
}

let hora = String(datos.hora_solicitada || '').trim() || null;
if (hora) {
  if (!/^([01]\d|2[0-3]):[0-5]\d$/.test(hora)) {
    errores.push('La hora solicitada debe tener el formato HH:MM');
  } else {
    const minutos = Number(hora.slice(0, 2)) * 60 + Number(hora.slice(3, 5));
    const enManana = minutos >= 480 && minutos < 720;
    const enTarde = minutos >= 810 && minutos < 1020;
    if (!enManana && !enTarde) {
      errores.push('La hora solicitada debe estar entre 08:00 y 12:00 o entre 13:30 y 17:00');
    }
    if (franja === 'manana' && !enManana) {
      errores.push('La hora solicitada no corresponde a la franja de la manana');
    }
    if (franja === 'tarde' && !enTarde) {
      errores.push('La hora solicitada no corresponde a la franja de la tarde');
    }
  }
}

let especialidad = String(datos.especialidad_solicitada || '').trim() || null;
if (especialidad) {
  const encontrada = medicosActivos.find(m => normalizar(m.especialidad) ===
normalizar(especialidad));
  if (encontrada) {
```

```
    especialidad = encontrada.especialidad;
  } else {
    errores.push('No hay medicos activos con la especialidad ' + especialidad);
    especialidad = null;
  }
}

// cambio 5: el medico solicitado se contrasta contra los activos.
// antes este comentario decia que la consulta siguiente verificaba
// que existiera y estuviera activo, pero ninguna lo hacia: un id
// inexistente violaba la llave foranea y terminaba en WF-00, y uno
// inactivo dejaba la solicitud en lista de espera para siempre.
//
// defecto 12.23: antes se convertia con Number(), y Number(true) es 1,
// de modo que "id_medico_solicitado": true asignaba la cita al medico 1
// sin que nadie lo pidiera. ahora solo se acepta un numero o un texto
// que contenga un entero positivo, igual que id_cita en WF-04 y WF-05.
let idMedico = datos.id_medico_solicitado;
let medicoPedido = null;
if (idMedico !== undefined && idMedico !== null && idMedico !== '') {
  const valorMedico = (typeof idMedico === 'number' || typeof idMedico === 'string')
    ? String(idMedico).trim()
    : '';
  if (!/^[1-9]\d*$/.test(valorMedico)) {
    errores.push('El medico solicitado debe ser un identificador valido');
    idMedico = null;
  } else {
    idMedico = Number(valorMedico);
    medicoPedido = medicosActivos.find(m => m.id === idMedico) || null;
    if (!medicoPedido) {
      errores.push('El medico solicitado no existe o no esta activo');
      idMedico = null;
    }
  }
} else {
  idMedico = null;
}

// cambio 5: si vienen los dos, deben coincidir. es la misma idea que
// rechazar la franja tarde con una hora de la manana.
if (medicoPedido && especialidad && medicoPedido.especialidad !== especialidad) {
  errores.push('El medico solicitado no pertenece a la especialidad ' + especialidad);
}

// el dpi se enmascara para que nunca aparezca completo en logs
```

```
const dpiEnmascarado = dpi.length >= 4 ? '*'.repeat(9) + dpi.slice(-4) : '****';

const datosLimpios = {
  ...datos,
  dpi: dpi,
  telefono: telefono,
  correo: correo,
  nombre: String(datos.nombre || '').trim(),
  apellido: String(datos.apellido || '').trim(),
  fecha_nacimiento: String(datos.fecha_nacimiento || '').trim(),
  fecha_preferida: String(datos.fecha_preferida || '').trim()
};

return [{
  json: {
    valido: errores.length === 0,
    errores: errores,
    rol: $('Verifica token').first().json.rol,
    dpi_enmascarado: dpiEnmascarado,
    franja_preferida: franja,
    hora_solicitada: hora,
    especialidad_solicitada: especialidad,
    id_medico_solicitado: idMedico,
    datos: datosLimpios
  }
}];
```

1.2.4.1 Se ejecuto el mismo test pero ya con el código ya corregido.

Defecto detectado y corregido gracias a las pruebas. La primera ejecución dio 94 pruebas superadas y 1 fallida: UT-VAL01-66 demostró que el nodo aceptaba "id_medico_solicitado": true como si fuera el médico 1, porque Number(true) da 1. Se corrigió en n8n validando el identificador con una expresión regular sobre su texto, igual que id_cita en WF-04 y WF-05 (defecto 12.12). Tras exportar WF-01 y regenerar los nodos, la misma prueba pasó: 95 pruebas, 95 superadas (defecto 12.23).

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf01.validadatos.test.js
    ✓ UT-VAL01-02 · DPI de 13 digitos (1 ms)
    ✓ UT-VAL01-03 · DPI de 14 digitos
    ✓ UT-VAL01-04 · DPI de con una letra
    ✓ UT-VAL01-05 · sin DPI: error
    ✓ UT-VAL01-06 · DPI con espacios: valido y sale recortado (defecto 12.11)
    ✓ UT-VAL01-07 · el DPI se enmascara dejando los ultimos 4 digitos (1 ms)
    ✓ UT-VAL01-08 · un DPI de menos de 4 caracteres se enmascara completo
  WF-01 · Valida datos · B. Telefono
    ✓ UT-VAL01-09 · sin telefono: valido, porque es opcional
    ✓ UT-VAL01-10 · telefono que empieza en 1
    ✓ UT-VAL01-11 · telefono que empieza en 2 (1 ms)
    ✓ UT-VAL01-12 · telefono que empieza en 8
    ✓ UT-VAL01-13 · telefono que empieza en 9 (1 ms)
    ✓ UT-VAL01-14 · telefono que tiene 7 digitos
    ✓ UT-VAL01-15 · telefono que tiene 9 digitos
    ✓ UT-VAL01-16 · telefono con espacios: valido y sale recortado (1 ms)
  WF-01 · Valida datos · C. Correo
    ✓ UT-VAL01-17 · correo valido (1 ms)
    ✓ UT-VAL01-18 · correo sin arroba
    ✓ UT-VAL01-19 · correo sin punto en el dominio
    ✓ UT-VAL01-20 · correo con un espacio (1 ms)
    ✓ UT-VAL01-21 · sin correo: error (1 ms)
  WF-01 · Valida datos · D. Fecha preferida
    ✓ UT-VAL01-22 · ayer: error de fecha en el pasado (1 ms)
    ✓ UT-VAL01-23 · hoy: valido
    ✓ UT-VAL01-24 · manana: valido
    ✓ UT-VAL01-25 · 30 de febrero (defecto 12.14): fecha invalida (1 ms)
    ✓ UT-VAL01-26 · en formato dd/mm/aaaa: fecha invalida (1 ms)
    ✓ UT-VAL01-27 · sin fecha preferida: fecha invalida
  WF-01 · Valida datos · E. Tipo de consulta
    ✓ UT-VAL01-28 · urgencia: valido (1 ms)
    ✓ UT-VAL01-28 · control: valido
    ✓ UT-VAL01-28 · general: valido
    ✓ UT-VAL01-29 · un tipo que no existe: error (1 ms)
    ✓ UT-VAL01-30 · General con mayuscula: error, la lista es exacta (1 ms)
  WF-01 · Valida datos · F. Campos obligatorios
    ✓ UT-VAL01-31 · sin nombre: error
    ✓ UT-VAL01-31 · sin apellido: error
    ✓ UT-VAL01-31 · sin fecha_nacimiento: error
    ✓ UT-VAL01-32 · nombre con solo espacios: cuenta como ausente (1 ms)
    ✓ UT-VAL01-33 · nombre y apellido con espacios: salen recortados
  WF-01 · Valida datos · G. Fecha de nacimiento
    ✓ UT-VAL01-34 · texto (defecto 12.14): error de formato
    ✓ UT-VAL01-35 · 30 de febrero: error de formato (1 ms)
    ✓ UT-VAL01-36 · manana: error, no puede ser futura
    ✓ UT-VAL01-37 · hoy: valido (1 ms)
    ✓ UT-VAL01-38 · 31/12/1899: error, anterior a 1900
    ✓ UT-VAL01-39 · 01/01/1900: valido (1 ms)
    ✓ UT-VAL01-40 · vacia: solo el error de campo faltante, sin el de formato
  WF-01 · Valida datos · H. Motivo
    ✓ UT-VAL01-41 · sin motivo: valido (47 ms)
    ✓ UT-VAL01-42 · motivo de 255 caracteres
    ✓ UT-VAL01-43 · motivo de 256 caracteres
```

```
  WF-01 · Valida datos · I. Franja y hora
    ✓ UT-VAL01-44 · sin franja: queda indiferente (1 ms)
    ✓ UT-VAL01-45 · franja manana: valida (1 ms)
    ✓ UT-VAL01-45 · franja tarde: valida
    ✓ UT-VAL01-45 · franja indiferente: valida
    ✓ UT-VAL01-46 · franja que no existe: error (1 ms)
    ✓ UT-VAL01-47 · franja con espacios: valida y recortada (1 ms)
    ✓ UT-VAL01-48 · sin hora: hora_solicitada queda nula
    ✓ UT-VAL01-49 · limite de la manana: 07:59
    ✓ UT-VAL01-49 · limite de la manana: 08:00 (1 ms)
    ✓ UT-VAL01-49 · limite de la manana: 11:59
    ✓ UT-VAL01-49 · limite de la manana: 12:00 (1 ms)
    ✓ UT-VAL01-50 · limite de la tarde: 13:29 (1 ms)
    ✓ UT-VAL01-50 · limite de la tarde: 13:30
    ✓ UT-VAL01-50 · limite de la tarde: 16:59
    ✓ UT-VAL01-50 · limite de la tarde: 17:00
    ✓ UT-VAL01-51 · hora del almuerzo (12:45): error (1 ms)
    ✓ UT-VAL01-52 · formato invalido: 8:00
    ✓ UT-VAL01-52 · formato invalido: 24:00
    ✓ UT-VAL01-52 · formato invalido: 08:60
    ✓ UT-VAL01-52 · formato invalido: ocho
    ✓ UT-VAL01-53 · franja tarde con 09:00: error (10 ms)
    ✓ UT-VAL01-54 · franja manana con 15:00: error (1 ms)
    ✓ UT-VAL01-55 · franja indiferente con 15:00: valido (1 ms)
  WF-01 · Valida datos · J. Especialidad
    ✓ UT-VAL01-56 · sin especialidad: queda nula (1 ms)
    ✓ UT-VAL01-57 · especialidad con solo espacios: nula y sin error
    ✓ UT-VAL01-58 · pediatría en minusculas y con tilde: se normaliza a Pediatria (1 ms)
    ✓ UT-VAL01-59 · especialidad sin medicos activos: error y queda nula
  WF-01 · Valida datos · K. Medico solicitado
    ✓ UT-VAL01-60 · medico ausente: queda nulo, sin error
    ✓ UT-VAL01-60 · medico nulo: queda nulo, sin error
    ✓ UT-VAL01-60 · medico texto vacio: queda nulo, sin error (1 ms)
    ✓ UT-VAL01-61 · medico 2 como numero: valido
    ✓ UT-VAL01-62 · medico "2" como texto: valido y convertido a numero (1 ms)
    ✓ UT-VAL01-63 · medico 0: identificador invalido
    ✓ UT-VAL01-63 · medico -1: identificador invalido
    ✓ UT-VAL01-64 · medico 2.5: identificador invalido
    ✓ UT-VAL01-64 · medico "abc": identificador invalido
    ✓ UT-VAL01-65 · medico 99, que no esta en el catalogo: error
    ✓ UT-VAL01-66 · medico true: debe rechazarse (hipotesis A) (1 ms)
  WF-01 · Valida datos · L. Combinacion medico y especialidad
    ✓ UT-VAL01-67 · medico 2 con Pediatria: valido
    ✓ UT-VAL01-68 · medico 2 con Ginecologia: error
  WF-01 · Valida datos · M. Catalogo de medicos
    ✓ UT-VAL01-69 · catalogo como texto json: funciona igual que como arreglo
    ✓ UT-VAL01-70 · catalogo nulo: cualquier medico o especialidad pedidos dan error (1 ms)
  WF-01 · Valida datos · N. Conjunto y contrato
    ✓ UT-VAL01-71 · cuerpo vacio: todos los errores juntos en una sola respuesta (1 ms)
    ✓ UT-VAL01-72 · Hash del token sin datos: no falla y valida como cuerpo vacio
    ✓ UT-VAL01-73 · solicitud valida: devuelve un item con los nueve campos (1 ms)
    ✓ UT-VAL01-74 · el rol sale de Verifica token
    ✓ UT-VAL01-75 · consulta los tres nodos anteriores (verify)
    ✓ UT-VAL01-76 · un campo no previsto se conserva en datos

  Test Suites: 1 passed, 1 total
  Tests:       95 passed, 95 total
  Snapshots:   0 total
  Time:        3.821 s
  Ran all test suites matching tests/sigac.wf01.validadatos.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.3 Cifra datos sensibles (WF-01)

El nodo Cifra datos sensibles protege los datos personales antes de guardarlos. Cifra el DPI y el teléfono con AES-256-GCM, con un vector de inicialización aleatorio en cada operación, y calcula la huella SHA-256 del DPI, que es determinista y permite encontrar a un paciente sin descifrar nada. La clave se lee de la variable de entorno SIGAC_CLAVE_CIFRADO y nunca aparece en el código.

Es el único nodo del sistema que corre en modo Run Once for Each Item: recibe $json y devuelve un objeto, no una lista. Como el cifrado es aleatorio, las pruebas no comparan contra un valor fijo: descifran el resultado con una clave de prueba y verifican que vuelva el dato original.

1.3.1 Código base del nodo Cifra datos sensibles

```
const crypto = require('crypto');

const clave = Buffer.from($env.SIGAC_CLAVE_CIFRADO, 'hex');
if (clave.length !== 32) {
  throw new Error('SIGAC_CLAVE_CIFRADO debe ser de 32 bytes (64 caracteres hex)');
}

function cifrar(texto) {
  if (!texto) return null;
  const iv = crypto.randomBytes(12);
  const cifrador = crypto.createCipheriv('aes-256-gcm', clave, iv);
  const cifrado = Buffer.concat([cifrador.update(String(texto), 'utf8'), cifrador.final()]);
  const etiqueta = cifrador.getAuthTag();
  return [iv.toString('hex'), cifrado.toString('hex'), etiqueta.toString('hex')].join(':');
}

const datos = $json.datos;

return {
  json: {
    ...$json,
    dpi_cifrado: cifrar(datos.dpi),
    dpi_huella: crypto.createHash('sha256').update(String(datos.dpi)).digest('hex'),
    telefono_cifrado: cifrar(datos.telefono)
  }
};
```

1.3.2 Código de test del nodo Cifra datos sensibles

```
// sigac.wf01.cifradatossensibles.test.js
// pruebas unitarias del nodo Cifra datos sensibles de WF-01.
// cifra el DPI y el telefono con AES-256-GCM y calcula la huella
// sha-256 del DPI.
// el nodo corre en modo "una vez por item": recibe $json y devuelve
// un objeto, no una lista. como el cifrado es aleatorio, las pruebas
```

```
// no comparan contra un valor fijo: descifran el resultado y
// verifican que vuelva el dato original.
// cada prueba lleva su ID de la tabla de pruebas: UT-CIF01-NN
'use strict';

const crypto = require('crypto');
const cifraDatos = require('../nodos/wf01_cifra_datos_sensibles.js');
const { ejecutar, sha256, CLAVE_PRUEBA } = require('../simulacion.js');

const DPI = '1234567890123';
const TELEFONO = '55551234';
const FORMATO = /^[0-9a-f]{24}:[0-9a-f]+:[0-9a-f]{32}$/;

// ejecuta el nodo con los datos y la clave indicados
function cifrar(datos, clave = CLAVE_PRUEBA, extra = {}) {
  return ejecutar(cifraDatos, {
    json: { ...extra, datos },
    env: { SIGAC_CLAVE_CIFRADO: clave }
  }).salida;
}

// la operacion inversa, para comprobar la ida y vuelta
function descifrar(valor, clave = CLAVE_PRUEBA) {
  const [iv, datos, etiqueta] = valor.split(':');
  const descifrador = crypto.createDecipheriv('aes-256-gcm', Buffer.from(clave, 'hex'),
Buffer.from(iv, 'hex'));
  descifrador.setAuthTag(Buffer.from(etiqueta, 'hex'));
  return Buffer.concat([descifrador.update(Buffer.from(datos, 'hex')),
descifrador.final()]).toString('utf8');
}

// ---------------------------------------------------------
// A. Clave de cifrado
describe('WF-01 · Cifra datos sensibles · A. Clave de cifrado', () => {
  test('UT-CIF01-01 · clave de 64 caracteres hexadecimales: cifra sin error', () => {
    expect(() => cifrar({ dpi: DPI })).not.toThrow();
  });

  test.each([
    ['UT-CIF01-02', '62 caracteres (31 bytes)', CLAVE_PRUEBA.slice(0, 62)],
    ['UT-CIF01-03', '66 caracteres (33 bytes)', CLAVE_PRUEBA + 'ff'],
    ['UT-CIF01-04', 'caracteres que no son hexadecimales', 'z'.repeat(64)]
  ])('%s · clave de %s: lanza error', (  ,     , clave) => {
    expect(() => cifrar({ dpi: DPI }, clave)).toThrow('SIGAC_CLAVE_CIFRADO debe ser de 32
bytes');
```

```
  });

  test('UT-CIF01-05 · variable de entorno sin definir: lanza error', () => {
    expect(() => ejecutar(cifraDatos, { json: { datos: { dpi: DPI } }, env: {} })).toThrow();
  });
});

// ---------------------------------------------------------
// B. Formato del cifrado
describe('WF-01 · Cifra datos sensibles · B. Formato del cifrado', () => {
  test('UT-CIF01-06 · formato iv:texto:etiqueta en hexadecimal', () => {
    expect(cifrar({ dpi: DPI }).json.dpi  cifrado).toMatch(FORMATO);
  });

  test('UT-CIF01-07 · iv de 24 caracteres y etiqueta de 32', () => {
    const [iv, , etiqueta] = cifrar({ dpi: DPI }).json.dpi_cifrado.split(':');
    expect(iv).toHaveLength(24);
    expect(etiqueta).toHaveLength(32);
  });

  test('UT-CIF01-08 · un DPI de 13 caracteres da 26 caracteres cifrados', () => {
    const [, texto] = cifrar({ dpi: DPI }).json.dpi_cifrado.split(':');
    expect(texto).toHaveLength(26);
  });
});

// ---------------------------------------------------------
// C. Ida y vuelta
describe('WF-01 · Cifra datos sensibles · C. Ida y vuelta', () => {
  test('UT-CIF01-09 · el DPI se recupera al descifrar', () => {
    expect(descifrar(cifrar({ dpi: DPI }).json.dpi_cifrado)).toBe(DPI);
  });

  test('UT-CIF01-10 · el telefono se recupera al descifrar', () => {
    expect(descifrar(cifrar({ dpi: DPI, telefono: TELEFONO
}).json.telefono_cifrado)).toBe(TELEFONO);
  });

  test('UT-CIF01-11 · un texto con tildes y ñ vuelve identico', () => {
    expect(descifrar(cifrar({ dpi: 'Peñate Gómez' }).json.dpi_cifrado)).toBe('Peñate Gómez');
  });

  test('UT-CIF01-12 · con otra clave no se puede descifrar', () => {
    const cifrado = cifrar({ dpi: DPI }).json.dpi  cifrado;
    const otraClave = 'ff'.repeat(32);
```

```
    expect(() => descifrar(cifrado, otraClave)).toThrow();
  });

  test('UT-CIF01-13 · un texto cifrado alterado se detecta al descifrar', () => {
    const [iv, texto, etiqueta] = cifrar({ dpi: DPI }).json.dpi_cifrado.split(':');
    const alterado = (texto[0] === '0' ? '1' : '0') + texto.slice(1);
    expect(() => descifrar([iv, alterado, etiqueta].join(':'))).toThrow();
  });
});

// ------------------------------------------------------------
// D. Aleatoriedad e IV
describe('WF-01 · Cifra datos sensibles · D. Aleatoriedad', () => {
  test('UT-CIF01-14 · el mismo DPI cifrado dos veces da textos distintos', () => {
    expect(cifrar({ dpi: DPI }).json.dpi  cifrado).not.toBe(cifrar({ dpi: DPI }).json.dpi  cifrado);
  });

  test('UT-CIF01-15 · el iv nunca se reutiliza', () => {
    const iv1 = cifrar({ dpi: DPI }).json.dpi  cifrado.split(':')[0];
    const iv2 = cifrar({ dpi: DPI }).json.dpi_cifrado.split(':')[0];
    expect(iv1).not.toBe(iv2);
  });
});

// ------------------------------------------------------------
// E. Huella del DPI
describe('WF-01 · Cifra datos sensibles · E. Huella del DPI', () => {
  test('UT-CIF01-16 · la huella es el sha-256 del DPI', () => {
    expect(cifrar({ dpi: DPI }).json.dpi_huella).toBe(sha256(DPI));
  });

  test('UT-CIF01-17 · el mismo DPI da siempre la misma huella', () => {
    expect(cifrar({ dpi: DPI }).json.dpi_huella).toBe(cifrar({ dpi: DPI }).json.dpi_huella);
  });

  test('UT-CIF01-18 · dos DPI distintos dan huellas distintas', () => {
    expect(cifrar({ dpi: DPI }).json.dpi  huella).not.toBe(cifrar({ dpi: '9999999999999'
}).json.dpi_huella);
  });

  test('UT-CIF01-19 · un DPI como numero da la misma huella que como texto', () => {
    expect(cifrar({ dpi: 1234567890123 }).json.dpi_huella).toBe(sha256(DPI));
  });
});
```

```
// ---------------------------------------------------------
// F. Valores vacios
describe('WF-01 · Cifra datos sensibles · F. Valores vacios', () => {
  test('UT-CIF01-20 · telefono vacio: queda nulo', () => {
    expect(cifrar({ dpi: DPI, telefono: '' }).json.telefono_cifrado).toBeNull();
  });

  test('UT-CIF01-21 · sin telefono: queda nulo', () => {
    expect(cifrar({ dpi: DPI }).json.telefono_cifrado).toBeNull();
  });

  test('UT-CIF01-22 · telefono como numero: se cifra igual que como texto', () => {
    expect(descifrar(cifrar({ dpi: DPI, telefono: 55551234
}).json.telefono_cifrado)).toBe(TELEFONO);
  });

  // observacion: Valida datos nunca deja pasar una solicitud sin DPI,
  // asi que este caso no ocurre en el flujo. la prueba documenta que
  // la proteccion depende de Valida datos y no de este nodo.
  test('UT-CIF01-23 · sin DPI: cifrado nulo y huella de la palabra "undefined"', () => {
    const salida = cifrar({ telefono: TELEFONO });
    expect(salida.json.dpi_cifrado).toBeNull();
    expect(salida.json.dpi_huella).toBe(sha256('undefined'));
  });
});

// ---------------------------------------------------------
// G. Contrato de salida
describe('WF-01 · Cifra datos sensibles · G. Contrato de salida', () => {
  test('UT-CIF01-24 · devuelve un objeto con json, no una lista', () => {
    const salida = cifrar({ dpi: DPI });
    expect(Array.isArray(salida)).toBe(false);
    expect(salida).toHaveProperty('json');
  });

  test('UT-CIF01-25 · conserva los campos que traia y agrega los tres nuevos', () => {
    const salida = cifrar({ dpi: DPI, telefono: TELEFONO }, CLAVE_PRUEBA, { valido: true,
rol: 'recepcion' });
    expect(salida.json.valido).toBe(true);
    expect(salida.json.rol).toBe('recepcion');
    expect(salida.json.datos).toEqual({ dpi: DPI, telefono: TELEFONO });
    expect(salida.json).toHaveProperty('dpi_cifrado');
    expect(salida.json).toHaveProperty('dpi_huella');
    expect(salida.json).toHaveProperty('telefono_cifrado');
  });
```

```
  test('UT-CIF01-26 · los campos cifrados no contienen el dato en claro', () => {
    const { dpi_cifrado, dpi_huella, telefono_cifrado } = cifrar({ dpi: DPI, telefono: TELEFONO
}).json;
    const protegidos = JSON.stringify({ dpi_cifrado, dpi_huella, telefono_cifrado });
    expect(protegidos).not.toContain(DPI);
    expect(protegidos).not.toContain(TELEFONO);
  });
});
```

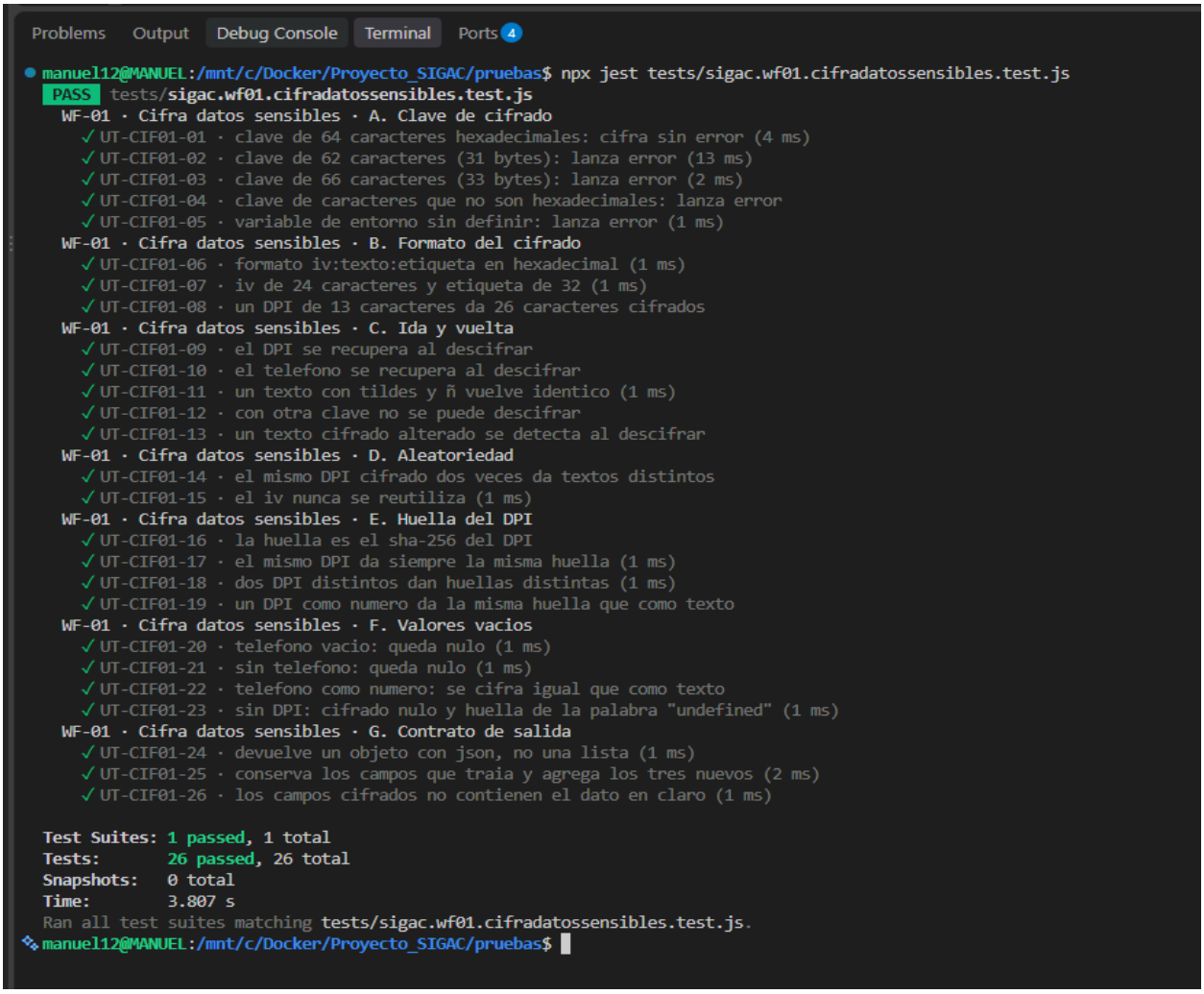
1.3.3 Tabla de pruebas unitarias realizadas al nodo Cifra datos sensibles (WF-01)

ID	Función	Caso de prueba	Entrada	Resultado esperado	Estado
UT-CIF01-01	Cifra datos sensibles	Clave válida (Happy Path)	Clave de 64 caracteres hexadecimales	Cifra sin error	Pass
UT-CIF01-02	Cifra datos sensibles	Clave corta	Clave de 62 caracteres (31 bytes)	Error «debe ser de 32 bytes»	Pass
UT-CIF01-03	Cifra datos sensibles	Clave larga	Clave de 66 caracteres (33 bytes)	Error «debe ser de 32 bytes»	Pass
UT-CIF01-04	Cifra datos sensibles	Clave no hexadecimal	64 caracteres z	Error «debe ser de 32 bytes»	Pass
UT-CIF01-05	Cifra datos sensibles	Clave ausente	Variable de entorno sin definir	Lanza un error	Pass
UT-CIF01-06	Cifra datos sensibles	Formato del cifrado	DPI válido	iv:texto:etiqueta en hexadecimal	Pass
UT-CIF01-07	Cifra datos sensibles	Tamaño del IV y la etiqueta	DPI válido	IV de 24 caracteres y etiqueta de 32	Pass
UT-CIF01-08	Cifra datos sensibles	Tamaño del texto cifrado	DPI de 13 caracteres	26 caracteres cifrados, sin relleno	Pass
UT-CIF01-09	Cifra datos sensibles	Ida y vuelta del DPI	DPI	Al descifrar vuelve el DPI original	Pass

UT-CIF01-10	Cifra datos sensibles	Ida y vuelta del teléfono	Teléfono	Al descifrar vuelve el teléfono original	Pass
UT-CIF01-11	Cifra datos sensibles	Texto con tildes y ñ	Peñate Gómez	Vuelve idéntico	Pass
UT-CIF01-12	Cifra datos sensibles	Descifrado con otra clave	Clave distinta	Falla: sin la clave no se recupera nada	Pass
UT-CIF01-13	Cifra datos sensibles	Texto cifrado alterado	Un carácter cambiado	Falla: GCM detecta la alteración	Pass
UT-CIF01-14	Cifra datos sensibles	Mismo DPI dos veces	El DPI cifrado dos veces	Textos cifrados distintos	Pass
UT-CIF01-15	Cifra datos sensibles	IV no reutilizado	El DPI cifrado dos veces	IV distintos	Pass
UT-CIF01-16	Cifra datos sensibles	Huella del DPI	DPI válido	SHA-256 del DPI	Pass
UT-CIF01-17	Cifra datos sensibles	Huella determinista	El mismo DPI dos veces	La misma huella	Pass
UT-CIF01-18	Cifra datos sensibles	DPI distintos	Dos DPI	Huellas distintas	Pass
UT-CIF01-19	Cifra datos sensibles	DPI como número	1234567890123 numérico	La misma huella que como texto	Pass
UT-CIF01-20	Cifra datos sensibles	Teléfono vacío	telefono: ""	telefono_cifrado: null	Pass
UT-CIF01-21	Cifra datos sensibles	Sin teléfono	Datos sin telefono	telefono_cifrado: null	Pass
UT-CIF01-22	Cifra datos sensibles	Teléfono como número	55551234 numérico	Se cifra igual que como texto	Pass

UT-CIF01-23	Cifra datos sensibles	Sin DPI (observación)	Datos sin dpi	Cifrado nulo; huella de la palabra "undefined"	Pass
UT-CIF01-24	Cifra datos sensibles	Contrato: modo por item	Cualquiera	Devuelve un objeto con json, no una lista	Pass
UT-CIF01-25	Cifra datos sensibles	Conserva los campos previos	$json con valido, rol y datos	Conserva todo y agrega los tres campos nuevos	Pass
UT-CIF01-26	Cifra datos sensibles	Datos en claro	DPI y teléfono válidos	Ningún campo protegido contiene el dato en claro	Pass

Observación del caso UT-CIF01-23. Si faltara el DPI, la huella se calcularía sobre la palabra "undefined", y todos los pacientes sin DPI compartirían esa misma huella. No ocurre en el flujo, porque Valida datos rechaza cualquier solicitud sin un DPI de 13 dígitos. La prueba deja documentado que la protección depende de Valida datos, y no de este nodo.



1.4 Registro exitoso (WF-01)

El nodo Registro exitoso arma la respuesta HTTP 201 que recibe recepción cuando la solicitud de cita quedó guardada y auditada. Toma el identificador y el estado de la solicitud del nodo Guarda solicitud, y el hash del eslabón de auditoría de Registra bitacora, que le llega como $json. El nodo no tiene ramas: su trabajo es reunir datos de dos nodos anteriores en una respuesta. Por eso las pruebas verifican el origen de cada campo, el comportamiento cuando un dato falta y que la respuesta no exponga más información de la necesaria.

1.4.1 Código base del nodo Registro exitoso

```
return [{
  json: {
    codigo: 201,
    estado: 'registrado',
    mensaje: 'Solicitud de cita registrada',
    id_solicitud: $('Guarda solicitud').first().json.id_solicitud,
    estado_solicitud: $('Guarda solicitud').first().json.estado,
    hash_auditoria: $json.hash_actual
  }
}];
```

1.4.2 Código de test del nodo Registro exitoso

```
// sigac.wf01.registroexitoso.test.js
// pruebas unitarias del nodo Registro exitoso de WF-01. arma la
// respuesta 201 con la solicitud guardada y el hash de auditoria.
// no tiene ramas: las pruebas verifican el origen de cada campo y
// el contrato de salida.
// cada prueba lleva su ID de la tabla de pruebas: UT-REG01-NN

'use strict';

const registroExitoso = require('../nodos/wf01_registro_exitoso.js');
const { ejecutar } = require('../simulacion.js');

// ejecuta el nodo con la salida simulada de Guarda solicitud y de
// Registra bitacora, que llega como $json
function registrar(guardaSolicitud, bitacora) {
  return ejecutar(registroExitoso, {
    anteriores: { 'Guarda solicitud': guardaSolicitud },
    json: bitacora
  });
}

const GUARDADA = { id_solicitud: 71, estado: 'pendiente' };
const BITACORA = { hash_actual: 'abc123' };
```

```
describe('WF-01 · Registro exitoso', () => {
  test('UT-REG01-01 · respuesta completa con los seis campos', () => {
    const { salida } = registrar(GUARDADA, BITACORA);
    expect(salida[0].json).toEqual({
      codigo: 201,
      estado: 'registrado',
      mensaje: 'Solicitud de cita registrada',
      id_solicitud: 71,
      estado_solicitud: 'pendiente',
      hash_auditoria: 'abc123'
    });
  });

  test('UT-REG01-02 · id_solicitud sale de Guarda solicitud', () => {
    const { salida } = registrar({ id_solicitud: 999, estado: 'pendiente' }, BITACORA);
    expect(salida[0].json.id_solicitud).toBe(999);
  });

  test('UT-REG01-03 · estado_solicitud sale del campo estado de Guarda solicitud', () => {
    const { salida } = registrar({ id_solicitud: 71, estado: 'en_espera' }, BITACORA);
    expect(salida[0].json.estado_solicitud).toBe('en_espera');
  });

  test('UT-REG01-04 · hash_auditoria sale de hash_actual de la bitacora', () => {
    const { salida } = registrar(GUARDADA, { hash_actual: 'f'.repeat(64) });
    expect(salida[0].json.hash_auditoria).toBe('f'.repeat(64));
  });

  test('UT-REG01-05 · consulta el nodo Guarda solicitud (verify)', () => {
    const { $ } = registrar(GUARDADA, BITACORA);
    expect($).toHaveBeenCalledWith('Guarda solicitud');
  });

  test('UT-REG01-06 · con varios items en Guarda solicitud, usa el primero', () => {
    const { salida } = registrar(
      [{ id_solicitud: 71, estado: 'pendiente' }, { id_solicitud: 72, estado: 'pendiente' }],
      BITACORA
    );
    expect(salida[0].json.id_solicitud).toBe(71);
  });

  test('UT-REG01-07 · Guarda solicitud sin estado: queda indefinido, sin error', () => {
    const { salida } = registrar({ id_solicitud: 71 }, BITACORA);
    expect(salida[0].json.estado_solicitud).toBeUndefined();
  });
```

```
  test('UT-REG01-08 · bitacora sin hash_actual: queda indefinido, sin error', () => {
    const { salida } = registrar(GUARDADA, {});
    expect(salida[0].json.hash_auditoria).toBeUndefined();
  });

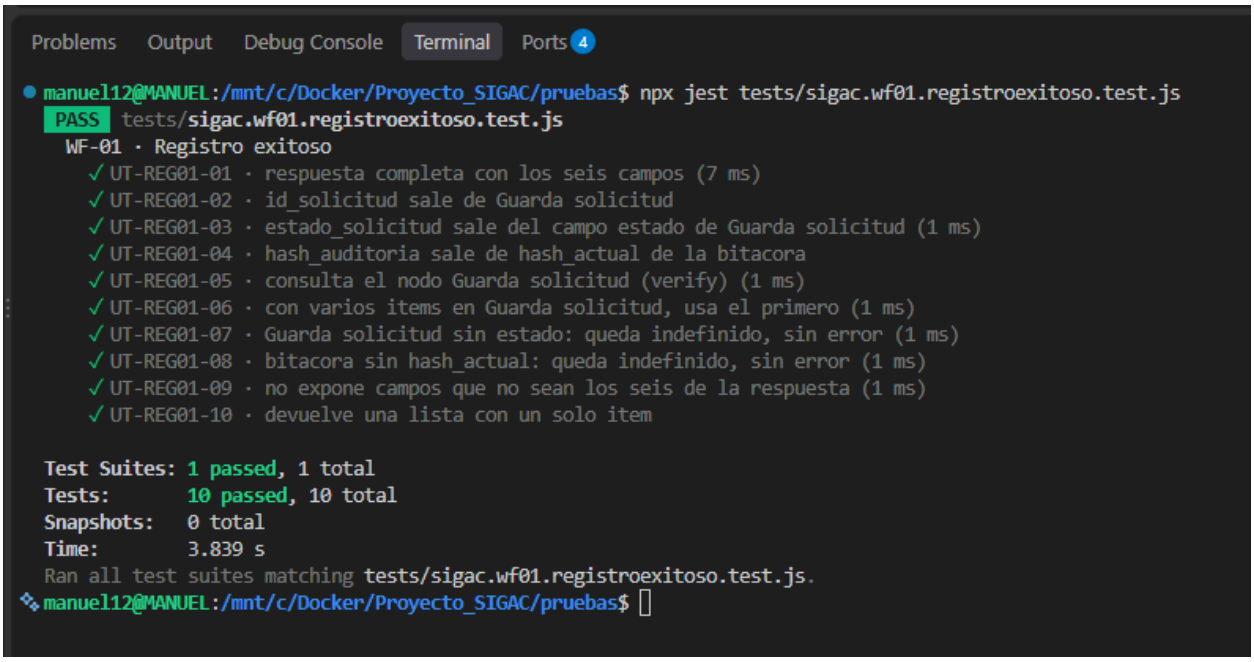
  test('UT-REG01-09 · no expone campos que no sean los seis de la respuesta', () => {
    const conDatos = { ...GUARDADA, id_paciente: 55, especialidad_solicitada: 'Pediatria', dpi:
'1234567890123' };
    const { salida } = registrar(conDatos, BITACORA);
    expect(Object.keys(salida[0].json).sort()).toEqual([
      'codigo', 'estado', 'estado_solicitud', 'hash_auditoria', 'id_solicitud', 'mensaje'
    ]);
    expect(JSON.stringify(salida)).not.toContain('1234567890123');
  });

  test('UT-REG01-10 · devuelve una lista con un solo item', () => {
    const { salida } = registrar(GUARDADA, BITACORA);
    expect(Array.isArray(salida)).toBe(true);
    expect(salida).toHaveLength(1);
  });
});
```

1.4.3 Tabla de pruebas unitarias realizadas al nodo Registro exitoso (WF-01)

ID	Función	Caso de prueba	Entrada	Resultado esperado	Estado
UT-REG01-01	Registro exitoso	Respuesta completa (Happy Path)	Solicitud 71 pendiente y hash abc123	Los seis campos exactos, con código 201	Pass
UT-REG01-02	Registro exitoso	Origen de id_solicitud	Guarda solicitud con id 999	id_solicitud: 999	Pass
UT-REG01-03	Registro exitoso	Origen de estado_solicitud	Guarda solicitud con estado en_espera	estado_solicitud: "en_espera"	Pass
UT-REG01-04	Registro exitoso	Origen de hash_auditoria	Bitácora con un hash de 64 caracteres	El mismo hash en la respuesta	Pass
UT-REG01-05	Registro exitoso	Nodo consultado (verify)	Solicitud válida	Consulta Guarda solicitud	Pass

UT-REG01-06	Registro exitoso	Varios items	Dos solicitudes, 71 y 72	Usa la primera: id_solicitud: 71	Pass
UT-REG01-07	Registro exitoso	Sin estado	Guarda solicitud sin estado	estado_solicitud indefinido, sin error	Pass
UT-REG01-08	Registro exitoso	Sin hash	Bitácora sin hash_actual	hash_auditoria indefinido, sin error	Pass
UT-REG01-09	Registro exitoso	Datos del paciente	Guarda solicitud con id de paciente, especialidad y DPI	Solo los seis campos; el DPI no aparece	Pass
UT-REG01-10	Registro exitoso	Contrato de salida	Solicitud válida	Una lista con un solo item	Pass



1.5 Rechazo 409 (WF-01)

El nodo Rechazo 409 responde cuando una solicitud de cita llega para un paciente que existe pero está inactivo o dado de baja. Lo alimenta la salida false del nodo ¿Paciente habilitado?, que detecta que Guarda solicitud no insertó nada. Se incorporó con la corrección del defecto 12.21. El código 409 (Conflict) indica que la petición es válida, con un token y un rol correctos, pero choca con el estado actual del paciente. El nodo devuelve una respuesta fija: no lee la entrada ni consulta otros nodos. Las pruebas verifican el contrato, que la respuesta no dependa de lo que reciba y que el mensaje no revele si el paciente está inactivo o dado de baja.

1.5.1 Código base del nodo Rechazo 409

```
return [{ json: { codigo: 409, estado: 'rechazado', mensaje: 'El paciente no esta habilitado para solicitar citas' } }];
```

1.5.2 Código de test del nodo Rechazo 409

```
// sigac.wf01.rechazo409.test.js
//
// pruebas unitarias del nodo Rechazo 409 de WF-01. responde cuando el
// paciente existe pero esta inactivo o dado de baja (defecto 12.21).
//
// es una respuesta fija: no lee la entrada ni consulta otros nodos.
// las pruebas verifican el contrato, que no dependa de lo que reciba
// y que el mensaje no revele mas de lo necesario.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-R40901-NN

'use strict';

const rechazo409 = require('../nodos/wf01_rechazo_409.js');
const { ejecutar } = require('../simulacion.js');

const RESPUESTA = {
  codigo: 409,
  estado: 'rechazado',
  mensaje: 'El paciente no esta habilitado para solicitar citas'
};

// lo que llega desde ¿Paciente habilitado? cuando Guarda solicitud no
// inserto nada: un item vacio, o con los datos que traia el flujo
const CON_DATOS = {
  dpi_cifrado: 'aa:bb:cc',
  dpi_huella: 'f'.repeat(64),
  datos: { dpi: '1234567890123', nombre: 'Ana', correo: 'ana@correo.com' }
};

describe('WF-01 · Rechazo 409', () => {
```

```
  test('UT-R40901-01 · respuesta completa', () => {
    const { salida } = ejecutar(rechazo409, { entrada: [{}] });
    expect(salida[0].json).toEqual(RESPUESTA);
  });

  test('UT-R40901-02 · codigo 409', () => {
    const { salida } = ejecutar(rechazo409, { entrada: [{}] });
    expect(salida[0].json.codigo).toBe(409);
  });

  test('UT-R40901-03 · estado rechazado, igual que todos los rechazos (defecto 12.17)', () => {
    const { salida } = ejecutar(rechazo409, { entrada: [{}] });
    expect(salida[0].json.estado).toBe('rechazado');
  });

  test('UT-R40901-04 · solo los campos codigo, estado y mensaje', () => {
    const { salida } = ejecutar(rechazo409, { entrada: [{}] });
    expect(Object.keys(salida[0].json).sort()).toEqual(['codigo', 'estado', 'mensaje']);
  });

  test('UT-R40901-05 · sin entrada: la misma respuesta', () => {
    const { salida } = ejecutar(rechazo409);
    expect(salida[0].json).toEqual(RESPUESTA);
  });

  test('UT-R40901-06 · con datos del paciente en la entrada: la misma respuesta, sin exponerlos',
() => {
    const { salida } = ejecutar(rechazo409, { entrada: [CON_DATOS], json: CON_DATOS });
    expect(salida[0].json).toEqual(RESPUESTA);
    const texto = JSON.stringify(salida);
    expect(texto).not.toContain('1234567890123');
    expect(texto).not.toContain('ana@correo.com');
  });

  test('UT-R40901-07 · no consulta ningun nodo anterior (verify)', () => {
    const { $ } = ejecutar(rechazo409, { entrada: [{}] });
    expect($).not.toHaveBeenCalled();
  });

  test('UT-R40901-08 · dos ejecuciones dan respuestas identicas', () => {
    const a = ejecutar(rechazo409, { entrada: [{}] }).salida;
    const b = ejecutar(rechazo409, { entrada: [CON_DATOS] }).salida;
    expect(a).toEqual(b);
  });
```

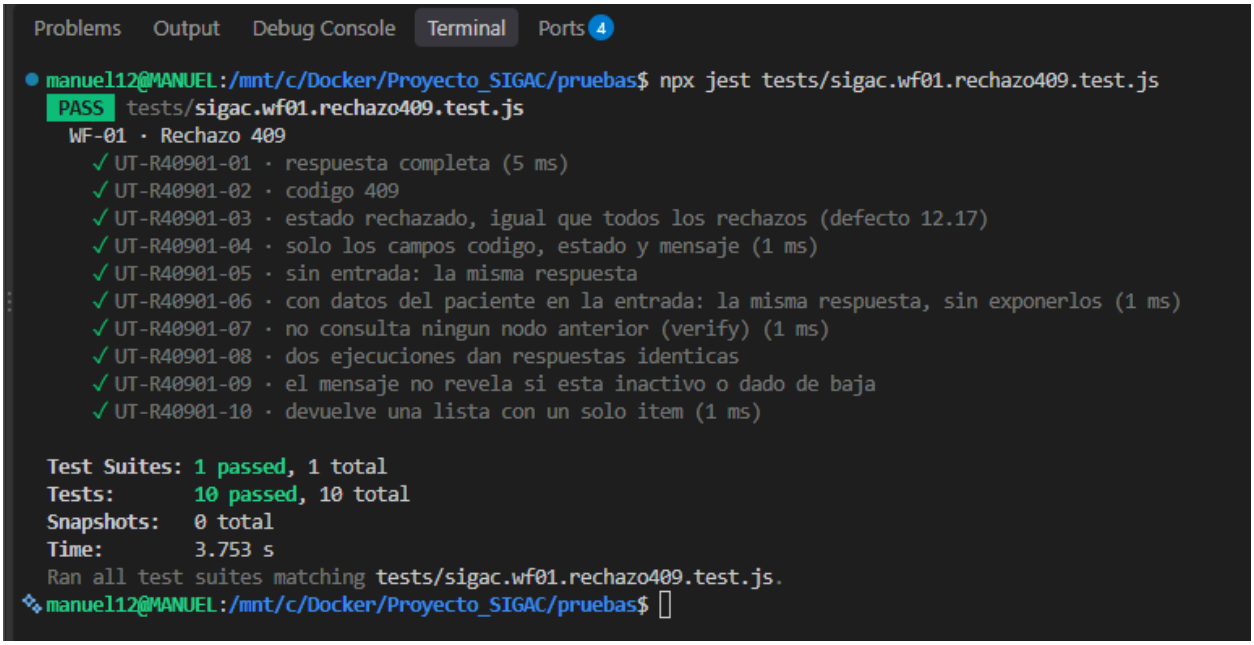
```
  test('UT-R40901-09 · el mensaje no revela si esta inactivo o dado de baja', () => {
    const { salida } = ejecutar(rechazo409, { entrada: [{}] });
    const mensaje = salida[0].json.mensaje.toLowerCase();
    expect(mensaje).not.toContain('baja');
    expect(mensaje).not.toContain('inactivo');
  });

  test('UT-R40901-10 · devuelve una lista con un solo item', () => {
    const { salida } = ejecutar(rechazo409, { entrada: [{}] });
    expect(Array.isArray(salida)).toBe(true);
    expect(salida).toHaveLength(1);
  });
});
```

1.5.3 Tabla de pruebas unitarias realizadas al nodo Rechazo 409 (WF-01)

ID	Función	Caso de prueba	Entrada	Resultado esperado	Estado
UT-R40901-01	Rechazo 409	Respuesta completa (Happy Path)	Item vacío	codigo, estado y mensaje exactos	Pass
UT-R40901-02	Rechazo 409	Código de respuesta	Item vacío	409	Pass
UT-R40901-03	Rechazo 409	Estado (defecto 12.17)	Item vacío	rechazado, igual que todos los rechazos	Pass
UT-R40901-04	Rechazo 409	Campos de la respuesta	Item vacío	Solo codigo, estado y mensaje	Pass
UT-R40901-05	Rechazo 409	Sin entrada	Ningún item	La misma respuesta	Pass
UT-R40901-06	Rechazo 409	Entrada con datos del paciente	DPI, nombre y correo	La misma respuesta, sin exponer ningún dato	Pass
UT-R40901-07	Rechazo 409	Nodos consultados (verify)	Item vacío	No consulta ningún nodo anterior	Pass

UT-R40901-08	Rechazo 409	Respuesta determinista	Dos ejecuciones con entradas distintas	Respuestas idénticas	Pass
UT-R40901-09	Rechazo 409	Mensaje genérico	Item vacío	No menciona «baja» ni «inactivo»	Pass
UT-R40901-10	Rechazo 409	Contrato de salida	Item vacío	Una lista con un solo item	Pass



1.6 Rechazo 400 (WF-01)

El nodo Rechazo 400 responde cuando la solicitud de cita no pasa la validación. Lo alimenta la salida false del nodo ¿Datos válidos?, y recibe como $json la salida de Valida datos, con la lista de todos los errores encontrados. La devuelve completa, para que recepción sepa exactamente qué corregir en un solo intento.

El código 400 (Bad Request) indica que la petición tiene datos inválidos. Las pruebas verifican el contrato de la respuesta, que la lista de errores llegue intacta y en el mismo orden, y que la respuesta no exponga los datos del paciente que venían en la entrada.

1.6.1 Código base del nodo Rechazo 400

```
return [{
  json: {
    codigo: 400,
    estado: 'rechazado',
    mensaje: 'Datos invalidos',
    errores: $json.errores
  }
}];
```

1.6.2 Código de test del nodo Rechazo 400

```
// sigac.wf01.rechazo400.test.js
// pruebas unitarias del nodo Rechazo 400 de WF-01. responde cuando
// Valida datos encontro errores, y devuelve la lista completa para
// que el emisor sepa que corregir. lee los errores de $json, que es
// la salida de Valida datos por la rama false de ¿Datos validos?
// cada prueba lleva su ID de la tabla de pruebas: UT-R40001-NN

'use strict';

const rechazo400 = require('../nodos/wf01_rechazo_400.js');
const { ejecutar } = require('../simulacion.js');

const ERRORES = ['El DPI debe tener exactamente 13 digitos', 'El correo no tiene un formato
valido'];

// ejecuta el nodo con la salida de Valida datos como $json
function rechazar(desdeValida) {
  return ejecutar(rechazo400, { entrada: [desdeValida], json: desdeValida });
}

describe('WF-01 · Rechazo 400', () => {
  test('UT-R40001-01 · respuesta completa', () => {
    expect(rechazar({ valido: false, errores: ERRORES }).salida[0].json).toEqual({
      codigo: 400,
      estado: 'rechazado',
      mensaje: 'Datos invalidos',
```

```
    errores: ERRORES
  });
});

test('UT-R40001-02 · codigo 400', () => {
  expect(rechazar({ errores: ERRORES }).salida[0].json.codigo).toBe(400);
});

test('UT-R40001-03 · estado rechazado (defecto 12.17)', () => {
  expect(rechazar({ errores: ERRORES }).salida[0].json.estado).toBe('rechazado');
});

test('UT-R40001-04 · mensaje Datos invalidos', () => {
  expect(rechazar({ errores: ERRORES }).salida[0].json.mensaje).toBe('Datos invalidos');
});

test('UT-R40001-05 · la lista de errores llega intacta', () => {
  expect(rechazar({ errores: ERRORES }).salida[0].json.errores).toEqual(ERRORES);
});

test('UT-R40001-06 · el orden de los errores se conserva', () => {
  const invertidos = [...ERRORES].reverse();
  expect(rechazar({ errores: invertidos }).salida[0].json.errores).toEqual(invertidos);
});

test('UT-R40001-07 · un solo error: lista de un elemento', () => {
  expect(rechazar({ errores: ['Falta el campo nombre'] }).salida[0].json.errores)
    .toEqual(['Falta el campo nombre']);
});

test('UT-R40001-08 · lista vacia: errores queda vacio', () => {
  expect(rechazar({ errores: [] }).salida[0].json.errores).toEqual([]);
});

test('UT-R40001-09 · sin errores en la entrada: queda indefinido, sin error', () => {
  expect(rechazar({ valido: false }).salida[0].json.errores).toBeUndefined();
});

test('UT-R40001-10 · con datos del paciente en la entrada: solo los cuatro campos', () => {
  const conDatos = { errores: ERRORES, datos: { dpi: '1234567890123', correo:
'ana@correo.com' } };
  const { salida } = rechazar(conDatos);
  expect(Object.keys(salida[0].json).sort()).toEqual(['codigo', 'errores', 'estado', 'mensaje']);
  expect(JSON.stringify(salida)).not.toContain('1234567890123');
});
```

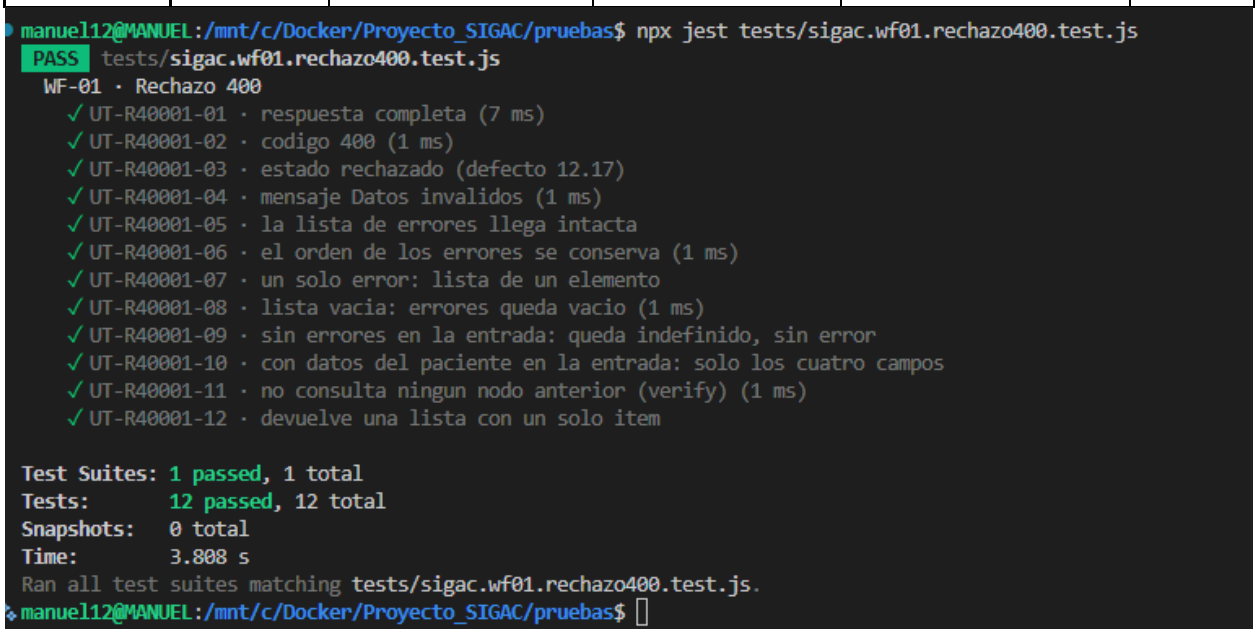
```
  test('UT-R40001-11 · no consulta ningun nodo anterior (verify)', () => {
    const { $ } = rechazar({ errores: ERRORES });
    expect($).not.toHaveBeenCalled();
  });

  test('UT-R40001-12 · devuelve una lista con un solo item', () => {
    const { salida } = rechazar({ errores: ERRORES });
    expect(Array.isArray(salida)).toBe(true);
    expect(salida).toHaveLength(1);
  });
});
```

1.6.3 Tabla de pruebas unitarias realizadas al nodo Rechazo 400 (WF-01)

ID	Función	Caso de prueba	Entrada	Resultado esperado	Estado
UT-R40001-01	Rechazo 400	Respuesta completa (Happy Path)	Dos errores de validación	codigo, estado, mensaje y errores exactos	Pass
UT-R40001-02	Rechazo 400	Código de respuesta	Dos errores	400	Pass
UT-R40001-03	Rechazo 400	Estado (defecto 12.17)	Dos errores	rechazado	Pass
UT-R40001-04	Rechazo 400	Mensaje	Dos errores	Datos invalidos	Pass
UT-R40001-05	Rechazo 400	Lista de errores	Dos errores	La misma lista, intacta	Pass
UT-R40001-06	Rechazo 400	Orden de los errores	Los dos errores en orden inverso	El mismo orden inverso	Pass
UT-R40001-07	Rechazo 400	Un solo error	["Falta el campo nombre"]	Lista de un elemento	Pass
UT-R40001-08	Rechazo 400	Lista vacía	errores: []	errores: []	Pass

UT-R40001-09	Rechazo 400	Sin errores	Entrada sin errores	errores indefinido, sin error	Pass
UT-R40001-10	Rechazo 400	Entrada con datos del paciente	Errores, DPI y correo	Solo los cuatro campos; el DPI no aparece	Pass
UT-R40001-11	Rechazo 400	Nodos consultados (verify)	Dos errores	No consulta ningún nodo anterior	Pass
UT-R40001-12	Rechazo 400	Contrato de salida	Dos errores	Una lista con un solo item	Pass



1.7 Rechazo 401 (WF-01)

El nodo Rechazo 401 responde cuando la petición no trae token, el token no existe o está desactivado. Lo alimenta la salida false del nodo ¿Token válido?.

El código 401 (Unauthorized) indica que no se pudo identificar a quien hace la petición. El mensaje es genérico a propósito: no revela cuál de los tres casos ocurrió, para no ayudar a quien intenta adivinar tokens válidos probando uno tras otro. Las pruebas verifican el contrato, que la respuesta sea siempre la misma sin importar la entrada, y que no devuelva ni la huella del token ni los datos que venían en la petición.

1.7.1 Código base del nodo Rechazo 401

```
return [{
  json: {
    codigo: 401,
    estado: 'rechazado',
    mensaje: 'Token ausente o invalido'
  }
}];
```

1.7.2 Código de test del nodo Rechazo 401

```
// sigac.wf01.rechazo401.test.js
// pruebas unitarias del nodo Rechazo 401 de WF-01. responde cuando el
// token no viene, no existe o esta desactivado. lo alimenta la salida
// false de ¿Token valido?
// el mensaje es generico a proposito: no revela cual de los tres casos
// ocurrio, para no ayudar a quien intenta adivinar tokens validos.
// cada prueba lleva su ID de la tabla de pruebas: UT-R40101-NN

'use strict';

const rechazo401 = require('../nodos/wf01_rechazo_401.js');
const { ejecutar } = require('../simulacion.js');

const RESPUESTA = {
  codigo: 401,
  estado: 'rechazado',
  mensaje: 'Token ausente o invalido'
};

// lo que puede llegar desde ¿Token valido?: la salida de Verifica
// token, vacia, junto con lo que traia el flujo
const CON_DATOS = {
  token_hash: 'a'.repeat(64),
  datos: { dpi: '1234567890123', correo: 'ana@correo.com' }
};

describe('WF-01 · Rechazo 401', () => {
```

```
  test('UT-R40101-01 · respuesta completa', () => {
    expect(ejecutar(rechazo401, { entrada: [{}] }).salida[0].json).toEqual(RESPUESTA);
  });

  test('UT-R40101-02 · codigo 401', () => {
    expect(ejecutar(rechazo401, { entrada: [{}] }).salida[0].json.codigo).toBe(401);
  });

  test('UT-R40101-03 · estado rechazado (defecto 12.17)', () => {
    expect(ejecutar(rechazo401, { entrada: [{}] }).salida[0].json.estado).toBe('rechazado');
  });

  test('UT-R40101-04 · mensaje Token ausente o invalido', () => {
    expect(ejecutar(rechazo401, { entrada: [{}] }).salida[0].json.mensaje).toBe('Token ausente o
invalido');
  });

  test('UT-R40101-05 · solo los campos codigo, estado y mensaje', () => {
    const { salida } = ejecutar(rechazo401, { entrada: [{}] });
    expect(Object.keys(salida[0].json).sort()).toEqual(['codigo', 'estado', 'mensaje']);
  });

  test('UT-R40101-06 · sin entrada: la misma respuesta', () => {
    expect(ejecutar(rechazo401).salida[0].json).toEqual(RESPUESTA);
  });

  test('UT-R40101-07 · con la huella del token y datos en la entrada: no los devuelve', () => {
    const { salida } = ejecutar(rechazo401, { entrada: [CON_DATOS], json: CON_DATOS });
    expect(salida[0].json).toEqual(RESPUESTA);
    const texto = JSON.stringify(salida);
    expect(texto).not.toContain('a'.repeat(64));
    expect(texto).not.toContain('1234567890123');
  });

  test('UT-R40101-08 · no consulta ningun nodo anterior (verify)', () => {
    const { $ } = ejecutar(rechazo401, { entrada: [{}] });
    expect($).not.toHaveBeenCalled();
  });

  test('UT-R40101-09 · dos ejecuciones con entradas distintas dan respuestas identicas', () => {
    const a = ejecutar(rechazo401, { entrada: [{}] }).salida;
    const b = ejecutar(rechazo401, { entrada: [CON_DATOS], json: CON_DATOS }).salida;
    expect(a).toEqual(b);
  });
```

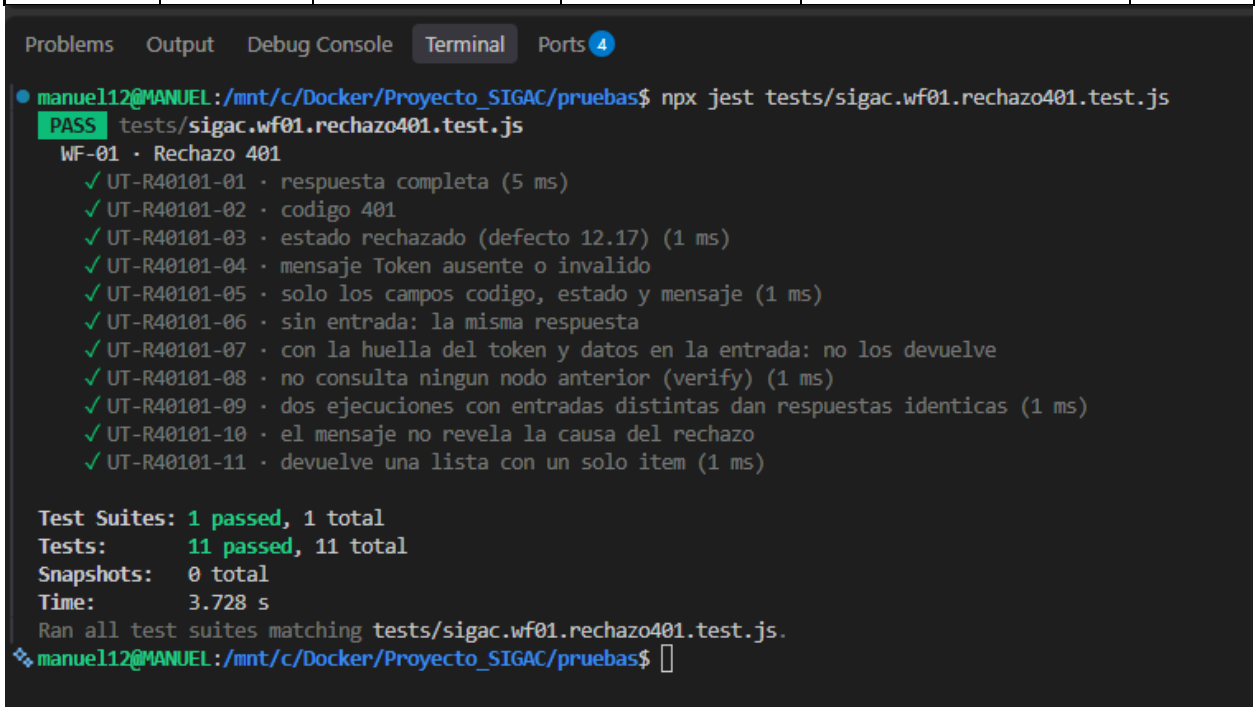
```
  test('UT-R40101-10 · el mensaje no revela la causa del rechazo', () => {
    const mensaje = ejecutar(rechazo401, { entrada: [{}] }).salida[0].json.mensaje.toLowerCase();
    for (const pista of ['inactivo', 'desactivado', 'expirado', 'rol']) {
      expect(mensaje).not.toContain(pista);
    }
  });

  test('UT-R40101-11 · devuelve una lista con un solo item', () => {
    const { salida } = ejecutar(rechazo401, { entrada: [{}] });
    expect(Array.isArray(salida)).toBe(true);
    expect(salida).toHaveLength(1);
  });
});
```

1.7.3 Tabla de pruebas unitarias realizadas al nodo Rechazo 401 (WF-01)

ID	Función	Caso de prueba	Entrada	Resultado esperado	Estado
UT-R40101-01	Rechazo 401	Respuesta completa (Happy Path)	Item vacío	codigo, estado y mensaje exactos	Pass
UT-R40101-02	Rechazo 401	Código de respuesta	Item vacío	401	Pass
UT-R40101-03	Rechazo 401	Estado (defecto 12.17)	Item vacío	rechazado	Pass
UT-R40101-04	Rechazo 401	Mensaje	Item vacío	Token ausente o invalido	Pass
UT-R40101-05	Rechazo 401	Campos de la respuesta	Item vacío	Solo codigo, estado y mensaje	Pass
UT-R40101-06	Rechazo 401	Sin entrada	Ningún item	La misma respuesta	Pass
UT-R40101-07	Rechazo 401	Entrada con la huella del token y datos	Huella, DPI y correo	La misma respuesta; no devuelve la huella ni el DPI	Pass
UT-R40101-08	Rechazo 401	Nodos consultados (verify)	Item vacío	No consulta ningún nodo anterior	Pass

UT-R40101-09	Rechazo 401	Respuesta determinista	Dos ejecuciones con entradas distintas	Respuestas idénticas	Pass
UT-R40101-10	Rechazo 401	Mensaje genérico	Item vacío	No menciona «inactivo», «desactivado», «expirado» ni «rol»	Pass
UT-R40101-11	Rechazo 401	Contrato de salida	Item vacío	Una lista con un solo item	Pass



1.8 Rechazo 403 (WF-01)

El nodo Rechazo 403 responde cuando el token es válido pero su rol no puede registrar solicitudes de cita: solo el rol recepcion puede hacerlo. Lo alimenta la salida false del nodo ¿Rol autorizado?.

El código 403 (Forbidden) se distingue del 401: aquí el sistema sí identificó a quien llama, pero su rol no tiene permiso para esta operación. El mensaje no revela qué rol se necesita ni cuál tiene quien llama. Las pruebas verifican el contrato, que la respuesta sea la misma para cualquier rol y que no devuelva los datos de la petición.

1.8.1 Código base del nodo Rechazo 403

```
return [{ json: { codigo: 403, estado: 'rechazado', mensaje: 'Operacion no permitida para este rol'
} }];
```

1.8.2 Código de test del nodo Rechazo 403

```
// sigac.wf01.rechazo403.test.js
// pruebas unitarias del nodo Rechazo 403 de WF-01. responde cuando el
// token es valido pero su rol no puede registrar solicitudes: solo
// recepcion puede. lo alimenta la salida false de ¿Rol autorizado?
// el mensaje no revela que rol se necesita ni cual tiene quien llama.
// cada prueba lleva su ID de la tabla de pruebas: UT-R40301-NN

'use strict';

const rechazo403 = require('../nodos/wf01_rechazo_403.js');
const { ejecutar } = require('../simulacion.js');

const RESPUESTA = {
  codigo: 403,
  estado: 'rechazado',
  mensaje: 'El rol no autoriza esta operacion'
};

// lo que llega desde ¿Rol autorizado?: la salida de Verifica token,
// con el rol de quien llama, junto con lo que traia el flujo
const CON_ROL = {
  rol: 'medico',
  datos: { dpi: '1234567890123', correo: 'ana@correo.com' }
};

describe('WF-01 · Rechazo 403', () => {
  test('UT-R40301-01 · respuesta completa', () => {
    expect(ejecutar(rechazo403, { entrada: [{}] }).salida[0].json).toEqual(RESPUESTA);
  });

  test('UT-R40301-02 · codigo 403', () => {
    expect(ejecutar(rechazo403, { entrada: [{}] }).salida[0].json.codigo).toBe(403);
```

```
});

test('UT-R40301-03 · estado rechazado (defecto 12.17)', () => {
  expect(ejecutar(rechazo403, { entrada: [{}] }).salida[0].json.estado).toBe('rechazado');
});

test('UT-R40301-04 · mensaje El rol no autoriza esta operacion', () => {
  expect(ejecutar(rechazo403, { entrada: [{}] }).salida[0].json.mensaje).toBe('El rol no autoriza
esta operacion');
});

test('UT-R40301-05 · solo los campos codigo, estado y mensaje', () => {
  const { salida } = ejecutar(rechazo403, { entrada: [{}] });
  expect(Object.keys(salida[0].json).sort()).toEqual(['codigo', 'estado', 'mensaje']);
});

test('UT-R40301-06 · sin entrada: la misma respuesta', () => {
  expect(ejecutar(rechazo403).salida[0].json).toEqual(RESPUESTA);
});

test('UT-R40301-07 · con el rol y datos en la entrada: no los devuelve', () => {
  const { salida } = ejecutar(rechazo403, { entrada: [CON_ROL], json: CON_ROL });
  expect(salida[0].json).toEqual(RESPUESTA);
  expect(JSON.stringify(salida)).not.toContain('1234567890123');
});

test('UT-R40301-08 · no consulta ningun nodo anterior (verify)', () => {
  const { $ } = ejecutar(rechazo403, { entrada: [{}] });
  expect($).not.toHaveBeenCalled();
});

test('UT-R40301-09 · dos ejecuciones con roles distintos dan respuestas identicas', () => {
  const comoMedico = ejecutar(rechazo403, { entrada: [{ rol: 'medico' }], json: { rol: 'medico' }
}).salida;
  const comoAdmin = ejecutar(rechazo403, { entrada: [{ rol: 'admin' }], json: { rol: 'admin' }
}).salida;
  expect(comoMedico).toEqual(comoAdmin);
});

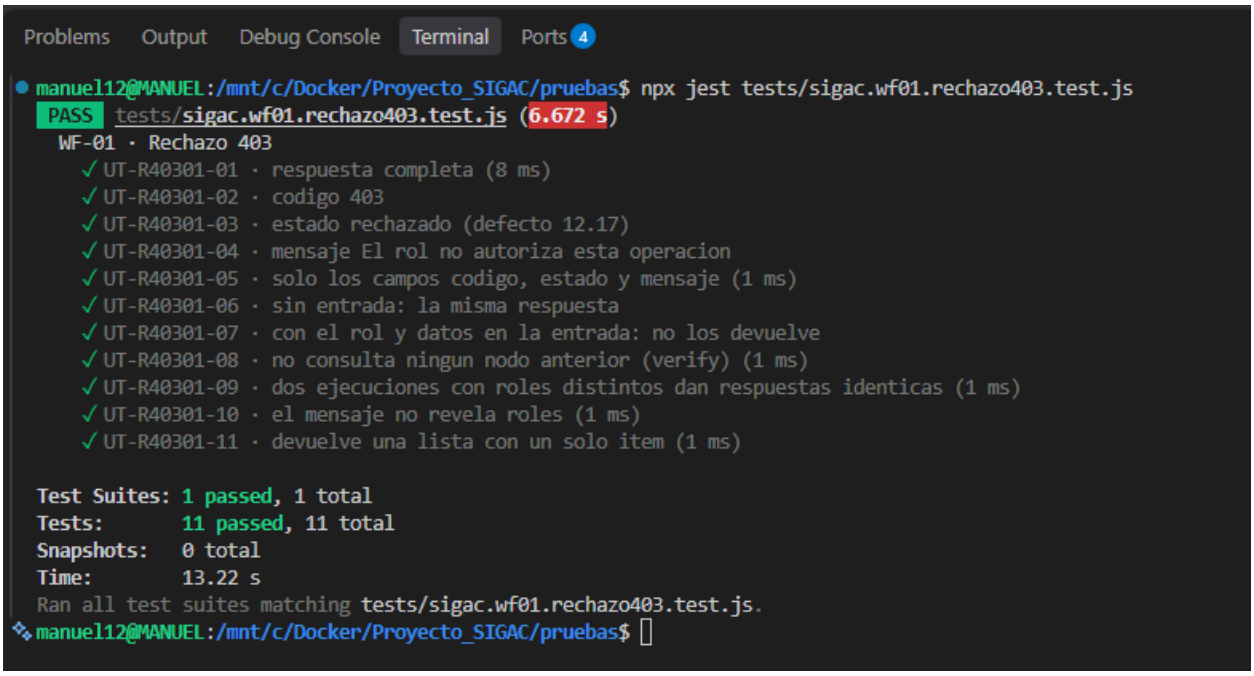
test('UT-R40301-10 · el mensaje no revela roles', () => {
  const mensaje = ejecutar(rechazo403, { entrada: [{}] }).salida[0].json.mensaje.toLowerCase();
  for (const rol of ['recepcion', 'medico', 'admin']) {
    expect(mensaje).not.toContain(rol);
  }
});
```

```
  test('UT-R40301-11 · devuelve una lista con un solo item', () => {
    const { salida } = ejecutar(rechazo403, { entrada: [{}] });
    expect(Array.isArray(salida)).toBe(true);
    expect(salida).toHaveLength(1);
  });
});
```

1.8.3 Tabla de pruebas unitarias realizadas al nodo Rechazo 403 (WF-01)

ID	Función	Caso de prueba	Entrada	Resultado esperado	Estado
UT-R40301-01	Rechazo 403	Respuesta completa (Happy Path)	Item vacío	codigo, estado y mensaje exactos	Pass
UT-R40301-02	Rechazo 403	Código de respuesta	Item vacío	403	Pass
UT-R40301-03	Rechazo 403	Estado (defecto 12.17)	Item vacío	rechazado	Pass
UT-R40301-04	Rechazo 403	Mensaje	Item vacío	El rol no autoriza esta operacion	Pass
UT-R40301-05	Rechazo 403	Campos de la respuesta	Item vacío	Solo codigo, estado y mensaje	Pass
UT-R40301-06	Rechazo 403	Sin entrada	Ningún item	La misma respuesta	Pass
UT-R40301-07	Rechazo 403	Entrada con el rol y datos	Rol medico, DPI y correo	La misma respuesta; no devuelve el DPI	Pass
UT-R40301-08	Rechazo 403	Nodos consultados (verify)	Item vacío	No consulta ningún nodo anterior	Pass
UT-R40301-09	Rechazo 403	Respuesta independiente del rol	Rol medico y rol admin	Respuestas idénticas	Pass
UT-R40301-10	Rechazo 403	Mensaje genérico	Item vacío	No menciona recepcion, medico ni admin	Pass

UT-R40301-11	Rechazo 403	Contrato de salida	Item vacío	Una lista con un solo item	Pass



1.9 Calcula prioridad (WF-02)

El nodo Calcula prioridad recibe el lote de solicitudes pendientes que leyó Lee pendientes y le asigna a cada una una prioridad según el tipo de consulta: 1 para urgencia, 3 para control y 5 para general. Cualquier tipo desconocido recibe la prioridad 5. También le asigna un turno, que es su posición dentro del lote.

Además propaga los cuatro campos de preferencia del paciente (franja, hora, especialidad y médico), que el nodo Reparte cupos necesita para respetar lo que el paciente pidió. Que esos campos se perdieran en este punto fue el defecto 12.8. Las pruebas verifican la tabla de prioridades, la propagación de cada campo, el turno y que los campos no previstos no pasen al siguiente nodo.

1.9.1 Código base del nodo Calcula prioridad

```
const tipos = { urgencia: 1, control: 3, general: 5 };

return $input.all().map((item, i) => ({
  json: {
    id_solicitud: item.json.id,
    id_paciente: item.json.id_paciente,
    tipo_consulta: item.json.tipo_consulta,
    fecha_preferida: item.json.fecha_preferida,
    prioridad: tipos[item.json.tipo_consulta] || 5,
    turno: i,
    franja_preferida: item.json.franja_preferida,
    hora_solicitada: item.json.hora_solicitada,
    especialidad_solicitada: item.json.especialidad_solicitada,
    id_medico_solicitado: item.json.id_medico_solicitado
  }
}));
```

1.9.2 Código de test del nodo Calcula prioridad

```
// sigac.wf02.calculaprioridad.test.js
// pruebas unitarias del nodo Calcula prioridad de WF-02. recibe el lote
// de solicitudes pendientes y a cada una le asigna una prioridad segun
// el tipo de consulta (1 la mas urgente, 5 la menos) y un turno, que es
// su posicion en el lote. propaga los cuatro campos de preferencia, que
// Reparte cupos necesita (defecto 12.8).
// cada prueba lleva su ID de la tabla de pruebas: UT-PRI02-NN

'use strict';

const calculaPrioridad = require('../nodos/wf02_calcula_prioridad.js');
const { ejecutar } = require('../simulacion.js');

// una solicitud tal como la devuelve Lee pendientes
```

```
const solicitud = (cambios = {}) => ({
  id: 71,
  id_paciente: 55,
  tipo_consulta: 'general',
  fecha_preferida: '2026-10-31',
  franja_preferida: 'tarde',
  hora_solicitada: '14:30:00',
  especialidad_solicitada: 'Pediatria',
  id_medico_solicitado: 2,
  motivo: 'control de rutina',
  estado: 'pendiente',
  ...cambios
});

// ejecuta el nodo con el lote indicado
const calcular = (...lote) => ejecutar(calculaPrioridad, { entrada: lote });

describe('WF-02 · Calcula prioridad', () => {
  test.each([
    ['UT-PRI02-01', 'urgencia', 1],
    ['UT-PRI02-02', 'control', 3],
    ['UT-PRI02-03', 'general', 5]
  ])('%s · %s: prioridad %i', (_, tipo_consulta, prioridad) => {
    const { salida } = calcular(solicitud({ tipo_consulta }));
    expect(salida[0].json.prioridad).toBe(prioridad);
  });

  test('UT-PRI02-04 · tipo desconocido: prioridad 5', () => {
    expect(calcular(solicitud({ tipo_consulta: 'revision'
})).salida[0].json.prioridad).toBe(5);
  });

  test('UT-PRI02-05 · sin tipo: prioridad 5', () => {
    expect(calcular(solicitud({ tipo_consulta: undefined
})).salida[0].json.prioridad).toBe(5);
  });

  test('UT-PRI02-06 · Urgencia con mayuscula: prioridad 5, la tabla es exacta',
() => {
    expect(calcular(solicitud({ tipo_consulta: 'Urgencia'
})).salida[0].json.prioridad).toBe(5);
  });

  test('UT-PRI02-07 · el id de la solicitud sale como id_solicitud', () => {
```

```
    expect(calcular(solicitud({ id: 999
})).salida[0].json.id_solicitud).toBe(999);
  });

  test('UT-PRI02-08 · paciente, tipo y fecha pasan intactos', () => {
    const { json } = calcular(solicitud()).salida[0];
    expect(json.id_paciente).toBe(55);
    expect(json.tipo_consulta).toBe('general');
    expect(json.fecha_preferida).toBe('2026-10-31');
  });

  test('UT-PRI02-09 · las cuatro preferencias pasan intactas (defecto 12.8)', ()
=> {
    const { json } = calcular(solicitud()).salida[0];
    expect(json.franja_preferida).toBe('tarde');
    expect(json.hora_solicitada).toBe('14:30:00');
    expect(json.especialidad_solicitada).toBe('Pediatria');
    expect(json.id_medico_solicitado).toBe(2);
  });

  test('UT-PRI02-10 · las preferencias nulas pasan como nulas (defecto 12.8)', ()
=> {
    const sinPreferencias = solicitud({
      franja_preferida: 'indiferente',
      hora_solicitada: null,
      especialidad_solicitada: null,
      id_medico_solicitado: null
    });
    const { json } = calcular(sinPreferencias).salida[0];
    expect(json.franja_preferida).toBe('indiferente');
    expect(json.hora_solicitada).toBeNull();
    expect(json.especialidad_solicitada).toBeNull();
    expect(json.id_medico_solicitado).toBeNull();
  });

  test('UT-PRI02-11 · el turno es la posicion en el lote', () => {
    const { salida } = calcular(solicitud({ id: 1 }), solicitud({ id: 2 }),
solicitud({ id: 3 }));
    expect(salida.map(s => s.json.turno)).toEqual([0, 1, 2]);
  });

  test('UT-PRI02-12 · un lote de tres da tres items en el mismo orden', () => {
    const { salida } = calcular(
      solicitud({ id: 10, tipo_consulta: 'general' }),
      solicitud({ id: 20, tipo_consulta: 'urgencia' }),
```

```
      solicitud({ id: 30, tipo_consulta: 'control' })
    );
    expect(salida).toHaveLength(3);
    expect(salida.map(s => s.json.id_solicitud)).toEqual([10, 20, 30]);
    expect(salida.map(s => s.json.prioridad)).toEqual([5, 1, 3]);
  });

  test('UT-PRI02-13 · lote vacio: lista vacia, sin error', () => {
    expect(calcular().salida).toEqual([]);
  });

  test('UT-PRI02-14 · campos no previstos como motivo o estado no se propagan',
() => {
    const { json } = calcular(solicitud()).salida[0];
    expect(json).not.toHaveProperty('motivo');
    expect(json).not.toHaveProperty('estado');
    expect(json).not.toHaveProperty('id');
  });

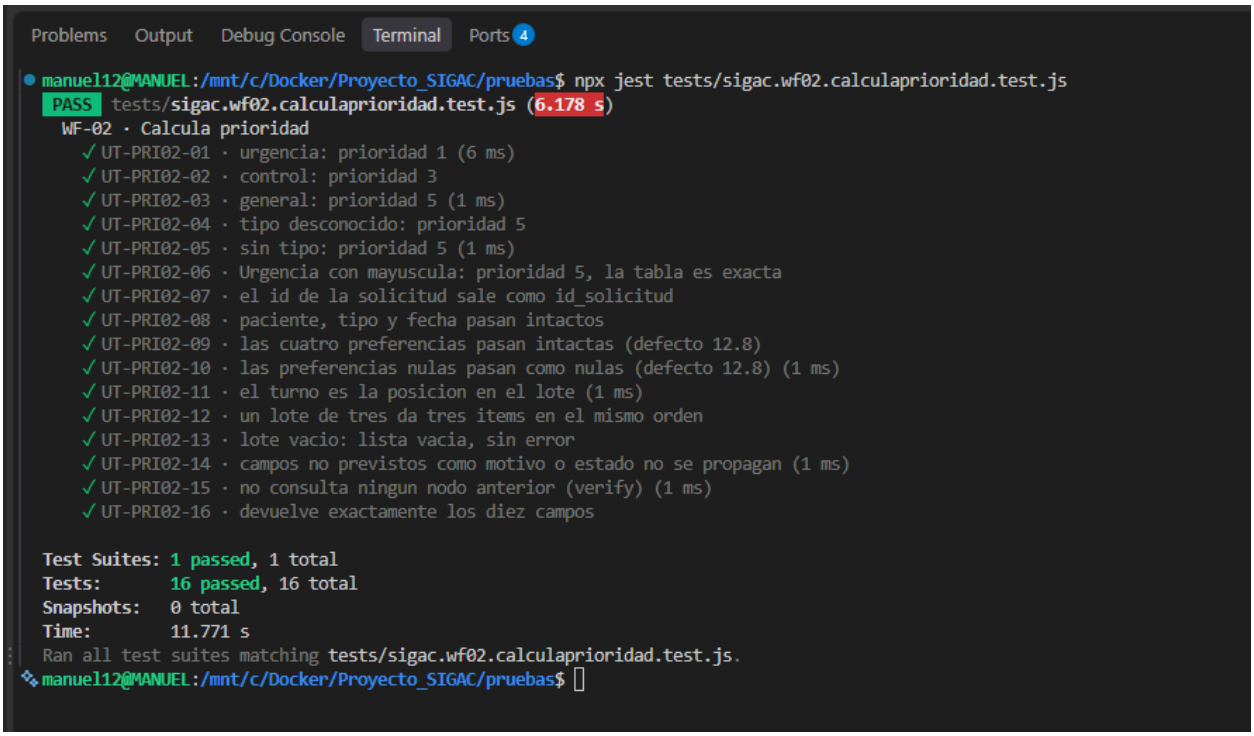
  test('UT-PRI02-15 · no consulta ningun nodo anterior (verify)', () => {
    const { $ } = calcular(solicitud());
    expect($).not.toHaveBeenCalled();
  });

  test('UT-PRI02-16 · devuelve exactamente los diez campos', () => {
    const { json } = calcular(solicitud()).salida[0];
    expect(Object.keys(json).sort()).toEqual([
      'especialidad_solicitada', 'fecha_preferida', 'franja_preferida',
'hora_solicitada',
      'id_medico_solicitado', 'id_paciente', 'id_solicitud', 'prioridad',
'tipo_consulta', 'turno'
    ]);
  });
});
```

1.9.3 Tabla de pruebas unitarias realizadas al nodo Calcula prioridad (WF-02)

ID	Función	Caso de prueba	Entrada	Resultado esperado	Estado
UT-PRI02-01	Calcula prioridad	Consulta de urgencia (Happy Path)	tipo_consulta: "urgencia"	prioridad: 1	Pass
UT-PRI02-02	Calcula prioridad	Consulta de control	tipo_consulta: "control"	prioridad: 3	Pass
UT-PRI02-03	Calcula prioridad	Consulta general	tipo_consulta: "general"	prioridad: 5	Pass
UT-PRI02-04	Calcula prioridad	Tipo desconocido	tipo_consulta: "revision"	prioridad: 5	Pass
UT-PRI02-05	Calcula prioridad	Sin tipo	Solicitud sin tipo_consulta	prioridad: 5	Pass
UT-PRI02-06	Calcula prioridad	Tipo con mayúscula	tipo_consulta: "Urgencia"	prioridad: 5: la tabla es exacta	Pass
UT-PRI02-07	Calcula prioridad	Origen de id_solicitud	Solicitud con id: 999	id_solicitud: 999	Pass
UT-PRI02-08	Calcula prioridad	Paciente, tipo y fecha	Paciente 55, general, 2026-10-31	Los tres campos intactos	Pass
UT-PRI02-09	Calcula prioridad	Preferencias con valor (defecto 12.8)	Franja tarde, hora 14:30:00, Pediatria, médico 2	Las cuatro preferencias intactas	Pass
UT-PRI02-10	Calcula prioridad	Preferencias nulas (defecto 12.8)	Franja indiferente; hora, especialidad y médico nulos	Las cuatro pasan igual, las nulas como nulas	Pass
UT-PRI02-11	Calcula prioridad	Turno en el lote	Lote de tres solicitudes	Turnos 0, 1 y 2	Pass
UT-PRI02-12	Calcula prioridad	Orden del lote	Solicitudes 10 (general), 20 (urgencia) y 30 (control)	Tres items en el mismo orden, con prioridades 5, 1 y 3	Pass

UT-PRI02-13	Calcula prioridad	Lote vacío	Ninguna solicitud	Lista vacía, sin error	Pass
UT-PRI02-14	Calcula prioridad	Campos no previstos	Solicitud con motivo y estado	No se propagan, ni tampoco id	Pass
UT-PRI02-15	Calcula prioridad	Nodos consultados (verify)	Una solicitud	No consulta ningún nodo anterior	Pass
UT-PRI02-16	Calcula prioridad	Contrato de salida	Una solicitud	Exactamente los diez campos de salida	Pass



1.10 Reparte cupos (WF-02)

El nodo Reparte cupos decide, para cada solicitud del lote, qué médico la atiende y a qué hora, o si queda sin cupo y pasa a lista de espera. Recibe las franjas de cada médico desde Busca cupo y las solicitudes desde Calcula prioridad.

Aplica la regla de que el sistema decide solo lo que el paciente no decidió: si pidió médico, solo ese; si no, la especialidad pedida, y sin especialidad, Medicina General. Respeta la franja y la hora pedidas, elige al médico con menos carga y asigna el primer hueco de 30 minutos libre. Todo se lleva en memoria, porque la base todavía no refleja las asignaciones del lote. Las pruebas verifican cada regla, los cupos por franja (defecto 12.15) y el reparto dentro de un mismo lote.

1.10.1 Código base del nodo Reparte cupos

```
const solicitudes = $('Calcula prioridad').all();
const salida = [];

for (const s of solicitudes) {
  const d = s.json;
  let candidatas = franjas.filter(f => f.libres > 0);

  if (d.id_medico_solicitado) {
    candidatas = candidatas.filter(f => f.id_medico ===
Number(d.id_medico_solicitado));
  } else {
    const esp = d.especialidad_solicitada || 'Medicina General';
    candidatas = candidatas.filter(f => f.especialidad === esp);
  }

  const franjaPedida = d.franja_preferida && d.franja_preferida !== 'indiferente'
    ? d.franja_preferida
    : null;
  if (franjaPedida) {
    candidatas = candidatas.filter(f => f.nombre === franjaPedida);
  }
  let horaExacta = null;
  if (d.hora_solicitada) {
    horaExacta = aMinutos(d.hora_solicitada);
    candidatas = candidatas.filter(f =>
      horaExacta >= f.inicio &&
      horaExacta < f.fin &&
      !f.tomadas.includes(horaExacta)
    );
  }
  candidatas.sort((a, b) => a.ocupados - b.ocupados);
  const franja = candidatas[0];
```

```
  if (!franja) {
    salida.push({ json: { ...d, id_medico: null, hora_inicio: null } });
    continue;
  }
  let minuto = null;
  if (horaExacta !== null) {
    minuto = horaExacta;
  } else {

      if (!franja.tomadas.includes(m)) {
        minuto = m;
        break;
      }
    }
    if (minuto === null) {
      salida.push({ json: { ...d, id_medico: null, hora_inicio: null } });
      continue;
    }
  }

  franja.libres -= 1;
  franja.ocupados += 1;
  franja.tomadas.push(minuto);

  salida.push({
    json: { ...d, id_medico: franja.id_medico, hora_inicio: aHora(minuto) }
  });
}

return salida;
```

1.10.2 Código de test del nodo Reparte cupos

```
// sigac.wf02.repartecupos.test.js
//
// pruebas unitarias del nodo Reparte cupos de WF-02. recibe el lote de
// solicitudes de Calcula prioridad y las franjas de Busca cupo, y le
// asigna a cada solicitud un medico y una hora, o la deja sin cupo para
// que vaya a lista de espera.
//
// reglas: el sistema decide solo lo que el paciente no decidio. si pidio
// medico, solo ese; si no, la especialidad pedida, y sin especialidad,
// Medicina General. respeta la franja y la hora pedidas, elige al medico
// con menos carga y asigna el primer hueco de 30 minutos libre. todo se
// lleva en memoria, porque la base todavia no refleja el lote.
//
// el catalogo simulado de franjas (como lo devuelve Busca cupo):
```

```
//   medico 1 Medicina General · medico 4 Medicina General · medico 2 Pediatria
//   manana 08:00-12:00 con 8 cupos · tarde 13:30-17:00 con 7 cupos
//
// cada prueba lleva su ID de la tabla de pruebas: UT-REP02-NN

'use strict';

const reparteCupos = require('../nodos/wf02_reparte_cupos.js');
const { ejecutar } = require('../simulacion.js');

// una fila de Busca cupo: una franja de un medico ese dia
const franja = (id_medico, especialidad, hora_inicio, hora_fin, cupos, ocupados =
0, horas_ocupadas = '') =>
  ({ id_medico, especialidad, hora_inicio, hora_fin, cupos, ocupados,
horas_ocupadas });

const manana = (id, esp, ocupados = 0, horas = '') => franja(id, esp, '08:00:00',
'12:00:00', 8, ocupados, horas);
const tarde = (id, esp, ocupados = 0, horas = '') => franja(id, esp, '13:30:00',
'17:00:00', 7, ocupados, horas);

// el catalogo: el medico 1 ya tiene carga y el 4 no
const CATALOGO = () => [
  manana(1, 'Medicina General', 2, '08:00,08:30'),
  tarde(1, 'Medicina General', 2, '13:30,14:00'),
  manana(4, 'Medicina General'),
  tarde(4, 'Medicina General'),
  manana(2, 'Pediatria'),
  tarde(2, 'Pediatria')
];

// una solicitud tal como sale de Calcula prioridad
const solicitud = (cambios = {}) => ({
  id_solicitud: 71,
  id_paciente: 55,
  tipo_consulta: 'general',
  fecha_preferida: '2026-10-27',
  prioridad: 5,
  turno: 0,
  franja_preferida: 'indiferente',
  hora_solicitada: null,
  especialidad_solicitada: null,
  id_medico_solicitado: null,
  ...cambios
});
```

```
// ejecuta el nodo con las franjas y el lote indicados
function repartir(franjas, ...lote) {
  return ejecutar(reparteCupos, {
    anteriores: { 'Busca cupo': franjas, 'Calcula prioridad': lote }
  });
}

// atajo: medico y hora asignados a cada solicitud del lote
const asignado = (franjas, ...lote) =>
  repartir(franjas, ...lote).salida.map(s => [s.json.id_medico,
s.json.hora_inicio]);

// ------------------------------------------------------------
// A. Quien atiende
// ------------------------------------------------------------
describe('WF-02 · Reparte cupos · A. Quien atiende', () => {
  test('UT-REP02-01 · sin preferencias: Medicina General, el medico con menos
carga', () => {
    expect(asignado(CATALOGO(), solicitud())).toEqual([[4, '08:00:00']]);
  });

  test('UT-REP02-02 · especialidad pedida: un medico de esa especialidad', () =>
{
    expect(asignado(CATALOGO(), solicitud({ especialidad_solicitada: 'Pediatria'
}))).toEqual([[2, '08:00:00']]);
  });

  test('UT-REP02-03 · medico pedido: solo ese, aunque otro tenga menos carga', ()
=> {
    expect(asignado(CATALOGO(), solicitud({ id_medico_solicitado: 1
}))[0][0]).toBe(1);
  });

  test('UT-REP02-04 · medico pedido como texto: se compara como numero', () => {
    expect(asignado(CATALOGO(), solicitud({ id_medico_solicitado: '2'
}))[0][0]).toBe(2);
  });

  test('UT-REP02-05 · especialidad sin medicos en las franjas: sin cupo', () => {
    expect(asignado(CATALOGO(), solicitud({ especialidad_solicitada:
'Ginecologia' }))).toEqual([[null, null]]);
  });
```

```
  test('UT-REP02-06 · medico pedido que no aparece en las franjas: sin cupo', ()
=> {
    expect(asignado(CATALOGO(), solicitud({ id_medico_solicitado: 5
}))).toEqual([[null, null]]);
  });
});

// ------------------------------------------------------------
// B. Franja y hora
// ------------------------------------------------------------
describe('WF-02 · Reparte cupos · B. Franja y hora', () => {
  test('UT-REP02-07 · franja tarde: el primer hueco de la tarde', () => {
    expect(asignado(CATALOGO(), solicitud({ franja_preferida: 'tarde'
}))).toEqual([[4, '13:30:00']]);
  });

  test('UT-REP02-08 · franja manana: el primer hueco de la manana', () => {
    expect(asignado(CATALOGO(), solicitud({ franja_preferida: 'manana'
}))).toEqual([[4, '08:00:00']]);
  });

  test('UT-REP02-09 · franja indiferente: no filtra por franja', () => {
    const soloTarde = [tarde(4, 'Medicina General')];
    expect(asignado(soloTarde, solicitud({ franja_preferida: 'indiferente'
}))).toEqual([[4, '13:30:00']]);
  });

  test('UT-REP02-10 · hora exacta libre: se asigna esa hora', () => {
    expect(asignado(CATALOGO(), solicitud({ hora_solicitada: '10:00'
}))).toEqual([[4, '10:00:00']]);
  });

  test('UT-REP02-11 · hora exacta ya tomada en la base: sin cupo con ese medico',
() => {
    expect(asignado(CATALOGO(), solicitud({ id_medico_solicitado: 1,
hora_solicitada: '08:30' })))
      .toEqual([[null, null]]);
  });

  test('UT-REP02-12 · hora exacta fuera de las franjas (12:30): sin cupo', () =>
{
    expect(asignado(CATALOGO(), solicitud({ hora_solicitada: '12:30'
}))).toEqual([[null, null]]);
  });
```

```
  test('UT-REP02-13 · hora exacta con segundos (HH:MM:SS): se interpreta igual',
() => {
    expect(asignado(CATALOGO(), solicitud({ hora_solicitada: '10:00:00'
}))).toEqual([[4, '10:00:00']]);
  });

  test('UT-REP02-14 · hora en la franja equivocada: sin cupo', () => {
    expect(asignado(CATALOGO(), solicitud({ franja_preferida: 'tarde',
hora_solicitada: '09:00' })))
      .toEqual([[null, null]]);
  });
});

// ------------------------------------------------------------
// C. Primer hueco libre (defecto 12.15)
// ------------------------------------------------------------
describe('WF-02 · Reparte cupos · C. Primer hueco libre', () => {
  test('UT-REP02-15 · salta las horas ya tomadas: 08:00 y 08:30 ocupadas da
09:00', () => {
    expect(asignado(CATALOGO(), solicitud({ id_medico_solicitado: 1,
franja_preferida: 'manana' })))
      .toEqual([[1, '09:00:00']]);
  });

  test('UT-REP02-16 · horas ocupadas con segundos: se interpretan igual', () => {
    const conSegundos = [manana(1, 'Medicina General', 1, '08:00:00')];
    expect(asignado(conSegundos, solicitud())).toEqual([[1, '08:30:00']]);
  });

  test('UT-REP02-17 · franja con cupos pero sin huecos de 30 minutos: sin cupo',
() => {
    const corta = [franja(1, 'Medicina General', '08:00:00', '09:00:00', 5, 2,
'08:00,08:30')];
    expect(asignado(corta, solicitud())).toEqual([[null, null]]);
  });

  test('UT-REP02-18 · franja llena (cupos iguales a ocupados): pasa al otro
medico', () => {
    const llena = [franja(1, 'Medicina General', '08:00:00', '12:00:00', 2, 2,
'08:00,08:30'), manana(4, 'Medicina General', 3)];
    expect(asignado(llena, solicitud())).toEqual([[4, '08:00:00']]);
  });

  test('UT-REP02-19 · cupos y ocupados como texto: se convierten a numero', () =>
{
```

```
    const comoTexto = [franja(1, 'Medicina General', '08:00:00', '12:00:00', '8',
'1', '08:00')];
    expect(asignado(comoTexto, solicitud())).toEqual([[1, '08:30:00']]);
  });
});

// ------------------------------------------------------------
// D. Lote en memoria
// ------------------------------------------------------------
describe('WF-02 · Reparte cupos · D. Lote en memoria', () => {
  test('UT-REP02-20 · dos solicitudes con el mismo medico: horas distintas', ()
=> {
    const uno = [manana(1, 'Medicina General')];
    expect(asignado(uno, solicitud({ id_solicitud: 1 }), solicitud({
id_solicitud: 2 })))
      .toEqual([[1, '08:00:00'], [1, '08:30:00']]);
  });

  test('UT-REP02-21 · el cupo se descuenta: la segunda solicitud va al otro
medico', () => {
    const unCupo = [franja(1, 'Medicina General', '08:00:00', '12:00:00', 1),
franja(4, 'Medicina General', '08:00:00', '12:00:00', 1)];
    expect(asignado(unCupo, solicitud({ id_solicitud: 1 }), solicitud({
id_solicitud: 2 })))
      .toEqual([[1, '08:00:00'], [4, '08:00:00']]);
  });

  test('UT-REP02-22 · la misma hora exacta pedida dos veces: solo la primera la
obtiene', () => {
    const uno = [manana(1, 'Medicina General')];
    expect(asignado(uno, solicitud({ hora_solicitada: '10:00' }), solicitud({
hora_solicitada: '10:00' })))
      .toEqual([[1, '10:00:00'], [null, null]]);
  });

  test('UT-REP02-23 · franjas repetidas por Busca cupo se cuentan una sola vez',
() => {
    // Busca cupo repite cada franja una vez por solicitud de entrada:
    // un cupo no debe convertirse en dos
    const repetida = franja(1, 'Medicina General', '08:00:00', '12:00:00', 1);
    expect(asignado([repetida, { ...repetida }], solicitud({ id_solicitud: 1 }),
solicitud({ id_solicitud: 2 })))
      .toEqual([[1, '08:00:00'], [null, null]]);
  });
```

```
  test('UT-REP02-24 · lote vacio: lista vacia', () => {
    expect(repartir(CATALOGO()).salida).toEqual([]);
  });
});

// ------------------------------------------------------------
// E. Entradas vacias y contrato
// ------------------------------------------------------------
describe('WF-02 · Reparte cupos · E. Entradas vacias y contrato', () => {
  test('UT-REP02-25 · sin franjas: todas sin cupo', () => {
    expect(asignado([], solicitud())).toEqual([[null, null]]);
  });

  test('UT-REP02-26 · fila vacia de Busca cupo (sin id_medico): se ignora', () =>
{
    expect(asignado([{}, manana(4, 'Medicina General')],
solicitud())).toEqual([[4, '08:00:00']]);
  });

  test('UT-REP02-27 · con cupo: conserva los campos de la solicitud y agrega
medico y hora', () => {
    const { salida } = repartir(CATALOGO(), solicitud({ id_solicitud: 99,
prioridad: 1 }));
    expect(salida[0].json).toEqual({ ...solicitud({ id_solicitud: 99, prioridad:
1 }), id_medico: 4, hora_inicio: '08:00:00' });
  });

  test('UT-REP02-28 · sin cupo: conserva los campos y deja medico y hora nulos',
() => {
    const { salida } = repartir([], solicitud({ id_solicitud: 99 }));
    expect(salida[0].json).toEqual({ ...solicitud({ id_solicitud: 99 }),
id_medico: null, hora_inicio: null });
  });

  test('UT-REP02-29 · consulta Busca cupo y Calcula prioridad (verify)', () => {
    const { $ } = repartir(CATALOGO(), solicitud());
    expect($).toHaveBeenCalledWith('Busca cupo');
    expect($).toHaveBeenCalledWith('Calcula prioridad');
  });
});

// ------------------------------------------------------------
// F. Traslapes (hipotesis C)
// ------------------------------------------------------------
// cada cita dura 30 minutos. el nodo solo evita repetir la hora exacta,
```

```
// asi que una cita a las 08:15 se cruza con otra a las 08:00. el indice
// uqcita_medico_horario de la base tampoco lo impide, porque solo
// compara la hora exacta.
describe('WF-02 · Reparte cupos · F. Traslapes', () => {
  test('UT-REP02-30 · hora pedida 08:15 con una cita a las 08:00: no debe
asignarse (hipotesis C)', () => {
    const conCita = [manana(1, 'Medicina General', 1, '08:00')];
    expect(asignado(conCita, solicitud({ hora_solicitada: '08:15'
}))).toEqual([[null, null]]);
  });

  test('UT-REP02-31 · cita existente a las 08:15: el hueco automatico no debe
cruzarse con ella (hipotesis C)', () => {
    const conCita = [manana(1, 'Medicina General', 1, '08:15')];
    expect(asignado(conCita, solicitud())).toEqual([[1, '09:00:00']]);
  });
});
```

```
  244 |   test('UT-REP02-31 · cita existente a las 08:15: el hueco automatico no debe cruzarse con ella (hipotesis C)', () => {
  245 |     const conCita = [manana(1, 'Medicina General', 1, '08:15')];
> 246 |     expect(asigned(conCita, solicitud())).toEqual([[1, '09:00:00']]);
      |                                           ^
  247 |   });
  248 | });
  249 |

      at Object.toEqual (tests/sigac.wf02.repartecupos.test.js:246:44)

Test Suites: 1 failed, 1 total
Tests:       2 failed, 29 passed, 31 total
Snapshots:   0 total
Time:        2.626 s, estimated 3 s
Ran all test suites matching tests/sigac.wf02.repartecupos.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.10.2.1 Defecto detectado por las pruebas (12.25).

Las pruebas UT-REP02-30 y UT-REP02-31 fallaron. En la primera, el nodo asignó la cita de las 08:15 al médico 1, aunque ya tenía otra a las 08:00. En la segunda, asignó el hueco de las 08:00, que se cruza con la cita existente de las 08:15. Cada cita dura 30 minutos, pero el nodo solo evita repetir la **hora exacta**, así que no detecta los traslapes.

1.10.3 Nuevo código base de Reparte Cupos

```
const vistas = new Set();
const franjas = [];

// convierte "HH:MM" o "HH:MM:SS" en minutos desde medianoche
function aMinutos(texto) {
  const [h, m] = String(texto).split(':').map(Number);
  return h * 60 + m;
}

for (const f of $('Busca cupo').all()) {
  if (!f.json.id_medico) continue;
  const clave = f.json.id_medico + '|' + f.json.hora_inicio;
  if (vistas.has(clave)) continue;
  vistas.add(clave);

  const inicio = aMinutos(f.json.hora_inicio);
  const fin    = aMinutos(f.json.hora_fin);

  // horas que ya estan ocupadas en la base dentro de esta franja.
  // antes solo se conocia el conteo, y el calculo suponia que las
  // citas ocupaban la franja en orden y sin huecos: una cita pedida
  // a las 10:00 podia terminar asignada otra vez a otro paciente.
  const tomadas = String(f.json.horas_ocupadas || '')
    .split(',')
    .filter(Boolean)
    .map(aMinutos);

  franjas.push({
    id_medico: f.json.id_medico,
    especialidad: f.json.especialidad,
    inicio: inicio,
    fin: fin,
    // la franja de la manana empieza antes del mediodia
    nombre: inicio < 720 ? 'manana' : 'tarde',
    libres: Number(f.json.cupos) - Number(f.json.ocupados),
    ocupados: Number(f.json.ocupados),
    tomadas: tomadas
  });
}

// defecto 12.25: cada cita dura 30 minutos. un minuto esta libre solo
// si no hay otra cita que empiece menos de 30 minutos antes o despues.
// antes se comparaba solo la hora exacta: una cita a las 08:15 se
// cruzaba con otra a las 08:00 del mismo medico, y el indice
```

```
// uqcita_medico_horario de la base tampoco lo impide, porque tambien
// compara solo la hora exacta.
function estaLibre(franja, minuto) {
  return !franja.tomadas.some(t => Math.abs(t - minuto) < 30);
}

// convierte minutos desde medianoche al formato HH:MM:SS
function aHora(min) {
  return String(Math.floor(min / 60)).padStart(2, '0') + ':' +
         String(min % 60).padStart(2, '0') + ':00';
}

// reparte las solicitudes entre las franjas con cupo disponible.
// se hace aqui y no en SQL porque todos los items se evaluan antes
// de que se inserte ninguna cita: la base todavia no refleja las
// asignaciones de este mismo lote.
//
// el sistema decide unicamente lo que el paciente no decidio: si
// pidio medico, especialidad u hora, se respeta; lo que no pidio,
// se resuelve aqui.
const solicitudes = $('Calcula prioridad').all();
const salida = [];

for (const s of solicitudes) {
  const d = s.json;

  // 1. candidatas: franjas con cupo que cumplan lo que el paciente pidio
  let candidatas = franjas.filter(f => f.libres > 0);

  if (d.id_medico_solicitado) {
    // pidio medico: solo ese. la especialidad se deduce de el.
    candidatas = candidatas.filter(f => f.id_medico ===
Number(d.id_medico_solicitado));
  } else {
    // sin medico pedido, la especialidad manda. si no la pidio,
    // va a Medicina General, que es el medico de primer contacto.
    const esp = d.especialidad_solicitada || 'Medicina General';
    candidatas = candidatas.filter(f => f.especialidad === esp);
  }

  const franjaPedida = d.franja_preferida && d.franja_preferida !== 'indiferente'
    ? d.franja_preferida
    : null;
  if (franjaPedida) {
    candidatas = candidatas.filter(f => f.nombre === franjaPedida);
```

```
}

// 2. si pidio hora exacta, solo sirve la franja que la contiene
//    y solo si no se cruza con otra cita, ni de la base ni de
//    este mismo lote
let horaExacta = null;
if (d.hora_solicitada) {
  horaExacta = aMinutos(d.hora_solicitada);
  candidatas = candidatas.filter(f =>
    horaExacta >= f.inicio &&
    horaExacta < f.fin &&
    estaLibre(f, horaExacta)
  );
}

// 3. entre las que quedan, la del medico con menos carga
candidatas.sort((a, b) => a.ocupados - b.ocupados);
const franja = candidatas[0];

if (!franja) {
  salida.push({ json: { ...d, id_medico: null, hora_inicio: null } });
  continue;
}

// 4. calcular el minuto concreto
let minuto = null;
if (horaExacta !== null) {
  minuto = horaExacta;
} else {
  // primer hueco de 30 minutos que no se cruce con otra cita. antes se usaba
  // inicio + 30 * ocupados, que choca con las horas pedidas por
  // otros pacientes y con las citas que ya existian en la base.
  for (let m = franja.inicio; m < franja.fin; m += 30) {
    if (estaLibre(franja, m)) {
      minuto = m;
      break;
    }
  }
  // si no queda ningun hueco libre en la franja, no hay cupo real
  if (minuto === null) {
    salida.push({ json: { ...d, id_medico: null, hora_inicio: null } });
    continue;
  }
}
```

```
  franja.libres -= 1;
  franja.ocupados += 1;
  franja.tomadas.push(minuto);

  salida.push({
    json: { ...d, id_medico: franja.id_medico, hora_inicio: aHora(minuto) }
  });
}

return salida;
```

1.10.4 Se corrió el mismo código test pero ya con la corrección del código base.

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf02.repartecupos.test.js
 PASS  tests/sigac.wf02.repartecupos.test.js
  WF-02 · Reparte cupos · A. Quien atiende
    ✓ UT-REP02-01 · sin preferencias: Medicina General, el medico con menos carga (6 ms)
    ✓ UT-REP02-02 · especialidad pedida: un medico de esa especialidad (1 ms)
    ✓ UT-REP02-03 · medico pedido: solo ese, aunque otro tenga menos carga (1 ms)
    ✓ UT-REP02-04 · medico pedido como texto: se compara como numero (1 ms)
    ✓ UT-REP02-05 · especialidad sin medicos en las franjas: sin cupo
    ✓ UT-REP02-06 · medico pedido que no aparece en las franjas: sin cupo
  WF-02 · Reparte cupos · B. Franja y hora
    ✓ UT-REP02-07 · franja tarde: el primer hueco de la tarde
    ✓ UT-REP02-08 · franja manana: el primer hueco de la manana (1 ms)
    ✓ UT-REP02-09 · franja indiferente: no filtra por franja
    ✓ UT-REP02-10 · hora exacta libre: se asigna esa hora (1 ms)
    ✓ UT-REP02-11 · hora exacta ya tomada en la base: sin cupo con ese medico
    ✓ UT-REP02-12 · hora exacta fuera de las franjas (12:30): sin cupo
    ✓ UT-REP02-13 · hora exacta con segundos (HH:MM:SS): se interpreta igual (1 ms)
    ✓ UT-REP02-14 · hora en la franja equivocada: sin cupo (1 ms)
  WF-02 · Reparte cupos · C. Primer hueco libre
    ✓ UT-REP02-15 · salta las horas ya tomadas: 08:00 y 08:30 ocupadas da 09:00 (1 ms)
    ✓ UT-REP02-16 · horas ocupadas con segundos: se interpretan igual
    ✓ UT-REP02-17 · franja con cupos pero sin huecos de 30 minutos: sin cupo
    ✓ UT-REP02-18 · franja llena (cupos iguales a ocupados): pasa al otro medico
    ✓ UT-REP02-19 · cupos y ocupados como texto: se convierten a numero (1 ms)
  WF-02 · Reparte cupos · D. Lote en memoria
    ✓ UT-REP02-20 · dos solicitudes con el mismo medico: horas distintas (1 ms)
    ✓ UT-REP02-21 · el cupo se descuenta: la segunda solicitud va al otro medico
    ✓ UT-REP02-22 · la misma hora exacta pedida dos veces: solo la primera la obtiene
    ✓ UT-REP02-23 · franjas repetidas por Busca cupo se cuentan una sola vez (1 ms)
    ✓ UT-REP02-24 · lote vacio: lista vacia
  WF-02 · Reparte cupos · E. Entradas vacias y contrato
    ✓ UT-REP02-25 · sin franjas: todas sin cupo
    ✓ UT-REP02-26 · fila vacia de Busca cupo (sin id_medico): se ignora (1 ms)
    ✓ UT-REP02-27 · con cupo: conserva los campos de la solicitud y agrega medico y hora (1 ms)
    ✓ UT-REP02-28 · sin cupo: conserva los campos y deja medico y hora nulos
    ✓ UT-REP02-29 · consulta Busca cupo y Calcula prioridad (verify)
  WF-02 · Reparte cupos · F. Traslapes
    ✓ UT-REP02-30 · hora pedida 08:15 con una cita a las 08:00: no debe asignarse (hipotesis C)
    ✓ UT-REP02-31 · cita existente a las 08:15: el hueco automatico no debe cruzarse con ella (hipotesis C) (4 ms)

Test Suites: 1 passed, 1 total
Tests:       31 passed, 31 total
Snapshots:   0 total
Time:        4.513 s
Ran all test suites matching tests/sigac.wf02.repartecupos.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.11 Hash del token (WF-04)

El nodo Hash del token de WF-04 es una versión propia de este flujo. Además de calcular la huella SHA-256 del token, extrae del cuerpo de la petición el identificador de la cita a cancelar y el motivo de la cancelación.

El id_cita solo se acepta si es un entero positivo, escrito como número o como texto. Cualquier otro valor se reemplaza por el centinela 0, que ninguna cita tiene, de modo que la consulta no encuentra nada y el flujo responde 404 en lugar de terminar en el manejador de errores (defecto 12.12). Si no viene motivo, se registra "sin motivo".

1.11.1 Código base del nodo Hash del token

```
const crypto = require('crypto');

const cabeceras = $input.first().json.headers || {};
const cuerpo    = $input.first().json.body    || {};

const token = cabeceras['x-sigac-token'] || '';

const tokenHash = token
  ? crypto.createHash('sha256').update(token).digest('hex')
  : 'sin-token';

const valorCita = String(cuerpo.id_cita ?? '').trim();
const idCita = /^[1-9]\d*$/.test(valorCita) ? Number(valorCita) : 0;

return [{
  json: {
    token_hash: tokenHash,
    id_cita: idCita,
    motivo: cuerpo.motivo || 'sin motivo'
  }
}];
```

1.11.2 Código de test del nodo Hash del token

```
// sigac.wf04.hashdeltoken.test.js
//
// pruebas unitarias del nodo Hash del token de WF-04. es una version
// propia de este flujo: ademas de calcular la huella del token, extrae del
cuerpo el id_cita y el motivo de la cancelacion.
//
// el id_cita debe ser un entero positivo escrito como numero o como
// texto. cualquier otra cosa se reemplaza por el centinela 0, que
// ninguna cita tiene, de modo que la consulta no encuentra nada y el
// flujo responde 404 (defecto 12.12).
//
// cada prueba lleva su ID de la tabla de pruebas: UT-HASH04-NN
```

```
'use strict';

const hashDelToken = require('../nodos/wf04_hash_del_token.js');
const { ejecutar, sha256 } = require('../simulacion.js');

// arma la peticion que entrega el Webhook y ejecuta el nodo
const hash = peticion => ejecutar(hashDelToken, { entrada: [peticion]
}).salida[0].json;
const conCita = id_cita => hash({ headers: { 'x-sigac-token': 'abc' }, body: {
id_cita } });

// ------------------------------------------------------------
// A. Token
// ------------------------------------------------------------
describe('WF-04 · Hash del token · A. Token', () => {
  test('UT-HASH04-01 · con token: calcula su huella sha-256', () => {
    expect(conCita(5).token_hash).toBe(sha256('abc'));
  });

  test('UT-HASH04-02 · sin cabecera de token: centinela sin-token', () => {
    expect(hash({ headers: {}, body: { id_cita: 5 } }).token_hash).toBe('sin-
token');
  });

  test('UT-HASH04-03 · token vacio: centinela sin-token', () => {
    expect(hash({ headers: { 'x-sigac-token': '' }, body: {}
}).token_hash).toBe('sin-token');
  });

  test('UT-HASH04-04 · peticion sin headers ni body: no falla', () => {
    const salida = hash({});
    expect(salida.token_hash).toBe('sin-token');
    expect(salida.id_cita).toBe(0);
  });

  test('UT-HASH04-05 · token con espacios: se usa tal cual, sin recortar', () =>
{
    expect(hash({ headers: { 'x-sigac-token': ' abc ' }, body: {}
}).token_hash).toBe(sha256(' abc '));
  });

  test('UT-HASH04-06 · el token en claro no aparece en la salida', () => {
    const salida = hash({ headers: { 'x-sigac-token': 'token-secreto-de-prueba'
}, body: { id_cita: 5 } });
    expect(JSON.stringify(salida)).not.toContain('token-secreto-de-prueba');
```

```
  });
});

// ------------------------------------------------------------
// B. id_cita (defecto 12.12)
// ------------------------------------------------------------
describe('WF-04 · Hash del token · B. id_cita', () => {
  test.each([
    ['UT-HASH04-07', 'numero entero', 12, 12],
    ['UT-HASH04-08', 'texto numerico', '12', 12],
    ['UT-HASH04-09', 'texto con espacios', ' 12 ', 12]
  ])('%s · id_cita como %s: se acepta', (_, __, valor, esperado) => {
    expect(conCita(valor).id_cita).toBe(esperado);
  });

  test.each([
    ['UT-HASH04-10', 'cero', 0],
    ['UT-HASH04-11', 'negativo', -1],
    ['UT-HASH04-12', 'decimal', 12.5],
    ['UT-HASH04-13', 'texto', 'abc'],
    ['UT-HASH04-14', 'booleano true', true],
    ['UT-HASH04-15', 'con cero a la izquierda', '012'],
    ['UT-HASH04-16', 'nulo', null],
    ['UT-HASH04-17', 'ausente', undefined]
  ])('%s · id_cita %s: centinela 0', (_, __, valor) => {
    expect(conCita(valor).id_cita).toBe(0);
  });
});

// ------------------------------------------------------------
// C. Motivo y contrato
// ------------------------------------------------------------
describe('WF-04 · Hash del token · C. Motivo y contrato', () => {
  test('UT-HASH04-18 · motivo presente: pasa tal cual', () => {
    expect(hash({ headers: {}, body: { id_cita: 5, motivo: 'viaje' }
}).motivo).toBe('viaje');
  });

  test.each([
    ['UT-HASH04-19', 'ausente', undefined],
    ['UT-HASH04-20', 'vacio', '']
  ])('%s · motivo %s: queda "sin motivo"', (_, __, motivo) => {
    expect(hash({ headers: {}, body: { id_cita: 5, motivo } }).motivo).toBe('sin
motivo');
  });
```

```
  test('UT-HASH04-21 · devuelve un solo item con exactamente token_hash, id_cita
y motivo', () => {
    const { salida } = ejecutar(hashDelToken, { entrada: [{ headers: {}, body: {
id_cita: 5 } }] });
    expect(salida).toHaveLength(1);
    expect(Object.keys(salida[0].json).sort()).toEqual(['id_cita', 'motivo',
'token_hash']);
  });

  test('UT-HASH04-22 · no consulta ningun nodo anterior (verify)', () => {
    const { $ } = ejecutar(hashDelToken, { entrada: [{ headers: {}, body: {} }]
});
    expect($).not.toHaveBeenCalled();
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf04.hashdeltoken.test.js
 PASS  tests/sigac.wf04.hashdeltoken.test.js
  WF-04 · Hash del token · A. Token
    ✓ UT-HASH04-01 · con token: calcula su huella sha-256 (5 ms)
    ✓ UT-HASH04-02 · sin cabecera de token: centinela sin-token (1 ms)
    ✓ UT-HASH04-03 · token vacio: centinela sin-token
    ✓ UT-HASH04-04 · peticion sin headers ni body: no falla
    ✓ UT-HASH04-05 · token con espacios: se usa tal cual, sin recortar (1 ms)
    ✓ UT-HASH04-06 · el token en claro no aparece en la salida (1 ms)
  WF-04 · Hash del token · B. id_cita
    ✓ UT-HASH04-07 · id_cita como numero entero: se acepta (1 ms)
    ✓ UT-HASH04-08 · id_cita como texto numerico: se acepta
    ✓ UT-HASH04-09 · id_cita como texto con espacios: se acepta (1 ms)
    ✓ UT-HASH04-10 · id_cita cero: centinela 0
    ✓ UT-HASH04-11 · id_cita negativo: centinela 0
    ✓ UT-HASH04-12 · id_cita decimal: centinela 0 (1 ms)
    ✓ UT-HASH04-13 · id_cita texto: centinela 0
    ✓ UT-HASH04-14 · id_cita booleano true: centinela 0
    ✓ UT-HASH04-15 · id_cita con cero a la izquierda: centinela 0 (1 ms)
    ✓ UT-HASH04-16 · id_cita nulo: centinela 0
    ✓ UT-HASH04-17 · id_cita ausente: centinela 0 (1 ms)
  WF-04 · Hash del token · C. Motivo y contrato
    ✓ UT-HASH04-18 · motivo presente: pasa tal cual (1 ms)
    ✓ UT-HASH04-19 · motivo ausente: queda "sin motivo"
    ✓ UT-HASH04-20 · motivo vacio: queda "sin motivo"
    ✓ UT-HASH04-21 · devuelve un solo item con exactamente token_hash, id_cita y motivo (1 ms)
    ✓ UT-HASH04-22 · no consulta ningun nodo anterior (verify) (1 ms)

Test Suites: 1 passed, 1 total
Tests:       22 passed, 22 total
Snapshots:   0 total
Time:        3.763 s
Ran all test suites matching tests/sigac.wf04.hashdeltoken.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.12 Rechazo 401 (WF-04)

El nodo Rechazo 401 de WF-04 responde cuando el token no viene, no existe, está desactivado o su rol no puede cancelar citas. En este flujo, el nodo ¿Rol autorizado? revisa el token y el rol a la vez, por eso un solo rechazo cubre los dos casos.

El mensaje es genérico a propósito: no distingue entre un token inválido y un rol sin permiso. Es una respuesta fija, que no lee la entrada ni consulta otros nodos.

1.12.1 Código base del nodo Rechazo 401

```
return [{
  json: {
    codigo: 401,
    estado: 'rechazado',
    mensaje: 'Token ausente o invalido, o rol sin autorizacion'
  }
}];
```

1.12.2 Código de test del nodo Rechazo 401

```
// sigac.wf04.rechazo401.test.js
//
// pruebas unitarias del nodo Rechazo 401 de WF-04. responde cuando el token no
viene, no existe, esta desactivado o su
// rol no puede cancelar citas. lo alimenta la salida false de
// ¿Rol autorizado?, que en WF-04 revisa token y rol a la vez..
//
// el mensaje es generico a proposito: no distingue entre un token
// invalido y un rol sin permiso.
//
// es una respuesta fija: no lee la entrada ni consulta otros nodos.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-R40104-NN

'use strict';

const rechazo = require('../nodos/wf04_rechazo_401.js');
const { ejecutar } = require('../simulacion.js');

const RESPUESTA = {
  codigo: 401,
  estado: 'rechazado',
  mensaje: 'Token ausente o invalido, o rol sin autorizacion'
};

// lo que puede llegar al nodo: la salida del nodo anterior junto con
// lo que traia el flujo, incluida informacion que no debe devolverse
const CON_DATOS = {
  token_hash: 'a'.repeat(64),
```

```
  rol: 'medico',
  id_cita: 3432,
  datos: { dpi: '1234567890123', correo: 'ana@correo.com' }
};

describe('WF-04 · Rechazo 401', () => {
  test('UT-R40104-01 · respuesta completa', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json).toEqual(RESPUESTA);
  });

  test('UT-R40104-02 · codigo 401', () => {
    expect(ejecutar(rechazo, { entrada: [{}] }).salida[0].json.codigo).toBe(401);
  });

  test('UT-R40104-03 · estado rechazado (defecto 12.17)', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.estado).toBe('rechazado');
  });

  test('UT-R40104-04 · mensaje Token ausente o invalido, o rol sin autorizacion',
() => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.mensaje).toBe('Token ausente o invalido, o rol sin
autorizacion');
  });

  test('UT-R40104-05 · solo los campos codigo, estado y mensaje', () => {
    const { salida } = ejecutar(rechazo, { entrada: [{}] });
    expect(Object.keys(salida[0].json).sort()).toEqual(['codigo', 'estado',
'mensaje']);
  });

  test('UT-R40104-06 · sin entrada: la misma respuesta', () => {
    expect(ejecutar(rechazo).salida[0].json).toEqual(RESPUESTA);
  });

  test('UT-R40104-07 · con datos en la entrada: la misma respuesta, sin
devolverlos', () => {
    const { salida } = ejecutar(rechazo, { entrada: [CON_DATOS], json: CON_DATOS
});
    expect(salida[0].json).toEqual(RESPUESTA);
    const texto = JSON.stringify(salida);
    expect(texto).not.toContain('a'.repeat(64));
    expect(texto).not.toContain('1234567890123');
```

```
    expect(texto).not.toContain('3432');
  });

  test('UT-R40104-08 · no consulta ningun nodo anterior (verify)', () => {
    const { $ } = ejecutar(rechazo, { entrada: [{}] });
    expect($).not.toHaveBeenCalled();
  });

  test('UT-R40104-09 · dos ejecuciones con entradas distintas dan respuestas
identicas', () => {
    const a = ejecutar(rechazo, { entrada: [{ rol: 'recepcion' }], json: { rol:
'recepcion' } }).salida;
    const b = ejecutar(rechazo, { entrada: [CON_DATOS], json: CON_DATOS
}).salida;
    expect(a).toEqual(b);
  });

  test('UT-R40104-10 · el mensaje no revela la causa del rechazo', () => {
    const mensaje = ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.mensaje.toLowerCase();
    for (const pista of ['inactivo', 'desactivado', 'expirado', 'recepcion',
'medico']) {
      expect(mensaje).not.toContain(pista);
    }
  });

  test('UT-R40104-11 · devuelve una lista con un solo item', () => {
    const { salida } = ejecutar(rechazo, { entrada: [{}] });
    expect(Array.isArray(salida)).toBe(true);
    expect(salida).toHaveLength(1);
  });
});
```

```
Problems  Output  Debug Console  Terminal  Ports 4

manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf04.rechazo401.test.js
 PASS  tests/sigac.wf04.rechazo401.test.js
  WF-04 · Rechazo 401
    ✓ UT-R40104-01 · respuesta completa (5 ms)
    ✓ UT-R40104-02 · codigo 401 (1 ms)
    ✓ UT-R40104-03 · estado rechazado (defecto 12.17) (4 ms)
    ✓ UT-R40104-04 · mensaje Token ausente o invalido, o rol sin autorizacion
    ✓ UT-R40104-05 · solo los campos codigo, estado y mensaje
    ✓ UT-R40104-06 · sin entrada: la misma respuesta (1 ms)
    ✓ UT-R40104-07 · con datos en la entrada: la misma respuesta, sin devolverlos (1 ms)
    ✓ UT-R40104-08 · no consulta ningun nodo anterior (verify)
    ✓ UT-R40104-09 · dos ejecuciones con entradas distintas dan respuestas identicas (1 ms)
    ✓ UT-R40104-10 · el mensaje no revela la causa del rechazo (1 ms)
    ✓ UT-R40104-11 · devuelve una lista con un solo item

Test Suites: 1 passed, 1 total
Tests:       11 passed, 11 total
Snapshots:   0 total
Time:        3.629 s
Ran all test suites matching tests/sigac.wf04.rechazo401.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.13 Rechazo 404 (WF-04)

El nodo Rechazo 404 de WF-04 responde cuando el nodo Cancela cita no modificó ninguna fila: la cita no existe, ya estaba cancelada o cerrada, o el id_cita llegó inválido y se convirtió en el centinela 0.

El mensaje no distingue entre una cita inexistente y una que ya no estaba confirmada. Es una respuesta fija.

1.13.1 Código base del nodo Rechazo 404

```
return [{
  json: {
    codigo: 404,
    estado: 'rechazado',
    mensaje: 'La cita no existe o no estaba confirmada'
  }
}];
```

1.13.2 Código de test del nodo Rechazo 404

```
// sigac.wf04.rechazo404.test.js
//
// pruebas unitarias del nodo Rechazo 404 de WF-04. responde cuando Cancela cita
no modifico ninguna fila: la cita no
// existe, ya estaba cancelada, cerrada, o el id_cita llego invalido y
// se convirtio en el centinela 0. lo alimenta la salida false de
// ¿Se cancelo?.
//
// el mensaje no distingue entre una cita inexistente y una que ya no
// estaba confirmada.
//
// es una respuesta fija: no lee la entrada ni consulta otros nodos.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-R40404-NN

'use strict';

const rechazo = require('../nodos/wf04_rechazo_404.js');
const { ejecutar } = require('../simulacion.js');

const RESPUESTA = {
  codigo: 404,
  estado: 'rechazado',
  mensaje: 'La cita no existe o no estaba confirmada'
};

// lo que puede llegar al nodo: la salida del nodo anterior junto con
// lo que traia el flujo, incluida informacion que no debe devolverse
const CON_DATOS = {
```

```
  token_hash: 'a'.repeat(64),
  rol: 'medico',
  id_cita: 3432,
  datos: { dpi: '1234567890123', correo: 'ana@correo.com' }
};

describe('WF-04 · Rechazo 404', () => {
  test('UT-R40404-01 · respuesta completa', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json).toEqual(RESPUESTA);
  });

  test('UT-R40404-02 · codigo 404', () => {
    expect(ejecutar(rechazo, { entrada: [{}] }).salida[0].json.codigo).toBe(404);
  });

  test('UT-R40404-03 · estado rechazado (defecto 12.17)', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.estado).toBe('rechazado');
  });

  test('UT-R40404-04 · mensaje La cita no existe o no estaba confirmada', () => {
    expect(ejecutar(rechazo, { entrada: [{}] }).salida[0].json.mensaje).toBe('La
cita no existe o no estaba confirmada');
  });

  test('UT-R40404-05 · solo los campos codigo, estado y mensaje', () => {
    const { salida } = ejecutar(rechazo, { entrada: [{}] });
    expect(Object.keys(salida[0].json).sort()).toEqual(['codigo', 'estado',
'mensaje']);
  });

  test('UT-R40404-06 · sin entrada: la misma respuesta', () => {
    expect(ejecutar(rechazo).salida[0].json).toEqual(RESPUESTA);
  });

  test('UT-R40404-07 · con datos en la entrada: la misma respuesta, sin
devolverlos', () => {
    const { salida } = ejecutar(rechazo, { entrada: [CON_DATOS], json: CON_DATOS
});
    expect(salida[0].json).toEqual(RESPUESTA);
    const texto = JSON.stringify(salida);
    expect(texto).not.toContain('a'.repeat(64));
    expect(texto).not.toContain('1234567890123');
    expect(texto).not.toContain('3432');
```

```
  });

  test('UT-R40404-08 · no consulta ningun nodo anterior (verify)', () => {
    const { $ } = ejecutar(rechazo, { entrada: [{}] });
    expect($).not.toHaveBeenCalled();
  });

  test('UT-R40404-09 · dos ejecuciones con entradas distintas dan respuestas
identicas', () => {
    const a = ejecutar(rechazo, { entrada: [{ rol: 'recepcion' }], json: { rol:
'recepcion' } }).salida;
    const b = ejecutar(rechazo, { entrada: [CON_DATOS], json: CON_DATOS
}).salida;
    expect(a).toEqual(b);
  });

  test('UT-R40404-10 · el mensaje no revela el estado real de la cita', () => {
    const mensaje = ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.mensaje.toLowerCase();
    for (const pista of ['cancelada', 'atendida', 'ausente', 'centinela']) {
      expect(mensaje).not.toContain(pista);
    }
  });

  test('UT-R40404-11 · devuelve una lista con un solo item', () => {
    const { salida } = ejecutar(rechazo, { entrada: [{}] });
    expect(Array.isArray(salida)).toBe(true);
    expect(salida).toHaveLength(1);
  });
});
```

```
Problems  Output  Debug Console  Terminal  Ports 4

manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf04.rechazo404.test.js
 PASS  tests/sigac.wf04.rechazo404.test.js
  WF-04 · Rechazo 404
    ✓ UT-R40404-01 · respuesta completa (5 ms)
    ✓ UT-R40404-02 · codigo 404
    ✓ UT-R40404-03 · estado rechazado (defecto 12.17)
    ✓ UT-R40404-04 · mensaje La cita no existe o no estaba confirmada
    ✓ UT-R40404-05 · solo los campos codigo, estado y mensaje
    ✓ UT-R40404-06 · sin entrada: la misma respuesta
    ✓ UT-R40404-07 · con datos en la entrada: la misma respuesta, sin devolverlos (1 ms)
    ✓ UT-R40404-08 · no consulta ningun nodo anterior (verify)
    ✓ UT-R40404-09 · dos ejecuciones con entradas distintas dan respuestas identicas
    ✓ UT-R40404-10 · el mensaje no revela el estado real de la cita (1 ms)
    ✓ UT-R40404-11 · devuelve una lista con un solo item (1 ms)

Test Suites: 1 passed, 1 total
Tests:       11 passed, 11 total
Snapshots:   0 total
Time:        3.81 s
Ran all test suites matching tests/sigac.wf04.rechazo404.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.14 Reasignación exitosa (WF-04)

El nodo Reasignación exitosa arma la respuesta 200 cuando la cita se canceló y su cupo pasó al siguiente paciente en lista de espera. Toma el identificador de la cita cancelada del nodo Cancela cita, y el de la cita nueva y la solicitud beneficiada del nodo Reasigna cupo, que le llega como $json.

Las pruebas verifican el origen de cada uno de los tres identificadores y que la respuesta no exponga el médico ni la fecha de la cita cancelada.

1.14.1 Código base del nodo Reasignación exitosa

```
return [{
  json: {
    codigo: 200,
    estado: 'reasignado',
    mensaje: 'Cita cancelada y cupo reasignado',
    id_cita_cancelada: $('Cancela cita').first().json.id_cita,
    id_cita_nueva: $json.id_cita_nueva,
    id_solicitud_beneficiada: $json.id_solicitud
  }
}];
```

1.14.2 Código de test del nodo Reasignación exitosa

```
// sigac.wf04.reasignacionexitosa.test.js
//
// pruebas unitarias del nodo Reasignacion exitosa de WF-04. arma la
// respuesta 200 cuando la cita se cancelo y su cupo paso al siguiente
// paciente en lista de espera. toma la cita cancelada de Cancela cita,
// y la cita nueva y la solicitud beneficiada de Reasigna cupo, que le
// llega como $json.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-REA04-NN

'use strict';

const reasignacionExitosa = require('../nodos/wf04_reasignacion_exitosa.js');
const { ejecutar } = require('../simulacion.js');

function responder(cancelada, reasignada) {
  return ejecutar(reasignacionExitosa, {
    anteriores: { 'Cancela cita': cancelada },
    json: reasignada
  });
}

const CANCELADA = { id_cita: 53, id_medico: 1, fecha_hora: '2026-09-21T15:30:00Z'
};
const REASIGNADA = { id_cita_nueva: 2393, id_solicitud: 13 };
```

```
describe('WF-04 · Reasignacion exitosa', () => {
  test('UT-REA04-01 · respuesta completa con los tres identificadores', () => {
    expect(responder(CANCELADA, REASIGNADA).salida[0].json).toEqual({
      codigo: 200,
      estado: 'reasignado',
      mensaje: 'Cita cancelada y cupo reasignado',
      id_cita_cancelada: 53,
      id_cita_nueva: 2393,
      id_solicitud_beneficiada: 13
    });
  });

  test('UT-REA04-02 · la cita cancelada sale de Cancela cita', () => {
    expect(responder({ id_cita: 999 },
REASIGNADA).salida[0].json.id_cita_cancelada).toBe(999);
  });

  test('UT-REA04-03 · la cita nueva sale de $json (Reasigna cupo)', () => {
    expect(responder(CANCELADA, { ...REASIGNADA, id_cita_nueva: 5000
}).salida[0].json.id_cita_nueva).toBe(5000);
  });

  test('UT-REA04-04 · la solicitud beneficiada sale del id_solicitud de $json',
() => {
    expect(responder(CANCELADA, { ...REASIGNADA, id_solicitud: 77
}).salida[0].json.id_solicitud_beneficiada).toBe(77);
  });

  test('UT-REA04-05 · consulta el nodo Cancela cita (verify)', () => {
    const { $ } = responder(CANCELADA, REASIGNADA);
    expect($).toHaveBeenCalledWith('Cancela cita');
  });

  test('UT-REA04-06 · no expone el medico ni la fecha de la cita cancelada', ()
=> {
    const { json } = responder(CANCELADA, REASIGNADA).salida[0];
    expect(json).not.toHaveProperty('id_medico');
    expect(json).not.toHaveProperty('fecha_hora');
  });

  test('UT-REA04-07 · sin datos de Reasigna cupo: los campos quedan indefinidos,
sin error', () => {
    const { json } = responder(CANCELADA, {}).salida[0];
    expect(json.id_cita_nueva).toBeUndefined();
```

```
    expect(json.id_solicitud_beneficiada).toBeUndefined();
  });

  test('UT-REA04-08 · devuelve una lista con un solo item de seis campos', () =>
{
    const { salida } = responder(CANCELADA, REASIGNADA);
    expect(salida).toHaveLength(1);
    expect(Object.keys(salida[0].json)).toHaveLength(6);
  });
});
```

```
Problems  Output  Debug Console  Terminal  Ports 4

manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf04.reasignacionexitosa.test.js
 PASS  tests/sigac.wf04.reasignacionexitosa.test.js
  WF-04 · Reasignacion exitosa
    ✓ UT-REA04-01 · respuesta completa con los tres identificadores (5 ms)
    ✓ UT-REA04-02 · la cita cancelada sale de Cancela cita
    ✓ UT-REA04-03 · la cita nueva sale de $json (Reasigna cupo)
    ✓ UT-REA04-04 · la solicitud beneficiada sale del id_solicitud de $json (1 ms)
    ✓ UT-REA04-05 · consulta el nodo Cancela cita (verify) (1 ms)
    ✓ UT-REA04-06 · no expone el medico ni la fecha de la cita cancelada (1 ms)
    ✓ UT-REA04-07 · sin datos de Reasigna cupo: los campos quedan indefinidos, sin error (3 ms)
    ✓ UT-REA04-08 · devuelve una lista con un solo item de seis campos (1 ms)

Test Suites: 1 passed, 1 total
Tests:       8 passed, 8 total
Snapshots:   0 total
Time:        3.805 s
Ran all test suites matching tests/sigac.wf04.reasignacionexitosa.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.15 Cancelado sin reasignar (WF-04)

El nodo Cancelado sin reasignar arma la respuesta 200 cuando la cita se canceló, pero nadie esperaba cupo para esa fecha. El horario queda libre para futuras solicitudes. Toma el identificador de la cita cancelada del nodo Cancela cita.

1.15.1 Código base del nodo Cancelado sin reasignar

```
return [{
  json: {
    codigo: 200,
    estado: 'cancelado',
    mensaje: 'Cita cancelada. No habia nadie en lista de espera para esa fecha',
    id_cita_cancelada: $('Cancela cita').first().json.id_cita
  }
}];
```

1.15.2 Código de test del nodo Cancelado sin reasignar

```
// sigac.wf04.canceladosinreasignar.test.js
//
// pruebas unitarias del nodo Cancelado sin reasignar de WF-04. arma la
// respuesta 200 cuando la cita se cancelo pero nadie esperaba cupo para
// esa fecha: el horario queda libre para futuras solicitudes.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-CAN04-NN

'use strict';

const canceladoSinReasignar =
require('../nodos/wf04_cancelado_sin_reasignar.js');
const { ejecutar } = require('../simulacion.js');

const responder = (cancelada, json = {}) =>
  ejecutar(canceladoSinReasignar, { anteriores: { 'Cancela cita': cancelada },
json });

describe('WF-04 · Cancelado sin reasignar', () => {
  test('UT-CAN04-01 · respuesta completa', () => {
    expect(responder({ id_cita: 53 }).salida[0].json).toEqual({
      codigo: 200,
      estado: 'cancelado',
      mensaje: 'Cita cancelada. No habia nadie en lista de espera para esa
fecha',
      id_cita_cancelada: 53
    });
  });

  test('UT-CAN04-02 · la cita cancelada sale de Cancela cita', () => {
```

```
    expect(responder({ id_cita: 999
}).salida[0].json.id_cita_cancelada).toBe(999);
  });

  test('UT-CAN04-03 · estado cancelado, distinto de reasignado', () => {
    expect(responder({ id_cita: 53 }).salida[0].json.estado).toBe('cancelado');
  });

  test('UT-CAN04-04 · consulta el nodo Cancela cita (verify)', () => {
    const { $ } = responder({ id_cita: 53 });
    expect($).toHaveBeenCalledWith('Cancela cita');
  });

  test('UT-CAN04-05 · no depende de $json: la salida vacia de Siguiente en espera
no la cambia', () => {
    expect(responder({ id_cita: 53 }, {}).salida).toEqual(responder({ id_cita: 53
}, { otro: 1 }).salida);
  });

  test('UT-CAN04-06 · no expone el medico ni la fecha de la cita cancelada', ()
=> {
    const { json } = responder({ id_cita: 53, id_medico: 1, fecha_hora: '2026-09-
21T15:30:00Z' }).salida[0];
    expect(Object.keys(json).sort()).toEqual(['codigo', 'estado',
'id_cita_cancelada', 'mensaje']);
  });

  test('UT-CAN04-07 · devuelve una lista con un solo item', () => {
    const { salida } = responder({ id_cita: 53 });
    expect(Array.isArray(salida)).toBe(true);
    expect(salida).toHaveLength(1);
  });
});
```

```
Problems  Output  Debug Console  Terminal  Ports 4

manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf04.canceladosinreasignar.test.js
 PASS  tests/sigac.wf04.canceladosinreasignar.test.js
  WF-04 · Cancelado sin reasignar
    ✓ UT-CAN04-01 · respuesta completa (5 ms)
    ✓ UT-CAN04-02 · la cita cancelada sale de Cancela cita
    ✓ UT-CAN04-03 · estado cancelado, distinto de reasignado
    ✓ UT-CAN04-04 · consulta el nodo Cancela cita (verify) (1 ms)
    ✓ UT-CAN04-05 · no depende de $json: la salida vacia de Siguiente en espera no la cambia (1 ms)
    ✓ UT-CAN04-06 · no expone el medico ni la fecha de la cita cancelada
    ✓ UT-CAN04-07 · devuelve una lista con un solo item

Test Suites: 1 passed, 1 total
Tests:       7 passed, 7 total
Snapshots:   0 total
Time:        4.301 s
Ran all test suites matching tests/sigac.wf04.canceladosinreasignar.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.16 Hash del token (WF-05)

El nodo Hash del token de WF-05 es una versión propia de este flujo. Además de calcular la huella SHA-256 del token, extrae del cuerpo el identificador de la cita a cerrar y valida el resultado del cierre.

El id_cita solo se acepta si es un entero positivo; cualquier otro valor se reemplaza por el centinela 0 (defecto 12.12). El resultado solo puede ser atendida o ausente; cualquier otro valor se reemplaza por el centinela invalido, que la consulta de cierre rechaza.

1.16.1 Código base del nodo Hash del token

```
const crypto = require('crypto');

const cabeceras = $input.first().json.headers || {};
const cuerpo    = $input.first().json.body    || {};

const token = cabeceras['x-sigac-token'] || '';

const tokenHash = token
  ? crypto.createHash('sha256').update(token).digest('hex')
  : 'sin-token';
const resultado = ['atendida', 'ausente'].includes(cuerpo.resultado)
  ? cuerpo.resultado
  : null;

const valorCita = String(cuerpo.id_cita ?? '').trim();
const idCita = /^[1-9]\d*$/.test(valorCita) ? Number(valorCita) : 0;

return [{
  json: {
    token_hash: tokenHash,
    id_cita: idCita,
    resultado: resultado || 'invalido'
  }
}];
```

1.16.2 Código de test del nodo Hash del token

```
// sigac.wf05.hashdeltoken.test.js
//
// pruebas unitarias del nodo Hash del token de WF-05. es una version
// propia de este flujo: ademas de calcular la huella del token, extrae del
cuerpo el id_cita y valida el resultado del cierre.
//
// el id_cita debe ser un entero positivo escrito como numero o como
// texto. cualquier otra cosa se reemplaza por el centinela 0, que
// ninguna cita tiene, de modo que la consulta no encuentra nada y el
// flujo responde 404 (defecto 12.12).
//
```

```
// cada prueba lleva su ID de la tabla de pruebas: UT-HASH05-NN

'use strict';

const hashDelToken = require('../nodos/wf05_hash_del_token.js');
const { ejecutar, sha256 } = require('../simulacion.js');

// arma la peticion que entrega el Webhook y ejecuta el nodo
const hash = peticion => ejecutar(hashDelToken, { entrada: [peticion]
}).salida[0].json;
const conCita = id_cita => hash({ headers: { 'x-sigac-token': 'abc' }, body: {
id_cita } });

// ------------------------------------------------------------
// A. Token
// ------------------------------------------------------------
describe('WF-05 · Hash del token · A. Token', () => {
  test('UT-HASH05-01 · con token: calcula su huella sha-256', () => {
    expect(conCita(5).token_hash).toBe(sha256('abc'));
  });

  test('UT-HASH05-02 · sin cabecera de token: centinela sin-token', () => {
    expect(hash({ headers: {}, body: { id_cita: 5 } }).token_hash).toBe('sin-
token');
  });

  test('UT-HASH05-03 · token vacio: centinela sin-token', () => {
    expect(hash({ headers: { 'x-sigac-token': '' }, body: {}
}).token_hash).toBe('sin-token');
  });

  test('UT-HASH05-04 · peticion sin headers ni body: no falla', () => {
    const salida = hash({});
    expect(salida.token_hash).toBe('sin-token');
    expect(salida.id_cita).toBe(0);
  });

  test('UT-HASH05-05 · token con espacios: se usa tal cual, sin recortar', () =>
{
    expect(hash({ headers: { 'x-sigac-token': ' abc ' }, body: {}
}).token_hash).toBe(sha256(' abc '));
  });

  test('UT-HASH05-06 · el token en claro no aparece en la salida', () => {
```

```
    const salida = hash({ headers: { 'x-sigac-token': 'token-secreto-de-prueba'
}, body: { id_cita: 5 } });
    expect(JSON.stringify(salida)).not.toContain('token-secreto-de-prueba');
  });
});

// ------------------------------------------------------------
// B. id_cita (defecto 12.12)
// ------------------------------------------------------------
describe('WF-05 · Hash del token · B. id_cita', () => {
  test.each([
    ['UT-HASH05-07', 'numero entero', 12, 12],
    ['UT-HASH05-08', 'texto numerico', '12', 12],
    ['UT-HASH05-09', 'texto con espacios', ' 12 ', 12]
  ])('%s · id_cita como %s: se acepta', (_, __, valor, esperado) => {
    expect(conCita(valor).id_cita).toBe(esperado);
  });

  test.each([
    ['UT-HASH05-10', 'cero', 0],
    ['UT-HASH05-11', 'negativo', -1],
    ['UT-HASH05-12', 'decimal', 12.5],
    ['UT-HASH05-13', 'texto', 'abc'],
    ['UT-HASH05-14', 'booleano true', true],
    ['UT-HASH05-15', 'con cero a la izquierda', '012'],
    ['UT-HASH05-16', 'nulo', null],
    ['UT-HASH05-17', 'ausente', undefined]
  ])('%s · id_cita %s: centinela 0', (_, __, valor) => {
    expect(conCita(valor).id_cita).toBe(0);
  });
});

// ------------------------------------------------------------
// C. Resultado y contrato
// ------------------------------------------------------------
describe('WF-05 · Hash del token · C. Resultado y contrato', () => {
  const resultado = valor => hash({ headers: {}, body: { id_cita: 5, resultado:
valor } }).resultado;

  test.each([
    ['UT-HASH05-18', 'atendida'],
    ['UT-HASH05-19', 'ausente']
  ])('%s · resultado %s: se acepta', (_, valor) => {
    expect(resultado(valor)).toBe(valor);
  });
```

```
  test.each([
    ['UT-HASH05-20', 'cancelada (no es un resultado de cierre)', 'cancelada'],
    ['UT-HASH05-21', 'con mayuscula', 'Atendida'],
    ['UT-HASH05-22', 'ausente del cuerpo', undefined],
    ['UT-HASH05-23', 'vacio', '']
  ])('%s · resultado %s: centinela invalido', (_, __, valor) => {
    expect(resultado(valor)).toBe('invalido');
  });

  test('UT-HASH05-24 · devuelve un solo item con exactamente token_hash, id_cita
y resultado', () => {
    const { salida } = ejecutar(hashDelToken, { entrada: [{ headers: {}, body: {
id_cita: 5 } }] });
    expect(salida).toHaveLength(1);
    expect(Object.keys(salida[0].json).sort()).toEqual(['id_cita', 'resultado',
'token_hash']);
  });

  test('UT-HASH05-25 · no consulta ningun nodo anterior (verify)', () => {
    const { $ } = ejecutar(hashDelToken, { entrada: [{ headers: {}, body: {} }]
});
    expect($).not.toHaveBeenCalled();
  });
});
```

```
Problems  Output  Debug Console  Terminal  Ports 4

manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf05.hashdeltoken.test.js
 PASS  tests/sigac.wf05.hashdeltoken.test.js
  WF-05 · Hash del token · A. Token
    ✓ UT-HASH05-01 · con token: calcula su huella sha-256 (7 ms)
    ✓ UT-HASH05-02 · sin cabecera de token: centinela sin-token
    ✓ UT-HASH05-03 · token vacio: centinela sin-token (1 ms)
    ✓ UT-HASH05-04 · peticion sin headers ni body: no falla
    ✓ UT-HASH05-05 · token con espacios: se usa tal cual, sin recortar (1 ms)
    ✓ UT-HASH05-06 · el token en claro no aparece en la salida (1 ms)
  WF-05 · Hash del token · B. id_cita
    ✓ UT-HASH05-07 · id_cita como numero entero: se acepta
    ✓ UT-HASH05-08 · id_cita como texto numerico: se acepta
    ✓ UT-HASH05-09 · id_cita como texto con espacios: se acepta (1 ms)
    ✓ UT-HASH05-10 · id_cita cero: centinela 0 (1 ms)
    ✓ UT-HASH05-11 · id_cita negativo: centinela 0
    ✓ UT-HASH05-12 · id_cita decimal: centinela 0
    ✓ UT-HASH05-13 · id_cita texto: centinela 0 (1 ms)
    ✓ UT-HASH05-14 · id_cita booleano true: centinela 0
    ✓ UT-HASH05-15 · id_cita con cero a la izquierda: centinela 0 (1 ms)
    ✓ UT-HASH05-16 · id_cita nulo: centinela 0
    ✓ UT-HASH05-17 · id_cita ausente: centinela 0
  WF-05 · Hash del token · C. Resultado y contrato
    ✓ UT-HASH05-18 · resultado atendida: se acepta
    ✓ UT-HASH05-19 · resultado ausente: se acepta
    ✓ UT-HASH05-20 · resultado cancelada (no es un resultado de cierre): centinela invalido (1 ms)
    ✓ UT-HASH05-21 · resultado con mayuscula: centinela invalido
    ✓ UT-HASH05-22 · resultado ausente del cuerpo: centinela invalido (1 ms)
    ✓ UT-HASH05-23 · resultado vacio: centinela invalido
    ✓ UT-HASH05-24 · devuelve un solo item con exactamente token_hash, id_cita y resultado (2 ms)
    ✓ UT-HASH05-25 · no consulta ningun nodo anterior (verify) (1 ms)

Test Suites: 1 passed, 1 total
Tests:       25 passed, 25 total
Snapshots:   0 total
Time:        3.856 s
Ran all test suites matching tests/sigac.wf05.hashdeltoken.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.17 Rechazo 401 (WF-05)

El nodo Rechazo 401 de WF-05 responde cuando el token no viene, no existe, está desactivado o su rol no puede cerrar citas: solo el médico puede hacerlo.

El mensaje es genérico a propósito: no distingue entre un token inválido y un rol sin permiso. Es una respuesta fija.

1.17.1 Código base del nodo Rechazo 401

```
return [{
  json: {
    codigo: 401,
    estado: 'rechazado',
    mensaje: 'Token ausente o invalido, o rol sin autorizacion'
  }
}];
```

1.17.2 Código de test del nodo Rechazo 401

```
// sigac.wf05.rechazo401.test.js
//
// pruebas unitarias del nodo Rechazo 401 de WF-05. responde cuando el token no
viene, no existe, esta desactivado o su
// rol no puede cerrar citas: solo el medico puede. lo alimenta la
// salida false de ¿Rol autorizado?.
//
// el mensaje es generico a proposito: no distingue entre un token
// invalido y un rol sin permiso.
//
// es una respuesta fija: no lee la entrada ni consulta otros nodos.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-R40105-NN

'use strict';

const rechazo = require('../nodos/wf05_rechazo_401.js');
const { ejecutar } = require('../simulacion.js');

const RESPUESTA = {
  codigo: 401,
  estado: 'rechazado',
  mensaje: 'Token ausente o invalido, o rol sin autorizacion'
};

// lo que puede llegar al nodo: la salida del nodo anterior junto con
// lo que traia el flujo, incluida informacion que no debe devolverse
const CON_DATOS = {
  token_hash: 'a'.repeat(64),
  rol: 'medico',
  id_cita: 3432,
```

```
  datos: { dpi: '1234567890123', correo: 'ana@correo.com' }
};

describe('WF-05 · Rechazo 401', () => {
  test('UT-R40105-01 · respuesta completa', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json).toEqual(RESPUESTA);
  });

  test('UT-R40105-02 · codigo 401', () => {
    expect(ejecutar(rechazo, { entrada: [{}] }).salida[0].json.codigo).toBe(401);
  });

  test('UT-R40105-03 · estado rechazado (defecto 12.17)', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.estado).toBe('rechazado');
  });

  test('UT-R40105-04 · mensaje Token ausente o invalido, o rol sin autorizacion',
() => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.mensaje).toBe('Token ausente o invalido, o rol sin
autorizacion');
  });

  test('UT-R40105-05 · solo los campos codigo, estado y mensaje', () => {
    const { salida } = ejecutar(rechazo, { entrada: [{}] });
    expect(Object.keys(salida[0].json).sort()).toEqual(['codigo', 'estado',
'mensaje']);
  });

  test('UT-R40105-06 · sin entrada: la misma respuesta', () => {
    expect(ejecutar(rechazo).salida[0].json).toEqual(RESPUESTA);
  });

  test('UT-R40105-07 · con datos en la entrada: la misma respuesta, sin
devolverlos', () => {
    const { salida } = ejecutar(rechazo, { entrada: [CON_DATOS], json: CON_DATOS
});
    expect(salida[0].json).toEqual(RESPUESTA);
    const texto = JSON.stringify(salida);
    expect(texto).not.toContain('a'.repeat(64));
    expect(texto).not.toContain('1234567890123');
    expect(texto).not.toContain('3432');
  });
```

```
  test('UT-R40105-08 · no consulta ningun nodo anterior (verify)', () => {
    const { $ } = ejecutar(rechazo, { entrada: [{}] });
    expect($).not.toHaveBeenCalled();
  });

  test('UT-R40105-09 · dos ejecuciones con entradas distintas dan respuestas
identicas', () => {
    const a = ejecutar(rechazo, { entrada: [{ rol: 'recepcion' }], json: { rol:
'recepcion' } }).salida;
    const b = ejecutar(rechazo, { entrada: [CON_DATOS], json: CON_DATOS
}).salida;
    expect(a).toEqual(b);
  });

  test('UT-R40105-10 · el mensaje no revela la causa del rechazo', () => {
    const mensaje = ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.mensaje.toLowerCase();
    for (const pista of ['inactivo', 'desactivado', 'expirado', 'recepcion',
'medico']) {
      expect(mensaje).not.toContain(pista);
    }
  });

  test('UT-R40105-11 · devuelve una lista con un solo item', () => {
    const { salida } = ejecutar(rechazo, { entrada: [{}] });
    expect(Array.isArray(salida)).toBe(true);
    expect(salida).toHaveLength(1);
  });
});
```

```
Problems  Output  Debug Console  Terminal  Ports 4

manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf05.rechazo401.test.js
 PASS  tests/sigac.wf05.rechazo401.test.js
  WF-05 · Rechazo 401
    ✓ UT-R40105-01 · respuesta completa (5 ms)
    ✓ UT-R40105-02 · codigo 401 (1 ms)
    ✓ UT-R40105-03 · estado rechazado (defecto 12.17)
    ✓ UT-R40105-04 · mensaje Token ausente o invalido, o rol sin autorizacion (1 ms)
    ✓ UT-R40105-05 · solo los campos codigo, estado y mensaje
    ✓ UT-R40105-06 · sin entrada: la misma respuesta
    ✓ UT-R40105-07 · con datos en la entrada: la misma respuesta, sin devolverlos (1 ms)
    ✓ UT-R40105-08 · no consulta ningun nodo anterior (verify) (1 ms)
    ✓ UT-R40105-09 · dos ejecuciones con entradas distintas dan respuestas identicas
    ✓ UT-R40105-10 · el mensaje no revela la causa del rechazo
    ✓ UT-R40105-11 · devuelve una lista con un solo item (1 ms)

Test Suites: 1 passed, 1 total
Tests:       11 passed, 11 total
Snapshots:   0 total
Time:        4.258 s
Ran all test suites matching tests/sigac.wf05.rechazo401.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.18 Rechazo 404 (WF-05)

El nodo Rechazo 404 de WF-05 responde cuando el nodo Cierra cita no modificó ninguna fila: la cita no existe, ya fue cerrada o cancelada, todavía es futura, o el resultado enviado no era atendida ni ausente.

El mensaje es único a propósito: no revela cuál de los casos ocurrió. Es una respuesta fija.

1.18.1 Código base del nodo Rechazo 404

```
return [{
  json: {
    codigo: 404,
    estado: 'rechazado',
    mensaje: 'La cita no existe o no esta en condiciones de cerrarse'
  }
}];
```

1.18.2 Código de test del nodo Rechazo 404

```
// sigac.wf05.rechazo404.test.js
//
// pruebas unitarias del nodo Rechazo 404 de WF-05. responde cuando Cierra cita
no modifico ninguna fila: la cita no
// existe, ya fue cerrada o cancelada, es futura, o el resultado no era
// atendida ni ausente. lo alimenta la salida false de ¿Se cerro?.
//
// mensaje unico a proposito: no revela cual de los casos ocurrio.
//
// es una respuesta fija: no lee la entrada ni consulta otros nodos.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-R40405-NN

'use strict';

const rechazo = require('../nodos/wf05_rechazo_404.js');
const { ejecutar } = require('../simulacion.js');

const RESPUESTA = {
  codigo: 404,
  estado: 'rechazado',
  mensaje: 'La cita no existe o no esta en condiciones de cerrarse'
};

// lo que puede llegar al nodo: la salida del nodo anterior junto con
// lo que traia el flujo, incluida informacion que no debe devolverse
const CON_DATOS = {
  token_hash: 'a'.repeat(64),
  rol: 'medico',
  id_cita: 3432,
  datos: { dpi: '1234567890123', correo: 'ana@correo.com' }
```

```
};

describe('WF-05 · Rechazo 404', () => {
  test('UT-R40405-01 · respuesta completa', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json).toEqual(RESPUESTA);
  });

  test('UT-R40405-02 · codigo 404', () => {
    expect(ejecutar(rechazo, { entrada: [{}] }).salida[0].json.codigo).toBe(404);
  });

  test('UT-R40405-03 · estado rechazado (defecto 12.17)', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.estado).toBe('rechazado');
  });

  test('UT-R40405-04 · mensaje La cita no existe o no esta en condiciones de
cerrarse', () => {
    expect(ejecutar(rechazo, { entrada: [{}] }).salida[0].json.mensaje).toBe('La
cita no existe o no esta en condiciones de cerrarse');
  });

  test('UT-R40405-05 · solo los campos codigo, estado y mensaje', () => {
    const { salida } = ejecutar(rechazo, { entrada: [{}] });
    expect(Object.keys(salida[0].json).sort()).toEqual(['codigo', 'estado',
'mensaje']);
  });

  test('UT-R40405-06 · sin entrada: la misma respuesta', () => {
    expect(ejecutar(rechazo).salida[0].json).toEqual(RESPUESTA);
  });

  test('UT-R40405-07 · con datos en la entrada: la misma respuesta, sin
devolverlos', () => {
    const { salida } = ejecutar(rechazo, { entrada: [CON_DATOS], json: CON_DATOS
});
    expect(salida[0].json).toEqual(RESPUESTA);
    const texto = JSON.stringify(salida);
    expect(texto).not.toContain('a'.repeat(64));
    expect(texto).not.toContain('1234567890123');
    expect(texto).not.toContain('3432');
  });

  test('UT-R40405-08 · no consulta ningun nodo anterior (verify)', () => {
```

```
    const { $ } = ejecutar(rechazo, { entrada: [{}] });
    expect($).not.toHaveBeenCalled();
  });

  test('UT-R40405-09 · dos ejecuciones con entradas distintas dan respuestas
identicas', () => {
    const a = ejecutar(rechazo, { entrada: [{ rol: 'recepcion' }], json: { rol:
'recepcion' } }).salida;
    const b = ejecutar(rechazo, { entrada: [CON_DATOS], json: CON_DATOS
}).salida;
    expect(a).toEqual(b);
  });

  test('UT-R40405-10 · el mensaje no revela cual de los casos ocurrio', () => {
    const mensaje = ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.mensaje.toLowerCase();
    for (const pista of ['futura', 'cancelada', 'atendida', 'ausente',
'invalido']) {
      expect(mensaje).not.toContain(pista);
    }
  });

  test('UT-R40405-11 · devuelve una lista con un solo item', () => {
    const { salida } = ejecutar(rechazo, { entrada: [{}] });
    expect(Array.isArray(salida)).toBe(true);
    expect(salida).toHaveLength(1);
  });
});
```

```
Problems  Output  Debug Console  Terminal  Ports 4

manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf05.rechazo404.test.js
 PASS  tests/sigac.wf05.rechazo404.test.js
  WF-05 · Rechazo 404
    ✓ UT-R40405-01 · respuesta completa (5 ms)
    ✓ UT-R40405-02 · codigo 404 (3 ms)
    ✓ UT-R40405-03 · estado rechazado (defecto 12.17)
    ✓ UT-R40405-04 · mensaje La cita no existe o no esta en condiciones de cerrarse
    ✓ UT-R40405-05 · solo los campos codigo, estado y mensaje
    ✓ UT-R40405-06 · sin entrada: la misma respuesta (1 ms)
    ✓ UT-R40405-07 · con datos en la entrada: la misma respuesta, sin devolverlos (1 ms)
    ✓ UT-R40405-08 · no consulta ningun nodo anterior (verify)
    ✓ UT-R40405-09 · dos ejecuciones con entradas distintas dan respuestas identicas
    ✓ UT-R40405-10 · el mensaje no revela cual de los casos ocurrio
    ✓ UT-R40405-11 · devuelve una lista con un solo item

Test Suites: 1 passed, 1 total
Tests:       11 passed, 11 total
Snapshots:   0 total
Time:        3.611 s
Ran all test suites matching tests/sigac.wf05.rechazo404.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.19 Cierre exitoso (WF-05)

El nodo Cierre exitoso arma la respuesta 200 cuando la cita quedó cerrada como atendida o ausente. Recibe como $json la fila que devolvió el nodo Cierra cita, de la que toma el identificador de la cita y su nuevo estado.

1.19.1 Código base del nodo Cierre exitoso

```
return [{
  json: {
    codigo: 200,
    estado: 'cerrada',
    mensaje: 'Cita cerrada correctamente',
    id_cita: $json.id_cita,
    resultado: $json.estado
  }
}];
```

1.19.2 Código de test del nodo Cierre exitoso

```
// sigac.wf05.cierreexitoso.test.js
//
// pruebas unitarias del nodo Cierre exitoso de WF-05. arma la respuesta
// 200 cuando la cita quedo cerrada como atendida o ausente. recibe como
// $json la fila que devolvio Cierra cita.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-CIE05-NN

'use strict';

const cierreExitoso = require('../nodos/wf05_cierre_exitoso.js');
const { ejecutar } = require('../simulacion.js');

const cerrar = fila => ejecutar(cierreExitoso, { entrada: [fila], json: fila });

describe('WF-05 · Cierre exitoso', () => {
  test('UT-CIE05-01 · respuesta completa', () => {
    expect(cerrar({ id_cita: 3432, estado: 'atendida'
}).salida[0].json).toEqual({
      codigo: 200,
      estado: 'cerrada',
      mensaje: 'Cita cerrada correctamente',
      id_cita: 3432,
      resultado: 'atendida'
    });
  });

  test.each([
    ['UT-CIE05-02', 'atendida'],
    ['UT-CIE05-03', 'ausente']
```

```
  ])('%s · resultado %s: sale del estado de la fila cerrada', (_, estado) => {
    expect(cerrar({ id_cita: 1, estado }).salida[0].json.resultado).toBe(estado);
  });

  test('UT-CIE05-04 · el id_cita sale de la fila cerrada', () => {
    expect(cerrar({ id_cita: 999, estado: 'atendida'
}).salida[0].json.id_cita).toBe(999);
  });

  test('UT-CIE05-05 · el estado de la respuesta es siempre cerrada, no el de la
cita', () => {
    expect(cerrar({ id_cita: 1, estado: 'ausente'
}).salida[0].json.estado).toBe('cerrada');
  });

  test('UT-CIE05-06 · no consulta ningun nodo anterior (verify)', () => {
    const { $ } = cerrar({ id_cita: 1, estado: 'atendida' });
    expect($).not.toHaveBeenCalled();
  });

  test('UT-CIE05-07 · no expone otros campos de la fila', () => {
    const { json } = cerrar({ id_cita: 1, estado: 'atendida', id_paciente: 55,
id_medico: 2 }).salida[0];
    expect(Object.keys(json).sort()).toEqual(['codigo', 'estado', 'id_cita',
'mensaje', 'resultado']);
  });

  test('UT-CIE05-08 · devuelve una lista con un solo item', () => {
    const { salida } = cerrar({ id_cita: 1, estado: 'atendida' });
    expect(salida).toHaveLength(1);
  });
});
```

```
Problems  Output  Debug Console  Terminal  Ports 4

manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf05.cierreexitoso.test.js
 PASS  tests/sigac.wf05.cierreexitoso.test.js
  WF-05 · Cierre exitoso
    ✓ UT-CIE05-01 · respuesta completa (5 ms)
    ✓ UT-CIE05-02 · resultado atendida: sale del estado de la fila cerrada (1 ms)
    ✓ UT-CIE05-03 · resultado ausente: sale del estado de la fila cerrada (1 ms)
    ✓ UT-CIE05-04 · el id_cita sale de la fila cerrada
    ✓ UT-CIE05-05 · el estado de la respuesta es siempre cerrada, no el de la cita
    ✓ UT-CIE05-06 · no consulta ningun nodo anterior (verify) (1 ms)
    ✓ UT-CIE05-07 · no expone otros campos de la fila
    ✓ UT-CIE05-08 · devuelve una lista con un solo item (1 ms)

Test Suites: 1 passed, 1 total
Tests:       8 passed, 8 total
Snapshots:   0 total
Time:        3.837 s
Ran all test suites matching tests/sigac.wf05.cierreexitoso.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.20 Compone el reporte (WF-06)

El nodo Compone el reporte arma el HTML del reporte semanal con los indicadores del sistema y la carga de cada médico, y lo entrega como binario para que Gotenberg lo convierta en PDF. Es el único nodo asíncrono del sistema: usa await con this.helpers.prepareBinaryData, una función propia de n8n que en las pruebas se simula con jest.fn. La semana se numera según la norma ISO (defecto 12.16), y la fecha se toma en hora de Guatemala. Por eso las pruebas fijan el reloj en fechas conocidas, incluidos los cambios de año.

1.20.1 Código base del nodo Compone el reporte

```
const ind = $('Reune indicadores').first().json;
const medicos = $('Carga por medico').all().map(m => m.json);

const ahora  = DateTime.now().setZone('America/Guatemala');
const fecha  = ahora.toFormat('dd/MM/yyyy');
const semana = ahora.weekNumber;
const anio   = ahora.weekYear;

const filasMedicos = medicos.map(m => `
  <tr>
    <td>${m.nombre}</td>
    <td>${m.especialidad}</td>
    <td class="n">${m.atendidas}</td>
    <td class="n">${m.confirmadas}</td>
    <td class="n">${m.ausentes}</td>
  </tr>`).join('');

const html = `<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="utf-8">
<style>
  body { font-family: Helvetica, Arial, sans-serif; color: #1a1a1a; margin: 40px;
}
  h1 { font-size: 20px; margin: 0 0 4px 0; color: #1B2A5E; }
  .sub { font-size: 12px; color: #666; margin-bottom: 24px; }
  h2 { font-size: 13px; margin: 24px 0 8px 0; color: #1B2A5E;
       border-bottom: 1px solid #ddd; padding-bottom: 4px; }
  table { width: 100%; border-collapse: collapse; font-size: 11px; }
  th { text-align: left; background: #f2f4f8; padding: 6px 8px; font-weight:
bold; }
  td { padding: 6px 8px; border-bottom: 1px solid #eee; }
  td.n, th.n { text-align: right; }
  .nota { font-size: 10px; color: #777; margin: 4px 0 12px 0; }
  .pie { margin-top: 32px; font-size: 9px; color: #888;
         border-top: 1px solid #ddd; padding-top: 8px; }
```

```
</style>
</head>
<body>

<h1>SIGAC — Reporte semanal de indicadores</h1>
<div class="sub">Semana ${semana} del ${anio} &middot; Generado el ${fecha}</div>

<h2>Actividad de los ultimos 7 dias</h2>
<table>
  <tr><td>Solicitudes recibidas</td><td class="n">${ind.sol_semana}</td></tr>
  <tr><td>Citas creadas</td><td class="n">${ind.citas_semana}</td></tr>
  <tr><td>Errores de severidad alta</td><td
class="n">${ind.errores_alta}</td></tr>
  <tr><td>Errores de severidad media</td><td
class="n">${ind.errores_media}</td></tr>
  <tr><td>Errores de severidad baja</td><td
class="n">${ind.errores_baja}</td></tr>
</table>

<h2>Citas por estado (acumulado)</h2>
<table>
  <tr><td>Confirmadas</td><td class="n">${ind.citas_confirmadas}</td></tr>
  <tr><td>Atendidas</td><td class="n">${ind.citas_atendidas}</td></tr>
  <tr><td>Canceladas</td><td class="n">${ind.citas_canceladas}</td></tr>
  <tr><td>Ausentes</td><td class="n">${ind.citas_ausentes}</td></tr>
</table>

<h2>Solicitudes por estado (acumulado)</h2>
<table>
  <tr><td>Pendientes de clasificar</td><td
class="n">${ind.sol_pendientes}</td></tr>
  <tr><td>Asignadas</td><td class="n">${ind.sol_asignadas}</td></tr>
  <tr><td>En lista de espera</td><td class="n">${ind.sol_en_espera}</td></tr>
  <tr><td>Rechazadas</td><td class="n">${ind.sol_rechazadas}</td></tr>
</table>

<h2>Carga por medico (acumulado)</h2>
<table>
  <tr>
    <th>Medico</th><th>Especialidad</th>
    <th class="n">Atendidas</th><th class="n">Confirmadas</th><th
class="n">Ausentes</th>
  </tr>
  ${filasMedicos}
</table>
```

```
<h2>Lista de espera (estado actual)</h2>
<table>
  <tr><td>Solicitudes aguardando cupo</td><td
class="n">${ind.espera_pendientes}</td></tr>
  <tr><td>Reasignaciones completadas</td><td
class="n">${ind.espera_atendidas}</td></tr>
</table>

<div class="pie">
  Documento generado automaticamente por SIGAC.
  Contiene unicamente informacion agregada: no incluye datos identificativos
  ni clinicos de ningun paciente. Pacientes registrados activos:
  ${ind.pacientes_activos}.
  <br><br>
  Los indicadores marcados como acumulado reflejan el estado total del sistema
  y no el periodo semanal. La base registra la fecha de creacion de cada cita,
  pero no la fecha en que cambia de estado, de modo que no es posible acotar
  por semana cuantas citas fueron atendidas o canceladas.
</div>

</body>
</html>`;

return [{
  json: {
    nombre_archivo: `semana_${anio}-${String(semana).padStart(2, '0')}.pdf`,
    semana: semana,
    anio: anio
  },
  binary: {
    'index.html': await this.helpers.prepareBinaryData(
      Buffer.from(html, 'utf8'),
      'index.html',
      'text/html'
    )
  }
}];
```

1.20.2 Código de test del nodo Compone el reporte

```
// sigac.wf06.componeelreporte.test.js
//
// pruebas unitarias del nodo Compone el reporte de WF-06. arma el HTML
// del reporte semanal con los indicadores y la carga por medico, y lo
// entrega como binario a Gotenberg para convertirlo en PDF.
//
```

```
// es el unico nodo asincrono del sistema: usa await con
// this.helpers.prepareBinaryData, propio de n8n, que aqui se simula con
// jest.fn. la semana se numera con la norma ISO (defecto 12.16), asi que
// el reloj se fija en fechas conocidas.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-REP06-NN

'use strict';

const componeReporte = require('../nodos/wf06_compone_el_reporte.js');
const { ejecutar } = require('../simulacion.js');

const INDICADORES = {
  sol_semana: 41, citas_semana: 37, errores_alta: 2, errores_media: 1,
errores_baja: 0,
  citas_confirmadas: 120, citas_atendidas: 80, citas_canceladas: 9,
citas_ausentes: 4,
  sol_pendientes: 3, sol_asignadas: 200, sol_en_espera: 12, sol_rechazadas: 0,
  espera_pendientes: 12, espera_atendidas: 5, pacientes_activos: 73
};

const MEDICOS = [
  { nombre: 'Ana Lucia Morales Chub', especialidad: 'Medicina General',
atendidas: 30, confirmadas: 40, ausentes: 1 },
  { nombre: 'Carlos Enrique Coc Ical', especialidad: 'Pediatria', atendidas: 20,
confirmadas: 25, ausentes: 2 }
];

// simula this.helpers.prepareBinaryData: guarda lo que recibe
function contextoN8n() {
  const prepareBinaryData = jest.fn(async (buffer, nombre, tipo) => ({
    data: buffer.toString('base64'), fileName: nombre, mimeType: tipo
  }));
  return { helpers: { prepareBinaryData } };
}

// ejecuta el nodo en una fecha dada y devuelve la salida, el html y el mock
async function componer(fechaIso, medicos = MEDICOS, indicadores = INDICADORES) {
  jest.setSystemTime(new Date(fechaIso));
  const contexto = contextoN8n();
  const { salida, $ } = ejecutar(componeReporte, {
    anteriores: { 'Reune indicadores': indicadores, 'Carga por medico': medicos
},
    contexto
  });
```

```
  const resultado = await salida;
  const html = Buffer.from(resultado[0].binary['index.html'].data,
'base64').toString('utf8');
  return { resultado, html, $, prepare: contexto.helpers.prepareBinaryData };
}

beforeAll(() => jest.useFakeTimers({ doNotFake: ['nextTick', 'setImmediate'] }));
afterAll(() => jest.useRealTimers());

// 23 de octubre de 2026, mediodia en Guatemala: semana ISO 43
const VIERNES_43 = '2026-10-23T18:00:00Z';

// ------------------------------------------------------------
// A. Semana ISO y nombre del archivo (defecto 12.16)
// ------------------------------------------------------------
describe('WF-06 · Compone el reporte · A. Semana ISO', () => {
  test('UT-REP06-01 · 23/10/2026: semana 43 de 2026', async () => {
    const { resultado } = await componer(VIERNES_43);
    expect(resultado[0].json).toEqual({ nombre_archivo: 'semana_2026-43.pdf',
semana: 43, anio: 2026 });
  });

  test('UT-REP06-02 · 01/01/2026 (jueves): semana 1, con el numero en dos
digitos', async () => {
    const { resultado } = await componer('2026-01-01T18:00:00Z');
    expect(resultado[0].json.nombre_archivo).toBe('semana_2026-01.pdf');
  });

  test('UT-REP06-03 · 01/01/2027 (viernes): pertenece a la semana 53 de 2026',
async () => {
    const { resultado } = await componer('2027-01-01T18:00:00Z');
    expect(resultado[0].json).toEqual({ nombre_archivo: 'semana_2026-53.pdf',
semana: 53, anio: 2026 });
  });

  test('UT-REP06-04 · 29/12/2025 (lunes): pertenece a la semana 1 de 2026', async
() => {
    const { resultado } = await componer('2025-12-29T18:00:00Z');
    expect(resultado[0].json.nombre_archivo).toBe('semana_2026-01.pdf');
  });

  test('UT-REP06-05 · la fecha se toma en hora de Guatemala, no en UTC', async ()
=> {
    // 24/10 a las 03:00 UTC son las 21:00 del 23/10 en Guatemala
    const { html } = await componer('2026-10-24T03:00:00Z');
```

```
    expect(html).toContain('Generado el 23/10/2026');
  });
});

// ------------------------------------------------------------
// B. Contenido del HTML
// ------------------------------------------------------------
describe('WF-06 · Compone el reporte · B. Contenido', () => {
  test('UT-REP06-06 · el encabezado muestra la semana, el anio y la fecha', async
() => {
    const { html } = await componer(VIERNES_43);
    expect(html).toContain('Semana 43 del 2026');
    expect(html).toContain('Generado el 23/10/2026');
  });

  test('UT-REP06-07 · incluye cada indicador con su valor', async () => {
    const { html } = await componer(VIERNES_43);
    // los indicadores de las tablas, cada uno en su celda
    const { pacientes_activos, ...deTablas } = INDICADORES;
    for (const valor of Object.values(deTablas)) {
      expect(html).toContain(`<td class="n">${valor}</td>`);
    }
    expect(html).toContain('Solicitudes recibidas</td><td class="n">41');
    // los pacientes activos van en el pie, como texto
    expect(html).toContain(`${pacientes_activos}.`);
  });

  test('UT-REP06-08 · una fila por medico, con sus cinco columnas', async () => {
    const { html } = await componer(VIERNES_43);
    expect(html).toContain('<td>Ana Lucia Morales Chub</td>');
    expect(html).toContain('<td>Pediatria</td>');
    expect((html.match(/<td class="n">30<\/td>/g) || []).length).toBe(1);
  });

  test('UT-REP06-09 · sin medicos: la tabla queda sin filas, sin error', async ()
=> {
    const { html } = await componer(VIERNES_43, []);
    expect(html).not.toContain('Ana Lucia');
    expect(html).toContain('Carga por medico');
  });

  test('UT-REP06-10 · el pie declara que los acumulados no son semanales', async
() => {
    const { html } = await componer(VIERNES_43);
    expect(html).toContain('no es posible acotar');
```

```
  });

  test('UT-REP06-11 · no incluye datos de pacientes, aunque llegaran en la
entrada', async () => {
    const conPaciente = [{ ...MEDICOS[0], dpi: '1234567890123', correo:
'ana@correo.com' }];
    const { html } = await componer(VIERNES_43, conPaciente);
    expect(html).not.toContain('1234567890123');
    expect(html).not.toContain('ana@correo.com');
  });

  test('UT-REP06-12 · es un documento HTML completo en espanol', async () => {
    const { html } = await componer(VIERNES_43);
    expect(html.startsWith('<!DOCTYPE html>')).toBe(true);
    expect(html).toContain('<html lang="es">');
    expect(html.trim().endsWith('</html>')).toBe(true);
  });
});

// ------------------------------------------------------------
// C. Binario para Gotenberg
// ------------------------------------------------------------
describe('WF-06 · Compone el reporte · C. Binario', () => {
  test('UT-REP06-13 · llama a prepareBinaryData una vez, como index.html de tipo
text/html', async () => {
    const { prepare } = await componer(VIERNES_43);
    expect(prepare).toHaveBeenCalledTimes(1);
    const [buffer, nombre, tipo] = prepare.mock.calls[0];
    expect(Buffer.isBuffer(buffer)).toBe(true);
    expect(nombre).toBe('index.html');
    expect(tipo).toBe('text/html');
  });

  test('UT-REP06-14 · el binario se entrega en la propiedad index.html', async ()
=> {
    const { resultado } = await componer(VIERNES_43);
    expect(Object.keys(resultado[0].binary)).toEqual(['index.html']);
  });

  test('UT-REP06-15 · el HTML viaja en UTF-8: la raya del titulo no se pierde',
async () => {
    const { html } = await componer(VIERNES_43);
    expect(html).toContain('SIGAC — Reporte semanal');
  });
```

```
  test('UT-REP06-16 · consulta Reune indicadores y Carga por medico (verify)',
async () => {
    const { $ } = await componer(VIERNES_43);
    expect($).toHaveBeenCalledWith('Reune indicadores');
    expect($).toHaveBeenCalledWith('Carga por medico');
  });

  test('UT-REP06-17 · es asincrono: devuelve una promesa', () => {
    jest.setSystemTime(new Date(VIERNES_43));
    const { salida } = ejecutar(componeReporte, {
      anteriores: { 'Reune indicadores': INDICADORES, 'Carga por medico': MEDICOS
},
      contexto: contextoN8n()
    });
    expect(salida).toBeInstanceOf(Promise);
    return salida;
  });

  test('UT-REP06-18 · devuelve un solo item', async () => {
    const { resultado } = await componer(VIERNES_43);
    expect(resultado).toHaveLength(1);
  });
});

// sigac.wf06.nombraelarchivo.test.js
//
// pruebas unitarias del nodo Nombra el archivo de WF-06. Gotenberg
// devuelve el PDF sin un nombre util; este nodo le pone el mismo nombre
// con que se guarda en disco, para que el correo lo adjunte con ese
// nombre y no con un identificador aleatorio.
//
// el nodo modifica el binario del item que recibe, asi que la entrada
// se arma a mano, con su propiedad binary.data.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-NOM06-NN

'use strict';

const nombraArchivo = require('../nodos/wf06_nombra_el_archivo.js');
const { nodosAnteriores } = require('../simulacion.js');
const { DateTime } = require('luxon');

// el item que entrega Gotenberg: el pdf en binary.data, sin nombre util
const pdf = () => ({
  json: {},
```

```
  binary: { data: { data: 'JVBERi0x', fileName: 'a1b2c3', fileExtension: 'bin',
mimeType: 'application/octet-stream' } }
});

function nombrar(item, nombre = 'semana_2026-43.pdf') {
  const $ = nodosAnteriores({ 'Compone el reporte': { nombre_archivo: nombre }
});
  const entrada = { first: () => item, all: () => [item] };
  return { salida: nombraArchivo.call({}, entrada, $, item.json, {}, DateTime), $
};
}

describe('WF-06 · Nombra el archivo', () => {
  test('UT-NOM06-01 · el archivo toma el nombre de Compone el reporte', () => {
    expect(nombrar(pdf()).salida[0].binary.data.fileName).toBe('semana_2026-
43.pdf');
  });

  test('UT-NOM06-02 · la extension queda como pdf', () => {
    expect(nombrar(pdf()).salida[0].binary.data.fileExtension).toBe('pdf');
  });

  test('UT-NOM06-03 · el tipo queda como application/pdf', () => {
    expect(nombrar(pdf()).salida[0].binary.data.mimeType).toBe('application/pdf')
;
  });

  test('UT-NOM06-04 · el contenido del pdf no se toca', () => {
    expect(nombrar(pdf()).salida[0].binary.data.data).toBe('JVBERi0x');
  });

  test('UT-NOM06-05 · otro nombre de semana se aplica igual', () => {
    expect(nombrar(pdf(), 'semana_2026-
01.pdf').salida[0].binary.data.fileName).toBe('semana_2026-01.pdf');
  });

  test('UT-NOM06-06 · devuelve el mismo item que recibio', () => {
    const item = pdf();
    expect(nombrar(item).salida[0]).toBe(item);
  });

  test('UT-NOM06-07 · consulta Compone el reporte (verify)', () => {
    expect(nombrar(pdf()).$).toHaveBeenCalledWith('Compone el reporte');
  });
```

```
  test('UT-NOM06-08 · sin binario en la entrada: lanza un error', () => {
    expect(() => nombrar({ json: {} })).toThrow();
  });

  test('UT-NOM06-09 · devuelve una lista con un solo item', () => {
    expect(nombrar(pdf()).salida).toHaveLength(1);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf06.componeelreporte.test.js
 PASS  tests/sigac.wf06.componeelreporte.test.js
  WF-06 · Compone el reporte · A. Semana ISO
    ✓ UT-REP06-01 · 23/10/2026: semana 43 de 2026 (33 ms)
    ✓ UT-REP06-02 · 01/01/2026 (jueves): semana 1, con el numero en dos digitos (1 ms)
    ✓ UT-REP06-03 · 01/01/2027 (viernes): pertenece a la semana 53 de 2026 (1 ms)
    ✓ UT-REP06-04 · 29/12/2025 (lunes): pertenece a la semana 1 de 2026 (1 ms)
    ✓ UT-REP06-05 · la fecha se toma en hora de Guatemala, no en UTC (1 ms)
  WF-06 · Compone el reporte · B. Contenido
    ✓ UT-REP06-06 · el encabezado muestra la semana, el anio y la fecha (1 ms)
    ✓ UT-REP06-07 · incluye cada indicador con su valor (2 ms)
    ✓ UT-REP06-08 · una fila por medico, con sus cinco columnas (1 ms)
    ✓ UT-REP06-09 · sin medicos: la tabla queda sin filas, sin error (1 ms)
    ✓ UT-REP06-10 · el pie declara que los acumulados no son semanales
    ✓ UT-REP06-11 · no incluye datos de pacientes, aunque llegaran en la entrada
    ✓ UT-REP06-12 · es un documento HTML completo en espanol
  WF-06 · Compone el reporte · C. Binario
    ✓ UT-REP06-13 · llama a prepareBinaryData una vez, como index.html de tipo text/html
    ✓ UT-REP06-14 · el binario se entrega en la propiedad index.html
    ✓ UT-REP06-15 · el HTML viaja en UTF-8: la raya del titulo no se pierde (1 ms)
    ✓ UT-REP06-16 · consulta Reune indicadores y Carga por medico (verify) (1 ms)
    ✓ UT-REP06-17 · es asincrono: devuelve una promesa (1 ms)
    ✓ UT-REP06-18 · devuelve un solo item

Test Suites: 1 passed, 1 total
Tests:       18 passed, 18 total
Snapshots:   0 total
Time:        3.803 s
Ran all test suites matching tests/sigac.wf06.componeelreporte.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.21 Nombra el archivo (WF-06)

El nodo Nombra el archivo le pone al PDF que devuelve Gotenberg el mismo nombre con que se guarda en disco, por ejemplo semana_2026-43.pdf. Sin este nodo, el correo adjuntaba el reporte con un identificador aleatorio.

El nodo modifica el binario del item que recibe, así que en las pruebas la entrada se arma a mano, con su propiedad binary.data.

1.21.1 Código base del nodo Nombra el archivo

```
const nombre = $('Compone el reporte').first().json.nombre_archivo;
const item = $input.first();

item.binary.data.fileName = nombre;
item.binary.data.fileExtension = 'pdf';
item.binary.data.mimeType = 'application/pdf';

return [item];
```

1.21.2 Código de test del nodo Nombra el archivo

```
// sigac.wf06.nombraelarchivo.test.js
//
// pruebas unitarias del nodo Nombra el archivo de WF-06. Gotenberg
// devuelve el PDF sin un nombre util; este nodo le pone el mismo nombre
// con que se guarda en disco, para que el correo lo adjunte con ese
// nombre y no con un identificador aleatorio.
//
// el nodo modifica el binario del item que recibe, asi que la entrada
// se arma a mano, con su propiedad binary.data.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-NOM06-NN

'use strict';

const nombraArchivo = require('../nodos/wf06_nombra_el_archivo.js');
const { nodosAnteriores } = require('../simulacion.js');
const { DateTime } = require('luxon');

const pdf = () => ({
  json: {},
  binary: { data: { data: 'JVBERi0x', fileName: 'a1b2c3', fileExtension: 'bin',
mimeType: 'application/octet-stream' } }
});

function nombrar(item, nombre = 'semana_2026-43.pdf') {
  const $ = nodosAnteriores({ 'Compone el reporte': { nombre_archivo: nombre }
});
  const entrada = { first: () => item, all: () => [item] };
```

```
  return { salida: nombraArchivo.call({}, entrada, $, item.json, {}, DateTime), $
};
}

describe('WF-06 · Nombra el archivo', () => {
  test('UT-NOM06-01 · el archivo toma el nombre de Compone el reporte', () => {
    expect(nombrar(pdf()).salida[0].binary.data.fileName).toBe('semana_2026-
43.pdf');
  });

  test('UT-NOM06-02 · la extension queda como pdf', () => {
    expect(nombrar(pdf()).salida[0].binary.data.fileExtension).toBe('pdf');
  });

  test('UT-NOM06-03 · el tipo queda como application/pdf', () => {
    expect(nombrar(pdf()).salida[0].binary.data.mimeType).toBe('application/pdf')
;
  });

  test('UT-NOM06-04 · el contenido del pdf no se toca', () => {
    expect(nombrar(pdf()).salida[0].binary.data.data).toBe('JVBERi0x');
  });

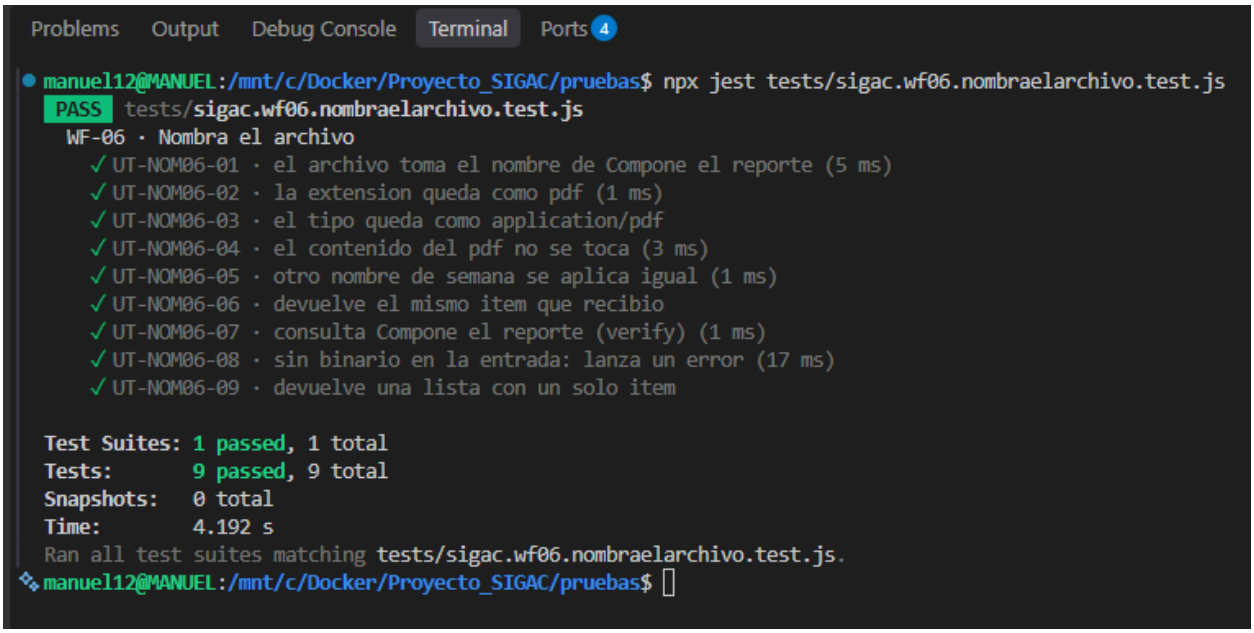
  test('UT-NOM06-05 · otro nombre de semana se aplica igual', () => {
    expect(nombrar(pdf(), 'semana_2026-
01.pdf').salida[0].binary.data.fileName).toBe('semana_2026-01.pdf');
  });

  test('UT-NOM06-06 · devuelve el mismo item que recibio', () => {
    const item = pdf();
    expect(nombrar(item).salida[0]).toBe(item);
  });

  test('UT-NOM06-07 · consulta Compone el reporte (verify)', () => {
    expect(nombrar(pdf()).$).toHaveBeenCalledWith('Compone el reporte');
  });

  test('UT-NOM06-08 · sin binario en la entrada: lanza un error', () => {
    expect(() => nombrar({ json: {} })).toThrow();
  });

  test('UT-NOM06-09 · devuelve una lista con un solo item', () => {
    expect(nombrar(pdf()).salida).toHaveLength(1);
  });
});
```


Problems
Output
Debug Console
Terminal
Ports 4
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf06.nombraelarchivo.test.js
PASS tests/sigac.wf06.nombraelarchivo.test.js
WF-06 · Nombra el archivo
✓ UT-NOM06-01 · el archivo toma el nombre de Compone el reporte (5 ms)
✓ UT-NOM06-02 · la extension queda como pdf (1 ms)
✓ UT-NOM06-03 · el tipo queda como application/pdf
✓ UT-NOM06-04 · el contenido del pdf no se toca (3 ms)
✓ UT-NOM06-05 · otro nombre de semana se aplica igual (1 ms)
✓ UT-NOM06-06 · devuelve el mismo item que recibio
✓ UT-NOM06-07 · consulta Compone el reporte (verify) (1 ms)
✓ UT-NOM06-08 · sin binario en la entrada: lanza un error (17 ms)
✓ UT-NOM06-09 · devuelve una lista con un solo item
Test Suites: 1 passed, 1 total
Tests: 9 passed, 9 total
Snapshots: 0 total
Time: 4.192 s
Ran all test suites matching tests/sigac.wf06.nombraelarchivo.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$

1.22 Hash del token (WF-07)

El nodo Hash del token de WF-07 recibe la petición del webhook, lee el token de la cabecera x-sigac-token y calcula su huella SHA-256, que es la que se compara contra la base de datos. El token nunca se compara ni se guarda en claro. Si la petición no trae token, el nodo usa el centinela sin-token, que nunca puede coincidir con una huella real.

Es idéntico al de WF-01, WF-08 y WF-09.

1.22.1 Código base del nodo Hash del token

```
const crypto = require('crypto');

const cabeceras = $input.first().json.headers || {};
const cuerpo    = $input.first().json.body    || {};

const token = cabeceras['x-sigac-token'] || '';

const tokenHash = token
  ? crypto.createHash('sha256').update(token).digest('hex')
  : 'sin-token';

return [{
  json: {
    token_hash: tokenHash,
    tiene_token: token.length > 0,
    datos: cuerpo
  }
}];
```

1.22.2 Código de test del nodo Hash del token

```
// sigac.wf07.hashdeltoken.test.js
//
// pruebas unitarias del nodo Hash del token de WF-07. lee el token de
// la cabecera x-sigac-token y calcula su huella sha-256, que es la que
// se compara contra la base. el token nunca se compara ni se guarda en
// claro. sin token, usa el centinela sin-token, que nunca puede
// coincidir con una huella real.
//
// tecnicas: particion de equivalencia, valores limite, cobertura de
// ramas (headers || {}, body || {}, token ? hash : centinela),
// propiedades del hash y contrato de salida.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-HASH07-NN

'use strict';

const hashDelToken = require('../nodos/wf07_hash_del_token.js');
const { ejecutar, sha256 } = require('../simulacion.js');
```

```
// arma la peticion que entrega el Webhook y ejecuta el nodo
const hash = peticion => ejecutar(hashDelToken, { entrada: [peticion] }).salida;

describe('WF-07 · Hash del token', () => {
  test('UT-HASH07-01 · con token: calcula su huella sha-256', () => {
    const salida = hash({ headers: { 'x-sigac-token': 'abc' }, body: {} });
    expect(salida[0].json.token_hash).toBe(sha256('abc'));
    expect(salida[0].json.tiene_token).toBe(true);
  });

  test('UT-HASH07-02 · sin cabecera de token: centinela sin-token', () => {
    const salida = hash({ headers: {}, body: {} });
    expect(salida[0].json.token_hash).toBe('sin-token');
    expect(salida[0].json.tiene_token).toBe(false);
  });

  test('UT-HASH07-03 · token vacio: centinela sin-token', () => {
    const salida = hash({ headers: { 'x-sigac-token': '' }, body: {} });
    expect(salida[0].json.token_hash).toBe('sin-token');
    expect(salida[0].json.tiene_token).toBe(false);
  });

  test('UT-HASH07-04 · peticion sin headers: no falla y usa el centinela', () =>
{
    const salida = hash({ body: {} });
    expect(salida[0].json.token_hash).toBe('sin-token');
  });

  test('UT-HASH07-05 · peticion sin body: datos queda como objeto vacio', () => {
    const salida = hash({ headers: { 'x-sigac-token': 'abc' } });
    expect(salida[0].json.datos).toEqual({});
  });

  test('UT-HASH07-06 · el cuerpo pasa intacto en datos', () => {
    const cuerpo = { dpi: '1234567890123', nombre: 'Ana', anidado: { a: 1 } };
    const salida = hash({ headers: {}, body: cuerpo });
    expect(salida[0].json.datos).toEqual(cuerpo);
  });

  test('UT-HASH07-07 · cabecera en mayusculas: no se lee (n8n las entrega en
minusculas)', () => {
    const salida = hash({ headers: { 'X-SIGAC-TOKEN': 'abc' }, body: {} });
    expect(salida[0].json.token_hash).toBe('sin-token');
  });
```

```
  test('UT-HASH07-08 · token con espacios: se usa tal cual, sin recortar', () =>
{
    const salida = hash({ headers: { 'x-sigac-token': ' abc ' }, body: {} });
    expect(salida[0].json.token_hash).toBe(sha256(' abc '));
    expect(salida[0].json.token_hash).not.toBe(sha256('abc'));
  });

  test('UT-HASH07-09 · el mismo token da siempre la misma huella', () => {
    const a = hash({ headers: { 'x-sigac-token': 'abc' }, body: {} });
    const b = hash({ headers: { 'x-sigac-token': 'abc' }, body: {} });
    expect(a[0].json.token_hash).toBe(b[0].json.token_hash);
  });

  test('UT-HASH07-10 · tokens distintos dan huellas distintas', () => {
    const a = hash({ headers: { 'x-sigac-token': 'abc' }, body: {} });
    const b = hash({ headers: { 'x-sigac-token': 'abd' }, body: {} });
    expect(a[0].json.token_hash).not.toBe(b[0].json.token_hash);
  });

  test('UT-HASH07-11 · la huella tiene 64 caracteres hexadecimales en minuscula',
() => {
    const salida = hash({ headers: { 'x-sigac-token': 'abc' }, body: {} });
    expect(salida[0].json.token_hash).toMatch(/^[0-9a-f]{64}$/);
  });

  test('UT-HASH07-12 · el token en claro no aparece en la salida', () => {
    // el token lleva guiones, que nunca aparecen en un hexadecimal:
    // si se encuentra en la salida, es porque el token salio en claro
    const token = 'token-secreto-de-prueba';
    const salida = hash({ headers: { 'x-sigac-token': token }, body: {} });
    expect(JSON.stringify(salida)).not.toContain(token);
  });

  test('UT-HASH07-13 · devuelve un solo item con exactamente tres campos', () =>
{
    const salida = hash({ headers: { 'x-sigac-token': 'abc' }, body: {} });
    expect(salida).toHaveLength(1);
    expect(Object.keys(salida[0].json).sort()).toEqual(['datos', 'tiene_token',
'token_hash']);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf07.hashdeltoken.test.js
 PASS  tests/sigac.wf07.hashdeltoken.test.js
  WF-07 · Hash del token
    ✓ UT-HASH07-01 · con token: calcula su huella sha-256 (6 ms)
    ✓ UT-HASH07-02 · sin cabecera de token: centinela sin-token (3 ms)
    ✓ UT-HASH07-03 · token vacio: centinela sin-token (1 ms)
    ✓ UT-HASH07-04 · peticion sin headers: no falla y usa el centinela
    ✓ UT-HASH07-05 · peticion sin body: datos queda como objeto vacio (1 ms)
    ✓ UT-HASH07-06 · el cuerpo pasa intacto en datos
    ✓ UT-HASH07-07 · cabecera en mayusculas: no se lee (n8n las entrega en minusculas) (1 ms)
    ✓ UT-HASH07-08 · token con espacios: se usa tal cual, sin recortar
    ✓ UT-HASH07-09 · el mismo token da siempre la misma huella (1 ms)
    ✓ UT-HASH07-10 · tokens distintos dan huellas distintas
    ✓ UT-HASH07-11 · la huella tiene 64 caracteres hexadecimales en minuscula
    ✓ UT-HASH07-12 · el token en claro no aparece en la salida (1 ms)
    ✓ UT-HASH07-13 · devuelve un solo item con exactamente tres campos (3 ms)

  Test Suites: 1 passed, 1 total
  Tests:       13 passed, 13 total
  Snapshots:   0 total
  Time:        4.159 s
  Ran all test suites matching tests/sigac.wf07.hashdeltoken.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.23 Valida datos (WF-07)

El nodo Valida datos de WF-07 revisa el cuerpo de la nota clínica: el id_cita, el diagnóstico (obligatorio) y las indicaciones (opcionales), con un máximo de 2000 caracteres cada uno. Los errores se acumulan y se devuelven todos juntos.

El médico que firma la nota no se lee del cuerpo de la petición, sino del token verificado. Así nadie puede escribir una nota a nombre de otro médico, aunque conozca el formato de la petición.

1.23.1 Código base del nodo Valida datos

```
const cuerpo = $('Hash del token').first().json.datos || {};
const medico = $('Verifica token').first().json;

const errores = [];
const MAX = 2000;

const idCita = Number(cuerpo.id_cita);
if (!Number.isInteger(idCita) || idCita <= 0) {
  errores.push('id_cita debe ser un entero positivo');
}

const diagnostico = typeof cuerpo.diagnostico === 'string' ?
cuerpo.diagnostico.trim() : '';
if (!diagnostico) {
  errores.push('diagnostico es obligatorio');
} else if (diagnostico.length > MAX) {
  errores.push(`diagnostico no puede exceder ${MAX} caracteres`);
}

let indicaciones = null;
if (cuerpo.indicaciones !== undefined && cuerpo.indicaciones !== null &&
cuerpo.indicaciones !== '') {
  if (typeof cuerpo.indicaciones !== 'string') {
    errores.push('indicaciones debe ser texto');
  } else {
    indicaciones = cuerpo.indicaciones.trim();
    if (indicaciones.length > MAX) {
      errores.push(`indicaciones no puede exceder ${MAX} caracteres`);
    }
  }
}

return [{
  json: {
    valido: errores.length === 0,
    errores,
```

```
    id_cita: idCita,
    id_medico: medico.id_medico,
    diagnostico,
    indicaciones
  }
}];
```

1.23.2 Código de test del nodo Valida datos

```
// sigac.wf07.validadatos.test.js
//
// pruebas unitarias del nodo Valida datos de WF-07. revisa el cuerpo de
// la nota clinica: id_cita, diagnostico (obligatorio) e indicaciones
// (opcionales), con un maximo de 2000 caracteres cada uno. los errores
// se acumulan y se devuelven todos juntos.
//
// el medico que firma NO se lee del cuerpo: sale de Verifica token. asi
// nadie puede escribir una nota a nombre de otro medico.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-VAL07-NN

'use strict';

const validaDatos = require('../nodos/wf07_valida_datos.js');
const { ejecutar } = require('../simulacion.js');

const valida = () => ({ id_cita: 3432, diagnostico: 'Faringitis aguda',
indicaciones: 'Reposo' });

function validar(cuerpo, medico = { rol: 'medico', id_medico: 2 }, hash = {
datos: cuerpo }) {
  return ejecutar(validaDatos, { anteriores: { 'Hash del token': hash, 'Verifica
token': medico } });
}

const salidaDe = cambios => validar({ ...valida(), ...cambios }).salida[0].json;

// ------------------------------------------------------------
// A. id_cita
// ------------------------------------------------------------
describe('WF-07 · Valida datos · A. id_cita', () => {
  test.each([
    ['UT-VAL07-01', 'numero', 3432, 3432],
    ['UT-VAL07-02', 'texto numerico', '3432', 3432]
  ])('%s · id_cita como %s: valido', (_, __, id_cita, esperado) => {
    const salida = salidaDe({ id_cita });
    expect(salida.valido).toBe(true);
```

```
    expect(salida.id_cita).toBe(esperado);
  });

  test.each([
    ['UT-VAL07-03', 'cero', 0],
    ['UT-VAL07-04', 'negativo', -1],
    ['UT-VAL07-05', 'decimal', 2.5],
    ['UT-VAL07-06', 'texto', 'abc'],
    ['UT-VAL07-07', 'ausente', undefined],
    ['UT-VAL07-08', 'vacio', '']
  ])('%s · id_cita %s: error', (_, __, id_cita) => {
    expect(salidaDe({ id_cita }).errores).toContain('id_cita debe ser un entero
positivo');
  });

  // hipotesis A: Number(true) es 1, asi que true se aceptaria como la
  // cita 1. un identificador se envia como numero, nunca como booleano.
  test('UT-VAL07-09 · id_cita true: debe rechazarse (hipotesis A)', () => {
    expect(salidaDe({ id_cita: true }).valido).toBe(false);
  });
});

// ------------------------------------------------------------
// B. Diagnostico
// ------------------------------------------------------------
describe('WF-07 · Valida datos · B. Diagnostico', () => {
  test('UT-VAL07-10 · diagnostico presente: valido', () => {
    expect(salidaDe({}).valido).toBe(true);
  });

  test.each([
    ['UT-VAL07-11', 'ausente', undefined],
    ['UT-VAL07-12', 'vacio', ''],
    ['UT-VAL07-13', 'con solo espacios', '   '],
    ['UT-VAL07-14', 'que no es texto', 12345]
  ])('%s · diagnostico %s: error, es obligatorio', (_, __, diagnostico) => {
    expect(salidaDe({ diagnostico }).errores).toContain('diagnostico es
obligatorio');
  });

  test('UT-VAL07-15 · diagnostico con espacios: sale recortado', () => {
    expect(salidaDe({ diagnostico: '  Faringitis  '
}).diagnostico).toBe('Faringitis');
  });
```

```
  test.each([
    ['UT-VAL07-16', 2000, true],
    ['UT-VAL07-17', 2001, false]
  ])('%s · diagnostico de %i caracteres', (_, largo, esperado) => {
    expect(salidaDe({ diagnostico: 'x'.repeat(largo) }).valido).toBe(esperado);
  });
});

// ---------------------------------------------------------
// C. Indicaciones (opcionales)
// ---------------------------------------------------------
describe('WF-07 · Valida datos · C. Indicaciones', () => {
  test.each([
    ['UT-VAL07-18', 'ausentes', undefined],
    ['UT-VAL07-19', 'nulas', null],
    ['UT-VAL07-20', 'vacias', '']
  ])('%s · indicaciones %s: valido y quedan nulas', (_, __, indicaciones) => {
    const salida = salidaDe({ indicaciones });
    expect(salida.valido).toBe(true);
    expect(salida.indicaciones).toBeNull();
  });

  test('UT-VAL07-21 · indicaciones con espacios: salen recortadas', () => {
    expect(salidaDe({ indicaciones: '  Reposo  ' }).indicaciones).toBe('Reposo');
  });

  test('UT-VAL07-22 · indicaciones con solo espacios: quedan como texto vacio, no
nulas', () => {
    expect(salidaDe({ indicaciones: '   ' }).indicaciones).toBe('');
  });

  test('UT-VAL07-23 · indicaciones que no son texto: error', () => {
    expect(salidaDe({ indicaciones: 5 }).errores).toContain('indicaciones debe
ser texto');
  });

  test.each([
    ['UT-VAL07-24', 2000, true],
    ['UT-VAL07-25', 2001, false]
  ])('%s · indicaciones de %i caracteres', (_, largo, esperado) => {
    expect(salidaDe({ indicaciones: 'x'.repeat(largo) }).valido).toBe(esperado);
  });
});

// ---------------------------------------------------------
```

```
// D. Firma, conjunto y contrato
// ------------------------------------------------------------
describe('WF-07 · Valida datos · D. Firma y contrato', () => {
  test('UT-VAL07-26 · el medico que firma sale de Verifica token', () => {
    expect(validar(valida(), { rol: 'medico', id_medico: 4
}).salida[0].json.id_medico).toBe(4);
  });

  test('UT-VAL07-27 · un id_medico en el cuerpo se ignora: nadie firma por otro',
() => {
    expect(validar({ ...valida(), id_medico: 1 }, { rol: 'medico', id_medico: 2
}).salida[0].json.id_medico).toBe(2);
  });

  test('UT-VAL07-28 · cuerpo vacio: los dos errores juntos', () => {
    expect(validar({}).salida[0].json.errores).toEqual(['id_cita debe ser un
entero positivo', 'diagnostico es obligatorio']);
  });

  test('UT-VAL07-29 · Hash del token sin datos: no falla y valida como cuerpo
vacio', () => {
    expect(validar(undefined, undefined,
{}).salida[0].json.errores).toHaveLength(2);
  });

  test('UT-VAL07-30 · consulta Hash del token y Verifica token (verify)', () => {
    const { $ } = validar(valida());
    expect($).toHaveBeenCalledWith('Hash del token');
    expect($).toHaveBeenCalledWith('Verifica token');
  });

  test('UT-VAL07-31 · devuelve un item con los seis campos', () => {
    const { salida } = validar(valida());
    expect(salida).toHaveLength(1);
    expect(Object.keys(salida[0].json).sort())
      .toEqual(['diagnostico', 'errores', 'id_cita', 'id_medico', 'indicaciones',
'valido']);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf07.validadatos.test.js
 FAIL  tests/sigac.wf07.validadatos.test.js
  WF-07 · Valida datos · A. id_cita
    ✓ UT-VAL07-01 · id_cita como numero: valido (6 ms)
    ✓ UT-VAL07-02 · id_cita como texto numerico: valido
    ✓ UT-VAL07-03 · id_cita cero: error (1 ms)
    ✓ UT-VAL07-04 · id_cita negativo: error
    ✓ UT-VAL07-05 · id_cita decimal: error (1 ms)
    ✓ UT-VAL07-06 · id_cita texto: error
    ✓ UT-VAL07-07 · id_cita ausente: error
    ✓ UT-VAL07-08 · id_cita vacio: error
    ✕ UT-VAL07-09 · id_cita true: debe rechazarse (hipotesis A) (3 ms)
  WF-07 · Valida datos · B. Diagnostico
    ✓ UT-VAL07-10 · diagnostico presente: valido (1 ms)
    ✓ UT-VAL07-11 · diagnostico ausente: error, es obligatorio
    ✓ UT-VAL07-12 · diagnostico vacio: error, es obligatorio
    ✓ UT-VAL07-13 · diagnostico con solo espacios: error, es obligatorio
    ✓ UT-VAL07-14 · diagnostico que no es texto: error, es obligatorio
    ✓ UT-VAL07-15 · diagnostico con espacios: sale recortado
    ✓ UT-VAL07-16 · diagnostico de 2000 caracteres
    ✓ UT-VAL07-17 · diagnostico de 2001 caracteres
  WF-07 · Valida datos · C. Indicaciones
    ✓ UT-VAL07-18 · indicaciones ausentes: valido y quedan nulas (1 ms)
    ✓ UT-VAL07-19 · indicaciones nulas: valido y quedan nulas (1 ms)
    ✓ UT-VAL07-20 · indicaciones vacias: valido y quedan nulas
    ✓ UT-VAL07-21 · indicaciones con espacios: salen recortadas (1 ms)
    ✓ UT-VAL07-22 · indicaciones con solo espacios: quedan como texto vacio, no nulas
    ✓ UT-VAL07-23 · indicaciones que no son texto: error
    ✓ UT-VAL07-24 · indicaciones de 2000 caracteres (1 ms)
    ✓ UT-VAL07-25 · indicaciones de 2001 caracteres
  WF-07 · Valida datos · D. Firma y contrato
    ✓ UT-VAL07-26 · el medico que firma sale de Verifica token (1 ms)
    ✓ UT-VAL07-27 · un id_medico en el cuerpo se ignora: nadie firma por otro
    ✓ UT-VAL07-28 · cuerpo vacio: los dos errores juntos
    ✓ UT-VAL07-29 · Hash del token sin datos: no falla y valida como cuerpo vacio
    ✓ UT-VAL07-30 · consulta Hash del token y Verifica token (verify) (1 ms)
    ✓ UT-VAL07-31 · devuelve un item con los seis campos (1 ms)

  ● WF-07 · Valida datos · A. id_cita › UT-VAL07-09 · id_cita true: debe rechazarse (hipotesis A)

    expect(received).toBe(expected) // Object.is equality

    Expected: false
    Received: true

      51 |   // cita 1. un identificador se envia como numero, nunca como booleano.
      52 |   test('UT-VAL07-09 · id_cita true: debe rechazarse (hipotesis A)', () => {
    > 53 |     expect(salidaDe({ id_cita: true }).valido).toBe(false);
         |                                                ^
      54 |   });
      55 | });
      56 |

      at Object.toBe (tests/sigac.wf07.validadatos.test.js:53:48)

Test Suites: 1 failed, 1 total
Tests:       1 failed, 30 passed, 31 total
Snapshots:   0 total
Time:        4.391 s
Ran all test suites matching tests/sigac.wf07.validadatos.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

Defecto detectado por las pruebas (12.24). La prueba UT-VAL07-09 falló: el nodo convierte el identificador con Number(), y Number(true) es 1, de modo que un true en lugar de un número se acepta como el identificador 1. Es el mismo defecto que el 12.23 de WF-01.

1.23.3 Nuevo código base de Valida Datos

```
const cuerpo = $('Hash del token').first().json.datos || {};
const medico = $('Verifica token').first().json;

const errores = [];
const MAX = 2000;

// defecto 12.24: antes se convertia con Number(), y Number(true) es 1,
// de modo que "id_cita": true se aceptaba como la cita 1. ahora solo se
// acepta un numero o un texto que contenga un entero positivo, igual
// que id_medico_solicitado en WF-01 (defecto 12.23).
const valorCita = (typeof cuerpo.id_cita === 'number' || typeof cuerpo.id_cita
=== 'string')
  ? String(cuerpo.id_cita).trim()
  : '';
const idCita = /^[1-9]\d*$/.test(valorCita) ? Number(valorCita) : null;
if (idCita === null) {
  errores.push('id_cita debe ser un entero positivo');
}

const diagnostico = typeof cuerpo.diagnostico === 'string' ?
cuerpo.diagnostico.trim() : '';
if (!diagnostico) {
  errores.push('diagnostico es obligatorio');
} else if (diagnostico.length > MAX) {
  errores.push(`diagnostico no puede exceder ${MAX} caracteres`);
}

// indicaciones es opcional: una consulta puede no requerir tratamiento
let indicaciones = null;
if (cuerpo.indicaciones !== undefined && cuerpo.indicaciones !== null &&
cuerpo.indicaciones !== '') {
  if (typeof cuerpo.indicaciones !== 'string') {
    errores.push('indicaciones debe ser texto');
  } else {
    indicaciones = cuerpo.indicaciones.trim();
    if (indicaciones.length > MAX) {
      errores.push(`indicaciones no puede exceder ${MAX} caracteres`);
    }
  }
}
```

```
return [{
  json: {
    valido: errores.length === 0,
    errores,
    id_cita: idCita,
    id_medico: medico.id_medico,
    diagnostico,
    indicaciones
  }
}];
```

1.23.4 Se corrió el mismo código test pero ya con la corrección del código base.

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf07.validadatos.test.js
 PASS  tests/sigac.wf07.validadatos.test.js
  WF-07 · Valida datos · A. id_cita
    ✓ UT-VAL07-01 · id_cita como numero: valido (5 ms)
    ✓ UT-VAL07-02 · id_cita como texto numerico: valido
    ✓ UT-VAL07-03 · id_cita cero: error (1 ms)
    ✓ UT-VAL07-04 · id_cita negativo: error (1 ms)
    ✓ UT-VAL07-05 · id_cita decimal: error
    ✓ UT-VAL07-06 · id_cita texto: error
    ✓ UT-VAL07-07 · id_cita ausente: error
    ✓ UT-VAL07-08 · id_cita vacio: error
    ✓ UT-VAL07-09 · id_cita true: debe rechazarse (hipotesis A)
  WF-07 · Valida datos · B. Diagnostico
    ✓ UT-VAL07-10 · diagnostico presente: valido (1 ms)
    ✓ UT-VAL07-11 · diagnostico ausente: error, es obligatorio
    ✓ UT-VAL07-12 · diagnostico vacio: error, es obligatorio
    ✓ UT-VAL07-13 · diagnostico con solo espacios: error, es obligatorio
    ✓ UT-VAL07-14 · diagnostico que no es texto: error, es obligatorio (4 ms)
    ✓ UT-VAL07-15 · diagnostico con espacios: sale recortado (1 ms)
    ✓ UT-VAL07-16 · diagnostico de 2000 caracteres
    ✓ UT-VAL07-17 · diagnostico de 2001 caracteres (1 ms)
  WF-07 · Valida datos · C. Indicaciones
    ✓ UT-VAL07-18 · indicaciones ausentes: valido y quedan nulas
    ✓ UT-VAL07-19 · indicaciones nulas: valido y quedan nulas
    ✓ UT-VAL07-20 · indicaciones vacias: valido y quedan nulas (1 ms)
    ✓ UT-VAL07-21 · indicaciones con espacios: salen recortadas
    ✓ UT-VAL07-22 · indicaciones con solo espacios: quedan como texto vacio, no nulas (1 ms)
    ✓ UT-VAL07-23 · indicaciones que no son texto: error
    ✓ UT-VAL07-24 · indicaciones de 2000 caracteres (1 ms)
    ✓ UT-VAL07-25 · indicaciones de 2001 caracteres
  WF-07 · Valida datos · D. Firma y contrato
    ✓ UT-VAL07-26 · el medico que firma sale de Verifica token (1 ms)
    ✓ UT-VAL07-27 · un id_medico en el cuerpo se ignora: nadie firma por otro
    ✓ UT-VAL07-28 · cuerpo vacio: los dos errores juntos (1 ms)
    ✓ UT-VAL07-29 · Hash del token sin datos: no falla y valida como cuerpo vacio
    ✓ UT-VAL07-30 · consulta Hash del token y Verifica token (verify) (1 ms)
    ✓ UT-VAL07-31 · devuelve un item con los seis campos (1 ms)

Test Suites: 1 passed, 1 total
Tests:       31 passed, 31 total
Snapshots:   0 total
Time:        4.086 s
Ran all test suites matching tests/sigac.wf07.validadatos.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.24 Cifra la nota (WF-07)

El nodo Cifra la nota cifra el diagnóstico y las indicaciones con AES-256-GCM, con la misma función y el mismo formato que WF-01: iv:textoCifrado:etiqueta. Su salida ya no lleva el texto clínico en claro.

Toma la cita del nodo Busca la cita y la nota del nodo Valida datos. Si no hay indicaciones, envía el centinela null, que la consulta de Guarda nota convierte en un nulo real. Como el cifrado es aleatorio, las pruebas descifran el resultado y verifican que vuelva el texto original.

1.24.1 Código base del nodo Cifra la nota

```
const crypto = require('crypto');

const clave = Buffer.from($env.SIGAC_CLAVE_CIFRADO, 'hex');
if (clave.length !== 32) {
  throw new Error('SIGAC_CLAVE_CIFRADO debe ser de 32 bytes (64 caracteres
hex)');
}

function cifrar(texto) {
  if (!texto) return null;
  const iv = crypto.randomBytes(12);
  const cifrador = crypto.createCipheriv('aes-256-gcm', clave, iv);
  const cifrado = Buffer.concat([cifrador.update(String(texto), 'utf8'),
cifrador.final()]);
  const etiqueta = cifrador.getAuthTag();
  return [iv.toString('hex'), cifrado.toString('hex'),
etiqueta.toString('hex')].join(':');
}

const cita = $input.first().json;
const nota = $('Valida datos').first().json;

return [{
  json: {
    id_expediente: cita.id_expediente,
    id_paciente: cita.id_paciente,
    id_cita: cita.id_cita,
    id_medico: nota.id_medico,
    diagnostico_cifrado: cifrar(nota.diagnostico),
    indicaciones_cifrado: cifrar(nota.indicaciones) ?? 'null'
  }
}];
```

1.24.2 Código de test del nodo Cifra la nota

```
// sigac.wf07.cifralanota.test.js
//
// pruebas unitarias del nodo Cifra la nota de WF-07. cifra el
```

```
// diagnostico y las indicaciones con AES-256-GCM, con la misma funcion y
// el mismo formato que WF-01: iv:textoCifrado:etiqueta. su salida ya no
// lleva el texto clinico en claro.
//
// toma la cita de $input (salida de Busca la cita) y la nota de
// Valida datos. si no hay indicaciones, envia el centinela 'null', que
// la consulta de Guarda nota convierte en nulo con nullif.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-CIF07-NN

'use strict';

const crypto = require('crypto');
const cifraNota = require('../nodos/wf07_cifra_la_nota.js');
const { ejecutar, CLAVE_PRUEBA } = require('../simulacion.js');

const CITA = { id_expediente: 40, id_paciente: 55, id_cita: 3432 };
const NOTA = { id_medico: 2, diagnostico: 'Faringitis aguda, sin fiebre',
indicaciones: 'Reposo, hidratacion' };
const FORMATO = /^[0-9a-f]{24}:[0-9a-f]+:[0-9a-f]{32}$/;

function cifrar(nota = NOTA, cita = CITA, clave = CLAVE_PRUEBA) {
  return ejecutar(cifraNota, {
    entrada: [cita],
    anteriores: { 'Valida datos': nota },
    env: { SIGAC_CLAVE_CIFRADO: clave }
  });
}

const salidaDe = (nota, cita) => cifrar(nota, cita).salida[0].json;

function descifrar(valor, clave = CLAVE_PRUEBA) {
  const [iv, datos, etiqueta] = valor.split(':');
  const d = crypto.createDecipheriv('aes-256-gcm', Buffer.from(clave, 'hex'),
Buffer.from(iv, 'hex'));
  d.setAuthTag(Buffer.from(etiqueta, 'hex'));
  return Buffer.concat([d.update(Buffer.from(datos, 'hex')),
d.final()]).toString('utf8');
}

// ---------------------------------------------------------------
// A. Clave de cifrado
// ---------------------------------------------------------------
describe('WF-07 · Cifra la nota · A. Clave', () => {
  test('UT-CIF07-01 · clave valida: cifra sin error', () => {
```

```
    expect(() => cifrar()).not.toThrow();
  });

  test.each([
    ['UT-CIF07-02', '62 caracteres', CLAVE_PRUEBA.slice(0, 62)],
    ['UT-CIF07-03', '66 caracteres', CLAVE_PRUEBA + 'ff'],
    ['UT-CIF07-04', 'no hexadecimal', 'z'.repeat(64)]
  ])('%s · clave de %s: lanza error', (_, __, clave) => {
    expect(() => cifrar(NOTA, CITA, clave)).toThrow('SIGAC_CLAVE_CIFRADO debe ser
de 32 bytes');
  });

  test('UT-CIF07-05 · variable de entorno sin definir: lanza error', () => {
    expect(() => ejecutar(cifraNota, { entrada: [CITA], anteriores: { 'Valida
datos': NOTA }, env: {} })).toThrow();
  });
});

// ------------------------------------------------------------
// B. Cifrado del texto clinico
// ------------------------------------------------------------
describe('WF-07 · Cifra la nota · B. Cifrado', () => {
  test('UT-CIF07-06 · diagnostico e indicaciones con formato iv:texto:etiqueta',
() => {
    const salida = salidaDe();
    expect(salida.diagnostico_cifrado).toMatch(FORMATO);
    expect(salida.indicaciones_cifrado).toMatch(FORMATO);
  });

  test('UT-CIF07-07 · el diagnostico se recupera al descifrar, con sus comas', ()
=> {
    expect(descifrar(salidaDe().diagnostico_cifrado)).toBe('Faringitis aguda, sin
fiebre');
  });

  test('UT-CIF07-08 · las indicaciones se recuperan al descifrar', () => {
    expect(descifrar(salidaDe().indicaciones_cifrado)).toBe('Reposo,
hidratacion');
  });

  test('UT-CIF07-09 · texto con tildes y ñ vuelve identico', () => {
    const salida = salidaDe({ ...NOTA, diagnostico: 'Otitis en niño, dolor leve'
});
    expect(descifrar(salida.diagnostico_cifrado)).toBe('Otitis en niño, dolor
leve');
```

```
  });

  test('UT-CIF07-10 · la misma nota cifrada dos veces da textos distintos', () =>
{
    expect(salidaDe().diagnostico_cifrado).not.toBe(salidaDe().diagnostico_cifrad
o);
  });

  test('UT-CIF07-11 · un texto cifrado alterado se detecta al descifrar', () => {
    const [iv, texto, etiqueta] = salidaDe().diagnostico_cifrado.split(':');
    const alterado = (texto[0] === '0' ? '1' : '0') + texto.slice(1);
    expect(() => descifrar([iv, alterado, etiqueta].join(':'))).toThrow();
  });
});

// ------------------------------------------------------------
// C. Indicaciones ausentes
// ------------------------------------------------------------
describe('WF-07 · Cifra la nota · C. Indicaciones ausentes', () => {
  test.each([
    ['UT-CIF07-12', 'nulas', null],
    ['UT-CIF07-13', 'vacias', '']
  ])('%s · indicaciones %s: centinela "null" para Guarda nota', (_, __,
indicaciones) => {
    expect(salidaDe({ ...NOTA, indicaciones
}).indicaciones_cifrado).toBe('null');
  });

  test('UT-CIF07-14 · sin diagnostico: queda nulo (Valida datos nunca lo deja
pasar)', () => {
    expect(salidaDe({ ...NOTA, diagnostico: ''
}).diagnostico_cifrado).toBeNull();
  });
});

// ------------------------------------------------------------
// D. Origen de cada campo y contrato
// ------------------------------------------------------------
describe('WF-07 · Cifra la nota · D. Origen y contrato', () => {
  test('UT-CIF07-15 · expediente, paciente y cita salen de la cita encontrada',
() => {
    const salida = salidaDe(NOTA, { id_expediente: 7, id_paciente: 8, id_cita: 9
});
    expect([salida.id_expediente, salida.id_paciente,
salida.id_cita]).toEqual([7, 8, 9]);
```

```
  });

  test('UT-CIF07-16 · el medico sale de Valida datos, es decir, del token', () =>
{
    expect(salidaDe({ ...NOTA, id_medico: 4 }).id_medico).toBe(4);
  });

  test('UT-CIF07-17 · la salida no lleva el texto clinico en claro', () => {
    const texto = JSON.stringify(cifrar().salida);
    expect(texto).not.toContain('Faringitis');
    expect(texto).not.toContain('Reposo');
  });

  test('UT-CIF07-18 · consulta Valida datos (verify)', () => {
    expect(cifrar().$).toHaveBeenCalledWith('Valida datos');
  });

  test('UT-CIF07-19 · devuelve un item con exactamente los seis campos', () => {
    const { salida } = cifrar();
    expect(salida).toHaveLength(1);
    expect(Object.keys(salida[0].json).sort()).toEqual([
      'diagnostico_cifrado', 'id_cita', 'id_expediente', 'id_medico',
'id_paciente', 'indicaciones_cifrado'
    ]);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf07.cifralanota.test.js
 PASS  tests/sigac.wf07.cifralanota.test.js
  WF-07 · Cifra la nota · A. Clave
    ✓ UT-CIF07-01 · clave valida: cifra sin error (5 ms)
    ✓ UT-CIF07-02 · clave de 62 caracteres: lanza error (17 ms)
    ✓ UT-CIF07-03 · clave de 66 caracteres: lanza error (2 ms)
    ✓ UT-CIF07-04 · clave de no hexadecimal: lanza error (1 ms)
    ✓ UT-CIF07-05 · variable de entorno sin definir: lanza error (2 ms)
  WF-07 · Cifra la nota · B. Cifrado
    ✓ UT-CIF07-06 · diagnostico e indicaciones con formato iv:texto:etiqueta (1 ms)
    ✓ UT-CIF07-07 · el diagnostico se recupera al descifrar, con sus comas (1 ms)
    ✓ UT-CIF07-08 · las indicaciones se recuperan al descifrar (1 ms)
    ✓ UT-CIF07-09 · texto con tildes y ñ vuelve identico (1 ms)
    ✓ UT-CIF07-10 · la misma nota cifrada dos veces da textos distintos (1 ms)
    ✓ UT-CIF07-11 · un texto cifrado alterado se detecta al descifrar (1 ms)
  WF-07 · Cifra la nota · C. Indicaciones ausentes
    ✓ UT-CIF07-12 · indicaciones nulas: centinela "null" para Guarda nota
    ✓ UT-CIF07-13 · indicaciones vacias: centinela "null" para Guarda nota (1 ms)
    ✓ UT-CIF07-14 · sin diagnostico: queda nulo (Valida datos nunca lo deja pasar)
  WF-07 · Cifra la nota · D. Origen y contrato
    ✓ UT-CIF07-15 · expediente, paciente y cita salen de la cita encontrada
    ✓ UT-CIF07-16 · el medico sale de Valida datos, es decir, del token
    ✓ UT-CIF07-17 · la salida no lleva el texto clinico en claro (1 ms)
    ✓ UT-CIF07-18 · consulta Valida datos (verify)
    ✓ UT-CIF07-19 · devuelve un item con exactamente los seis campos (1 ms)

Test Suites: 1 passed, 1 total
Tests:       19 passed, 19 total
Snapshots:   0 total
Time:        3.852 s
Ran all test suites matching tests/sigac.wf07.cifralanota.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.25 Nota registrada (WF-07)

El nodo Nota registrada arma la respuesta 201 con los identificadores de la nota guardada y el hash del eslabón de auditoría. El texto clínico nunca se devuelve, ni cifrado ni en claro.

1.25.1 Código base del nodo Nota registrada

```
const nota = $('Guarda nota').first().json;
const bitacora = $input.first().json;

return [{
  json: {
    codigo: 201,
    estado: 'registrada',
    mensaje: 'Nota clinica registrada',
    id_nota: nota.id,
    id_cita: nota.id_cita,
    creado_en: nota.creado_en,
    hash_auditoria: bitacora.hash_actual
  }
}];
```

1.25.2 Código de test del nodo Nota registrada

```
// sigac.wf07.notaregistrada.test.js
//
// pruebas unitarias del nodo Nota registrada de WF-07. arma la respuesta
// 201 con los identificadores de la nota y el hash de auditoria. el
// texto clinico nunca se devuelve.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-NOT07-NN

'use strict';

const notaRegistrada = require('../nodos/wf07_nota_registrada.js');
const { ejecutar } = require('../simulacion.js');

const GUARDADA = { id: 3, id_cita: 3432, creado_en: '2026-10-23T20:30:08.200Z' };
const BITACORA = { hash_actual: 'abc123' };

const responder = (guardada = GUARDADA, bitacora = BITACORA) =>
  ejecutar(notaRegistrada, { entrada: [bitacora], anteriores: { 'Guarda nota':
guardada } });

describe('WF-07 · Nota registrada', () => {
  test('UT-NOT07-01 · respuesta completa', () => {
    expect(responder().salida[0].json).toEqual({
      codigo: 201,
      estado: 'registrada',
      mensaje: 'Nota clinica registrada',
```

```
      id_nota: 3,
      id_cita: 3432,
      creado_en: '2026-10-23T20:30:08.200Z',
      hash_auditoria: 'abc123'
    });
  });

  test('UT-NOT07-02 · el id de la nota sale del id de Guarda nota', () => {
    expect(responder({ ...GUARDADA, id: 99 }).salida[0].json.id_nota).toBe(99);
  });

  test('UT-NOT07-03 · el hash sale de la bitacora ($input)', () => {
    expect(responder(GUARDADA, { hash_actual: 'f'.repeat(64)
}).salida[0].json.hash_auditoria).toBe('f'.repeat(64));
  });

  test('UT-NOT07-04 · no devuelve el texto clinico aunque venga en Guarda nota',
() => {
    const conTexto = { ...GUARDADA, diagnostico_cifrado: 'aa:bb:cc', diagnostico:
'Faringitis' };
    const texto = JSON.stringify(responder(conTexto).salida);
    expect(texto).not.toContain('Faringitis');
    expect(texto).not.toContain('aa:bb:cc');
  });

  test('UT-NOT07-05 · no devuelve el medico que firmo', () => {
    expect(responder({ ...GUARDADA, id_medico: 2
}).salida[0].json).not.toHaveProperty('id_medico');
  });

  test('UT-NOT07-06 · sin hash en la bitacora: queda indefinido, sin error', ()
=> {
    expect(responder(GUARDADA,
{}).salida[0].json.hash_auditoria).toBeUndefined();
  });

  test('UT-NOT07-07 · consulta Guarda nota (verify)', () => {
    expect(responder().$).toHaveBeenCalledWith('Guarda nota');
  });

  test('UT-NOT07-08 · devuelve una lista con un solo item de siete campos', () =>
{
    const { salida } = responder();
    expect(salida).toHaveLength(1);
    expect(Object.keys(salida[0].json)).toHaveLength(7);
```

```
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf07.notaregistrada.test.js
 PASS  tests/sigac.wf07.notaregistrada.test.js
  WF-07 · Nota registrada
    ✓ UT-NOT07-01 · respuesta completa (6 ms)
    ✓ UT-NOT07-02 · el id de la nota sale del id de Guarda nota
    ✓ UT-NOT07-03 · el hash sale de la bitacora ($input) (1 ms)
    ✓ UT-NOT07-04 · no devuelve el texto clinico aunque venga en Guarda nota (1 ms)
    ✓ UT-NOT07-05 · no devuelve el medico que firmo (4 ms)
    ✓ UT-NOT07-06 · sin hash en la bitacora: queda indefinido, sin error
    ✓ UT-NOT07-07 · consulta Guarda nota (verify)
    ✓ UT-NOT07-08 · devuelve una lista con un solo item de siete campos

Test Suites: 1 passed, 1 total
Tests:       8 passed, 8 total
Snapshots:   0 total
Time:        4.447 s
Ran all test suites matching tests/sigac.wf07.notaregistrada.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.26 Rechazo 400 (WF-07)

El nodo Rechazo 400 de WF-07 responde cuando la nota clínica no pasa la validación. Recibe la salida de Valida datos con la lista de todos los errores encontrados y la devuelve completa.

1.26.1 Código base del nodo Rechazo 400

```
return [{ json: { codigo: 400, estado: 'rechazado', mensaje: 'Datos invalidos',
errores: $input.first().json.errores } }];
```

1.26.2 Código de test del nodo Rechazo 400

```
// sigac.wf07.rechazo400.test.js
//
// pruebas unitarias del nodo Rechazo 400 de WF-07. responde cuando
// Valida datos encontro errores, y devuelve la lista completa para
// que el emisor sepa que corregir. lee los errores de $input.first().json,
// que es la salida de Valida datos por la rama false de ¿Datos validos?
//
// cada prueba lleva su ID de la tabla de pruebas: UT-R40007-NN

'use strict';

const rechazo400 = require('../nodos/wf07_rechazo_400.js');
const { ejecutar } = require('../simulacion.js');

const ERRORES = ['El DPI debe tener exactamente 13 digitos', 'El correo no tiene
un formato valido'];

// ejecuta el nodo con la salida de Valida datos, como $input y como $json
function rechazar(desdeValida) {
  return ejecutar(rechazo400, { entrada: [desdeValida], json: desdeValida });
}

describe('WF-07 · Rechazo 400', () => {
  test('UT-R40007-01 · respuesta completa', () => {
    expect(rechazar({ valido: false, errores: ERRORES
}).salida[0].json).toEqual({
      codigo: 400,
      estado: 'rechazado',
      mensaje: 'Datos invalidos',
      errores: ERRORES
    });
  });

  test('UT-R40007-02 · codigo 400', () => {
    expect(rechazar({ errores: ERRORES }).salida[0].json.codigo).toBe(400);
  });

  test('UT-R40007-03 · estado rechazado (defecto 12.17)', () => {
```

```
    expect(rechazar({ errores: ERRORES
}).salida[0].json.estado).toBe('rechazado');
  });

  test('UT-R40007-04 · mensaje Datos invalidos', () => {
    expect(rechazar({ errores: ERRORES }).salida[0].json.mensaje).toBe('Datos
invalidos');
  });

  test('UT-R40007-05 · la lista de errores llega intacta', () => {
    expect(rechazar({ errores: ERRORES
}).salida[0].json.errores).toEqual(ERRORES);
  });

  test('UT-R40007-06 · el orden de los errores se conserva', () => {
    const invertidos = [...ERRORES].reverse();
    expect(rechazar({ errores: invertidos
}).salida[0].json.errores).toEqual(invertidos);
  });

  test('UT-R40007-07 · un solo error: lista de un elemento', () => {
    expect(rechazar({ errores: ['Falta el campo nombre']
}).salida[0].json.errores)
      .toEqual(['Falta el campo nombre']);
  });

  test('UT-R40007-08 · lista vacia: errores queda vacio', () => {
    expect(rechazar({ errores: [] }).salida[0].json.errores).toEqual([]);
  });

  test('UT-R40007-09 · sin errores en la entrada: queda indefinido, sin error',
() => {
    expect(rechazar({ valido: false }).salida[0].json.errores).toBeUndefined();
  });

  test('UT-R40007-10 · con datos del paciente en la entrada: solo los cuatro
campos', () => {
    const conDatos = { errores: ERRORES, datos: { dpi: '1234567890123', correo:
'ana@correo.com' } };
    const { salida } = rechazar(conDatos);
    expect(Object.keys(salida[0].json).sort()).toEqual(['codigo', 'errores',
'estado', 'mensaje']);
    expect(JSON.stringify(salida)).not.toContain('1234567890123');
  });
```

```
  test('UT-R40007-11 · no consulta ningun nodo anterior (verify)', () => {
    const { $ } = rechazar({ errores: ERRORES });
    expect($).not.toHaveBeenCalled();
  });

  test('UT-R40007-12 · devuelve una lista con un solo item', () => {
    const { salida } = rechazar({ errores: ERRORES });
    expect(Array.isArray(salida)).toBe(true);
    expect(salida).toHaveLength(1);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf07.rechazo400.test.js
 PASS  tests/sigac.wf07.rechazo400.test.js
  WF-07 · Rechazo 400
    ✓ UT-R40007-01 · respuesta completa (6 ms)
    ✓ UT-R40007-02 · codigo 400 (1 ms)
    ✓ UT-R40007-03 · estado rechazado (defecto 12.17)
    ✓ UT-R40007-04 · mensaje Datos invalidos (2 ms)
    ✓ UT-R40007-05 · la lista de errores llega intacta
    ✓ UT-R40007-06 · el orden de los errores se conserva (1 ms)
    ✓ UT-R40007-07 · un solo error: lista de un elemento
    ✓ UT-R40007-08 · lista vacia: errores queda vacio (1 ms)
    ✓ UT-R40007-09 · sin errores en la entrada: queda indefinido, sin error
    ✓ UT-R40007-10 · con datos del paciente en la entrada: solo los cuatro campos
    ✓ UT-R40007-11 · no consulta ningun nodo anterior (verify) (1 ms)
    ✓ UT-R40007-12 · devuelve una lista con un solo item

Test Suites: 1 passed, 1 total
Tests:       12 passed, 12 total
Snapshots:   0 total
Time:        3.584 s
Ran all test suites matching tests/sigac.wf07.rechazo400.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.27 Rechazo 401 (WF-07)

El nodo Rechazo 401 de WF-07 responde cuando el token no viene, no existe o está desactivado. El mensaje es genérico a propósito, para no ayudar a quien intenta adivinar tokens válidos. Es una respuesta fija.

1.27.1 Código base del nodo Rechazo 401

```
return [{ json: { codigo: 401, estado: 'rechazado', mensaje: 'No autorizado' }
}];
```

1.27.2 Código de test del nodo Rechazo 401

```
// sigac.wf07.rechazo401.test.js
//
// pruebas unitarias del nodo Rechazo 401 de WF-07. responde cuando el token no
viene, no existe o esta desactivado. lo
// alimenta la salida false de ¿Token valido?.
//
// el mensaje es generico a proposito: no revela cual de los tres casos
// ocurrio, para no ayudar a quien intenta adivinar tokens validos.
//
// es una respuesta fija: no lee la entrada ni consulta otros nodos.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-R40107-NN

'use strict';

const rechazo = require('../nodos/wf07_rechazo_401.js');
const { ejecutar } = require('../simulacion.js');

const RESPUESTA = {
  codigo: 401,
  estado: 'rechazado',
  mensaje: 'No autorizado'
};

// lo que puede llegar al nodo: la salida del nodo anterior junto con
// lo que traia el flujo, incluida informacion que no debe devolverse
const CON_DATOS = {
  token_hash: 'a'.repeat(64),
  rol: 'medico',
  id_cita: 3432,
  datos: { dpi: '1234567890123', correo: 'ana@correo.com' }
};

describe('WF-07 · Rechazo 401', () => {
  test('UT-R40107-01 · respuesta completa', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json).toEqual(RESPUESTA);
```

```
  });

  test('UT-R40107-02 · codigo 401', () => {
    expect(ejecutar(rechazo, { entrada: [{}] }).salida[0].json.codigo).toBe(401);
  });

  test('UT-R40107-03 · estado rechazado (defecto 12.17)', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.estado).toBe('rechazado');
  });

  test('UT-R40107-04 · mensaje No autorizado', () => {
    expect(ejecutar(rechazo, { entrada: [{}] }).salida[0].json.mensaje).toBe('No
autorizado');
  });

  test('UT-R40107-05 · solo los campos codigo, estado y mensaje', () => {
    const { salida } = ejecutar(rechazo, { entrada: [{}] });
    expect(Object.keys(salida[0].json).sort()).toEqual(['codigo', 'estado',
'mensaje']);
  });

  test('UT-R40107-06 · sin entrada: la misma respuesta', () => {
    expect(ejecutar(rechazo).salida[0].json).toEqual(RESPUESTA);
  });

  test('UT-R40107-07 · con datos en la entrada: la misma respuesta, sin
devolverlos', () => {
    const { salida } = ejecutar(rechazo, { entrada: [CON_DATOS], json: CON_DATOS
});
    expect(salida[0].json).toEqual(RESPUESTA);
    const texto = JSON.stringify(salida);
    expect(texto).not.toContain('a'.repeat(64));
    expect(texto).not.toContain('1234567890123');
    expect(texto).not.toContain('3432');
  });

  test('UT-R40107-08 · no consulta ningun nodo anterior (verify)', () => {
    const { $ } = ejecutar(rechazo, { entrada: [{}] });
    expect($).not.toHaveBeenCalled();
  });

  test('UT-R40107-09 · dos ejecuciones con entradas distintas dan respuestas
identicas', () => {
```

```
    const a = ejecutar(rechazo, { entrada: [{ rol: 'recepcion' }], json: { rol:
'recepcion' } }).salida;
    const b = ejecutar(rechazo, { entrada: [CON_DATOS], json: CON_DATOS
}).salida;
    expect(a).toEqual(b);
  });

  test('UT-R40107-10 · el mensaje no revela la causa del rechazo', () => {
    const mensaje = ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.mensaje.toLowerCase();
    for (const pista of ['inactivo', 'desactivado', 'expirado', 'rol']) {
      expect(mensaje).not.toContain(pista);
    }
  });

  test('UT-R40107-11 · devuelve una lista con un solo item', () => {
    const { salida } = ejecutar(rechazo, { entrada: [{}] });
    expect(Array.isArray(salida)).toBe(true);
    expect(salida).toHaveLength(1);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf07.rechazo401.test.js
 PASS  tests/sigac.wf07.rechazo401.test.js
  WF-07 · Rechazo 401
    ✓ UT-R40107-01 · respuesta completa (5 ms)
    ✓ UT-R40107-02 · codigo 401 (2 ms)
    ✓ UT-R40107-03 · estado rechazado (defecto 12.17)
    ✓ UT-R40107-04 · mensaje No autorizado (1 ms)
    ✓ UT-R40107-05 · solo los campos codigo, estado y mensaje
    ✓ UT-R40107-06 · sin entrada: la misma respuesta
    ✓ UT-R40107-07 · con datos en la entrada: la misma respuesta, sin devolverlos (1 ms)
    ✓ UT-R40107-08 · no consulta ningun nodo anterior (verify)
    ✓ UT-R40107-09 · dos ejecuciones con entradas distintas dan respuestas identicas
    ✓ UT-R40107-10 · el mensaje no revela la causa del rechazo
    ✓ UT-R40107-11 · devuelve una lista con un solo item (1 ms)

Test Suites: 1 passed, 1 total
Tests:       11 passed, 11 total
Snapshots:   0 total
Time:        3.877 s
Ran all test suites matching tests/sigac.wf07.rechazo401.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ ~
```

1.28 Rechazo 403 (WF-07)

El nodo Rechazo 403 de WF-07 responde cuando el token es válido pero no es de un médico: solo un médico puede escribir notas clínicas. El mensaje no revela qué rol se necesita ni cuál tiene quien llama. Es una respuesta fija.

1.28.1 Código base del nodo Rechazo 403

```
return [{ json: { codigo: 403, estado: 'rechazado', mensaje: 'Operacion no
permitida para este rol' } }];
```

1.28.2 Código de test del nodo Rechazo 403

```
// sigac.wf07.rechazo403.test.js
//
// pruebas unitarias del nodo Rechazo 403 de WF-07. responde cuando el token es
valido pero no es de un medico: solo un
// medico puede escribir notas clinicas. lo alimenta la salida false
// de ¿Es medico?.
//
// el mensaje no revela que rol se necesita ni cual tiene quien llama.
//
// es una respuesta fija: no lee la entrada ni consulta otros nodos.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-R40307-NN

'use strict';

const rechazo = require('../nodos/wf07_rechazo_403.js');
const { ejecutar } = require('../simulacion.js');

const RESPUESTA = {
  codigo: 403,
  estado: 'rechazado',
  mensaje: 'Operacion no permitida para este rol'
};

// lo que puede llegar al nodo: la salida del nodo anterior junto con
// lo que traia el flujo, incluida informacion que no debe devolverse
const CON_DATOS = {
  token_hash: 'a'.repeat(64),
  rol: 'medico',
  id_cita: 3432,
  datos: { dpi: '1234567890123', correo: 'ana@correo.com' }
};

describe('WF-07 · Rechazo 403', () => {
  test('UT-R40307-01 · respuesta completa', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json).toEqual(RESPUESTA);
```

```
  });

  test('UT-R40307-02 · codigo 403', () => {
    expect(ejecutar(rechazo, { entrada: [{}] }).salida[0].json.codigo).toBe(403);
  });

  test('UT-R40307-03 · estado rechazado (defecto 12.17)', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.estado).toBe('rechazado');
  });

  test('UT-R40307-04 · mensaje Operacion no permitida para este rol', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.mensaje).toBe('Operacion no permitida para este rol');
  });

  test('UT-R40307-05 · solo los campos codigo, estado y mensaje', () => {
    const { salida } = ejecutar(rechazo, { entrada: [{}] });
    expect(Object.keys(salida[0].json).sort()).toEqual(['codigo', 'estado',
'mensaje']);
  });

  test('UT-R40307-06 · sin entrada: la misma respuesta', () => {
    expect(ejecutar(rechazo).salida[0].json).toEqual(RESPUESTA);
  });

  test('UT-R40307-07 · con datos en la entrada: la misma respuesta, sin
devolverlos', () => {
    const { salida } = ejecutar(rechazo, { entrada: [CON_DATOS], json: CON_DATOS
});
    expect(salida[0].json).toEqual(RESPUESTA);
    const texto = JSON.stringify(salida);
    expect(texto).not.toContain('a'.repeat(64));
    expect(texto).not.toContain('1234567890123');
    expect(texto).not.toContain('3432');
  });

  test('UT-R40307-08 · no consulta ningun nodo anterior (verify)', () => {
    const { $ } = ejecutar(rechazo, { entrada: [{}] });
    expect($).not.toHaveBeenCalled();
  });

  test('UT-R40307-09 · dos ejecuciones con entradas distintas dan respuestas
identicas', () => {
```

```
    const a = ejecutar(rechazo, { entrada: [{ rol: 'recepcion' }], json: { rol:
'recepcion' } }).salida;
    const b = ejecutar(rechazo, { entrada: [CON_DATOS], json: CON_DATOS
}).salida;
    expect(a).toEqual(b);
  });

  test('UT-R40307-10 · el mensaje no revela roles', () => {
    const mensaje = ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.mensaje.toLowerCase();
    for (const pista of ['recepcion', 'medico', 'admin']) {
      expect(mensaje).not.toContain(pista);
    }
  });

  test('UT-R40307-11 · devuelve una lista con un solo item', () => {
    const { salida } = ejecutar(rechazo, { entrada: [{}] });
    expect(Array.isArray(salida)).toBe(true);
    expect(salida).toHaveLength(1);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf07.rechazo403.test.js
 PASS  tests/sigac.wf07.rechazo403.test.js
  WF-07 · Rechazo 403
    ✓ UT-R40307-01 · respuesta completa (7 ms)
    ✓ UT-R40307-02 · codigo 403 (1 ms)
    ✓ UT-R40307-03 · estado rechazado (defecto 12.17)
    ✓ UT-R40307-04 · mensaje Operacion no permitida para este rol
    ✓ UT-R40307-05 · solo los campos codigo, estado y mensaje
    ✓ UT-R40307-06 · sin entrada: la misma respuesta
    ✓ UT-R40307-07 · con datos en la entrada: la misma respuesta, sin devolverlos
    ✓ UT-R40307-08 · no consulta ningun nodo anterior (verify)
    ✓ UT-R40307-09 · dos ejecuciones con entradas distintas dan respuestas identicas (1 ms)
    ✓ UT-R40307-10 · el mensaje no revela roles (1 ms)
    ✓ UT-R40307-11 · devuelve una lista con un solo item (2 ms)

Test Suites: 1 passed, 1 total
Tests:       11 passed, 11 total
Snapshots:   0 total
Time:        3.67 s
Ran all test suites matching tests/sigac.wf07.rechazo403.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.29 Rechazo 404 (WF-07)

El nodo Rechazo 404 de WF-07 responde cuando el nodo Busca la cita no encontró una cita del médico, con expediente y ya atendida o vencida. Usa el mismo mensaje para una cita inexistente y para una cita de otro médico: no revela si la cita existe pero pertenece a otro. Es una respuesta fija.

1.29.1 Código base del nodo Rechazo 404

```
return [{ json: { codigo: 404, estado: 'rechazado', mensaje: 'Cita no encontrada
o no admite nota' } }];
```

1.29.2 Código de test del nodo Rechazo 404

```
// sigac.wf07.rechazo404.test.js
//
// pruebas unitarias del nodo Rechazo 404 de WF-07. responde cuando Busca la cita
no encontro una cita del medico, con
// expediente, atendida o ya vencida. lo alimenta la salida false de
// ¿Cita valida?.
//
// el mismo mensaje para una cita inexistente y para una cita de otro
// medico: no revela si la cita existe pero pertenece a otro.
//
// es una respuesta fija: no lee la entrada ni consulta otros nodos.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-R40407-NN

'use strict';

const rechazo = require('../nodos/wf07_rechazo_404.js');
const { ejecutar } = require('../simulacion.js');

const RESPUESTA = {
  codigo: 404,
  estado: 'rechazado',
  mensaje: 'Cita no encontrada o no admite nota'
};

// lo que puede llegar al nodo: la salida del nodo anterior junto con
// lo que traia el flujo, incluida informacion que no debe devolverse
const CON_DATOS = {
  token_hash: 'a'.repeat(64),
  rol: 'medico',
  id_cita: 3432,
  datos: { dpi: '1234567890123', correo: 'ana@correo.com' }
};

describe('WF-07 · Rechazo 404', () => {
  test('UT-R40407-01 · respuesta completa', () => {
```

```
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json).toEqual(RESPUESTA);
  });

  test('UT-R40407-02 · codigo 404', () => {
    expect(ejecutar(rechazo, { entrada: [{}] }).salida[0].json.codigo).toBe(404);
  });

  test('UT-R40407-03 · estado rechazado (defecto 12.17)', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.estado).toBe('rechazado');
  });

  test('UT-R40407-04 · mensaje Cita no encontrada o no admite nota', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.mensaje).toBe('Cita no encontrada o no admite nota');
  });

  test('UT-R40407-05 · solo los campos codigo, estado y mensaje', () => {
    const { salida } = ejecutar(rechazo, { entrada: [{}] });
    expect(Object.keys(salida[0].json).sort()).toEqual(['codigo', 'estado',
'mensaje']);
  });

  test('UT-R40407-06 · sin entrada: la misma respuesta', () => {
    expect(ejecutar(rechazo).salida[0].json).toEqual(RESPUESTA);
  });

  test('UT-R40407-07 · con datos en la entrada: la misma respuesta, sin
devolverlos', () => {
    const { salida } = ejecutar(rechazo, { entrada: [CON_DATOS], json: CON_DATOS
});
    expect(salida[0].json).toEqual(RESPUESTA);
    const texto = JSON.stringify(salida);
    expect(texto).not.toContain('a'.repeat(64));
    expect(texto).not.toContain('1234567890123');
    expect(texto).not.toContain('3432');
  });

  test('UT-R40407-08 · no consulta ningun nodo anterior (verify)', () => {
    const { $ } = ejecutar(rechazo, { entrada: [{}] });
    expect($).not.toHaveBeenCalled();
  });
```

```
  test('UT-R40407-09 · dos ejecuciones con entradas distintas dan respuestas
identicas', () => {
    const a = ejecutar(rechazo, { entrada: [{ rol: 'recepcion' }], json: { rol:
'recepcion' } }).salida;
    const b = ejecutar(rechazo, { entrada: [CON_DATOS], json: CON_DATOS
}).salida;
    expect(a).toEqual(b);
  });

  test('UT-R40407-10 · el mensaje no revela si la cita es de otro medico', () =>
{
    const mensaje = ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.mensaje.toLowerCase();
    for (const pista of ['otro medico', 'ajena', 'pertenece']) {
      expect(mensaje).not.toContain(pista);
    }
  });

  test('UT-R40407-11 · devuelve una lista con un solo item', () => {
    const { salida } = ejecutar(rechazo, { entrada: [{}] });
    expect(Array.isArray(salida)).toBe(true);
    expect(salida).toHaveLength(1);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf07.rechazo404.test.js
 PASS  tests/sigac.wf07.rechazo404.test.js
  WF-07 · Rechazo 404
    ✓ UT-R40407-01 · respuesta completa (7 ms)
    ✓ UT-R40407-02 · codigo 404
    ✓ UT-R40407-03 · estado rechazado (defecto 12.17) (6 ms)
    ✓ UT-R40407-04 · mensaje Cita no encontrada o no admite nota
    ✓ UT-R40407-05 · solo los campos codigo, estado y mensaje (1 ms)
    ✓ UT-R40407-06 · sin entrada: la misma respuesta (1 ms)
    ✓ UT-R40407-07 · con datos en la entrada: la misma respuesta, sin devolverlos (2 ms)
    ✓ UT-R40407-08 · no consulta ningun nodo anterior (verify) (1 ms)
    ✓ UT-R40407-09 · dos ejecuciones con entradas distintas dan respuestas identicas (3 ms)
    ✓ UT-R40407-10 · el mensaje no revela si la cita es de otro medico
    ✓ UT-R40407-11 · devuelve una lista con un solo item

Test Suites: 1 passed, 1 total
Tests:       11 passed, 11 total
Snapshots:   0 total
Time:        4.078 s
Ran all test suites matching tests/sigac.wf07.rechazo404.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.30 Hash del token (WF-08)

El nodo Hash del token de WF-08 lee el token de la cabecera x-sigac-token y calcula su huella SHA-256, que es la que se compara contra la base de datos. El token nunca se compara ni se guarda en claro. Sin token, usa el centinela sin-token.

Es idéntico al de WF-01, WF-07 y WF-09.

1.30.1 Código base del nodo Hash del token

```
const crypto = require('crypto');

const cabeceras = $input.first().json.headers || {};
const cuerpo    = $input.first().json.body    || {};

const token = cabeceras['x-sigac-token'] || '';
const tokenHash = token
  ? crypto.createHash('sha256').update(token).digest('hex')
  : 'sin-token';

return [{
  json: {
    token_hash: tokenHash,
    tiene_token: token.length > 0,
    datos: cuerpo
  }
}];
```

1.30.2 Código de test del nodo Hash del token

```
// sigac.wf08.hashdeltoken.test.js
//
// pruebas unitarias del nodo Hash del token de WF-08. lee el token de
// la cabecera x-sigac-token y calcula su huella sha-256, que es la que
// se compara contra la base. el token nunca se compara ni se guarda en
// claro. sin token, usa el centinela sin-token, que nunca puede
// coincidir con una huella real.
//
// tecnicas: particion de equivalencia, valores limite, cobertura de
// ramas (headers || {}, body || {}, token ? hash : centinela),
// propiedades del hash y contrato de salida.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-HASH08-NN

'use strict';

const hashDelToken = require('../nodos/wf08_hash_del_token.js');
const { ejecutar, sha256 } = require('../simulacion.js');

// arma la peticion que entrega el Webhook y ejecuta el nodo
const hash = peticion => ejecutar(hashDelToken, { entrada: [peticion] }).salida;
```

```
describe('WF-08 · Hash del token', () => {
  test('UT-HASH08-01 · con token: calcula su huella sha-256', () => {
    const salida = hash({ headers: { 'x-sigac-token': 'abc' }, body: {} });
    expect(salida[0].json.token_hash).toBe(sha256('abc'));
    expect(salida[0].json.tiene_token).toBe(true);
  });

  test('UT-HASH08-02 · sin cabecera de token: centinela sin-token', () => {
    const salida = hash({ headers: {}, body: {} });
    expect(salida[0].json.token_hash).toBe('sin-token');
    expect(salida[0].json.tiene_token).toBe(false);
  });

  test('UT-HASH08-03 · token vacio: centinela sin-token', () => {
    const salida = hash({ headers: { 'x-sigac-token': '' }, body: {} });
    expect(salida[0].json.token_hash).toBe('sin-token');
    expect(salida[0].json.tiene_token).toBe(false);
  });

  test('UT-HASH08-04 · peticion sin headers: no falla y usa el centinela', () =>
{
    const salida = hash({ body: {} });
    expect(salida[0].json.token_hash).toBe('sin-token');
  });

  test('UT-HASH08-05 · peticion sin body: datos queda como objeto vacio', () => {
    const salida = hash({ headers: { 'x-sigac-token': 'abc' } });
    expect(salida[0].json.datos).toEqual({});
  });

  test('UT-HASH08-06 · el cuerpo pasa intacto en datos', () => {
    const cuerpo = { dpi: '1234567890123', nombre: 'Ana', anidado: { a: 1 } };
    const salida = hash({ headers: {}, body: cuerpo });
    expect(salida[0].json.datos).toEqual(cuerpo);
  });

  test('UT-HASH08-07 · cabecera en mayusculas: no se lee (n8n las entrega en
minusculas)', () => {
    const salida = hash({ headers: { 'X-SIGAC-TOKEN': 'abc' }, body: {} });
    expect(salida[0].json.token_hash).toBe('sin-token');
  });

  test('UT-HASH08-08 · token con espacios: se usa tal cual, sin recortar', () =>
{
```

```
    const salida = hash({ headers: { 'x-sigac-token': ' abc ' }, body: {} });
    expect(salida[0].json.token_hash).toBe(sha256(' abc '));
    expect(salida[0].json.token_hash).not.toBe(sha256('abc'));
  });

  test('UT-HASH08-09 · el mismo token da siempre la misma huella', () => {
    const a = hash({ headers: { 'x-sigac-token': 'abc' }, body: {} });
    const b = hash({ headers: { 'x-sigac-token': 'abc' }, body: {} });
    expect(a[0].json.token_hash).toBe(b[0].json.token_hash);
  });

  test('UT-HASH08-10 · tokens distintos dan huellas distintas', () => {
    const a = hash({ headers: { 'x-sigac-token': 'abc' }, body: {} });
    const b = hash({ headers: { 'x-sigac-token': 'abd' }, body: {} });
    expect(a[0].json.token_hash).not.toBe(b[0].json.token_hash);
  });

  test('UT-HASH08-11 · la huella tiene 64 caracteres hexadecimales en minuscula',
() => {
    const salida = hash({ headers: { 'x-sigac-token': 'abc' }, body: {} });
    expect(salida[0].json.token_hash).toMatch(/^[0-9a-f]{64}$/);
  });

  test('UT-HASH08-12 · el token en claro no aparece en la salida', () => {
    // el token lleva guiones, que nunca aparecen en un hexadecimal:
    // si se encuentra en la salida, es porque el token salio en claro
    const token = 'token-secreto-de-prueba';
    const salida = hash({ headers: { 'x-sigac-token': token }, body: {} });
    expect(JSON.stringify(salida)).not.toContain(token);
  });

  test('UT-HASH08-13 · devuelve un solo item con exactamente tres campos', () =>
{
    const salida = hash({ headers: { 'x-sigac-token': 'abc' }, body: {} });
    expect(salida).toHaveLength(1);
    expect(Object.keys(salida[0].json).sort()).toEqual(['datos', 'tiene_token',
'token_hash']);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf08.hashdeltoken.test.js
 PASS  tests/sigac.wf08.hashdeltoken.test.js
  WF-08 · Hash del token
    ✓ UT-HASH08-01 · con token: calcula su huella sha-256 (5 ms)
    ✓ UT-HASH08-02 · sin cabecera de token: centinela sin-token
    ✓ UT-HASH08-03 · token vacio: centinela sin-token
    ✓ UT-HASH08-04 · peticion sin headers: no falla y usa el centinela (1 ms)
    ✓ UT-HASH08-05 · peticion sin body: datos queda como objeto vacio (1 ms)
    ✓ UT-HASH08-06 · el cuerpo pasa intacto en datos
    ✓ UT-HASH08-07 · cabecera en mayusculas: no se lee (n8n las entrega en minusculas)
    ✓ UT-HASH08-08 · token con espacios: se usa tal cual, sin recortar
    ✓ UT-HASH08-09 · el mismo token da siempre la misma huella
    ✓ UT-HASH08-10 · tokens distintos dan huellas distintas (1 ms)
    ✓ UT-HASH08-11 · la huella tiene 64 caracteres hexadecimales en minuscula (1 ms)
    ✓ UT-HASH08-12 · el token en claro no aparece en la salida (1 ms)
    ✓ UT-HASH08-13 · devuelve un solo item con exactamente tres campos (1 ms)

  Test Suites: 1 passed, 1 total
  Tests:       13 passed, 13 total
  Snapshots:   0 total
  Time:        3.8 s
  Ran all test suites matching tests/sigac.wf08.hashdeltoken.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.31 Valida datos (WF-08)

El nodo Valida datos de WF-08 revisa el cuerpo de la consulta de historial. El médico solo envía el id de la cita: el paciente se deduce de la cita, nunca se recibe directamente, y el médico sale del token verificado. Así un médico solo puede consultar el historial de pacientes con quienes tiene o tuvo una cita.

1.31.1 Código base del nodo Valida datos

```
const cuerpo = $('Hash del token').first().json.datos || {};
const medico = $('Verifica token').first().json;
const errores = [];

const idCita = Number(cuerpo.id_cita);
if (!Number.isInteger(idCita) || idCita <= 0) {
  errores.push('id_cita debe ser un entero positivo');
}

return [{
  json: {
    valido: errores.length === 0,
    errores,
    id_cita: idCita,
    id_medico: medico.id_medico
  }
}];
```

1.31.2 Código de test del nodo Valida datos

```
// sigac.wf08.validadatos.test.js
//
// pruebas unitarias del nodo Valida datos de WF-08. el medico solo envia
// el id de la cita: el paciente se deduce de la cita, nunca se recibe
// directamente. el medico sale de Verifica token.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-VAL08-NN

'use strict';

const validaDatos = require('../nodos/wf08_valida_datos.js');
const { ejecutar } = require('../simulacion.js');

function validar(cuerpo, medico = { rol: 'medico', id_medico: 1 }, hash = {
datos: cuerpo }) {
  return ejecutar(validaDatos, { anteriores: { 'Hash del token': hash, 'Verifica
token': medico } });
}

const salidaDe = cuerpo => validar(cuerpo).salida[0].json;
```

```
describe('WF-08 · Valida datos', () => {
  test.each([
    ['UT-VAL08-01', 'numero', 3432, 3432],
    ['UT-VAL08-02', 'texto numerico', '3432', 3432]
  ])('%s · id_cita como %s: valido', (_, __, id_cita, esperado) => {
    const salida = salidaDe({ id_cita });
    expect(salida.valido).toBe(true);
    expect(salida.id_cita).toBe(esperado);
  });

  test.each([
    ['UT-VAL08-03', 'cero', 0],
    ['UT-VAL08-04', 'negativo', -1],
    ['UT-VAL08-05', 'decimal', 2.5],
    ['UT-VAL08-06', 'texto', 'abc'],
    ['UT-VAL08-07', 'ausente', undefined]
  ])('%s · id_cita %s: error', (_, __, id_cita) => {
    expect(salidaDe({ id_cita }).errores).toEqual(['id_cita debe ser un entero
positivo']);
  });

  // hipotesis A: Number(true) es 1
  test('UT-VAL08-08 · id_cita true: debe rechazarse (hipotesis A)', () => {
    expect(salidaDe({ id_cita: true }).valido).toBe(false);
  });

  test('UT-VAL08-09 · un id_paciente en el cuerpo se ignora: el paciente sale de
la cita', () => {
    const salida = salidaDe({ id_cita: 3432, id_paciente: 55 });
    expect(salida).not.toHaveProperty('id_paciente');
  });

  test('UT-VAL08-10 · el medico sale de Verifica token', () => {
    expect(validar({ id_cita: 1, id_medico: 9 }, { rol: 'medico', id_medico: 2
}).salida[0].json.id_medico).toBe(2);
  });

  test('UT-VAL08-11 · Hash del token sin datos: no falla y da el error de
id_cita', () => {
    expect(validar(undefined, undefined, {}).salida[0].json.valido).toBe(false);
  });

  test('UT-VAL08-12 · consulta Hash del token y Verifica token (verify)', () => {
    const { $ } = validar({ id_cita: 1 });
    expect($).toHaveBeenCalledWith('Hash del token');
```

```
    expect($).toHaveBeenCalledWith('Verifica token');
  });

  test('UT-VAL08-13 · devuelve un item con los cuatro campos', () => {
    const { salida } = validar({ id_cita: 1 });
    expect(salida).toHaveLength(1);
    expect(Object.keys(salida[0].json).sort()).toEqual(['errores', 'id_cita',
'id_medico', 'valido']);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf08.validadatos.test.js
 FAIL  tests/sigac.wf08.validadatos.test.js
  WF-08 · Valida datos
    ✓ UT-VAL08-01 · id_cita como numero: valido (7 ms)
    ✓ UT-VAL08-02 · id_cita como texto numerico: valido (1 ms)
    ✓ UT-VAL08-03 · id_cita cero: error (1 ms)
    ✓ UT-VAL08-04 · id_cita negativo: error (1 ms)
    ✓ UT-VAL08-05 · id_cita decimal: error (1 ms)
    ✓ UT-VAL08-06 · id_cita texto: error (1 ms)
    ✓ UT-VAL08-07 · id_cita ausente: error (1 ms)
    ✕ UT-VAL08-08 · id_cita true: debe rechazarse (hipotesis A) (2 ms)
    ✓ UT-VAL08-09 · un id_paciente en el cuerpo se ignora: el paciente sale de la cita (1 ms)
    ✓ UT-VAL08-10 · el medico sale de Verifica token (2 ms)
    ✓ UT-VAL08-11 · Hash del token sin datos: no falla y da el error de id_cita (1 ms)
    ✓ UT-VAL08-12 · consulta Hash del token y Verifica token (verify) (1 ms)
    ✓ UT-VAL08-13 · devuelve un item con los cuatro campos

  ● WF-08 · Valida datos › UT-VAL08-08 · id_cita true: debe rechazarse (hipotesis A)

    expect(received).toBe(expected) // Object.is equality

    Expected: false
    Received: true

      40 |   // hipotesis A: Number(true) es 1
      41 |   test('UT-VAL08-08 · id_cita true: debe rechazarse (hipotesis A)', () => {
    > 42 |     expect(salidaDe({ id_cita: true }).valido).toBe(false);
         |                                                ^
      43 |   });
      44 |
      45 |   test('UT-VAL08-09 · un id_paciente en el cuerpo se ignora: el paciente sale de la cita', () => {

      at Object.toBe (tests/sigac.wf08.validadatos.test.js:42:48)

Test Suites: 1 failed, 1 total
Tests:       1 failed, 12 passed, 13 total
Snapshots:   0 total
Time:        3.781 s
Ran all test suites matching tests/sigac.wf08.validadatos.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.31.2.1 Defecto detectado por las pruebas (12.24).

La prueba UT-VAL08-08 falló: el nodo convierte el identificador con Number(), y Number(true) es 1, de modo que un true en lugar de un número se acepta como el identificador 1. Es el mismo defecto que el 12.23 de WF-01. Se corrige en n8n y, tras exportar el flujo y regenerar los nodos, la misma prueba confirma la corrección.

1.31.3 Nuevo código base de Valida Datos.

```
const cuerpo = $('Hash del token').first().json.datos || {};
const medico = $('Verifica token').first().json;

const errores = [];

// defecto 12.24: antes se convertia con Number(), y Number(true) es 1,
// de modo que "id_cita": true se aceptaba como la cita 1. ahora solo se
// acepta un numero o un texto que contenga un entero positivo, igual
// que id_medico_solicitado en WF-01 (defecto 12.23).
const valorCita = (typeof cuerpo.id_cita === 'number' || typeof cuerpo.id_cita
=== 'string')
  ? String(cuerpo.id_cita).trim()
  : '';
const idCita = /^[1-9]\d*$/.test(valorCita) ? Number(valorCita) : null;
if (idCita === null) {
  errores.push('id_cita debe ser un entero positivo');
}
return [{
  json: {
    valido: errores.length === 0,
    errores,
    id_cita: idCita,
    id_medico: medico.id_medico
  }
}];
```

1.31.4 Se corrió el mismo código test pero ya con la corrección del código base.

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf08.validadatos.test.js
 PASS  tests/sigac.wf08.validadatos.test.js
  WF-08 · Valida datos
    ✓ UT-VAL08-01 · id_cita como numero: valido (4 ms)
    ✓ UT-VAL08-02 · id_cita como texto numerico: valido (1 ms)
    ✓ UT-VAL08-03 · id_cita cero: error (1 ms)
    ✓ UT-VAL08-04 · id_cita negativo: error (1 ms)
    ✓ UT-VAL08-05 · id_cita decimal: error
    ✓ UT-VAL08-06 · id_cita texto: error
    ✓ UT-VAL08-07 · id_cita ausente: error
    ✓ UT-VAL08-08 · id_cita true: debe rechazarse (hipotesis A)
    ✓ UT-VAL08-09 · un id_paciente en el cuerpo se ignora: el paciente sale de la cita (5 ms)
    ✓ UT-VAL08-10 · el medico sale de Verifica token (1 ms)
    ✓ UT-VAL08-11 · Hash del token sin datos: no falla y da el error de id_cita
    ✓ UT-VAL08-12 · consulta Hash del token y Verifica token (verify)
    ✓ UT-VAL08-13 · devuelve un item con los cuatro campos (1 ms)

Test Suites: 1 passed, 1 total
Tests:       13 passed, 13 total
Snapshots:   0 total
Time:        3.916 s
Ran all test suites matching tests/sigac.wf08.validadatos.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.32 Descifra (WF-08)

El nodo Descifra descifra el historial clínico del paciente: es la función inversa de la que cifra en WF-01 y WF-07. Además verifica la integridad: si un valor cifrado se alteró en la base, AES-GCM lo detecta y el nodo lanza un error, que registra WF-00.

Junta todas las notas en un solo item, en orden cronológico y con la fecha en hora de Guatemala, para que la bitácora registre una sola lectura. Las pruebas cifran las notas con el propio nodo Cifra la nota de WF-07, de modo que también verifican que los dos nodos usan el mismo formato.

1.32.1 Código base del nodo Descifra

```
const crypto = require('crypto');

const clave = Buffer.from($env.SIGAC_CLAVE_CIFRADO, 'hex');
if (clave.length !== 32) {
  throw new Error('SIGAC_CLAVE_CIFRADO debe ser de 32 bytes (64 caracteres
hex)');
}

function descifrar(valor) {
  if (!valor) return null;
  const partes = String(valor).split(':');
  if (partes.length !== 3) {
    throw new Error('formato de cifrado no reconocido');
  }
  const [ivHex, datosHex, etiquetaHex] = partes;
  const descifrador = crypto.createDecipheriv('aes-256-gcm', clave,
Buffer.from(ivHex, 'hex'));
  descifrador.setAuthTag(Buffer.from(etiquetaHex, 'hex'));
  return Buffer.concat([
    descifrador.update(Buffer.from(datosHex, 'hex')),
    descifrador.final()
  ]).toString('utf8');
}

const filas = $input.all().map(i => i.json).filter(f => f.id);

const notas = filas
  .map(f => ({
    id_nota: f.id,
    fecha: DateTime.fromJSDate(new Date(f.creado_en))
                   .setZone('America/Guatemala')
                   .toFormat("dd/MM/yyyy HH:mm"),
    medico: f.medico,
    especialidad: f.especialidad,
    diagnostico: descifrar(f.diagnostico_cifrado),
```

```
    indicaciones: descifrar(f.indicaciones_cifrado)
  }))
  .reverse();

const cita = $('Busca la cita').first().json;
const medico = $('Valida datos').first().json;

return [{
  json: {
    id_paciente: cita.id_paciente,
    id_cita: cita.id_cita,
    id_medico: medico.id_medico,
    total: notas.length,
    notas
  }
}];
```

1.32.2 Código de test del nodo Descifra

```
// sigac.wf08.descifra.test.js
//
// pruebas unitarias del nodo Descifra de WF-08. descifra el historial
// clinico del paciente: es la funcion inversa de la que cifra en WF-01 y
// WF-07. ademas verifica la integridad: si un valor cifrado se altero
// en la base, AES-GCM lo detecta y el nodo lanza un error.
//
// junta todas las notas en un solo item, en orden cronologico, con la
// fecha en hora de Guatemala.
//
// las pruebas cifran las notas con el propio nodo Cifra la nota de
// WF-07, de modo que tambien verifican que los dos nodos hablan el
// mismo formato.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-DES08-NN

'use strict';

const descifra = require('../nodos/wf08_descifra.js');
const cifraNota = require('../nodos/wf07_cifra_la_nota.js');
const cifraDatos = require('../nodos/wf01_cifra_datos_sensibles.js');
const { ejecutar, CLAVE_PRUEBA } = require('../simulacion.js');

// cifra un texto con el nodo de WF-07
function cifrarConWF07(texto) {
  return ejecutar(cifraNota, {
    entrada: [{ id_expediente: 1, id_paciente: 1, id_cita: 1 }],
```

```
    anteriores: { 'Valida datos': { id_medico: 1, diagnostico: texto,
indicaciones: null } }
  }).salida[0].json.diagnostico_cifrado;
}

// una fila de Lee notas, tal como la devuelve la base
const fila = (id, creado_en, diagnostico, indicaciones = null) => ({
  id,
  creado_en,
  medico: 'Carlos Enrique Coc Ical',
  especialidad: 'Pediatria',
  diagnostico_cifrado: cifrarConWF07(diagnostico),
  indicaciones_cifrado: indicaciones === null ? null :
cifrarConWF07(indicaciones)
});

function leer(filas, clave = CLAVE_PRUEBA) {
  return ejecutar(descifra, {
    entrada: filas,
    anteriores: {
      'Busca la cita': { id_paciente: 55, id_cita: 3432, id_expediente: 40 },
      'Valida datos': { id_medico: 2 }
    },
    env: { SIGAC_CLAVE_CIFRADO: clave }
  });
}

const salidaDe = (...filas) => leer(filas).salida[0].json;

// ------------------------------------------------------------
// A. Descifrado
// ------------------------------------------------------------
describe('WF-08 · Descifra · A. Descifrado', () => {
  test('UT-DES08-01 · descifra lo que cifro WF-07: vuelve el texto original', ()
=> {
    const salida = salidaDe(fila(3, '2026-10-23T20:30:08Z', 'Faringitis aguda,
sin fiebre', 'Reposo, hidratacion'));
    expect(salida.notas[0].diagnostico).toBe('Faringitis aguda, sin fiebre');
    expect(salida.notas[0].indicaciones).toBe('Reposo, hidratacion');
  });

  test('UT-DES08-02 · descifra tambien el formato de WF-01: es el mismo cifrado',
() => {
    const dpiCifrado = ejecutar(cifraDatos, { json: { datos: { dpi:
'1234567890123' } } }).salida.json.dpi_cifrado;
```

```
    const salida = salidaDe({ ...fila(1, '2026-10-23T20:30:08Z', 'x'),
diagnostico_cifrado: dpiCifrado });
    expect(salida.notas[0].diagnostico).toBe('1234567890123');
  });

  test('UT-DES08-03 · indicaciones nulas: quedan nulas', () => {
    expect(salidaDe(fila(3, '2026-10-23T20:30:08Z',
'Control')).notas[0].indicaciones).toBeNull();
  });

  test('UT-DES08-04 · texto con tildes y ñ vuelve identico', () => {
    expect(salidaDe(fila(3, '2026-10-23T20:30:08Z', 'Otitis en
niño')).notas[0].diagnostico).toBe('Otitis en niño');
  });
});

// ------------------------------------------------------------
// B. Integridad y clave
// ------------------------------------------------------------
describe('WF-08 · Descifra · B. Integridad', () => {
  test('UT-DES08-05 · un valor alterado en la base: lanza error (lo registra WF-
00)', () => {
    const f = fila(3, '2026-10-23T20:30:08Z', 'Faringitis');
    const [iv, texto, etiqueta] = f.diagnostico_cifrado.split(':');
    f.diagnostico_cifrado = [iv, (texto[0] === '0' ? '1' : '0') + texto.slice(1),
etiqueta].join(':');
    expect(() => leer([f])).toThrow();
  });

  test('UT-DES08-06 · un valor sin las tres partes: formato no reconocido', () =>
{
    const f = { ...fila(3, '2026-10-23T20:30:08Z', 'x'), diagnostico_cifrado:
'aa:bb' };
    expect(() => leer([f])).toThrow('formato de cifrado no reconocido');
  });

  test('UT-DES08-07 · un hash antiguo de 64 caracteres (sin cifrar): formato no
reconocido', () => {
    const f = { ...fila(3, '2026-10-23T20:30:08Z', 'x'), diagnostico_cifrado:
'a'.repeat(64) };
    expect(() => leer([f])).toThrow('formato de cifrado no reconocido');
  });

  test('UT-DES08-08 · con otra clave: no se puede descifrar', () => {
```

```
    expect(() => leer([fila(3, '2026-10-23T20:30:08Z', 'x')],
'ff'.repeat(32))).toThrow();
  });

  test('UT-DES08-09 · clave de largo incorrecto: lanza error', () => {
    expect(() => leer([fila(3, '2026-10-23T20:30:08Z', 'x')],
'abcd')).toThrow('SIGAC_CLAVE_CIFRADO debe ser de 32 bytes');
  });
});

// ------------------------------------------------------------
// C. Orden, fecha y paciente sin notas
// ------------------------------------------------------------
describe('WF-08 · Descifra · C. Orden y fecha', () => {
  test('UT-DES08-10 · la base las trae de la mas reciente a la mas antigua; salen
en orden cronologico', () => {
    const salida = salidaDe(
      fila(5, '2026-10-23T20:00:00Z', 'tercera'),
      fila(4, '2026-10-22T20:00:00Z', 'segunda'),
      fila(3, '2026-10-21T20:00:00Z', 'primera')
    );
    expect(salida.notas.map(n => n.diagnostico)).toEqual(['primera', 'segunda',
'tercera']);
  });

  test('UT-DES08-11 · la fecha se muestra en hora de Guatemala (UTC-6)', () => {
    expect(salidaDe(fila(3, '2026-10-23T20:30:08Z',
'x')).notas[0].fecha).toBe('23/10/2026 14:30');
  });

  test('UT-DES08-12 · una nota de madrugada en UTC queda en el dia anterior en
Guatemala', () => {
    expect(salidaDe(fila(3, '2026-10-24T03:15:00Z',
'x')).notas[0].fecha).toBe('23/10/2026 21:15');
  });

  test('UT-DES08-13 · paciente sin notas (item vacio de Always Output Data):
lista vacia', () => {
    const salida = salidaDe({});
    expect(salida.total).toBe(0);
    expect(salida.notas).toEqual([]);
  });

  test('UT-DES08-14 · el total coincide con la cantidad de notas', () => {
```

```
    expect(salidaDe(fila(1, '2026-10-21T20:00:00Z', 'a'), fila(2, '2026-10-
22T20:00:00Z', 'b')).total).toBe(2);
  });
});

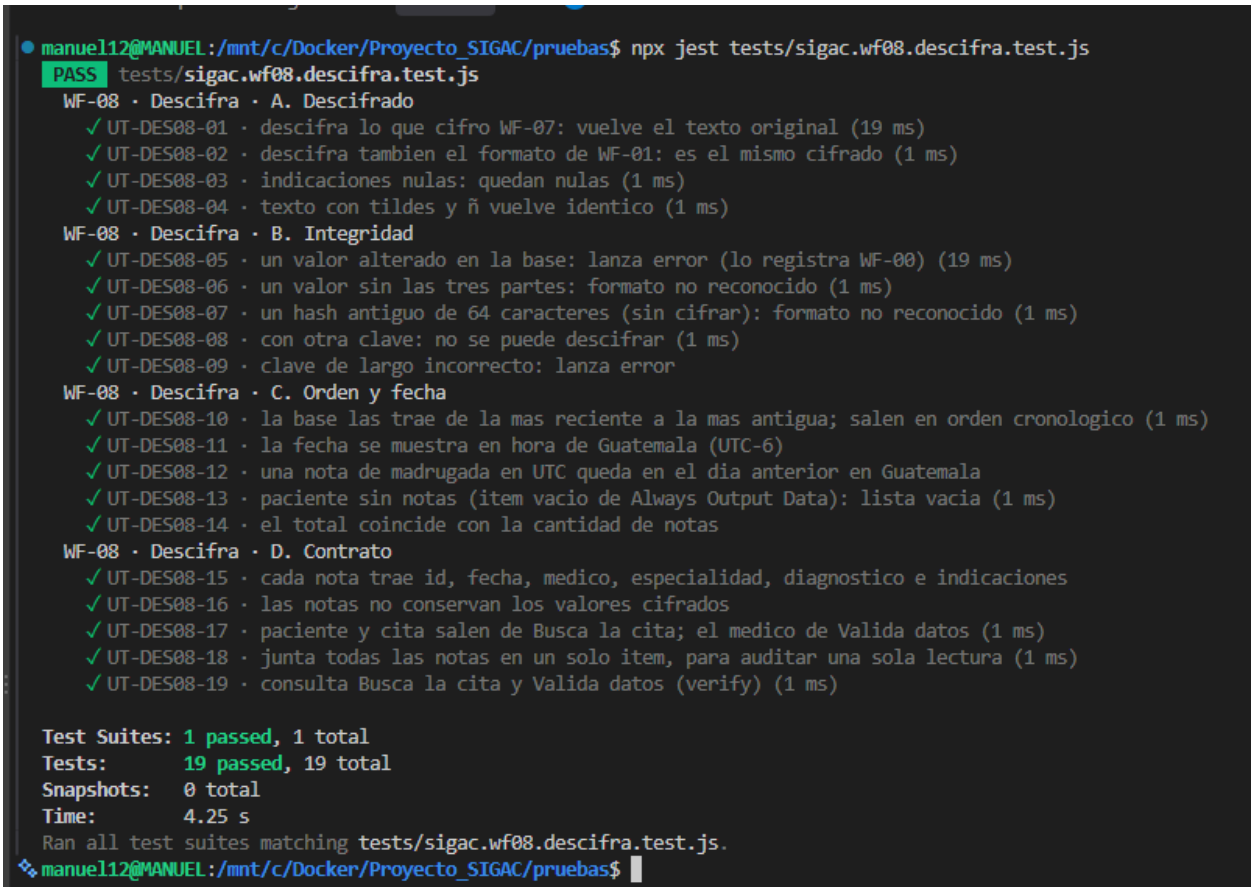
// ------------------------------------------------------------
// D. Contrato
// ------------------------------------------------------------
describe('WF-08 · Descifra · D. Contrato', () => {
  test('UT-DES08-15 · cada nota trae id, fecha, medico, especialidad, diagnostico
e indicaciones', () => {
    const nota = salidaDe(fila(3, '2026-10-23T20:30:08Z', 'x')).notas[0];
    expect(Object.keys(nota).sort()).toEqual(['diagnostico', 'especialidad',
'fecha', 'id_nota', 'indicaciones', 'medico']);
    expect(nota.medico).toBe('Carlos Enrique Coc Ical');
  });

  test('UT-DES08-16 · las notas no conservan los valores cifrados', () => {
    expect(JSON.stringify(salidaDe(fila(3, '2026-10-23T20:30:08Z',
'x')))).not.toContain('_cifrado');
  });

  test('UT-DES08-17 · paciente y cita salen de Busca la cita; el medico de Valida
datos', () => {
    const salida = salidaDe(fila(3, '2026-10-23T20:30:08Z', 'x'));
    expect([salida.id_paciente, salida.id_cita, salida.id_medico]).toEqual([55,
3432, 2]);
  });

  test('UT-DES08-18 · junta todas las notas en un solo item, para auditar una
sola lectura', () => {
    const { salida } = leer([fila(1, '2026-10-21T20:00:00Z', 'a'), fila(2, '2026-
10-22T20:00:00Z', 'b')]);
    expect(salida).toHaveLength(1);
  });

  test('UT-DES08-19 · consulta Busca la cita y Valida datos (verify)', () => {
    const { $ } = leer([fila(3, '2026-10-23T20:30:08Z', 'x')]);
    expect($).toHaveBeenCalledWith('Busca la cita');
    expect($).toHaveBeenCalledWith('Valida datos');
  });
});
```


manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf08.descifra.test.js
PASS tests/sigac.wf08.descifra.test.js
WF-08 · Descifra · A. Descifrado
✓ UT-DES08-01 · descifra lo que cifro WF-07: vuelve el texto original (19 ms)
✓ UT-DES08-02 · descifra tambien el formato de WF-01: es el mismo cifrado (1 ms)
✓ UT-DES08-03 · indicaciones nulas: quedan nulas (1 ms)
✓ UT-DES08-04 · texto con tildes y ñ vuelve identico (1 ms)
WF-08 · Descifra · B. Integridad
✓ UT-DES08-05 · un valor alterado en la base: lanza error (lo registra WF-00) (19 ms)
✓ UT-DES08-06 · un valor sin las tres partes: formato no reconocido (1 ms)
✓ UT-DES08-07 · un hash antiguo de 64 caracteres (sin cifrar): formato no reconocido (1 ms)
✓ UT-DES08-08 · con otra clave: no se puede descifrar (1 ms)
✓ UT-DES08-09 · clave de largo incorrecto: lanza error
WF-08 · Descifra · C. Orden y fecha
✓ UT-DES08-10 · la base las trae de la mas reciente a la mas antigua; salen en orden cronologico (1 ms)
✓ UT-DES08-11 · la fecha se muestra en hora de Guatemala (UTC-6)
✓ UT-DES08-12 · una nota de madrugada en UTC queda en el dia anterior en Guatemala
✓ UT-DES08-13 · paciente sin notas (item vacio de Always Output Data): lista vacia (1 ms)
✓ UT-DES08-14 · el total coincide con la cantidad de notas
WF-08 · Descifra · D. Contrato
✓ UT-DES08-15 · cada nota trae id, fecha, medico, especialidad, diagnostico e indicaciones
✓ UT-DES08-16 · las notas no conservan los valores cifrados
✓ UT-DES08-17 · paciente y cita salen de Busca la cita; el medico de Valida datos (1 ms)
✓ UT-DES08-18 · junta todas las notas en un solo item, para auditar una sola lectura (1 ms)
✓ UT-DES08-19 · consulta Busca la cita y Valida datos (verify) (1 ms)
Test Suites: 1 passed, 1 total
Tests: 19 passed, 19 total
Snapshots: 0 total
Time: 4.25 s
Ran all test suites matching tests/sigac.wf08.descifra.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$

1.33 Historial listo (WF-08)

El nodo Historial listo arma la respuesta 200 con el historial descifrado. Es la única respuesta del sistema que entrega información clínica en claro, y solo se llega a ella después de que la lectura quedó registrada en la bitácora. Si el paciente no tiene notas, responde igualmente 200 con una lista vacía.

1.33.1 Código base del nodo Historial listo

```
const historial = $('Descifra').first().json;
const bitacora = $input.first().json;

return [{
  json: {
    codigo: 200,
    estado: 'consultado',
    mensaje: historial.total === 0
      ? 'El paciente no tiene notas clinicas previas'
      : 'Historial clinico del paciente',
    id_cita: historial.id_cita,
    total: historial.total,
    notas: historial.notas,
    hash_auditoria: bitacora.hash_actual
  }
}];
```

1.33.2 Código de test del nodo Historial listo

```
// sigac.wf08.historiallisto.test.js
//
// pruebas unitarias del nodo Historial listo de WF-08. arma la respuesta
// 200 con el historial descifrado. es la unica respuesta del sistema que
// entrega informacion clinica en claro, y solo llega aqui despues de que
// la lectura quedo registrada en la bitacora.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-HIS08-NN

'use strict';

const historialListo = require('../nodos/wf08_historial_listo.js');
const { ejecutar } = require('../simulacion.js');

const NOTAS = [
  { id_nota: 3, fecha: '23/10/2026 14:30', medico: 'Carlos Enrique Coc Ical',
especialidad: 'Pediatria', diagnostico: 'Faringitis', indicaciones: 'Reposo' }
];

const responder = (historial, bitacora = { hash_actual: 'abc123' }) =>
  ejecutar(historialListo, { entrada: [bitacora], anteriores: { Descifra:
historial } });
```

```
const conNotas = { id_paciente: 55, id_cita: 3432, id_medico: 2, total: 1, notas:
NOTAS };
const sinNotas = { id_paciente: 55, id_cita: 3432, id_medico: 2, total: 0, notas:
[] };

describe('WF-08 · Historial listo', () => {
  test('UT-HIS08-01 · respuesta completa con notas', () => {
    expect(responder(conNotas).salida[0].json).toEqual({
      codigo: 200,
      estado: 'consultado',
      mensaje: 'Historial clinico del paciente',
      id_cita: 3432,
      total: 1,
      notas: NOTAS,
      hash_auditoria: 'abc123'
    });
  });

  test('UT-HIS08-02 · sin notas: mensaje de que no tiene notas previas', () => {
    const { json } = responder(sinNotas).salida[0];
    expect(json.mensaje).toBe('El paciente no tiene notas clinicas previas');
    expect(json.notas).toEqual([]);
    expect(json.total).toBe(0);
  });

  test('UT-HIS08-03 · sin notas tambien responde 200: la consulta fue valida', ()
=> {
    expect(responder(sinNotas).salida[0].json.codigo).toBe(200);
  });

  test('UT-HIS08-04 · el hash de auditoria sale de la bitacora ($input)', () => {
    expect(responder(conNotas, { hash_actual: 'f'.repeat(64)
}).salida[0].json.hash_auditoria).toBe('f'.repeat(64));
  });

  test('UT-HIS08-05 · no devuelve el id del paciente ni del medico', () => {
    const { json } = responder(conNotas).salida[0];
    expect(json).not.toHaveProperty('id_paciente');
    expect(json).not.toHaveProperty('id_medico');
  });

  test('UT-HIS08-06 · las notas pasan intactas desde Descifra', () => {
    expect(responder(conNotas).salida[0].json.notas).toBe(NOTAS);
  });
```

```
  test('UT-HIS08-07 · consulta Descifra (verify)', () => {
    expect(responder(conNotas).$).toHaveBeenCalledWith('Descifra');
  });

  test('UT-HIS08-08 · devuelve una lista con un solo item', () => {
    expect(responder(conNotas).salida).toHaveLength(1);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf08.historiallisto.test.js
 PASS  tests/sigac.wf08.historiallisto.test.js
  WF-08 · Historial listo
    ✓ UT-HIS08-01 · respuesta completa con notas (5 ms)
    ✓ UT-HIS08-02 · sin notas: mensaje de que no tiene notas previas (3 ms)
    ✓ UT-HIS08-03 · sin notas tambien responde 200: la consulta fue valida (1 ms)
    ✓ UT-HIS08-04 · el hash de auditoria sale de la bitacora ($input)
    ✓ UT-HIS08-05 · no devuelve el id del paciente ni del medico
    ✓ UT-HIS08-06 · las notas pasan intactas desde Descifra
    ✓ UT-HIS08-07 · consulta Descifra (verify) (1 ms)
    ✓ UT-HIS08-08 · devuelve una lista con un solo item

Test Suites: 1 passed, 1 total
Tests:       8 passed, 8 total
Snapshots:   0 total
Time:        3.777 s
Ran all test suites matching tests/sigac.wf08.historiallisto.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.34 Rechazo 400 (WF-08)

El nodo Rechazo 400 de WF-08 responde cuando la consulta de historial no pasa la validación. Devuelve completa la lista de errores que recibió de Valida datos.

1.34.1 Código base del nodo Rechazo 400

```
return [{ json: { codigo: 400, estado: 'rechazado', mensaje: 'Datos invalidos',
errores: $input.first().json.errores } }];
```

1.34.2 Código de test del nodo Rechazo 400

```
// sigac.wf08.rechazo400.test.js
//
// pruebas unitarias del nodo Rechazo 400 de WF-08. responde cuando
// Valida datos encontro errores, y devuelve la lista completa para
// que el emisor sepa que corregir. lee los errores de $input.first().json,
// que es la salida de Valida datos por la rama false de ¿Datos validos?
//
// cada prueba lleva su ID de la tabla de pruebas: UT-R40008-NN

'use strict';

const rechazo400 = require('../nodos/wf08_rechazo_400.js');
const { ejecutar } = require('../simulacion.js');

const ERRORES = ['El DPI debe tener exactamente 13 digitos', 'El correo no tiene
un formato valido'];

// ejecuta el nodo con la salida de Valida datos, como $input y como $json
function rechazar(desdeValida) {
  return ejecutar(rechazo400, { entrada: [desdeValida], json: desdeValida });
}

describe('WF-08 · Rechazo 400', () => {
  test('UT-R40008-01 · respuesta completa', () => {
    expect(rechazar({ valido: false, errores: ERRORES
}).salida[0].json).toEqual({
      codigo: 400,
      estado: 'rechazado',
      mensaje: 'Datos invalidos',
      errores: ERRORES
    });
  });

  test('UT-R40008-02 · codigo 400', () => {
    expect(rechazar({ errores: ERRORES }).salida[0].json.codigo).toBe(400);
  });

  test('UT-R40008-03 · estado rechazado (defecto 12.17)', () => {
```

```
    expect(rechazar({ errores: ERRORES
}).salida[0].json.estado).toBe('rechazado');
  });

  test('UT-R40008-04 · mensaje Datos invalidos', () => {
    expect(rechazar({ errores: ERRORES }).salida[0].json.mensaje).toBe('Datos
invalidos');
  });

  test('UT-R40008-05 · la lista de errores llega intacta', () => {
    expect(rechazar({ errores: ERRORES
}).salida[0].json.errores).toEqual(ERRORES);
  });

  test('UT-R40008-06 · el orden de los errores se conserva', () => {
    const invertidos = [...ERRORES].reverse();
    expect(rechazar({ errores: invertidos
}).salida[0].json.errores).toEqual(invertidos);
  });

  test('UT-R40008-07 · un solo error: lista de un elemento', () => {
    expect(rechazar({ errores: ['Falta el campo nombre']
}).salida[0].json.errores)
      .toEqual(['Falta el campo nombre']);
  });

  test('UT-R40008-08 · lista vacia: errores queda vacio', () => {
    expect(rechazar({ errores: [] }).salida[0].json.errores).toEqual([]);
  });

  test('UT-R40008-09 · sin errores en la entrada: queda indefinido, sin error',
() => {
    expect(rechazar({ valido: false }).salida[0].json.errores).toBeUndefined();
  });

  test('UT-R40008-10 · con datos del paciente en la entrada: solo los cuatro
campos', () => {
    const conDatos = { errores: ERRORES, datos: { dpi: '1234567890123', correo:
'ana@correo.com' } };
    const { salida } = rechazar(conDatos);
    expect(Object.keys(salida[0].json).sort()).toEqual(['codigo', 'errores',
'estado', 'mensaje']);
    expect(JSON.stringify(salida)).not.toContain('1234567890123');
  });
```

```
  test('UT-R40008-11 · no consulta ningun nodo anterior (verify)', () => {
    const { $ } = rechazar({ errores: ERRORES });
    expect($).not.toHaveBeenCalled();
  });

  test('UT-R40008-12 · devuelve una lista con un solo item', () => {
    const { salida } = rechazar({ errores: ERRORES });
    expect(Array.isArray(salida)).toBe(true);
    expect(salida).toHaveLength(1);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf08.rechazo400.test.js
 PASS  tests/sigac.wf08.rechazo400.test.js
  WF-08 · Rechazo 400
    ✓ UT-R40008-01 · respuesta completa (4 ms)
    ✓ UT-R40008-02 · codigo 400 (1 ms)
    ✓ UT-R40008-03 · estado rechazado (defecto 12.17) (3 ms)
    ✓ UT-R40008-04 · mensaje Datos invalidos
    ✓ UT-R40008-05 · la lista de errores llega intacta (1 ms)
    ✓ UT-R40008-06 · el orden de los errores se conserva (1 ms)
    ✓ UT-R40008-07 · un solo error: lista de un elemento
    ✓ UT-R40008-08 · lista vacia: errores queda vacio
    ✓ UT-R40008-09 · sin errores en la entrada: queda indefinido, sin error (1 ms)
    ✓ UT-R40008-10 · con datos del paciente en la entrada: solo los cuatro campos (1 ms)
    ✓ UT-R40008-11 · no consulta ningun nodo anterior (verify)
    ✓ UT-R40008-12 · devuelve una lista con un solo item

Test Suites: 1 passed, 1 total
Tests:       12 passed, 12 total
Snapshots:   0 total
Time:        3.73 s
Ran all test suites matching tests/sigac.wf08.rechazo400.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.35 Rechazo 401 (WF-08)

El nodo Rechazo 401 de WF-08 responde cuando el token no viene, no existe o está desactivado. El mensaje es genérico a propósito. Es una respuesta fija.

1.35.1 Código base del nodo Rechazo 401

```
return [{ json: { codigo: 401, estado: 'rechazado', mensaje: 'No autorizado' }
}];
```

1.35.2 Código de test del nodo Rechazo 401

```
// sigac.wf08.rechazo401.test.js
//
// pruebas unitarias del nodo Rechazo 401 de WF-08. responde cuando el token no
viene, no existe o esta desactivado. lo
// alimenta la salida false de ¿Token valido?.
//
// el mensaje es generico a proposito: no revela cual de los tres casos
// ocurrio, para no ayudar a quien intenta adivinar tokens validos.
//
// es una respuesta fija: no lee la entrada ni consulta otros nodos.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-R40108-NN

'use strict';

const rechazo = require('../nodos/wf08_rechazo_401.js');
const { ejecutar } = require('../simulacion.js');

const RESPUESTA = {
  codigo: 401,
  estado: 'rechazado',
  mensaje: 'No autorizado'
};

// lo que puede llegar al nodo: la salida del nodo anterior junto con
// lo que traia el flujo, incluida informacion que no debe devolverse
const CON_DATOS = {
  token_hash: 'a'.repeat(64),
  rol: 'medico',
  id_cita: 3432,
  datos: { dpi: '1234567890123', correo: 'ana@correo.com' }
};

describe('WF-08 · Rechazo 401', () => {
  test('UT-R40108-01 · respuesta completa', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json).toEqual(RESPUESTA);
  });
```

```
  test('UT-R40108-02 · codigo 401', () => {
    expect(ejecutar(rechazo, { entrada: [{}] }).salida[0].json.codigo).toBe(401);
  });

  test('UT-R40108-03 · estado rechazado (defecto 12.17)', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.estado).toBe('rechazado');
  });

  test('UT-R40108-04 · mensaje No autorizado', () => {
    expect(ejecutar(rechazo, { entrada: [{}] }).salida[0].json.mensaje).toBe('No
autorizado');
  });

  test('UT-R40108-05 · solo los campos codigo, estado y mensaje', () => {
    const { salida } = ejecutar(rechazo, { entrada: [{}] });
    expect(Object.keys(salida[0].json).sort()).toEqual(['codigo', 'estado',
'mensaje']);
  });

  test('UT-R40108-06 · sin entrada: la misma respuesta', () => {
    expect(ejecutar(rechazo).salida[0].json).toEqual(RESPUESTA);
  });

  test('UT-R40108-07 · con datos en la entrada: la misma respuesta, sin
devolverlos', () => {
    const { salida } = ejecutar(rechazo, { entrada: [CON_DATOS], json: CON_DATOS
});
    expect(salida[0].json).toEqual(RESPUESTA);
    const texto = JSON.stringify(salida);
    expect(texto).not.toContain('a'.repeat(64));
    expect(texto).not.toContain('1234567890123');
    expect(texto).not.toContain('3432');
  });

  test('UT-R40108-08 · no consulta ningun nodo anterior (verify)', () => {
    const { $ } = ejecutar(rechazo, { entrada: [{}] });
    expect($).not.toHaveBeenCalled();
  });

  test('UT-R40108-09 · dos ejecuciones con entradas distintas dan respuestas
identicas', () => {
    const a = ejecutar(rechazo, { entrada: [{ rol: 'recepcion' }], json: { rol:
'recepcion' } }).salida;
```

```
    const b = ejecutar(rechazo, { entrada: [CON_DATOS], json: CON_DATOS
}).salida;
    expect(a).toEqual(b);
  });

  test('UT-R40108-10 · el mensaje no revela la causa del rechazo', () => {
    const mensaje = ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.mensaje.toLowerCase();
    for (const pista of ['inactivo', 'desactivado', 'expirado', 'rol']) {
      expect(mensaje).not.toContain(pista);
    }
  });

  test('UT-R40108-11 · devuelve una lista con un solo item', () => {
    const { salida } = ejecutar(rechazo, { entrada: [{}] });
    expect(Array.isArray(salida)).toBe(true);
    expect(salida).toHaveLength(1);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf08.rechazo401.test.js
 PASS  tests/sigac.wf08.rechazo401.test.js
  WF-08 · Rechazo 401
    ✓ UT-R40108-01 · respuesta completa (5 ms)
    ✓ UT-R40108-02 · codigo 401
    ✓ UT-R40108-03 · estado rechazado (defecto 12.17) (1 ms)
    ✓ UT-R40108-04 · mensaje No autorizado
    ✓ UT-R40108-05 · solo los campos codigo, estado y mensaje (1 ms)
    ✓ UT-R40108-06 · sin entrada: la misma respuesta
    ✓ UT-R40108-07 · con datos en la entrada: la misma respuesta, sin devolverlos
    ✓ UT-R40108-08 · no consulta ningun nodo anterior (verify) (1 ms)
    ✓ UT-R40108-09 · dos ejecuciones con entradas distintas dan respuestas identicas
    ✓ UT-R40108-10 · el mensaje no revela la causa del rechazo
    ✓ UT-R40108-11 · devuelve una lista con un solo item (1 ms)

Test Suites: 1 passed, 1 total
Tests:       11 passed, 11 total
Snapshots:   0 total
Time:        3.736 s
Ran all test suites matching tests/sigac.wf08.rechazo401.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.36 Rechazo 403 (WF-08)

El nodo Rechazo 403 de WF-08 responde cuando el token es válido pero no es de un médico: solo un médico puede consultar historiales clínicos. El mensaje no revela qué rol se necesita. Es una respuesta fija.

1.36.1 Código base del nodo Rechazo 403

```
return [{ json: { codigo: 403, estado: 'rechazado', mensaje: 'Operacion no
permitida para este rol' } }];
```

1.36.2 Código de test del nodo Rechazo 403

```
// sigac.wf08.rechazo403.test.js
//
// pruebas unitarias del nodo Rechazo 403 de WF-08. responde cuando el token es
valido pero no es de un medico: solo un
// medico puede consultar historiales. lo alimenta la salida false de
// ¿Es medico?.
//
// el mensaje no revela que rol se necesita ni cual tiene quien llama.
//
// es una respuesta fija: no lee la entrada ni consulta otros nodos.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-R40308-NN

'use strict';

const rechazo = require('../nodos/wf08_rechazo_403.js');
const { ejecutar } = require('../simulacion.js');

const RESPUESTA = {
  codigo: 403,
  estado: 'rechazado',
  mensaje: 'Operacion no permitida para este rol'
};

// lo que puede llegar al nodo: la salida del nodo anterior junto con
// lo que traia el flujo, incluida informacion que no debe devolverse
const CON_DATOS = {
  token_hash: 'a'.repeat(64),
  rol: 'medico',
  id_cita: 3432,
  datos: { dpi: '1234567890123', correo: 'ana@correo.com' }
};

describe('WF-08 · Rechazo 403', () => {
  test('UT-R40308-01 · respuesta completa', () => {
```

```
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json).toEqual(RESPUESTA);
  });

  test('UT-R40308-02 · codigo 403', () => {
    expect(ejecutar(rechazo, { entrada: [{}] }).salida[0].json.codigo).toBe(403);
  });

  test('UT-R40308-03 · estado rechazado (defecto 12.17)', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.estado).toBe('rechazado');
  });

  test('UT-R40308-04 · mensaje Operacion no permitida para este rol', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.mensaje).toBe('Operacion no permitida para este rol');
  });

  test('UT-R40308-05 · solo los campos codigo, estado y mensaje', () => {
    const { salida } = ejecutar(rechazo, { entrada: [{}] });
    expect(Object.keys(salida[0].json).sort()).toEqual(['codigo', 'estado',
'mensaje']);
  });

  test('UT-R40308-06 · sin entrada: la misma respuesta', () => {
    expect(ejecutar(rechazo).salida[0].json).toEqual(RESPUESTA);
  });

  test('UT-R40308-07 · con datos en la entrada: la misma respuesta, sin
devolverlos', () => {
    const { salida } = ejecutar(rechazo, { entrada: [CON_DATOS], json: CON_DATOS
});
    expect(salida[0].json).toEqual(RESPUESTA);
    const texto = JSON.stringify(salida);
    expect(texto).not.toContain('a'.repeat(64));
    expect(texto).not.toContain('1234567890123');
    expect(texto).not.toContain('3432');
  });

  test('UT-R40308-08 · no consulta ningun nodo anterior (verify)', () => {
    const { $ } = ejecutar(rechazo, { entrada: [{}] });
    expect($).not.toHaveBeenCalled();
  });
```

```
  test('UT-R40308-09 · dos ejecuciones con entradas distintas dan respuestas
identicas', () => {
    const a = ejecutar(rechazo, { entrada: [{ rol: 'recepcion' }], json: { rol:
'recepcion' } }).salida;
    const b = ejecutar(rechazo, { entrada: [CON_DATOS], json: CON_DATOS
}).salida;
    expect(a).toEqual(b);
  });

  test('UT-R40308-10 · el mensaje no revela roles', () => {
    const mensaje = ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.mensaje.toLowerCase();
    for (const pista of ['recepcion', 'medico', 'admin']) {
      expect(mensaje).not.toContain(pista);
    }
  });

  test('UT-R40308-11 · devuelve una lista con un solo item', () => {
    const { salida } = ejecutar(rechazo, { entrada: [{}] });
    expect(Array.isArray(salida)).toBe(true);
    expect(salida).toHaveLength(1);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf08.rechazo403.test.js
 PASS  tests/sigac.wf08.rechazo403.test.js
  WF-08 · Rechazo 403
    ✓ UT-R40308-01 · respuesta completa (7 ms)
    ✓ UT-R40308-02 · codigo 403
    ✓ UT-R40308-03 · estado rechazado (defecto 12.17)
    ✓ UT-R40308-04 · mensaje Operacion no permitida para este rol
    ✓ UT-R40308-05 · solo los campos codigo, estado y mensaje
    ✓ UT-R40308-06 · sin entrada: la misma respuesta (1 ms)
    ✓ UT-R40308-07 · con datos en la entrada: la misma respuesta, sin devolverlos
    ✓ UT-R40308-08 · no consulta ningun nodo anterior (verify)
    ✓ UT-R40308-09 · dos ejecuciones con entradas distintas dan respuestas identicas (1 ms)
    ✓ UT-R40308-10 · el mensaje no revela roles
    ✓ UT-R40308-11 · devuelve una lista con un solo item (1 ms)

Test Suites: 1 passed, 1 total
Tests:       11 passed, 11 total
Snapshots:   0 total
Time:        3.678 s
Ran all test suites matching tests/sigac.wf08.rechazo403.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.37 Rechazo 404 (WF-08)

El nodo Rechazo 404 de WF-08 responde cuando el nodo Busca la cita no encontró una cita propia del médico, confirmada o atendida. Usa el mismo mensaje para una cita inexistente y para una cita de otro médico. Es una respuesta fija.

1.37.1 Código base del nodo Rechazo 404

```
return [{ json: { codigo: 404, estado: 'rechazado', mensaje: 'Cita no encontrada
o no permite consultar historial' } }];
```

1.37.2 Código de test del nodo Rechazo 404

```
// sigac.wf08.rechazo404.test.js
//
// pruebas unitarias del nodo Rechazo 404 de WF-08. responde cuando Busca la cita
no encontro una cita propia del medico,
// confirmada o atendida. lo alimenta la salida false de ¿Cita valida?.
//
// el mismo mensaje para una cita inexistente y para una cita de otro
// medico: no revela si la cita existe pero pertenece a otro.
//
// es una respuesta fija: no lee la entrada ni consulta otros nodos.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-R40408-NN

'use strict';

const rechazo = require('../nodos/wf08_rechazo_404.js');
const { ejecutar } = require('../simulacion.js');

const RESPUESTA = {
  codigo: 404,
  estado: 'rechazado',
  mensaje: 'Cita no encontrada o no permite consultar historial'
};

// lo que puede llegar al nodo: la salida del nodo anterior junto con
// lo que traia el flujo, incluida informacion que no debe devolverse
const CON_DATOS = {
  token_hash: 'a'.repeat(64),
  rol: 'medico',
  id_cita: 3432,
  datos: { dpi: '1234567890123', correo: 'ana@correo.com' }
};

describe('WF-08 · Rechazo 404', () => {
  test('UT-R40408-01 · respuesta completa', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json).toEqual(RESPUESTA);
```

```
  });

  test('UT-R40408-02 · codigo 404', () => {
    expect(ejecutar(rechazo, { entrada: [{}] }).salida[0].json.codigo).toBe(404);
  });

  test('UT-R40408-03 · estado rechazado (defecto 12.17)', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.estado).toBe('rechazado');
  });

  test('UT-R40408-04 · mensaje Cita no encontrada o no permite consultar
historial', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.mensaje).toBe('Cita no encontrada o no permite consultar
historial');
  });

  test('UT-R40408-05 · solo los campos codigo, estado y mensaje', () => {
    const { salida } = ejecutar(rechazo, { entrada: [{}] });
    expect(Object.keys(salida[0].json).sort()).toEqual(['codigo', 'estado',
'mensaje']);
  });

  test('UT-R40408-06 · sin entrada: la misma respuesta', () => {
    expect(ejecutar(rechazo).salida[0].json).toEqual(RESPUESTA);
  });

  test('UT-R40408-07 · con datos en la entrada: la misma respuesta, sin
devolverlos', () => {
    const { salida } = ejecutar(rechazo, { entrada: [CON_DATOS], json: CON_DATOS
});
    expect(salida[0].json).toEqual(RESPUESTA);
    const texto = JSON.stringify(salida);
    expect(texto).not.toContain('a'.repeat(64));
    expect(texto).not.toContain('1234567890123');
    expect(texto).not.toContain('3432');
  });

  test('UT-R40408-08 · no consulta ningun nodo anterior (verify)', () => {
    const { $ } = ejecutar(rechazo, { entrada: [{}] });
    expect($).not.toHaveBeenCalled();
  });
```

```
  test('UT-R40408-09 · dos ejecuciones con entradas distintas dan respuestas
identicas', () => {
    const a = ejecutar(rechazo, { entrada: [{ rol: 'recepcion' }], json: { rol:
'recepcion' } }).salida;
    const b = ejecutar(rechazo, { entrada: [CON_DATOS], json: CON_DATOS
}).salida;
    expect(a).toEqual(b);
  });

  test('UT-R40408-10 · el mensaje no revela si la cita es de otro medico', () =>
{
    const mensaje = ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.mensaje.toLowerCase();
    for (const pista of ['otro medico', 'ajena', 'pertenece']) {
      expect(mensaje).not.toContain(pista);
    }
  });

  test('UT-R40408-11 · devuelve una lista con un solo item', () => {
    const { salida } = ejecutar(rechazo, { entrada: [{}] });
    expect(Array.isArray(salida)).toBe(true);
    expect(salida).toHaveLength(1);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf08.rechazo404.test.js
 PASS  tests/sigac.wf08.rechazo404.test.js
  WF-08 · Rechazo 404
    ✓ UT-R40408-01 · respuesta completa (6 ms)
    ✓ UT-R40408-02 · codigo 404 (1 ms)
    ✓ UT-R40408-03 · estado rechazado (defecto 12.17)
    ✓ UT-R40408-04 · mensaje Cita no encontrada o no permite consultar historial
    ✓ UT-R40408-05 · solo los campos codigo, estado y mensaje
    ✓ UT-R40408-06 · sin entrada: la misma respuesta (1 ms)
    ✓ UT-R40408-07 · con datos en la entrada: la misma respuesta, sin devolverlos (1 ms)
    ✓ UT-R40408-08 · no consulta ningun nodo anterior (verify)
    ✓ UT-R40408-09 · dos ejecuciones con entradas distintas dan respuestas identicas
    ✓ UT-R40408-10 · el mensaje no revela si la cita es de otro medico
    ✓ UT-R40408-11 · devuelve una lista con un solo item (1 ms)

Test Suites: 1 passed, 1 total
Tests:       11 passed, 11 total
Snapshots:   0 total
Time:        3.741 s
Ran all test suites matching tests/sigac.wf08.rechazo404.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.38 Hash del token (WF-09)

El nodo Hash del token de WF-09 lee el token de la cabecera x-sigac-token y calcula su huella SHA-256, que es la que se compara contra la base de datos. El token nunca se compara ni se guarda en claro. Sin token, usa el centinela sin-token.

Es idéntico al de WF-01, WF-07 y WF-08.

1.38.1 Código base del nodo Hash del token

```
const crypto = require('crypto');

const cabeceras = $input.first().json.headers || {};
const cuerpo    = $input.first().json.body    || {};

const token = cabeceras['x-sigac-token'] || '';

const tokenHash = token
  ? crypto.createHash('sha256').update(token).digest('hex')
  : 'sin-token';

return [{
  json: {
    token_hash: tokenHash,
    tiene_token: token.length > 0,
    datos: cuerpo
  }
}];
```

1.38.2 Código de test del nodo Hash del token

```
// sigac.wf09.hashdeltoken.test.js
//
// pruebas unitarias del nodo Hash del token de WF-09. lee el token de
// la cabecera x-sigac-token y calcula su huella sha-256, que es la que
// se compara contra la base. el token nunca se compara ni se guarda en
// claro. sin token, usa el centinela sin-token, que nunca puede
// coincidir con una huella real.
//
// tecnicas: particion de equivalencia, valores limite, cobertura de
// ramas (headers || {}, body || {}, token ? hash : centinela),
// propiedades del hash y contrato de salida.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-HASH09-NN

'use strict';

const hashDelToken = require('../nodos/wf09_hash_del_token.js');
const { ejecutar, sha256 } = require('../simulacion.js');

// arma la peticion que entrega el Webhook y ejecuta el nodo
```

```
const hash = peticion => ejecutar(hashDelToken, { entrada: [peticion] }).salida;

describe('WF-09 · Hash del token', () => {
  test('UT-HASH09-01 · con token: calcula su huella sha-256', () => {
    const salida = hash({ headers: { 'x-sigac-token': 'abc' }, body: {} });
    expect(salida[0].json.token_hash).toBe(sha256('abc'));
    expect(salida[0].json.tiene_token).toBe(true);
  });

  test('UT-HASH09-02 · sin cabecera de token: centinela sin-token', () => {
    const salida = hash({ headers: {}, body: {} });
    expect(salida[0].json.token_hash).toBe('sin-token');
    expect(salida[0].json.tiene_token).toBe(false);
  });

  test('UT-HASH09-03 · token vacio: centinela sin-token', () => {
    const salida = hash({ headers: { 'x-sigac-token': '' }, body: {} });
    expect(salida[0].json.token_hash).toBe('sin-token');
    expect(salida[0].json.tiene_token).toBe(false);
  });

  test('UT-HASH09-04 · peticion sin headers: no falla y usa el centinela', () =>
{
    const salida = hash({ body: {} });
    expect(salida[0].json.token_hash).toBe('sin-token');
  });

  test('UT-HASH09-05 · peticion sin body: datos queda como objeto vacio', () => {
    const salida = hash({ headers: { 'x-sigac-token': 'abc' } });
    expect(salida[0].json.datos).toEqual({});
  });

  test('UT-HASH09-06 · el cuerpo pasa intacto en datos', () => {
    const cuerpo = { dpi: '1234567890123', nombre: 'Ana', anidado: { a: 1 } };
    const salida = hash({ headers: {}, body: cuerpo });
    expect(salida[0].json.datos).toEqual(cuerpo);
  });

  test('UT-HASH09-07 · cabecera en mayusculas: no se lee (n8n las entrega en
minusculas)', () => {
    const salida = hash({ headers: { 'X-SIGAC-TOKEN': 'abc' }, body: {} });
    expect(salida[0].json.token_hash).toBe('sin-token');
  });
```

```
  test('UT-HASH09-08 · token con espacios: se usa tal cual, sin recortar', () =>
{
    const salida = hash({ headers: { 'x-sigac-token': ' abc ' }, body: {} });
    expect(salida[0].json.token_hash).toBe(sha256(' abc '));
    expect(salida[0].json.token_hash).not.toBe(sha256('abc'));
  });

  test('UT-HASH09-09 · el mismo token da siempre la misma huella', () => {
    const a = hash({ headers: { 'x-sigac-token': 'abc' }, body: {} });
    const b = hash({ headers: { 'x-sigac-token': 'abc' }, body: {} });
    expect(a[0].json.token_hash).toBe(b[0].json.token_hash);
  });

  test('UT-HASH09-10 · tokens distintos dan huellas distintas', () => {
    const a = hash({ headers: { 'x-sigac-token': 'abc' }, body: {} });
    const b = hash({ headers: { 'x-sigac-token': 'abd' }, body: {} });
    expect(a[0].json.token_hash).not.toBe(b[0].json.token_hash);
  });

  test('UT-HASH09-11 · la huella tiene 64 caracteres hexadecimales en minuscula',
() => {
    const salida = hash({ headers: { 'x-sigac-token': 'abc' }, body: {} });
    expect(salida[0].json.token_hash).toMatch(/^[0-9a-f]{64}$/);
  });

  test('UT-HASH09-12 · el token en claro no aparece en la salida', () => {
    // el token lleva guiones, que nunca aparecen en un hexadecimal:
    // si se encuentra en la salida, es porque el token salio en claro
    const token = 'token-secreto-de-prueba';
    const salida = hash({ headers: { 'x-sigac-token': token }, body: {} });
    expect(JSON.stringify(salida)).not.toContain(token);
  });

  test('UT-HASH09-13 · devuelve un solo item con exactamente tres campos', () =>
{
    const salida = hash({ headers: { 'x-sigac-token': 'abc' }, body: {} });
    expect(salida).toHaveLength(1);
    expect(Object.keys(salida[0].json).sort()).toEqual(['datos', 'tiene_token',
'token_hash']);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf09.hashdeltoken.test.js
 PASS  tests/sigac.wf09.hashdeltoken.test.js
  WF-09 · Hash del token
    ✓ UT-HASH09-01 · con token: calcula su huella sha-256 (6 ms)
    ✓ UT-HASH09-02 · sin cabecera de token: centinela sin-token (5 ms)
    ✓ UT-HASH09-03 · token vacio: centinela sin-token (1 ms)
    ✓ UT-HASH09-04 · peticion sin headers: no falla y usa el centinela (1 ms)
    ✓ UT-HASH09-05 · peticion sin body: datos queda como objeto vacio (2 ms)
    ✓ UT-HASH09-06 · el cuerpo pasa intacto en datos
    ✓ UT-HASH09-07 · cabecera en mayusculas: no se lee (n8n las entrega en minusculas)
    ✓ UT-HASH09-08 · token con espacios: se usa tal cual, sin recortar (1 ms)
    ✓ UT-HASH09-09 · el mismo token da siempre la misma huella (1 ms)
    ✓ UT-HASH09-10 · tokens distintos dan huellas distintas
    ✓ UT-HASH09-11 · la huella tiene 64 caracteres hexadecimales en minuscula (1 ms)
    ✓ UT-HASH09-12 · el token en claro no aparece en la salida (1 ms)
    ✓ UT-HASH09-13 · devuelve un solo item con exactamente tres campos (1 ms)

Test Suites: 1 passed, 1 total
Tests:       13 passed, 13 total
Snapshots:   0 total
Time:        4.239 s
Ran all test suites matching tests/sigac.wf09.hashdeltoken.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ ~
```

1.39 Valida datos (WF-09)

El nodo Valida datos de WF-09 revisa la petición de administración: la acción (agregar, desactivar, dar_baja o reactivar), la entidad (paciente, médico o recepcionista) y los campos que exige cada combinación. Calcula además la ruta que tomará el Switch y la bandera baja, que distingue una baja definitiva de una desactivación.

Los pacientes no se pueden agregar desde aquí: nacen en WF-01 con su primera solicitud, donde se valida y cifra su DPI. Los errores se acumulan y se devuelven todos juntos.

1.39.1 Código base del nodo Valida datos

```
const datos = $('Hash del token').first().json.datos || {};
const errores = [];

const acciones  = ['agregar', 'desactivar', 'dar_baja', 'reactivar'];
const entidades = ['paciente', 'medico', 'recepcionista'];

const accion  = String(datos.accion  || '').trim().toLowerCase();
const entidad = String(datos.entidad || '').trim().toLowerCase();

if (!acciones.includes(accion)) {
  errores.push('La accion debe ser agregar, desactivar, dar_baja o reactivar');
}
if (!entidades.includes(entidad)) {
  errores.push('La entidad debe ser paciente, medico o recepcionista');
}
let nombre = null;
let especialidad = null;
let colegiado = null;

if (accion === 'agregar') {
  if (entidad === 'paciente') {
    errores.push('Los pacientes se registran con su primera solicitud de cita, no
desde administracion');
  }

  if (entidad === 'medico' || entidad === 'recepcionista') {
    nombre = String(datos.nombre || '').trim() || null;
    if (!nombre) {
      errores.push('Falta el campo nombre');
    } else if (nombre.length > 120) {
      errores.push('El nombre no puede exceder 120 caracteres');
    }
  }

  if (entidad === 'medico') {
    especialidad = String(datos.especialidad || '').trim() || null;
```

```
    if (!especialidad) {
      errores.push('Falta el campo especialidad');
    } else if (especialidad.length > 60) {
      errores.push('La especialidad no puede exceder 60 caracteres');
    }

    colegiado = String(datos.colegiado || '').trim() || null;
    if (!colegiado) {
      errores.push('Falta el campo colegiado');
    } else if (!/^[A-Za-z0-9-]{1,20}$/.test(colegiado)) {
      errores.push('El colegiado debe tener hasta 20 letras, digitos o guiones');
    }
  }
}

let id = null;
if (['desactivar', 'dar_baja', 'reactivar'].includes(accion)) {
  id = Number(datos.id);
  if (!Number.isInteger(id) || id < 1) {
    errores.push('El id debe ser un entero positivo');
    id = null;
  }
}
let ruta = null;
if (errores.length === 0) {
  const tramo = accion === 'agregar'   ? 'agregar'
              : accion === 'reactivar' ? 'reactivar'
              : 'estado';
  ruta = entidad + '_' + tramo;
}
return [{
  json: {
    valido: errores.length === 0,
    errores: errores,
    rol: $('Verifica token').first().json.rol,
    ruta: ruta,
    accion: accion,
    entidad: entidad,
    id: id,
    baja: accion === 'dar_baja',
    nombre: nombre,
    especialidad: especialidad,
    colegiado: colegiado
  }
}];
```

1.39.2 Código de test del nodo Valida datos

```
// sigac.wf09.validadatos.test.js
//
// pruebas unitarias del nodo Valida datos de WF-09. valida la peticion de
// administracion: accion, entidad y los campos de cada combinacion, y
// calcula la ruta que tomara el Switch. los errores se acumulan.
//
//   agregar     medico, recepcionista             nombre (+ especialidad y
colegiado)
//   desactivar  paciente, medico, recepcionista    id
//   dar_baja    paciente, medico, recepcionista    id
//   reactivar   paciente, medico, recepcionista    id
//
// cada prueba lleva su ID de la tabla de pruebas: UT-VAL09-NN

'use strict';

const validaDatos = require('../nodos/wf09_valida_datos.js');
const { ejecutar } = require('../simulacion.js');

function validar(cuerpo, hash = { datos: cuerpo }) {
  return ejecutar(validaDatos, {
    anteriores: { 'Hash del token': hash, 'Verifica token': { rol: 'admin' } }
  });
}

const salidaDe = cuerpo => validar(cuerpo).salida[0].json;

const MEDICO = { accion: 'agregar', entidad: 'medico', nombre: 'Caal Tiul, Jorge
Luis', especialidad: 'Pediatria', colegiado: 'COL-90001' };

// ------------------------------------------------------------
// A. Accion y entidad
// ------------------------------------------------------------
describe('WF-09 · Valida datos · A. Accion y entidad', () => {
  test('UT-VAL09-01 · accion inexistente: error', () => {
    expect(salidaDe({ accion: 'borrar', entidad: 'medico', id: 1 }).errores)
      .toContain('La accion debe ser agregar, desactivar, dar_baja o reactivar');
  });

  test('UT-VAL09-02 · entidad inexistente: error', () => {
    expect(salidaDe({ accion: 'desactivar', entidad: 'admin', id: 1 }).errores)
      .toContain('La entidad debe ser paciente, medico o recepcionista');
  });
```

```
  test('UT-VAL09-03 · cuerpo vacio: los dos errores juntos', () => {
    expect(salidaDe({}).errores).toEqual([
      'La accion debe ser agregar, desactivar, dar_baja o reactivar',
      'La entidad debe ser paciente, medico o recepcionista'
    ]);
  });

  test('UT-VAL09-04 · accion y entidad en mayusculas y con espacios: se aceptan',
() => {
    const salida = salidaDe({ accion: ' Desactivar ', entidad: ' MEDICO ', id: 2
});
    expect(salida.valido).toBe(true);
    expect(salida.accion).toBe('desactivar');
    expect(salida.entidad).toBe('medico');
  });
});

// ------------------------------------------------------------
// B. Rutas del Switch
// ------------------------------------------------------------
describe('WF-09 · Valida datos · B. Rutas', () => {
  test.each([
    ['agregar', 'recepcionista', { nombre: 'Rosa' }, 'recepcionista_agregar'],
    ['desactivar', 'recepcionista', { id: 1 }, 'recepcionista_estado'],
    ['dar_baja', 'recepcionista', { id: 1 }, 'recepcionista_estado'],
    ['reactivar', 'recepcionista', { id: 1 }, 'recepcionista_reactivar'],
    ['desactivar', 'paciente', { id: 1 }, 'paciente_estado'],
    ['dar_baja', 'paciente', { id: 1 }, 'paciente_estado'],
    ['reactivar', 'paciente', { id: 1 }, 'paciente_reactivar'],
    ['agregar', 'medico', { nombre: 'X', especialidad: 'Pediatria', colegiado:
'COL-1' }, 'medico_agregar'],
    ['desactivar', 'medico', { id: 1 }, 'medico_estado'],
    ['dar_baja', 'medico', { id: 1 }, 'medico_estado'],
    ['reactivar', 'medico', { id: 1 }, 'medico_reactivar']
  ])('UT-VAL09-05 · %s %s: ruta correcta', (accion, entidad, campos, ruta) => {
    expect(salidaDe({ accion, entidad, ...campos }).ruta).toBe(ruta);
  });

  test('UT-VAL09-06 · con errores, la ruta queda nula', () => {
    expect(salidaDe({ accion: 'desactivar', entidad: 'medico', id: 'abc'
}).ruta).toBeNull();
  });

  test.each([
    ['UT-VAL09-07', 'dar_baja', true],
```

```
    ['UT-VAL09-08', 'desactivar', false],
    ['UT-VAL09-09', 'reactivar', false]
  ])('%s · %s: bandera baja %p', (_, accion, baja) => {
    expect(salidaDe({ accion, entidad: 'paciente', id: 1 }).baja).toBe(baja);
  });
});

// ------------------------------------------------------------
// C. Agregar
// ------------------------------------------------------------
describe('WF-09 · Valida datos · C. Agregar', () => {
  test('UT-VAL09-10 · agregar paciente: error, nacen en WF-01', () => {
    expect(salidaDe({ accion: 'agregar', entidad: 'paciente', nombre: 'Ana'
}).errores)
      .toContain('Los pacientes se registran con su primera solicitud de cita, no
desde administracion');
  });

  test('UT-VAL09-11 · medico completo: valido, con coma en el nombre', () => {
    const salida = salidaDe(MEDICO);
    expect(salida.valido).toBe(true);
    expect(salida.nombre).toBe('Caal Tiul, Jorge Luis');
  });

  test.each([
    ['UT-VAL09-12', 'nombre', 'Falta el campo nombre'],
    ['UT-VAL09-13', 'especialidad', 'Falta el campo especialidad'],
    ['UT-VAL09-14', 'colegiado', 'Falta el campo colegiado']
  ])('%s · medico sin %s: error', (_, campo, mensaje) => {
    const cuerpo = { ...MEDICO };
    delete cuerpo[campo];
    expect(salidaDe(cuerpo).errores).toContain(mensaje);
  });

  test('UT-VAL09-15 · recepcionista solo necesita nombre', () => {
    expect(salidaDe({ accion: 'agregar', entidad: 'recepcionista', nombre: 'Rosa'
}).valido).toBe(true);
  });

  test.each([
    ['UT-VAL09-16', 120, true],
    ['UT-VAL09-17', 121, false]
  ])('%s · nombre de %i caracteres', (_, largo, esperado) => {
    expect(salidaDe({ ...MEDICO, nombre: 'x'.repeat(largo)
}).valido).toBe(esperado);
```

```
  });

  test.each([
    ['UT-VAL09-18', 60, true],
    ['UT-VAL09-19', 61, false]
  ])('%s · especialidad de %i caracteres', (_, largo, esperado) => {
    expect(salidaDe({ ...MEDICO, especialidad: 'x'.repeat(largo)
}).valido).toBe(esperado);
  });

  test.each([
    ['UT-VAL09-20', 'COL-90001', true],
    ['UT-VAL09-21', 'x'.repeat(20), true],
    ['UT-VAL09-22', 'x'.repeat(21), false],
    ['UT-VAL09-23', 'COL 90001', false],
    ['UT-VAL09-24', 'COL_90001', false]
  ])('%s · colegiado %s', (_, colegiado, esperado) => {
    expect(salidaDe({ ...MEDICO, colegiado }).valido).toBe(esperado);
  });

  test('UT-VAL09-25 · nombre, especialidad y colegiado salen recortados', () => {
    const salida = salidaDe({ ...MEDICO, nombre: '  Jorge  ', especialidad: '
Pediatria ', colegiado: ' COL-1 ' });
    expect([salida.nombre, salida.especialidad,
salida.colegiado]).toEqual(['Jorge', 'Pediatria', 'COL-1']);
  });

  test('UT-VAL09-26 · agregar no necesita id: queda nulo', () => {
    expect(salidaDe(MEDICO).id).toBeNull();
  });
});

// ------------------------------------------------------------
// D. id para desactivar, dar_baja y reactivar
// ------------------------------------------------------------
describe('WF-09 · Valida datos · D. id', () => {
  test.each([
    ['UT-VAL09-27', 'numero', 6, 6],
    ['UT-VAL09-28', 'texto numerico', '6', 6]
  ])('%s · id como %s: valido', (_, __, id, esperado) => {
    expect(salidaDe({ accion: 'desactivar', entidad: 'medico', id
}).id).toBe(esperado);
  });

  test.each([
```

```
    ['UT-VAL09-29', 'cero', 0],
    ['UT-VAL09-30', 'negativo', -1],
    ['UT-VAL09-31', 'decimal', 2.5],
    ['UT-VAL09-32', 'texto', 'abc'],
    ['UT-VAL09-33', 'ausente', undefined]
  ])('%s · id %s: error', (_, __, id) => {
    expect(salidaDe({ accion: 'desactivar', entidad: 'medico', id
}).errores).toContain('El id debe ser un entero positivo');
  });

  // hipotesis A: Number(true) es 1, asi que true desactivaria a la persona 1
  test('UT-VAL09-34 · id true: debe rechazarse (hipotesis A)', () => {
    expect(salidaDe({ accion: 'dar_baja', entidad: 'medico', id: true
}).valido).toBe(false);
  });

  test('UT-VAL09-35 · desactivar ignora nombre, especialidad y colegiado', () =>
{
    const salida = salidaDe({ accion: 'desactivar', entidad: 'medico', id: 2,
nombre: 'X', colegiado: 'COL-1' });
    expect([salida.nombre, salida.especialidad, salida.colegiado]).toEqual([null,
null, null]);
  });
});

// ------------------------------------------------------------
// E. Contrato
// ------------------------------------------------------------
describe('WF-09 · Valida datos · E. Contrato', () => {
  test('UT-VAL09-36 · el rol sale de Verifica token', () => {
    expect(salidaDe(MEDICO).rol).toBe('admin');
  });

  test('UT-VAL09-37 · Hash del token sin datos: no falla', () => {
    expect(validar(undefined, {}).salida[0].json.valido).toBe(false);
  });

  test('UT-VAL09-38 · consulta Hash del token y Verifica token (verify)', () => {
    const { $ } = validar(MEDICO);
    expect($).toHaveBeenCalledWith('Hash del token');
    expect($).toHaveBeenCalledWith('Verifica token');
  });

  test('UT-VAL09-39 · devuelve un item con los once campos', () => {
    const { salida } = validar(MEDICO);
```

```
    expect(salida).toHaveLength(1);
    expect(Object.keys(salida[0].json).sort()).toEqual([
      'accion', 'baja', 'colegiado', 'entidad', 'errores', 'especialidad', 'id',
'nombre', 'rol', 'ruta', 'valido'
    ]);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf09.validadatos.test.js
 FAIL  tests/sigac.wf09.validadatos.test.js
  WF-09 · Valida datos · A. Accion y entidad
    ✓ UT-VAL09-01 · accion inexistente: error (5 ms)
    ✓ UT-VAL09-02 · entidad inexistente: error
    ✓ UT-VAL09-03 · cuerpo vacio: los dos errores juntos (1 ms)
    ✓ UT-VAL09-04 · accion y entidad en mayusculas y con espacios: se aceptan
  WF-09 · Valida datos · B. Rutas
    ✓ UT-VAL09-05 · agregar recepcionista: ruta correcta
    ✓ UT-VAL09-05 · desactivar recepcionista: ruta correcta
    ✓ UT-VAL09-05 · dar_baja recepcionista: ruta correcta
    ✓ UT-VAL09-05 · reactivar recepcionista: ruta correcta
    ✓ UT-VAL09-05 · desactivar paciente: ruta correcta
    ✓ UT-VAL09-05 · dar_baja paciente: ruta correcta (1 ms)
    ✓ UT-VAL09-05 · reactivar paciente: ruta correcta
    ✓ UT-VAL09-05 · agregar medico: ruta correcta
    ✓ UT-VAL09-05 · desactivar medico: ruta correcta
    ✓ UT-VAL09-05 · dar_baja medico: ruta correcta (1 ms)
    ✓ UT-VAL09-05 · reactivar medico: ruta correcta
    ✓ UT-VAL09-06 · con errores, la ruta queda nula (1 ms)
    ✓ UT-VAL09-07 · dar_baja: bandera baja true
    ✓ UT-VAL09-08 · desactivar: bandera baja false (1 ms)
    ✓ UT-VAL09-09 · reactivar: bandera baja false
  WF-09 · Valida datos · C. Agregar
    ✓ UT-VAL09-10 · agregar paciente: error, nacen en WF-01 (1 ms)
    ✓ UT-VAL09-11 · medico completo: valido, con coma en el nombre
    ✓ UT-VAL09-12 · medico sin nombre: error
    ✓ UT-VAL09-13 · medico sin especialidad: error
    ✓ UT-VAL09-14 · medico sin colegiado: error (1 ms)
    ✓ UT-VAL09-15 · recepcionista solo necesita nombre
    ✓ UT-VAL09-16 · nombre de 120 caracteres
    ✓ UT-VAL09-17 · nombre de 121 caracteres
    ✓ UT-VAL09-18 · especialidad de 60 caracteres
    ✓ UT-VAL09-19 · especialidad de 61 caracteres (1 ms)
    ✓ UT-VAL09-20 · colegiado COL-90001
    ✓ UT-VAL09-21 · colegiado xxxxxxxxxxxxxxxxxx
    ✓ UT-VAL09-22 · colegiado xxxxxxxxxxxxxxxxxxx
    ✓ UT-VAL09-23 · colegiado COL 90001
    ✓ UT-VAL09-24 · colegiado COL_90001
    ✓ UT-VAL09-25 · nombre, especialidad y colegiado salen recortados (5 ms)
    ✓ UT-VAL09-26 · agregar no necesita id: queda nulo (1 ms)
  WF-09 · Valida datos · D. id
    ✓ UT-VAL09-27 · id como numero: valido (1 ms)
    ✓ UT-VAL09-28 · id como texto numerico: valido
    ✓ UT-VAL09-29 · id cero: error (1 ms)
    ✓ UT-VAL09-30 · id negativo: error
    ✓ UT-VAL09-31 · id decimal: error (1 ms)
    ✓ UT-VAL09-32 · id texto: error
    ✓ UT-VAL09-33 · id ausente: error (1 ms)
    ✕ UT-VAL09-34 · id true: debe rechazarse (hipotesis A) (2 ms)
    ✓ UT-VAL09-35 · desactivar ignora nombre, especialidad y colegiado (4 ms)
  WF-09 · Valida datos · E. Contrato
    ✓ UT-VAL09-36 · el rol sale de Verifica token
    ✓ UT-VAL09-37 · Hash del token sin datos: no falla
    ✓ UT-VAL09-38 · consulta Hash del token y Verifica token (verify) (4 ms)
    ✓ UT-VAL09-39 · devuelve un item con los once campos (1 ms)

  ● WF-09 · Valida datos · D. id › UT-VAL09-34 · id true: debe rechazarse (hipotesis A)
```

```
  ● WF-09 · Valida datos · D. id › UT-VAL09-34 · id true: debe rechazarse (hipotesis A)

    expect(received).toBe(expected) // Object.is equality

    Expected: false
    Received: true

      175 |   // hipotesis A: Number(true) es 1, asi que true desactivaria a la persona 1
      176 |   test('UT-VAL09-34 · id true: debe rechazarse (hipotesis A)', () => {
    > 177 |     expect(salidaDe({ accion: 'dar_baja', entidad: 'medico', id: true }).valido).toBe(false);
          |                                                                                  ^
      178 |   });
      179 |
      180 |   test('UT-VAL09-35 · desactivar ignora nombre, especialidad y colegiado', () => {

      at Object.toBe (tests/sigac.wf09.validadatos.test.js:177:82)

Test Suites: 1 failed, 1 total
Tests:       1 failed, 48 passed, 49 total
Snapshots:   0 total
Time:        3.705 s
Ran all test suites matching tests/sigac.wf09.validadatos.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.39.2.1 Defecto detectado por las pruebas (12.24).

La prueba UT-VAL09-34 falló: el nodo convierte el identificador con Number(), y Number(true) es 1, de modo que un true en lugar de un número se acepta como el identificador 1. Es el mismo defecto que el 12.23 de WF-01. Se corrige en n8n y, tras exportar el flujo y regenerar los nodos, la misma prueba confirma la corrección.

npx jest tests/sigac.wf09.validadatos.test.js

1.39.3 Nuevo código base de Valida Datos.

```
const datos = $('Hash del token').first().json.datos || {};
const errores = [];

const acciones  = ['agregar', 'desactivar', 'dar_baja', 'reactivar'];
const entidades = ['paciente', 'medico', 'recepcionista'];

const accion  = String(datos.accion  || '').trim().toLowerCase();
const entidad = String(datos.entidad || '').trim().toLowerCase();

if (!acciones.includes(accion)) {
  errores.push('La accion debe ser agregar, desactivar, dar_baja o reactivar');
}
if (!entidades.includes(entidad)) {
  errores.push('La entidad debe ser paciente, medico o recepcionista');
}

// ---- agregar ----
let nombre = null;
```

```
let especialidad = null;
let colegiado = null;

if (accion === 'agregar') {
  if (entidad === 'paciente') {
    errores.push('Los pacientes se registran con su primera solicitud de cita, no
desde administracion');
  }

  if (entidad === 'medico' || entidad === 'recepcionista') {
    nombre = String(datos.nombre || '').trim() || null;
    if (!nombre) {
      errores.push('Falta el campo nombre');
    } else if (nombre.length > 120) {
      errores.push('El nombre no puede exceder 120 caracteres');
    }
  }

  if (entidad === 'medico') {
    especialidad = String(datos.especialidad || '').trim() || null;
    if (!especialidad) {
      errores.push('Falta el campo especialidad');
    } else if (especialidad.length > 60) {
      errores.push('La especialidad no puede exceder 60 caracteres');
    }

    // el colegiado es unico en la tabla medico. aqui solo se valida
    // la forma; si ya existe, lo detecta el insert y responde 409.
    colegiado = String(datos.colegiado || '').trim() || null;
    if (!colegiado) {
      errores.push('Falta el campo colegiado');
    } else if (!/^[A-Za-z0-9-]{1,20}$/.test(colegiado)) {
      errores.push('El colegiado debe tener hasta 20 letras, digitos o guiones');
    }
  }
}

// ---- desactivar, dar_baja, reactivar ----
// el id debe ser un entero positivo. aqui se rechaza con 400 en lugar
// de usar el centinela 0 de WF-04 y WF-05, porque este flujo ya tiene
// su propio rechazo de datos.
//
// defecto 12.24: antes se convertia con Number(), y Number(true) es 1,
// de modo que "id": true desactivaba o daba de baja a la persona 1. ahora
// solo se acepta un numero o un texto que contenga un entero positivo,
```

```
// igual que id_medico_solicitado en WF-01 (defecto 12.23).
let id = null;
if (['desactivar', 'dar_baja', 'reactivar'].includes(accion)) {
  const valorId = (typeof datos.id === 'number' || typeof datos.id === 'string')
    ? String(datos.id).trim()
    : '';
  if (/^[1-9]\d*$/.test(valorId)) {
    id = Number(valorId);
  } else {
    errores.push('El id debe ser un entero positivo');
  }
}

// la ruta es la salida que tomara el Switch. desactivar y dar_baja
// comparten camino: solo difieren en la bandera baja, que decide si
// se escribe baja_en.
let ruta = null;
if (errores.length === 0) {
  const tramo = accion === 'agregar'   ? 'agregar'
              : accion === 'reactivar' ? 'reactivar'
              : 'estado';
  ruta = entidad + '_' + tramo;
}

return [{
  json: {
    valido: errores.length === 0,
    errores: errores,
    rol: $('Verifica token').first().json.rol,
    ruta: ruta,
    accion: accion,
    entidad: entidad,
    id: id,
    baja: accion === 'dar_baja',
    nombre: nombre,
    especialidad: especialidad,
    colegiado: colegiado
  }
}];
```

1.39.4 Se corrió el mismo código test pero ya con la corrección del código base.

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf09.validadatos.test.js
 PASS  tests/sigac.wf09.validadatos.test.js
  WF-09 · Valida datos · A. Accion y entidad
    ✓ UT-VAL09-01 · accion inexistente: error (5 ms)
    ✓ UT-VAL09-02 · entidad inexistente: error (1 ms)
    ✓ UT-VAL09-03 · cuerpo vacio: los dos errores juntos (4 ms)
    ✓ UT-VAL09-04 · accion y entidad en mayusculas y con espacios: se aceptan (1 ms)
  WF-09 · Valida datos · B. Rutas
    ✓ UT-VAL09-05 · agregar recepcionista: ruta correcta
    ✓ UT-VAL09-05 · desactivar recepcionista: ruta correcta
    ✓ UT-VAL09-05 · dar_baja recepcionista: ruta correcta
    ✓ UT-VAL09-05 · reactivar recepcionista: ruta correcta
    ✓ UT-VAL09-05 · desactivar paciente: ruta correcta
    ✓ UT-VAL09-05 · dar_baja paciente: ruta correcta
    ✓ UT-VAL09-05 · reactivar paciente: ruta correcta
    ✓ UT-VAL09-05 · agregar medico: ruta correcta (1 ms)
    ✓ UT-VAL09-05 · desactivar medico: ruta correcta
    ✓ UT-VAL09-05 · dar_baja medico: ruta correcta (1 ms)
    ✓ UT-VAL09-05 · reactivar medico: ruta correcta (2 ms)
    ✓ UT-VAL09-06 · con errores, la ruta queda nula
    ✓ UT-VAL09-07 · dar_baja: bandera baja true (2 ms)
    ✓ UT-VAL09-08 · desactivar: bandera baja false
    ✓ UT-VAL09-09 · reactivar: bandera baja false
  WF-09 · Valida datos · C. Agregar
    ✓ UT-VAL09-10 · agregar paciente: error, nacen en WF-01 (1 ms)
    ✓ UT-VAL09-11 · medico completo: valido, con coma en el nombre
    ✓ UT-VAL09-12 · medico sin nombre: error
    ✓ UT-VAL09-13 · medico sin especialidad: error (1 ms)
    ✓ UT-VAL09-14 · medico sin colegiado: error
    ✓ UT-VAL09-15 · recepcionista solo necesita nombre
    ✓ UT-VAL09-16 · nombre de 120 caracteres
    ✓ UT-VAL09-17 · nombre de 121 caracteres
    ✓ UT-VAL09-18 · especialidad de 60 caracteres
    ✓ UT-VAL09-19 · especialidad de 61 caracteres
    ✓ UT-VAL09-20 · colegiado COL-90001
    ✓ UT-VAL09-21 · colegiado xxxxxxxxxxxxxxxxxxxx (1 ms)
    ✓ UT-VAL09-22 · colegiado xxxxxxxxxxxxxxxxxxxxx
    ✓ UT-VAL09-23 · colegiado COL 90001
    ✓ UT-VAL09-24 · colegiado COL_90001
    ✓ UT-VAL09-25 · nombre, especialidad y colegiado salen recortados (1 ms)
    ✓ UT-VAL09-26 · agregar no necesita id: queda nulo (1 ms)
  WF-09 · Valida datos · D. id
    ✓ UT-VAL09-27 · id como numero: valido (1 ms)
    ✓ UT-VAL09-28 · id como texto numerico: valido (1 ms)
    ✓ UT-VAL09-29 · id cero: error
    ✓ UT-VAL09-30 · id negativo: error (1 ms)
    ✓ UT-VAL09-31 · id decimal: error
    ✓ UT-VAL09-32 · id texto: error (1 ms)
    ✓ UT-VAL09-33 · id ausente: error
    ✓ UT-VAL09-34 · id true: debe rechazarse (hipotesis A)
    ✓ UT-VAL09-35 · desactivar ignora nombre, especialidad y colegiado
  WF-09 · Valida datos · E. Contrato
    ✓ UT-VAL09-36 · el rol sale de Verifica token
    ✓ UT-VAL09-37 · Hash del token sin datos: no falla
    ✓ UT-VAL09-38 · consulta Hash del token y Verifica token (verify) (1 ms)
    ✓ UT-VAL09-39 · devuelve un item con los once campos (1 ms)

Test Suites: 1 passed, 1 total
Tests:       49 passed, 49 total
Snapshots:   0 total
Time:        4.273 s
Ran all test suites matching tests/sigac.wf09.validadatos.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.40 Rechazo 400 (WF-09)

El nodo Rechazo 400 de WF-09 responde cuando la petición de administración no pasa la validación. Devuelve completa la lista de errores que recibió de Valida datos.

1.40.1 Código base del nodo Rechazo 400

```
return [{ json: { codigo: 400, estado: 'rechazado', mensaje: 'Datos invalidos',
errores: $input.first().json.errores } }];
```

1.40.2 Código de test del nodo Rechazo 400

```
// sigac.wf09.rechazo400.test.js
//
// pruebas unitarias del nodo Rechazo 400 de WF-09. responde cuando
// Valida datos encontro errores, y devuelve la lista completa para
// que el emisor sepa que corregir. lee los errores de $input.first().json,
// que es la salida de Valida datos por la rama false de ¿Datos validos?
//
// cada prueba lleva su ID de la tabla de pruebas: UT-R40009-NN

'use strict';

const rechazo400 = require('../nodos/wf09_rechazo_400.js');
const { ejecutar } = require('../simulacion.js');

const ERRORES = ['El DPI debe tener exactamente 13 digitos', 'El correo no tiene
un formato valido'];

// ejecuta el nodo con la salida de Valida datos, como $input y como $json
function rechazar(desdeValida) {
  return ejecutar(rechazo400, { entrada: [desdeValida], json: desdeValida });
}

describe('WF-09 · Rechazo 400', () => {
  test('UT-R40009-01 · respuesta completa', () => {
    expect(rechazar({ valido: false, errores: ERRORES
}).salida[0].json).toEqual({
      codigo: 400,
      estado: 'rechazado',
      mensaje: 'Datos invalidos',
      errores: ERRORES
    });
  });

  test('UT-R40009-02 · codigo 400', () => {
    expect(rechazar({ errores: ERRORES }).salida[0].json.codigo).toBe(400);
  });

  test('UT-R40009-03 · estado rechazado (defecto 12.17)', () => {
```

```
    expect(rechazar({ errores: ERRORES
}).salida[0].json.estado).toBe('rechazado');
  });

  test('UT-R40009-04 · mensaje Datos invalidos', () => {
    expect(rechazar({ errores: ERRORES }).salida[0].json.mensaje).toBe('Datos
invalidos');
  });

  test('UT-R40009-05 · la lista de errores llega intacta', () => {
    expect(rechazar({ errores: ERRORES
}).salida[0].json.errores).toEqual(ERRORES);
  });

  test('UT-R40009-06 · el orden de los errores se conserva', () => {
    const invertidos = [...ERRORES].reverse();
    expect(rechazar({ errores: invertidos
}).salida[0].json.errores).toEqual(invertidos);
  });

  test('UT-R40009-07 · un solo error: lista de un elemento', () => {
    expect(rechazar({ errores: ['Falta el campo nombre']
}).salida[0].json.errores)
      .toEqual(['Falta el campo nombre']);
  });

  test('UT-R40009-08 · lista vacia: errores queda vacio', () => {
    expect(rechazar({ errores: [] }).salida[0].json.errores).toEqual([]);
  });

  test('UT-R40009-09 · sin errores en la entrada: queda indefinido, sin error',
() => {
    expect(rechazar({ valido: false }).salida[0].json.errores).toBeUndefined();
  });

  test('UT-R40009-10 · con datos del paciente en la entrada: solo los cuatro
campos', () => {
    const conDatos = { errores: ERRORES, datos: { dpi: '1234567890123', correo:
'ana@correo.com' } };
    const { salida } = rechazar(conDatos);
    expect(Object.keys(salida[0].json).sort()).toEqual(['codigo', 'errores',
'estado', 'mensaje']);
    expect(JSON.stringify(salida)).not.toContain('1234567890123');
  });
```

```
  test('UT-R40009-11 · no consulta ningun nodo anterior (verify)', () => {
    const { $ } = rechazar({ errores: ERRORES });
    expect($).not.toHaveBeenCalled();
  });

  test('UT-R40009-12 · devuelve una lista con un solo item', () => {
    const { salida } = rechazar({ errores: ERRORES });
    expect(Array.isArray(salida)).toBe(true);
    expect(salida).toHaveLength(1);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf09.rechazo400.test.js
 PASS  tests/sigac.wf09.rechazo400.test.js
  WF-09 · Rechazo 400
    ✓ UT-R40009-01 · respuesta completa (5 ms)
    ✓ UT-R40009-02 · codigo 400
    ✓ UT-R40009-03 · estado rechazado (defecto 12.17) (3 ms)
    ✓ UT-R40009-04 · mensaje Datos invalidos (1 ms)
    ✓ UT-R40009-05 · la lista de errores llega intacta
    ✓ UT-R40009-06 · el orden de los errores se conserva
    ✓ UT-R40009-07 · un solo error: lista de un elemento
    ✓ UT-R40009-08 · lista vacia: errores queda vacio (1 ms)
    ✓ UT-R40009-09 · sin errores en la entrada: queda indefinido, sin error
    ✓ UT-R40009-10 · con datos del paciente en la entrada: solo los cuatro campos
    ✓ UT-R40009-11 · no consulta ningun nodo anterior (verify)
    ✓ UT-R40009-12 · devuelve una lista con un solo item (1 ms)

Test Suites: 1 passed, 1 total
Tests:       12 passed, 12 total
Snapshots:   0 total
Time:        3.845 s
Ran all test suites matching tests/sigac.wf09.rechazo400.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.41 Rechazo 401 (WF-09)

El nodo Rechazo 401 de WF-09 responde cuando el token no viene, no existe o está desactivado. El mensaje es genérico a propósito. Es una respuesta fija.

1.41.1 Código base del nodo Rechazo 401

```
return [{ json: { codigo: 401, estado: 'rechazado', mensaje: 'No autorizado' }
}];
```

1.41.2 Código de test del nodo Rechazo 401

```
// sigac.wf09.rechazo401.test.js
//
// pruebas unitarias del nodo Rechazo 401 de WF-09. responde cuando el token no
viene, no existe o esta desactivado. lo
// alimenta la salida false de ¿Token valido?.
//
// el mensaje es generico a proposito: no revela cual de los tres casos
// ocurrio, para no ayudar a quien intenta adivinar tokens validos.
//
// es una respuesta fija: no lee la entrada ni consulta otros nodos.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-R40109-NN

'use strict';

const rechazo = require('../nodos/wf09_rechazo_401.js');
const { ejecutar } = require('../simulacion.js');

const RESPUESTA = {
  codigo: 401,
  estado: 'rechazado',
  mensaje: 'No autorizado'
};

// lo que puede llegar al nodo: la salida del nodo anterior junto con
// lo que traia el flujo, incluida informacion que no debe devolverse
const CON_DATOS = {
  token_hash: 'a'.repeat(64),
  rol: 'medico',
  id_cita: 3432,
  datos: { dpi: '1234567890123', correo: 'ana@correo.com' }
};

describe('WF-09 · Rechazo 401', () => {
  test('UT-R40109-01 · respuesta completa', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json).toEqual(RESPUESTA);
  });
```

```
  test('UT-R40109-02 · codigo 401', () => {
    expect(ejecutar(rechazo, { entrada: [{}] }).salida[0].json.codigo).toBe(401);
  });

  test('UT-R40109-03 · estado rechazado (defecto 12.17)', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.estado).toBe('rechazado');
  });

  test('UT-R40109-04 · mensaje No autorizado', () => {
    expect(ejecutar(rechazo, { entrada: [{}] }).salida[0].json.mensaje).toBe('No
autorizado');
  });

  test('UT-R40109-05 · solo los campos codigo, estado y mensaje', () => {
    const { salida } = ejecutar(rechazo, { entrada: [{}] });
    expect(Object.keys(salida[0].json).sort()).toEqual(['codigo', 'estado',
'mensaje']);
  });

  test('UT-R40109-06 · sin entrada: la misma respuesta', () => {
    expect(ejecutar(rechazo).salida[0].json).toEqual(RESPUESTA);
  });

  test('UT-R40109-07 · con datos en la entrada: la misma respuesta, sin
devolverlos', () => {
    const { salida } = ejecutar(rechazo, { entrada: [CON_DATOS], json: CON_DATOS
});
    expect(salida[0].json).toEqual(RESPUESTA);
    const texto = JSON.stringify(salida);
    expect(texto).not.toContain('a'.repeat(64));
    expect(texto).not.toContain('1234567890123');
    expect(texto).not.toContain('3432');
  });

  test('UT-R40109-08 · no consulta ningun nodo anterior (verify)', () => {
    const { $ } = ejecutar(rechazo, { entrada: [{}] });
    expect($).not.toHaveBeenCalled();
  });

  test('UT-R40109-09 · dos ejecuciones con entradas distintas dan respuestas
identicas', () => {
    const a = ejecutar(rechazo, { entrada: [{ rol: 'recepcion' }], json: { rol:
'recepcion' } }).salida;
```

```
    const b = ejecutar(rechazo, { entrada: [CON_DATOS], json: CON_DATOS
}).salida;
    expect(a).toEqual(b);
  });

  test('UT-R40109-10 · el mensaje no revela la causa del rechazo', () => {
    const mensaje = ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.mensaje.toLowerCase();
    for (const pista of ['inactivo', 'desactivado', 'expirado', 'rol']) {
      expect(mensaje).not.toContain(pista);
    }
  });

  test('UT-R40109-11 · devuelve una lista con un solo item', () => {
    const { salida } = ejecutar(rechazo, { entrada: [{}] });
    expect(Array.isArray(salida)).toBe(true);
    expect(salida).toHaveLength(1);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf09.rechazo401.test.js
 PASS  tests/sigac.wf09.rechazo401.test.js
  WF-09 · Rechazo 401
    ✓ UT-R40109-01 · respuesta completa (5 ms)
    ✓ UT-R40109-02 · codigo 401 (4 ms)
    ✓ UT-R40109-03 · estado rechazado (defecto 12.17) (1 ms)
    ✓ UT-R40109-04 · mensaje No autorizado
    ✓ UT-R40109-05 · solo los campos codigo, estado y mensaje (1 ms)
    ✓ UT-R40109-06 · sin entrada: la misma respuesta
    ✓ UT-R40109-07 · con datos en la entrada: la misma respuesta, sin devolverlos
    ✓ UT-R40109-08 · no consulta ningun nodo anterior (verify)
    ✓ UT-R40109-09 · dos ejecuciones con entradas distintas dan respuestas identicas (1 ms)
    ✓ UT-R40109-10 · el mensaje no revela la causa del rechazo (1 ms)
    ✓ UT-R40109-11 · devuelve una lista con un solo item

Test Suites: 1 passed, 1 total
Tests:       11 passed, 11 total
Snapshots:   0 total
Time:        3.752 s
Ran all test suites matching tests/sigac.wf09.rechazo401.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.42 Rechazo 403 (WF-09)

El nodo Rechazo 403 de WF-09 responde cuando el token es válido pero no es de administrador: solo el admin puede administrar personas. El mensaje no revela qué rol se necesita. Es una respuesta fija.

1.42.1 Código base del nodo Rechazo 403

```
return [{ json: { codigo: 403, estado: 'rechazado', mensaje: 'Operacion no
permitida para este rol' } }];
```

1.42.2 Código de test del nodo Rechazo 403

```
// sigac.wf09.rechazo403.test.js
//
// pruebas unitarias del nodo Rechazo 403 de WF-09. responde cuando el token es
valido pero no es de administrador: solo
// el admin puede administrar personas. lo alimenta la salida false de
// ¿Es admin?.
//
// el mensaje no revela que rol se necesita ni cual tiene quien llama.
//
// es una respuesta fija: no lee la entrada ni consulta otros nodos.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-R40309-NN

'use strict';

const rechazo = require('../nodos/wf09_rechazo_403.js');
const { ejecutar } = require('../simulacion.js');

const RESPUESTA = {
  codigo: 403,
  estado: 'rechazado',
  mensaje: 'Operacion no permitida para este rol'
};

// lo que puede llegar al nodo: la salida del nodo anterior junto con
// lo que traia el flujo, incluida informacion que no debe devolverse
const CON_DATOS = {
  token_hash: 'a'.repeat(64),
  rol: 'medico',
  id_cita: 3432,
  datos: { dpi: '1234567890123', correo: 'ana@correo.com' }
};

describe('WF-09 · Rechazo 403', () => {
  test('UT-R40309-01 · respuesta completa', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json).toEqual(RESPUESTA);
```

```
  });

  test('UT-R40309-02 · codigo 403', () => {
    expect(ejecutar(rechazo, { entrada: [{}] }).salida[0].json.codigo).toBe(403);
  });

  test('UT-R40309-03 · estado rechazado (defecto 12.17)', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.estado).toBe('rechazado');
  });

  test('UT-R40309-04 · mensaje Operacion no permitida para este rol', () => {
    expect(ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.mensaje).toBe('Operacion no permitida para este rol');
  });

  test('UT-R40309-05 · solo los campos codigo, estado y mensaje', () => {
    const { salida } = ejecutar(rechazo, { entrada: [{}] });
    expect(Object.keys(salida[0].json).sort()).toEqual(['codigo', 'estado',
'mensaje']);
  });

  test('UT-R40309-06 · sin entrada: la misma respuesta', () => {
    expect(ejecutar(rechazo).salida[0].json).toEqual(RESPUESTA);
  });

  test('UT-R40309-07 · con datos en la entrada: la misma respuesta, sin
devolverlos', () => {
    const { salida } = ejecutar(rechazo, { entrada: [CON_DATOS], json: CON_DATOS
});
    expect(salida[0].json).toEqual(RESPUESTA);
    const texto = JSON.stringify(salida);
    expect(texto).not.toContain('a'.repeat(64));
    expect(texto).not.toContain('1234567890123');
    expect(texto).not.toContain('3432');
  });

  test('UT-R40309-08 · no consulta ningun nodo anterior (verify)', () => {
    const { $ } = ejecutar(rechazo, { entrada: [{}] });
    expect($).not.toHaveBeenCalled();
  });

  test('UT-R40309-09 · dos ejecuciones con entradas distintas dan respuestas
identicas', () => {
```

```
    const a = ejecutar(rechazo, { entrada: [{ rol: 'recepcion' }], json: { rol:
'recepcion' } }).salida;
    const b = ejecutar(rechazo, { entrada: [CON_DATOS], json: CON_DATOS
}).salida;
    expect(a).toEqual(b);
  });

  test('UT-R40309-10 · el mensaje no revela roles', () => {
    const mensaje = ejecutar(rechazo, { entrada: [{}]
}).salida[0].json.mensaje.toLowerCase();
    for (const pista of ['recepcion', 'medico', 'admin']) {
      expect(mensaje).not.toContain(pista);
    }
  });

  test('UT-R40309-11 · devuelve una lista con un solo item', () => {
    const { salida } = ejecutar(rechazo, { entrada: [{}] });
    expect(Array.isArray(salida)).toBe(true);
    expect(salida).toHaveLength(1);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf09.rechazo403.test.js
 PASS  tests/sigac.wf09.rechazo403.test.js
  WF-09 · Rechazo 403
    ✓ UT-R40309-01 · respuesta completa (5 ms)
    ✓ UT-R40309-02 · codigo 403
    ✓ UT-R40309-03 · estado rechazado (defecto 12.17)
    ✓ UT-R40309-04 · mensaje Operacion no permitida para este rol (1 ms)
    ✓ UT-R40309-05 · solo los campos codigo, estado y mensaje
    ✓ UT-R40309-06 · sin entrada: la misma respuesta (1 ms)
    ✓ UT-R40309-07 · con datos en la entrada: la misma respuesta, sin devolverlos (1 ms)
    ✓ UT-R40309-08 · no consulta ningun nodo anterior (verify)
    ✓ UT-R40309-09 · dos ejecuciones con entradas distintas dan respuestas identicas (1 ms)
    ✓ UT-R40309-10 · el mensaje no revela roles (1 ms)
    ✓ UT-R40309-11 · devuelve una lista con un solo item (1 ms)

Test Suites: 1 passed, 1 total
Tests:       11 passed, 11 total
Snapshots:   0 total
Time:        3.605 s
Ran all test suites matching tests/sigac.wf09.rechazo403.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.43 Genera token (WF-09)

El nodo Genera token, de la ruta recepcionista_agregar, genera el token de la recepcionista nueva: 32 bytes aleatorios en hexadecimal y su huella SHA-256. A la base solo llega la huella; el token en claro viaja una sola vez, en la respuesta, y el flujo no guarda ejecuciones exitosas.

1.43.1 Código base del nodo Genera token

```
const crypto = require('crypto');

const token = crypto.randomBytes(32).toString('hex');
const tokenHash = crypto.createHash('sha256').update(token).digest('hex');

return [{
  json: {
    ...$input.first().json,
    token: token,
    token_hash: tokenHash
  }
}];
```

1.43.2 Código de test del nodo Genera token

```
// sigac.wf09.generatoken.test.js
//
// pruebas unitarias del nodo Genera token de WF-09, en la ruta
// recepcionista_agregar. genera el token de la persona: 32 bytes aleatorios en
// hexadecimal y su huella sha-256. a la base solo va la huella; el
// token en claro viaja una sola vez, en la respuesta.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-GTK09-NN

'use strict';

const generaToken = require('../nodos/wf09_genera_token.js');
const { ejecutar, sha256 } = require('../simulacion.js');

const DESDE_VALIDA = { valido: true, ruta: 'recepcionista_agregar', id: 3,
nombre: 'Rosa' };
const generar = (entrada = DESDE_VALIDA) => ejecutar(generaToken, { entrada:
[entrada] });
const salidaDe = entrada => generar(entrada).salida[0].json;

describe('WF-09 · Genera token', () => {
  test('UT-GTK09-01 · el token tiene 64 caracteres hexadecimales (32 bytes)', ()
=> {
    expect(salidaDe().token).toMatch(/^[0-9a-f]{64}$/);
  });

  test('UT-GTK09-02 · la huella es el sha-256 del token', () => {
```

```
    const salida = salidaDe();
    expect(salida.token_hash).toBe(sha256(salida.token));
  });

  test('UT-GTK09-03 · la huella tiene 64 caracteres hexadecimales', () => {
    expect(salidaDe().token_hash).toMatch(/^[0-9a-f]{64}$/);
  });

  test('UT-GTK09-04 · la huella es distinta del token', () => {
    const salida = salidaDe();
    expect(salida.token_hash).not.toBe(salida.token);
  });

  test('UT-GTK09-05 · dos ejecuciones dan tokens distintos', () => {
    expect(salidaDe().token).not.toBe(salidaDe().token);
  });

  test('UT-GTK09-06 · 100 tokens seguidos: ninguno se repite', () => {
    const tokens = new Set(Array.from({ length: 100 }, () => salidaDe().token));
    expect(tokens.size).toBe(100);
  });

  test('UT-GTK09-07 · conserva los campos que traia de Valida datos', () => {
    const salida = salidaDe();
    expect(salida.ruta).toBe('recepcionista_agregar');
    expect(salida.id).toBe(3);
    expect(salida.nombre).toBe('Rosa');
  });

  test('UT-GTK09-08 · un token que viniera en la entrada se reemplaza', () => {
    const salida = salidaDe({ ...DESDE_VALIDA, token: 'impuesto', token_hash:
'impuesto' });
    expect(salida.token).not.toBe('impuesto');
    expect(salida.token_hash).toBe(sha256(salida.token));
  });

  test('UT-GTK09-09 · entrada vacia: genera igual, sin error', () => {
    expect(salidaDe({}).token).toMatch(/^[0-9a-f]{64}$/);
  });

  test('UT-GTK09-10 · no consulta ningun nodo anterior (verify)', () => {
    expect(generar().$).not.toHaveBeenCalled();
  });

  test('UT-GTK09-11 · devuelve una lista con un solo item', () => {
```

```
    expect(generar().salida).toHaveLength(1);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf09.generatoken.test.js
 PASS  tests/sigac.wf09.generatoken.test.js
  WF-09 · Genera token
    ✓ UT-GTK09-01 · el token tiene 64 caracteres hexadecimales (32 bytes) (5 ms)
    ✓ UT-GTK09-02 · la huella es el sha-256 del token
    ✓ UT-GTK09-03 · la huella tiene 64 caracteres hexadecimales (1 ms)
    ✓ UT-GTK09-04 · la huella es distinta del token (1 ms)
    ✓ UT-GTK09-05 · dos ejecuciones dan tokens distintos
    ✓ UT-GTK09-06 · 100 tokens seguidos: ninguno se repite (2 ms)
    ✓ UT-GTK09-07 · conserva los campos que traia de Valida datos (1 ms)
    ✓ UT-GTK09-08 · un token que viniera en la entrada se reemplaza (2 ms)
    ✓ UT-GTK09-09 · entrada vacia: genera igual, sin error (2 ms)
    ✓ UT-GTK09-10 · no consulta ningun nodo anterior (verify) (1 ms)
    ✓ UT-GTK09-11 · devuelve una lista con un solo item (1 ms)

Test Suites: 1 passed, 1 total
Tests:       11 passed, 11 total
Snapshots:   0 total
Time:        3.961 s
Ran all test suites matching tests/sigac.wf09.generatoken.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.44 Recepcionista agregada (WF-09)

El nodo Recepcionista agregada arma la respuesta 201 con el identificador de la recepcionista nueva, su token en claro (la única vez que se muestra) y el hash de auditoría. El identificador y el hash salen del nodo Agrega recepcionista; el token, del nodo Genera token. La huella del token nunca se devuelve.

1.44.1 Código base del nodo Recepcionista agregada

```
const r = $input.first().json;

return [{
  json: {
    codigo: 201,
    estado: 'registrado',
    mensaje: 'Recepcionista agregada',
    id_recepcionista: r.id_recepcionista,
    token: $('Genera token').first().json.token,
    aviso: 'Guarde este token ahora: no se volvera a mostrar',
    hash_auditoria: r.hash_auditoria
  }
}];
```

1.44.2 Código de test del nodo Recepcionista agregada

```
// sigac.wf09.recepcionistaagregada.test.js
//
// pruebas unitarias del nodo Recepcionista agregada de WF-09. arma la
// respuesta 201 con el id de la recepcionista nueva, su token en claro
// (la unica vez que se muestra) y el hash de auditoria. el id y el hash
// salen de Agrega recepcionista ($input); el token, de Genera token.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-RAG09-NN
'use strict';
const recepcionistaAgregada = require('../nodos/wf09_recepcionista_agregada.js');
const { ejecutar } = require('../simulacion.js');

const TOKEN = 'c'.repeat(64);
const AGREGADA = { id_recepcionista: 3, hash_auditoria: 'abc123' };

const responder = (agregada = AGREGADA, generado = { token: TOKEN, token_hash:
'h'.repeat(64) }) =>
  ejecutar(recepcionistaAgregada, { entrada: [agregada], anteriores: { 'Genera
token': generado } });

describe('WF-09 · Recepcionista agregada', () => {
  test('UT-RAG09-01 · respuesta completa', () => {
    expect(responder().salida[0].json).toEqual({
      codigo: 201,
      estado: 'registrado',
```

```
      mensaje: 'Recepcionista agregada',
      id_recepcionista: 3,
      token: TOKEN,
      aviso: 'Guarde este token ahora: no se volvera a mostrar',
      hash_auditoria: 'abc123'
    });
  });

  test('UT-RAG09-02 · el id sale de Agrega recepcionista', () => {
    expect(responder({ ...AGREGADA, id_recepcionista: 99
}).salida[0].json.id_recepcionista).toBe(99);
  });

  test('UT-RAG09-03 · el token sale de Genera token', () => {
    expect(responder(AGREGADA, { token: 'd'.repeat(64)
}).salida[0].json.token).toBe('d'.repeat(64));
  });

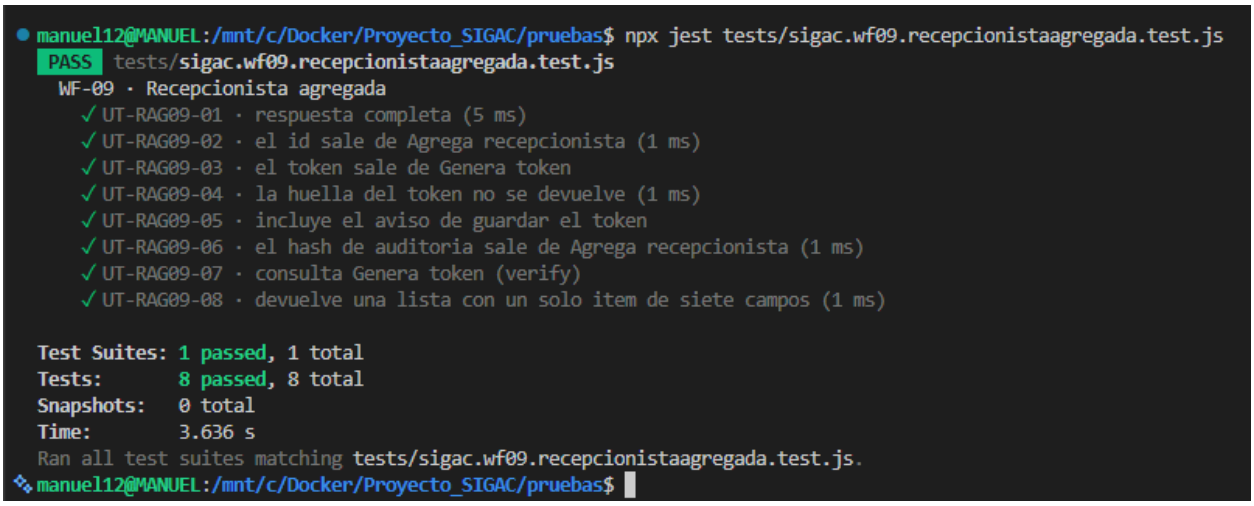
  test('UT-RAG09-04 · la huella del token no se devuelve', () => {
    const texto = JSON.stringify(responder().salida);
    expect(texto).not.toContain('h'.repeat(64));
  });

  test('UT-RAG09-05 · incluye el aviso de guardar el token', () => {
    expect(responder().salida[0].json.aviso).toContain('no se volvera a
mostrar');
  });

  test('UT-RAG09-06 · el hash de auditoria sale de Agrega recepcionista', () => {
    expect(responder({ ...AGREGADA, hash_auditoria: 'f'.repeat(64)
}).salida[0].json.hash_auditoria).toBe('f'.repeat(64));
  });

  test('UT-RAG09-07 · consulta Genera token (verify)', () => {
    expect(responder().$).toHaveBeenCalledWith('Genera token');
  });

  test('UT-RAG09-08 · devuelve una lista con un solo item de siete campos', () =>
{
    const { salida } = responder();
    expect(salida).toHaveLength(1);
    expect(Object.keys(salida[0].json)).toHaveLength(7);
  });
});
```


manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf09.recepcionistaagregada.test.js
PASS tests/sigac.wf09.recepcionistaagregada.test.js
WF-09 · Recepcionista agregada
✓ UT-RAG09-01 · respuesta completa (5 ms)
✓ UT-RAG09-02 · el id sale de Agrega recepcionista (1 ms)
✓ UT-RAG09-03 · el token sale de Genera token
✓ UT-RAG09-04 · la huella del token no se devuelve (1 ms)
✓ UT-RAG09-05 · incluye el aviso de guardar el token
✓ UT-RAG09-06 · el hash de auditoria sale de Agrega recepcionista (1 ms)
✓ UT-RAG09-07 · consulta Genera token (verify)
✓ UT-RAG09-08 · devuelve una lista con un solo item de siete campos (1 ms)
Test Suites: 1 passed, 1 total
Tests: 8 passed, 8 total
Snapshots: 0 total
Time: 3.636 s
Ran all test suites matching tests/sigac.wf09.recepcionistaagregada.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$

1.45 Estado recepcionista (WF-09)

El nodo Estado recepcionista traduce el resultado del nodo Cambia estado recepcionista al código de respuesta: no_existe da 404, ya_baja da 409 y ok da 200, con el mensaje de desactivada o dada de baja y la cantidad de tokens desactivados. El conteo llega como texto desde PostgreSQL y se convierte a número.

1.45.1 Código base del nodo Estado recepcionista

```
const r = $input.first().json;
const id = $('Valida datos').first().json.id;

if (r.resultado === 'no_existe') {
  return [{ json: { codigo: 404, estado: 'rechazado', mensaje: 'No existe la
recepcionista ' + id } }];
}

if (r.resultado === 'ya_baja') {
  return [{ json: { codigo: 409, estado: 'rechazado', mensaje: 'La recepcionista
' + id + ' ya fue dada de baja; la baja es definitiva' } }];
}

return [{
  json: {
    codigo: 200,
    estado: 'actualizado',
    mensaje: r.dada_de_baja
      ? 'Recepcionista ' + id + ' dada de baja'
      : 'Recepcionista ' + id + ' desactivada',
    tokens_desactivados: Number(r.tokens_desactivados),
    hash_auditoria: r.hash_auditoria
  }
}];
```

1.45.2 Código de test del nodo Estado recepcionista

```
// sigac.wf09.estadorecepcionista.test.js
//
// pruebas unitarias del nodo Estado recepcionista de WF-09. traduce el
// resultado de Cambia estado recepcionista al codigo de respuesta:
//   no_existe -> 404 · ya_baja -> 409 · ok -> 200 (desactivada o dada de baja)
//
// cada prueba lleva su ID de la tabla de pruebas: UT-ERC09-NN

'use strict';

const estadoRecepcionista = require('../nodos/wf09_estado_recepcionista.js');
const { ejecutar } = require('../simulacion.js');

const responder = (resultado, id = 3) =>
```

```
  ejecutar(estadoRecepcionista, { entrada: [resultado], anteriores: { 'Valida
datos': { id } } });
const salidaDe = (resultado, id) => responder(resultado, id).salida[0].json;

describe('WF-09 · Estado recepcionista', () => {
  test('UT-ERC09-01 · no_existe: 404 con el id', () => {
    expect(salidaDe({ resultado: 'no_existe' }, 99)).toEqual({
      codigo: 404, estado: 'rechazado', mensaje: 'No existe la recepcionista 99'
    });
  });

  test('UT-ERC09-02 · ya_baja: 409, la baja es definitiva', () => {
    expect(salidaDe({ resultado: 'ya_baja' }, 2)).toEqual({
      codigo: 409, estado: 'rechazado', mensaje: 'La recepcionista 2 ya fue dada
de baja; la baja es definitiva'
    });
  });

  test('UT-ERC09-03 · ok sin baja: 200, desactivada', () => {
    expect(salidaDe({ resultado: 'ok', dada_de_baja: false, tokens_desactivados:
'1', hash_auditoria: 'abc' }, 3)).toEqual({
      codigo: 200, estado: 'actualizado', mensaje: 'Recepcionista 3 desactivada',
tokens_desactivados: 1, hash_auditoria: 'abc'
    });
  });

  test('UT-ERC09-04 · ok con baja: 200, dada de baja', () => {
    expect(salidaDe({ resultado: 'ok', dada_de_baja: true, tokens_desactivados:
'1' }).mensaje).toBe('Recepcionista 3 dada de baja');
  });

  test('UT-ERC09-05 · tokens_desactivados llega como texto desde postgres: sale
como numero', () => {
    expect(salidaDe({ resultado: 'ok', dada_de_baja: false, tokens_desactivados:
'2' }).tokens_desactivados).toBe(2);
  });

  test('UT-ERC09-06 · ok sin tokens activos: tokens_desactivados 0', () => {
    expect(salidaDe({ resultado: 'ok', dada_de_baja: false, tokens_desactivados:
'0' }).tokens_desactivados).toBe(0);
  });

  test('UT-ERC09-07 · los rechazos no incluyen hash ni tokens', () => {
    expect(Object.keys(salidaDe({ resultado: 'no_existe'
})).sort()).toEqual(['codigo', 'estado', 'mensaje']);
```

```
    expect(Object.keys(salidaDe({ resultado: 'ya_baja'
})).sort()).toEqual(['codigo', 'estado', 'mensaje']);
  });

  test('UT-ERC09-08 · el id del mensaje sale de Valida datos (verify)', () => {
    const { $ } = responder({ resultado: 'no_existe' });
    expect($).toHaveBeenCalledWith('Valida datos');
  });

  test('UT-ERC09-09 · devuelve una lista con un solo item', () => {
    expect(responder({ resultado: 'ok', tokens_desactivados: '1'
}).salida).toHaveLength(1);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf09.estadorecepcionista.test.js
 PASS  tests/sigac.wf09.estadorecepcionista.test.js
  WF-09 · Estado recepcionista
    ✓ UT-ERC09-01 · no_existe: 404 con el id (5 ms)
    ✓ UT-ERC09-02 · ya_baja: 409, la baja es definitiva (1 ms)
    ✓ UT-ERC09-03 · ok sin baja: 200, desactivada
    ✓ UT-ERC09-04 · ok con baja: 200, dada de baja (2 ms)
    ✓ UT-ERC09-05 · tokens_desactivados llega como texto desde postgres: sale como numero
    ✓ UT-ERC09-06 · ok sin tokens activos: tokens_desactivados 0 (1 ms)
    ✓ UT-ERC09-07 · los rechazos no incluyen hash ni tokens
    ✓ UT-ERC09-08 · el id del mensaje sale de Valida datos (verify) (1 ms)
    ✓ UT-ERC09-09 · devuelve una lista con un solo item (1 ms)

Test Suites: 1 passed, 1 total
Tests:       9 passed, 9 total
Snapshots:   0 total
Time:        3.598 s
Ran all test suites matching tests/sigac.wf09.estadorecepcionista.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.46 Genera token nuevo (WF-09)

El nodo Genera token nuevo, de la ruta recepcionista_reactivar, genera el token nuevo de la recepcionista que se reactiva: 32 bytes aleatorios en hexadecimal y su huella SHA-256. El token anterior queda inactivo como rastro. Es una copia exacta del nodo Genera token.

1.46.1 Código base del nodo Genera token nuevo

```
const crypto = require('crypto');

const token = crypto.randomBytes(32).toString('hex');
const tokenHash = crypto.createHash('sha256').update(token).digest('hex');

return [{
  json: {
    ...$input.first().json,
    token: token,
    token_hash: tokenHash
  }
}];
```

1.46.2 Código de test del nodo Genera token nuevo

```
// sigac.wf09.generatokennuevo.test.js
//
// pruebas unitarias del nodo Genera token nuevo de WF-09, en la ruta
// recepcionista_reactivar. genera el token de la persona: 32 bytes aleatorios en
// hexadecimal y su huella sha-256. a la base solo va la huella; el
// token en claro viaja una sola vez, en la respuesta.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-GTN09-NN

'use strict';

const generaToken = require('../nodos/wf09_genera_token_nuevo.js');
const { ejecutar, sha256 } = require('../simulacion.js');

const DESDE_VALIDA = { valido: true, ruta: 'recepcionista_reactivar', id: 3,
nombre: 'Rosa' };
const generar = (entrada = DESDE_VALIDA) => ejecutar(generaToken, { entrada:
[entrada] });
const salidaDe = entrada => generar(entrada).salida[0].json;

describe('WF-09 · Genera token nuevo', () => {
  test('UT-GTN09-01 · el token tiene 64 caracteres hexadecimales (32 bytes)', ()
=> {
    expect(salidaDe().token).toMatch(/^[0-9a-f]{64}$/);
  });

  test('UT-GTN09-02 · la huella es el sha-256 del token', () => {
```

```
    const salida = salidaDe();
    expect(salida.token_hash).toBe(sha256(salida.token));
  });

  test('UT-GTN09-03 · la huella tiene 64 caracteres hexadecimales', () => {
    expect(salidaDe().token_hash).toMatch(/^[0-9a-f]{64}$/);
  });

  test('UT-GTN09-04 · la huella es distinta del token', () => {
    const salida = salidaDe();
    expect(salida.token_hash).not.toBe(salida.token);
  });

  test('UT-GTN09-05 · dos ejecuciones dan tokens distintos', () => {
    expect(salidaDe().token).not.toBe(salidaDe().token);
  });

  test('UT-GTN09-06 · 100 tokens seguidos: ninguno se repite', () => {
    const tokens = new Set(Array.from({ length: 100 }, () => salidaDe().token));
    expect(tokens.size).toBe(100);
  });

  test('UT-GTN09-07 · conserva los campos que traia de Valida datos', () => {
    const salida = salidaDe();
    expect(salida.ruta).toBe('recepcionista_reactivar');
    expect(salida.id).toBe(3);
    expect(salida.nombre).toBe('Rosa');
  });

  test('UT-GTN09-08 · un token que viniera en la entrada se reemplaza', () => {
    const salida = salidaDe({ ...DESDE_VALIDA, token: 'impuesto', token_hash:
'impuesto' });
    expect(salida.token).not.toBe('impuesto');
    expect(salida.token_hash).toBe(sha256(salida.token));
  });

  test('UT-GTN09-09 · entrada vacia: genera igual, sin error', () => {
    expect(salidaDe({}).token).toMatch(/^[0-9a-f]{64}$/);
  });

  test('UT-GTN09-10 · no consulta ningun nodo anterior (verify)', () => {
    expect(generar().$).not.toHaveBeenCalled();
  });

  test('UT-GTN09-11 · devuelve una lista con un solo item', () => {
```

```
    expect(generar().salida).toHaveLength(1);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf09.generatokennuevo.test.js
 PASS  tests/sigac.wf09.generatokennuevo.test.js
  WF-09 · Genera token nuevo
    ✓ UT-GTN09-01 · el token tiene 64 caracteres hexadecimales (32 bytes) (4 ms)
    ✓ UT-GTN09-02 · la huella es el sha-256 del token (1 ms)
    ✓ UT-GTN09-03 · la huella tiene 64 caracteres hexadecimales
    ✓ UT-GTN09-04 · la huella es distinta del token
    ✓ UT-GTN09-05 · dos ejecuciones dan tokens distintos (3 ms)
    ✓ UT-GTN09-06 · 100 tokens seguidos: ninguno se repite (4 ms)
    ✓ UT-GTN09-07 · conserva los campos que traia de Valida datos (1 ms)
    ✓ UT-GTN09-08 · un token que viniera en la entrada se reemplaza (1 ms)
    ✓ UT-GTN09-09 · entrada vacia: genera igual, sin error (1 ms)
    ✓ UT-GTN09-10 · no consulta ningun nodo anterior (verify) (1 ms)
    ✓ UT-GTN09-11 · devuelve una lista con un solo item (1 ms)

Test Suites: 1 passed, 1 total
Tests:       11 passed, 11 total
Snapshots:   0 total
Time:        3.716 s
Ran all test suites matching tests/sigac.wf09.generatokennuevo.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.47 Recepcionista reactivada (WF-09)

El nodo Recepcionista reactivada traduce el resultado del nodo Reactiva recepcionista: no_existe da 404, ya_baja y ya_activa dan 409, y ok da 200 con el token nuevo. Solo en el caso ok se devuelve el token: en los rechazos no se guardó nada y el token generado se descarta.

1.47.1 Código base del nodo Recepcionista reactivada

```
const r = $input.first().json;
const id = $('Valida datos').first().json.id;

if (r.resultado === 'no_existe') {
  return [{ json: { codigo: 404, estado: 'rechazado', mensaje: 'No existe la
recepcionista ' + id } }];
}

if (r.resultado === 'ya_baja') {
  return [{ json: { codigo: 409, estado: 'rechazado', mensaje: 'La recepcionista
' + id + ' fue dada de baja; la baja es definitiva' } }];
}

if (r.resultado === 'ya_activa') {
  return [{ json: { codigo: 409, estado: 'rechazado', mensaje: 'La recepcionista
' + id + ' ya esta activa' } }];
}

return [{
  json: {
    codigo: 200,
    estado: 'actualizado',
    mensaje: 'Recepcionista ' + id + ' reactivada con token nuevo',
    token: $('Genera token nuevo').first().json.token,
    aviso: 'Guarde este token ahora: no se volvera a mostrar. El token anterior
sigue desactivado',
    hash_auditoria: r.hash_auditoria
  }
}];
```

1.47.2 Código de test del nodo Recepcionista reactivada

```
// sigac.wf09.recepcionistareactivada.test.js
//
// pruebas unitarias del nodo Recepcionista reactivada de WF-09. traduce
// el resultado de Reactiva recepcionista al codigo de respuesta:
//   no_existe -> 404 · ya_baja -> 409 · ya_activa -> 409 · ok -> 200 con token
nuevo
// solo en el caso ok se devuelve el token: en los rechazos no se guardo
// nada y el token generado se descarta.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-RRE09-NN
```

```
'use strict';

const recepcionistaReactivada =
require('../nodos/wf09_recepcionista_reactivada.js');
const { ejecutar } = require('../simulacion.js');

const TOKEN = 'e'.repeat(64);
const responder = (resultado, id = 3) => ejecutar(recepcionistaReactivada, {
  entrada: [resultado],
  anteriores: { 'Valida datos': { id }, 'Genera token nuevo': { token: TOKEN } }
});
const salidaDe = (resultado, id) => responder(resultado, id).salida[0].json;

describe('WF-09 · Recepcionista reactivada', () => {
  test('UT-RRE09-01 · no_existe: 404', () => {
    expect(salidaDe({ resultado: 'no_existe' }, 99)).toEqual({ codigo: 404,
estado: 'rechazado', mensaje: 'No existe la recepcionista 99' });
  });

  test('UT-RRE09-02 · ya_baja: 409, la baja es definitiva', () => {
    expect(salidaDe({ resultado: 'ya_baja' }, 2)).toEqual({ codigo: 409, estado:
'rechazado', mensaje: 'La recepcionista 2 fue dada de baja; la baja es
definitiva' });
  });

  test('UT-RRE09-03 · ya_activa: 409', () => {
    expect(salidaDe({ resultado: 'ya_activa' }, 3)).toEqual({ codigo: 409,
estado: 'rechazado', mensaje: 'La recepcionista 3 ya esta activa' });
  });

  test('UT-RRE09-04 · ok: 200 con el token nuevo y el aviso', () => {
    expect(salidaDe({ resultado: 'ok', hash_auditoria: 'abc' }, 3)).toEqual({
      codigo: 200,
      estado: 'actualizado',
      mensaje: 'Recepcionista 3 reactivada con token nuevo',
      token: TOKEN,
      aviso: 'Guarde este token ahora: no se volvera a mostrar. El token anterior
sigue desactivado',
      hash_auditoria: 'abc'
    });
  });

  test.each(['no_existe', 'ya_baja', 'ya_activa'])('UT-RRE09-05 · %s: el token
generado no se devuelve', resultado => {
```

```
    expect(JSON.stringify(responder({ resultado }).salida)).not.toContain(TOKEN);
  });

  test('UT-RRE09-06 · el token sale de Genera token nuevo (verify)', () => {
    const { $ } = responder({ resultado: 'ok' });
    expect($).toHaveBeenCalledWith('Genera token nuevo');
  });

  test('UT-RRE09-07 · en los rechazos no se consulta Genera token nuevo', () => {
    const { $ } = responder({ resultado: 'ya_activa' });
    expect($).not.toHaveBeenCalledWith('Genera token nuevo');
  });

  test('UT-RRE09-08 · devuelve una lista con un solo item', () => {
    expect(responder({ resultado: 'ok' }).salida).toHaveLength(1);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf09.recepcionistareactivada.test.js
 PASS  tests/sigac.wf09.recepcionistareactivada.test.js
  WF-09 · Recepcionista reactivada
    ✓ UT-RRE09-01 · no_existe: 404 (5 ms)
    ✓ UT-RRE09-02 · ya_baja: 409, la baja es definitiva (1 ms)
    ✓ UT-RRE09-03 · ya_activa: 409 (1 ms)
    ✓ UT-RRE09-04 · ok: 200 con el token nuevo y el aviso
    ✓ UT-RRE09-05 · no_existe: el token generado no se devuelve
    ✓ UT-RRE09-05 · ya_baja: el token generado no se devuelve (1 ms)
    ✓ UT-RRE09-05 · ya_activa: el token generado no se devuelve
    ✓ UT-RRE09-06 · el token sale de Genera token nuevo (verify) (1 ms)
    ✓ UT-RRE09-07 · en los rechazos no se consulta Genera token nuevo (1 ms)
    ✓ UT-RRE09-08 · devuelve una lista con un solo item

Test Suites: 1 passed, 1 total
Tests:       10 passed, 10 total
Snapshots:   0 total
Time:        3.599 s
Ran all test suites matching tests/sigac.wf09.recepcionistareactivada.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.48 Estado paciente (WF-09)

El nodo Estado paciente traduce el resultado del nodo Cambia estado paciente: no_existe da 404; ya_baja da 409; con_pendientes da 409 e informa cuántas citas futuras y solicitudes en curso tiene el paciente, para que recepción sepa qué resolver antes; y ok da 200, desactivado o dado de baja.

1.48.1 Código base del nodo Estado paciente

```
const r = $input.first().json;
const id = $('Valida datos').first().json.id;
// count(*) llega como texto desde postgres, por eso el Number()
const citas = Number(r.citas_futuras);
const solicitudes = Number(r.solicitudes_pendientes);

if (r.resultado === 'no_existe') {
  return [{ json: { codigo: 404, estado: 'rechazado', mensaje: 'No existe el
paciente ' + id } }];
}

if (r.resultado === 'ya_baja') {
  return [{ json: { codigo: 409, estado: 'rechazado', mensaje: 'El paciente ' +
id + ' ya fue dado de baja; la baja es definitiva' } }];
}
if (r.resultado === 'con_pendientes') {
  return [{
    json: {
      codigo: 409,
      estado: 'rechazado',
      mensaje: 'El paciente ' + id + ' tiene citas o solicitudes en curso; deben
cancelarse o resolverse primero',
      citas_futuras: citas,
      solicitudes_pendientes: solicitudes
    }
  }];
}

return [{
  json: {
    codigo: 200,
    estado: 'actualizado',
    mensaje: r.dado_de_baja
      ? 'Paciente ' + id + ' dado de baja'
      : 'Paciente ' + id + ' desactivado',
    hash_auditoria: r.hash_auditoria
  }
}];
```

1.48.2 Código de test del nodo Estado paciente

```
// sigac.wf09.estadopaciente.test.js
//
// pruebas unitarias del nodo Estado paciente de WF-09. traduce el
// resultado de Cambia estado paciente al codigo de respuesta:
//   no_existe -> 404 · ya_baja -> 409 · con_pendientes -> 409 con conteos
//   ok -> 200 (desactivado o dado de baja)
//
// cada prueba lleva su ID de la tabla de pruebas: UT-EPA09-NN

'use strict';

const estadoPaciente = require('../nodos/wf09_estado_paciente.js');
const { ejecutar } = require('../simulacion.js');

const responder = (resultado, id = 55) =>
  ejecutar(estadoPaciente, { entrada: [resultado], anteriores: { 'Valida datos':
{ id } } });
const salidaDe = (resultado, id) => responder(resultado, id).salida[0].json;

describe('WF-09 · Estado paciente', () => {
  test('UT-EPA09-01 · no_existe: 404', () => {
    expect(salidaDe({ resultado: 'no_existe', citas_futuras: '0',
solicitudes_pendientes: '0' }, 999))
      .toEqual({ codigo: 404, estado: 'rechazado', mensaje: 'No existe el
paciente 999' });
  });

  test('UT-EPA09-02 · ya_baja: 409, la baja es definitiva', () => {
    expect(salidaDe({ resultado: 'ya_baja' }, 56))
      .toEqual({ codigo: 409, estado: 'rechazado', mensaje: 'El paciente 56 ya
fue dado de baja; la baja es definitiva' });
  });

  test('UT-EPA09-03 · con_pendientes: 409 con los dos conteos', () => {
    expect(salidaDe({ resultado: 'con_pendientes', citas_futuras: '4',
solicitudes_pendientes: '7' }, 22)).toEqual({
      codigo: 409,
      estado: 'rechazado',
      mensaje: 'El paciente 22 tiene citas o solicitudes en curso; deben
cancelarse o resolverse primero',
      citas_futuras: 4,
      solicitudes_pendientes: 7
    });
  });
```

```
  test('UT-EPA09-04 · con_pendientes solo por una cita: los conteos lo muestran',
() => {
    const salida = salidaDe({ resultado: 'con_pendientes', citas_futuras: '1',
solicitudes_pendientes: '0' }, 19);
    expect([salida.citas_futuras, salida.solicitudes_pendientes]).toEqual([1,
0]);
  });

  test('UT-EPA09-05 · los conteos llegan como texto desde postgres: salen como
numero', () => {
    const salida = salidaDe({ resultado: 'con_pendientes', citas_futuras: '2',
solicitudes_pendientes: '3' });
    expect(typeof salida.citas_futuras).toBe('number');
    expect(typeof salida.solicitudes_pendientes).toBe('number');
  });

  test('UT-EPA09-06 · ok sin baja: 200, desactivado', () => {
    expect(salidaDe({ resultado: 'ok', dado_de_baja: false, hash_auditoria: 'abc'
}, 55))
      .toEqual({ codigo: 200, estado: 'actualizado', mensaje: 'Paciente 55
desactivado', hash_auditoria: 'abc' });
  });

  test('UT-EPA09-07 · ok con baja: 200, dado de baja', () => {
    expect(salidaDe({ resultado: 'ok', dado_de_baja: true },
56).mensaje).toBe('Paciente 56 dado de baja');
  });

  test('UT-EPA09-08 · la respuesta exitosa no incluye los conteos', () => {
    const salida = salidaDe({ resultado: 'ok', dado_de_baja: false,
citas_futuras: '0', solicitudes_pendientes: '0' });
    expect(salida).not.toHaveProperty('citas_futuras');
  });

  test('UT-EPA09-09 · consulta Valida datos (verify)', () => {
    expect(responder({ resultado: 'ok' }).$).toHaveBeenCalledWith('Valida
datos');
  });

  test('UT-EPA09-10 · devuelve una lista con un solo item', () => {
    expect(responder({ resultado: 'ok' }).salida).toHaveLength(1);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf09.estadopaciente.test.js
 PASS  tests/sigac.wf09.estadopaciente.test.js
  WF-09 · Estado paciente
    ✓ UT-EPA09-01 · no_existe: 404 (6 ms)
    ✓ UT-EPA09-02 · ya_baja: 409, la baja es definitiva
    ✓ UT-EPA09-03 · con_pendientes: 409 con los dos conteos (1 ms)
    ✓ UT-EPA09-04 · con_pendientes solo por una cita: los conteos lo muestran
    ✓ UT-EPA09-05 · los conteos llegan como texto desde postgres: salen como numero (1 ms)
    ✓ UT-EPA09-06 · ok sin baja: 200, desactivado
    ✓ UT-EPA09-07 · ok con baja: 200, dado de baja
    ✓ UT-EPA09-08 · la respuesta exitosa no incluye los conteos (1 ms)
    ✓ UT-EPA09-09 · consulta Valida datos (verify) (1 ms)
    ✓ UT-EPA09-10 · devuelve una lista con un solo item

Test Suites: 1 passed, 1 total
Tests:       10 passed, 10 total
Snapshots:   0 total
Time:        3.752 s
Ran all test suites matching tests/sigac.wf09.estadopaciente.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.49 Paciente reactivado (WF-09)

El nodo Paciente reactivado traduce el resultado del nodo Reactiva paciente: no_existe da 404, ya_baja y ya_activo dan 409, y ok da 200. A diferencia del médico y la recepcionista, el paciente no tiene token, porque no accede al sistema.

1.49.1 Código base del nodo Paciente reactivado

```
const r = $input.first().json;
const id = $('Valida datos').first().json.id;

if (r.resultado === 'no_existe') {
  return [{ json: { codigo: 404, estado: 'rechazado', mensaje: 'No existe el
paciente ' + id } }];
}

if (r.resultado === 'ya_baja') {
  return [{ json: { codigo: 409, estado: 'rechazado', mensaje: 'El paciente ' +
id + ' fue dado de baja; la baja es definitiva' } }];
}

if (r.resultado === 'ya_activo') {
  return [{ json: { codigo: 409, estado: 'rechazado', mensaje: 'El paciente ' +
id + ' ya esta activo' } }];
}

return [{
  json: {
    codigo: 200,
    estado: 'actualizado',
    mensaje: 'Paciente ' + id + ' reactivado',
    hash_auditoria: r.hash_auditoria
  }
}];
```

1.49.2 Código de test del nodo Paciente reactivado

```
// sigac.wf09.pacientereactivado.test.js
//
// pruebas unitarias del nodo Paciente reactivado de WF-09. traduce el
// resultado de Reactiva paciente al codigo de respuesta:
//   no_existe -> 404 · ya_baja -> 409 · ya_activo -> 409 · ok -> 200
// el paciente no tiene token: no accede al sistema.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-PRE09-NN

'use strict';

const pacienteReactivado = require('../nodos/wf09_paciente_reactivado.js');
const { ejecutar } = require('../simulacion.js');
```

```
const responder = (resultado, id = 55) =>
  ejecutar(pacienteReactivado, { entrada: [resultado], anteriores: { 'Valida
datos': { id } } });
const salidaDe = (resultado, id) => responder(resultado, id).salida[0].json;

describe('WF-09 · Paciente reactivado', () => {
  test('UT-PRE09-01 · no_existe: 404', () => {
    expect(salidaDe({ resultado: 'no_existe' }, 999)).toEqual({ codigo: 404,
estado: 'rechazado', mensaje: 'No existe el paciente 999' });
  });

  test('UT-PRE09-02 · ya_baja: 409, la baja es definitiva', () => {
    expect(salidaDe({ resultado: 'ya_baja' }, 56)).toEqual({ codigo: 409, estado:
'rechazado', mensaje: 'El paciente 56 fue dado de baja; la baja es definitiva'
});
  });

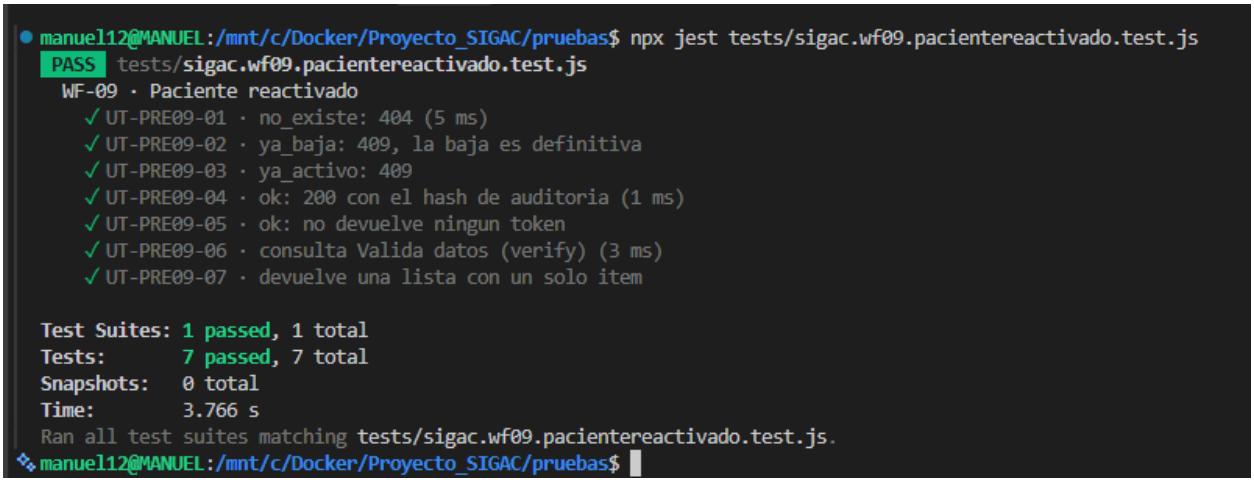
  test('UT-PRE09-03 · ya_activo: 409', () => {
    expect(salidaDe({ resultado: 'ya_activo' }, 55)).toEqual({ codigo: 409,
estado: 'rechazado', mensaje: 'El paciente 55 ya esta activo' });
  });

  test('UT-PRE09-04 · ok: 200 con el hash de auditoria', () => {
    expect(salidaDe({ resultado: 'ok', hash_auditoria: 'abc' }, 55))
      .toEqual({ codigo: 200, estado: 'actualizado', mensaje: 'Paciente 55
reactivado', hash_auditoria: 'abc' });
  });

  test('UT-PRE09-05 · ok: no devuelve ningun token', () => {
    expect(salidaDe({ resultado: 'ok' })).not.toHaveProperty('token');
  });

  test('UT-PRE09-06 · consulta Valida datos (verify)', () => {
    expect(responder({ resultado: 'ok' }).$).toHaveBeenCalledWith('Valida
datos');
  });

  test('UT-PRE09-07 · devuelve una lista con un solo item', () => {
    expect(responder({ resultado: 'ok' }).salida).toHaveLength(1);
  });
});
```


manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf09.pacientereactivado.test.js
PASS tests/sigac.wf09.pacientereactivado.test.js
WF-09 · Paciente reactivado
✓ UT-PRE09-01 · no_existe: 404 (5 ms)
✓ UT-PRE09-02 · ya_baja: 409, la baja es definitiva
✓ UT-PRE09-03 · ya_activo: 409
✓ UT-PRE09-04 · ok: 200 con el hash de auditoria (1 ms)
✓ UT-PRE09-05 · ok: no devuelve ningun token
✓ UT-PRE09-06 · consulta Valida datos (verify) (3 ms)
✓ UT-PRE09-07 · devuelve una lista con un solo item
Test Suites: 1 passed, 1 total
Tests: 7 passed, 7 total
Snapshots: 0 total
Time: 3.766 s
Ran all test suites matching tests/sigac.wf09.pacientereactivado.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$

1.50 Genera token medico (WF-09)

El nodo Genera token medico, de la ruta medico_agregar, genera el token del médico nuevo: 32 bytes aleatorios en hexadecimal y su huella SHA-256. Es una copia exacta del nodo Genera token.

1.50.1 Código base del nodo Genera token medico

```
const crypto = require('crypto');

const token = crypto.randomBytes(32).toString('hex');
const tokenHash = crypto.createHash('sha256').update(token).digest('hex');

return [{
  json: {
    ...$input.first().json,
    token: token,
    token_hash: tokenHash
  }
}];
```

1.50.2 Código de test del nodo Genera token medico

```
// sigac.wf09.generatokenmedico.test.js
//
// pruebas unitarias del nodo Genera token medico de WF-09, en la ruta
// medico_agregar. genera el token de la persona: 32 bytes aleatorios en
// hexadecimal y su huella sha-256. a la base solo va la huella; el
// token en claro viaja una sola vez, en la respuesta.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-GTM09-NN

'use strict';

const generaToken = require('../nodos/wf09_genera_token_medico.js');
const { ejecutar, sha256 } = require('../simulacion.js');

const DESDE_VALIDA = { valido: true, ruta: 'medico_agregar', id: 3, nombre:
'Rosa' };
const generar = (entrada = DESDE_VALIDA) => ejecutar(generaToken, { entrada:
[entrada] });
const salidaDe = entrada => generar(entrada).salida[0].json;

describe('WF-09 · Genera token medico', () => {
  test('UT-GTM09-01 · el token tiene 64 caracteres hexadecimales (32 bytes)', ()
=> {
    expect(salidaDe().token).toMatch(/^[0-9a-f]{64}$/);
  });

  test('UT-GTM09-02 · la huella es el sha-256 del token', () => {
```

```
    const salida = salidaDe();
    expect(salida.token_hash).toBe(sha256(salida.token));
  });

  test('UT-GTM09-03 · la huella tiene 64 caracteres hexadecimales', () => {
    expect(salidaDe().token_hash).toMatch(/^[0-9a-f]{64}$/);
  });

  test('UT-GTM09-04 · la huella es distinta del token', () => {
    const salida = salidaDe();
    expect(salida.token_hash).not.toBe(salida.token);
  });

  test('UT-GTM09-05 · dos ejecuciones dan tokens distintos', () => {
    expect(salidaDe().token).not.toBe(salidaDe().token);
  });

  test('UT-GTM09-06 · 100 tokens seguidos: ninguno se repite', () => {
    const tokens = new Set(Array.from({ length: 100 }, () => salidaDe().token));
    expect(tokens.size).toBe(100);
  });

  test('UT-GTM09-07 · conserva los campos que traia de Valida datos', () => {
    const salida = salidaDe();
    expect(salida.ruta).toBe('medico_agregar');
    expect(salida.id).toBe(3);
    expect(salida.nombre).toBe('Rosa');
  });

  test('UT-GTM09-08 · un token que viniera en la entrada se reemplaza', () => {
    const salida = salidaDe({ ...DESDE_VALIDA, token: 'impuesto', token_hash:
'impuesto' });
    expect(salida.token).not.toBe('impuesto');
    expect(salida.token_hash).toBe(sha256(salida.token));
  });

  test('UT-GTM09-09 · entrada vacia: genera igual, sin error', () => {
    expect(salidaDe({}).token).toMatch(/^[0-9a-f]{64}$/);
  });

  test('UT-GTM09-10 · no consulta ningun nodo anterior (verify)', () => {
    expect(generar().$).not.toHaveBeenCalled();
  });

  test('UT-GTM09-11 · devuelve una lista con un solo item', () => {
```

```
    expect(generar().salida).toHaveLength(1);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf09.generatokenmedico.test.js
 PASS  tests/sigac.wf09.generatokenmedico.test.js
  WF-09 · Genera token medico
    ✓ UT-GTM09-01 · el token tiene 64 caracteres hexadecimales (32 bytes) (6 ms)
    ✓ UT-GTM09-02 · la huella es el sha-256 del token (1 ms)
    ✓ UT-GTM09-03 · la huella tiene 64 caracteres hexadecimales (1 ms)
    ✓ UT-GTM09-04 · la huella es distinta del token (1 ms)
    ✓ UT-GTM09-05 · dos ejecuciones dan tokens distintos (1 ms)
    ✓ UT-GTM09-06 · 100 tokens seguidos: ninguno se repite (13 ms)
    ✓ UT-GTM09-07 · conserva los campos que traia de Valida datos (2 ms)
    ✓ UT-GTM09-08 · un token que viniera en la entrada se reemplaza
    ✓ UT-GTM09-09 · entrada vacia: genera igual, sin error (1 ms)
    ✓ UT-GTM09-10 · no consulta ningun nodo anterior (verify) (1 ms)
    ✓ UT-GTM09-11 · devuelve una lista con un solo item (1 ms)

Test Suites: 1 passed, 1 total
Tests:       11 passed, 11 total
Snapshots:   0 total
Time:        3.806 s
Ran all test suites matching tests/sigac.wf09.generatokenmedico.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.51 Medico agregado (WF-09)

El nodo Medico agregado traduce el resultado del nodo Agrega medico: colegiado_duplicado da 409, y ok da 201 con el identificador del médico, su especialidad, las diez franjas del horario estándar y su token en claro. Solo en el caso ok se devuelve el token.

1.51.1 Código base del nodo Medico agregado

```
const r = $input.first().json;
const datos = $('Valida datos').first().json;

if (r.resultado === 'colegiado_duplicado') {
  return [{ json: { codigo: 409, estado: 'rechazado', mensaje: 'Ya existe un
medico con el colegiado ' + datos.colegiado } }];
}

return [{
  json: {
    codigo: 201,
    estado: 'registrado',
    mensaje: 'Medico agregado con el horario estandar',
    id_medico: r.id_medico,
    especialidad: datos.especialidad,
    franjas_de_horario: Number(r.franjas),
    token: $('Genera token medico').first().json.token,
    aviso: 'Guarde este token ahora: no se volvera a mostrar',
    hash_auditoria: r.hash_auditoria
  }
}];
```

1.51.2 Código de test del nodo Medico agregado

```
// sigac.wf09.medicoagregado.test.js
//
// pruebas unitarias del nodo Medico agregado de WF-09. traduce el
// resultado de Agrega medico:
//   colegiado_duplicado -> 409 · ok -> 201 con el token, la especialidad
//   y las franjas del horario estandar
// solo en el caso ok se devuelve el token.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-MAG09-NN

'use strict';

const medicoAgregado = require('../nodos/wf09_medico_agregado.js');
const { ejecutar } = require('../simulacion.js');

const TOKEN = 'b'.repeat(64);
const VALIDA = { especialidad: 'Pediatria', colegiado: 'COL-90001' };
```

```
const responder = (resultado, valida = VALIDA) => ejecutar(medicoAgregado, {
  entrada: [resultado],
  anteriores: { 'Valida datos': valida, 'Genera token medico': { token: TOKEN } }
});
const salidaDe = (resultado, valida) => responder(resultado,
valida).salida[0].json;

describe('WF-09 · Medico agregado', () => {
  test('UT-MAG09-01 · ok: 201 completo', () => {
    expect(salidaDe({ resultado: 'ok', id_medico: 6, franjas: '10',
hash_auditoria: 'abc' })).toEqual({
      codigo: 201,
      estado: 'registrado',
      mensaje: 'Medico agregado con el horario estandar',
      id_medico: 6,
      especialidad: 'Pediatria',
      franjas_de_horario: 10,
      token: TOKEN,
      aviso: 'Guarde este token ahora: no se volvera a mostrar',
      hash_auditoria: 'abc'
    });
  });

  test('UT-MAG09-02 · colegiado_duplicado: 409 con el colegiado', () => {
    expect(salidaDe({ resultado: 'colegiado_duplicado' })).toEqual({
      codigo: 409, estado: 'rechazado', mensaje: 'Ya existe un medico con el
colegiado COL-90001'
    });
  });

  test('UT-MAG09-03 · colegiado duplicado: el token generado no se devuelve', ()
=> {
    expect(JSON.stringify(responder({ resultado: 'colegiado_duplicado'
}).salida)).not.toContain(TOKEN);
  });

  test('UT-MAG09-04 · las franjas llegan como texto desde postgres: salen como
numero', () => {
    expect(salidaDe({ resultado: 'ok', franjas: '10'
}).franjas_de_horario).toBe(10);
  });

  test('UT-MAG09-05 · la especialidad sale de Valida datos', () => {
    expect(salidaDe({ resultado: 'ok', franjas: '10' }, { ...VALIDA,
especialidad: 'Ginecologia' }).especialidad).toBe('Ginecologia');
```

```
  });

  test('UT-MAG09-06 · el token sale de Genera token medico (verify)', () => {
    expect(responder({ resultado: 'ok', franjas: '10'
}).$).toHaveBeenCalledWith('Genera token medico');
  });

  test('UT-MAG09-07 · ok: incluye el aviso de guardar el token', () => {
    expect(salidaDe({ resultado: 'ok', franjas: '10' }).aviso).toContain('no se
volvera a mostrar');
  });

  test('UT-MAG09-08 · devuelve una lista con un solo item', () => {
    expect(responder({ resultado: 'ok', franjas: '10' }).salida).toHaveLength(1);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf09.medicoagregado.test.js
 PASS  tests/sigac.wf09.medicoagregado.test.js
  WF-09 · Medico agregado
    ✓ UT-MAG09-01 · ok: 201 completo (7 ms)
    ✓ UT-MAG09-02 · colegiado_duplicado: 409 con el colegiado (4 ms)
    ✓ UT-MAG09-03 · colegiado duplicado: el token generado no se devuelve (1 ms)
    ✓ UT-MAG09-04 · las franjas llegan como texto desde postgres: salen como numero (2 ms)
    ✓ UT-MAG09-05 · la especialidad sale de Valida datos (1 ms)
    ✓ UT-MAG09-06 · el token sale de Genera token medico (verify) (2 ms)
    ✓ UT-MAG09-07 · ok: incluye el aviso de guardar el token (1 ms)
    ✓ UT-MAG09-08 · devuelve una lista con un solo item

Test Suites: 1 passed, 1 total
Tests:       8 passed, 8 total
Snapshots:   0 total
Time:        3.728 s
Ran all test suites matching tests/sigac.wf09.medicoagregado.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.52 Genera token medico nuevo (WF-09)

El nodo Genera token medico nuevo, de la ruta medico_reactivar, genera el token nuevo del médico que se reactiva: 32 bytes aleatorios en hexadecimal y su huella SHA-256. Es una copia exacta del nodo Genera token.

1.52.1 Código base del nodo Genera token medico nuevo

```
const crypto = require('crypto');

const token = crypto.randomBytes(32).toString('hex');
const tokenHash = crypto.createHash('sha256').update(token).digest('hex');

return [{
  json: {
    ...$input.first().json,
    token: token,
    token_hash: tokenHash
  }
}];
```

1.52.2 Código de test del nodo Genera token medico nuevo

```
// sigac.wf09.generatokenmediconuevo.test.js
//
// pruebas unitarias del nodo Genera token medico nuevo de WF-09, en la ruta
// medico_reactivar. genera el token de la persona: 32 bytes aleatorios en
// hexadecimal y su huella sha-256. a la base solo va la huella; el
// token en claro viaja una sola vez, en la respuesta.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-GMN09-NN

'use strict';

const generaToken = require('../nodos/wf09_genera_token_medico_nuevo.js');
const { ejecutar, sha256 } = require('../simulacion.js');

const DESDE_VALIDA = { valido: true, ruta: 'medico_reactivar', id: 3, nombre:
'Rosa' };
const generar = (entrada = DESDE_VALIDA) => ejecutar(generaToken, { entrada:
[entrada] });
const salidaDe = entrada => generar(entrada).salida[0].json;

describe('WF-09 · Genera token medico nuevo', () => {
  test('UT-GMN09-01 · el token tiene 64 caracteres hexadecimales (32 bytes)', ()
=> {
    expect(salidaDe().token).toMatch(/^[0-9a-f]{64}$/);
  });

  test('UT-GMN09-02 · la huella es el sha-256 del token', () => {
```

```
    const salida = salidaDe();
    expect(salida.token_hash).toBe(sha256(salida.token));
  });

  test('UT-GMN09-03 · la huella tiene 64 caracteres hexadecimales', () => {
    expect(salidaDe().token_hash).toMatch(/^[0-9a-f]{64}$/);
  });

  test('UT-GMN09-04 · la huella es distinta del token', () => {
    const salida = salidaDe();
    expect(salida.token_hash).not.toBe(salida.token);
  });

  test('UT-GMN09-05 · dos ejecuciones dan tokens distintos', () => {
    expect(salidaDe().token).not.toBe(salidaDe().token);
  });

  test('UT-GMN09-06 · 100 tokens seguidos: ninguno se repite', () => {
    const tokens = new Set(Array.from({ length: 100 }, () => salidaDe().token));
    expect(tokens.size).toBe(100);
  });

  test('UT-GMN09-07 · conserva los campos que traia de Valida datos', () => {
    const salida = salidaDe();
    expect(salida.ruta).toBe('medico_reactivar');
    expect(salida.id).toBe(3);
    expect(salida.nombre).toBe('Rosa');
  });

  test('UT-GMN09-08 · un token que viniera en la entrada se reemplaza', () => {
    const salida = salidaDe({ ...DESDE_VALIDA, token: 'impuesto', token_hash:
'impuesto' });
    expect(salida.token).not.toBe('impuesto');
    expect(salida.token_hash).toBe(sha256(salida.token));
  });

  test('UT-GMN09-09 · entrada vacia: genera igual, sin error', () => {
    expect(salidaDe({}).token).toMatch(/^[0-9a-f]{64}$/);
  });

  test('UT-GMN09-10 · no consulta ningun nodo anterior (verify)', () => {
    expect(generar().$).not.toHaveBeenCalled();
  });

  test('UT-GMN09-11 · devuelve una lista con un solo item', () => {
```

```
    expect(generar().salida).toHaveLength(1);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf09.generatokenmediconuevo.test.js
 PASS  tests/sigac.wf09.generatokenmediconuevo.test.js
  WF-09 · Genera token medico nuevo
    ✓ UT-GMN09-01 · el token tiene 64 caracteres hexadecimales (32 bytes) (6 ms)
    ✓ UT-GMN09-02 · la huella es el sha-256 del token
    ✓ UT-GMN09-03 · la huella tiene 64 caracteres hexadecimales (1 ms)
    ✓ UT-GMN09-04 · la huella es distinta del token
    ✓ UT-GMN09-05 · dos ejecuciones dan tokens distintos (1 ms)
    ✓ UT-GMN09-06 · 100 tokens seguidos: ninguno se repite (8 ms)
    ✓ UT-GMN09-07 · conserva los campos que traia de Valida datos (1 ms)
    ✓ UT-GMN09-08 · un token que viniera en la entrada se reemplaza (1 ms)
    ✓ UT-GMN09-09 · entrada vacia: genera igual, sin error (2 ms)
    ✓ UT-GMN09-10 · no consulta ningun nodo anterior (verify) (2 ms)
    ✓ UT-GMN09-11 · devuelve una lista con un solo item

Test Suites: 1 passed, 1 total
Tests:       11 passed, 11 total
Snapshots:   0 total
Time:        3.851 s
Ran all test suites matching tests/sigac.wf09.generatokenmediconuevo.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ ~
```

1.53 Medico reactivado (WF-09)

El nodo Medico reactivado traduce el resultado del nodo Reactiva medico: no_existe da 404, ya_baja y ya_activo dan 409, y ok da 200 con el token nuevo. La respuesta advierte que las citas reasignadas o canceladas al desactivarlo no se restauran.

1.53.1 Código base del nodo Medico reactivado

```
const r = $input.first().json;
const id = $('Valida datos').first().json.id;

if (r.resultado === 'no_existe') {
  return [{ json: { codigo: 404, estado: 'rechazado', mensaje: 'No existe el
medico ' + id } }];
}

if (r.resultado === 'ya_baja') {
  return [{ json: { codigo: 409, estado: 'rechazado', mensaje: 'El medico ' + id
+ ' fue dado de baja; la baja es definitiva' } }];
}

if (r.resultado === 'ya_activo') {
  return [{ json: { codigo: 409, estado: 'rechazado', mensaje: 'El medico ' + id
+ ' ya esta activo' } }];
}

return [{
  json: {
    codigo: 200,
    estado: 'actualizado',
    mensaje: 'Medico ' + id + ' reactivado con token nuevo',
    token: $('Genera token medico nuevo').first().json.token,
    aviso: 'Guarde este token ahora: no se volvera a mostrar. Sus citas
anteriores no se restauran',
    hash_auditoria: r.hash_auditoria
  }
}];
```

1.53.2 Código de test del nodo Medico reactivado

```
// sigac.wf09.medicoreactivado.test.js
//
// pruebas unitarias del nodo Medico reactivado de WF-09. traduce el
// resultado de Reactiva medico al codigo de respuesta:
//   no_existe -> 404 · ya_baja -> 409 · ya_activo -> 409 · ok -> 200 con token
nuevo
// las citas reasignadas o canceladas al desactivarlo no se restauran.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-MRE09-NN
```

```
'use strict';

const medicoReactivado = require('../nodos/wf09_medico_reactivado.js');
const { ejecutar } = require('../simulacion.js');

const TOKEN = 'a1'.repeat(32);
const responder = (resultado, id = 6) => ejecutar(medicoReactivado, {
  entrada: [resultado],
  anteriores: { 'Valida datos': { id }, 'Genera token medico nuevo': { token:
TOKEN } }
});
const salidaDe = (resultado, id) => responder(resultado, id).salida[0].json;

describe('WF-09 · Medico reactivado', () => {
  test('UT-MRE09-01 · no_existe: 404', () => {
    expect(salidaDe({ resultado: 'no_existe' }, 99)).toEqual({ codigo: 404,
estado: 'rechazado', mensaje: 'No existe el medico 99' });
  });

  test('UT-MRE09-02 · ya_baja: 409, la baja es definitiva', () => {
    expect(salidaDe({ resultado: 'ya_baja' }, 6)).toEqual({ codigo: 409, estado:
'rechazado', mensaje: 'El medico 6 fue dado de baja; la baja es definitiva' });
  });

  test('UT-MRE09-03 · ya_activo: 409', () => {
    expect(salidaDe({ resultado: 'ya_activo' }, 6)).toEqual({ codigo: 409,
estado: 'rechazado', mensaje: 'El medico 6 ya esta activo' });
  });

  test('UT-MRE09-04 · ok: 200 con el token nuevo', () => {
    expect(salidaDe({ resultado: 'ok', hash_auditoria: 'abc' }, 2)).toEqual({
      codigo: 200,
      estado: 'actualizado',
      mensaje: 'Medico 2 reactivado con token nuevo',
      token: TOKEN,
      aviso: 'Guarde este token ahora: no se volvera a mostrar. Sus citas
anteriores no se restauran',
      hash_auditoria: 'abc'
    });
  });

  test('UT-MRE09-05 · el aviso advierte que las citas anteriores no se
restauran', () => {
    expect(salidaDe({ resultado: 'ok' }).aviso).toContain('no se restauran');
  });
```

```
  test.each(['no_existe', 'ya_baja', 'ya_activo'])('UT-MRE09-06 · %s: el token
generado no se devuelve', resultado => {
    expect(JSON.stringify(responder({ resultado }).salida)).not.toContain(TOKEN);
  });

  test('UT-MRE09-07 · el token sale de Genera token medico nuevo (verify)', () =>
{
    expect(responder({ resultado: 'ok' }).$).toHaveBeenCalledWith('Genera token
medico nuevo');
  });

  test('UT-MRE09-08 · devuelve una lista con un solo item', () => {
    expect(responder({ resultado: 'ok' }).salida).toHaveLength(1);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf09.medicoreactivado.test.js
 PASS  tests/sigac.wf09.medicoreactivado.test.js
  WF-09 · Medico reactivado
    ✓ UT-MRE09-01 · no_existe: 404 (6 ms)
    ✓ UT-MRE09-02 · ya_baja: 409, la baja es definitiva (3 ms)
    ✓ UT-MRE09-03 · ya_activo: 409 (1 ms)
    ✓ UT-MRE09-04 · ok: 200 con el token nuevo
    ✓ UT-MRE09-05 · el aviso advierte que las citas anteriores no se restauran (1 ms)
    ✓ UT-MRE09-06 · no_existe: el token generado no se devuelve (1 ms)
    ✓ UT-MRE09-06 · ya_baja: el token generado no se devuelve (1 ms)
    ✓ UT-MRE09-06 · ya_activo: el token generado no se devuelve
    ✓ UT-MRE09-07 · el token sale de Genera token medico nuevo (verify) (1 ms)
    ✓ UT-MRE09-08 · devuelve una lista con un solo item (1 ms)

Test Suites: 1 passed, 1 total
Tests:       10 passed, 10 total
Snapshots:   0 total
Time:        3.865 s
Ran all test suites matching tests/sigac.wf09.medicoreactivado.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.54 Reparte reemplazos (WF-09)

El nodo Reparte reemplazos decide, cita por cita, a qué médico pasan las citas futuras del médico que se desactiva o se da de baja, o si se cancelan. No toca la base: trabaja sobre la fotografía que armó el nodo Lee situacion del medico.

La cita conserva su fecha y hora; solo cambia el médico. Primero se busca en la misma especialidad y, si no hay, en Medicina General. El candidato debe tener una franja ese día que cubra la hora, no tener otra cita a esa hora y no haber llenado el cupo de la franja. Entre varios gana el de menor carga del día, y en empate, el id menor. Cada reasignación se anota en memoria para la cita siguiente.

1.54.1 Código base del nodo Reparte reemplazos

```
const lectura = $input.first().json;
const datos = $('Valida datos').first().json;

function leer(valor, porDefecto) {
  if (valor === null || valor === undefined) return porDefecto;
  return typeof valor === 'string' ? JSON.parse(valor) : valor;
}

const medico = leer(lectura.medico, null);
const citas = leer(lectura.citas, []);
const candidatos = leer(lectura.candidatos, []);

if (!medico) {
  return [{ json: { resultado: 'no_existe' } }];
}
if (medico.baja_en) {
  return [{ json: { resultado: 'ya_baja' } }];
}

function minutos(hora) {
  return Number(hora.slice(0, 2)) * 60 + Number(hora.slice(3, 5));
}

const ocupacion = {};
for (const k of candidatos) {
  ocupacion[k.id] = {};
  for (const o of leer(k.ocupadas, [])) {
    if (!ocupacion[k.id][o.fecha]) ocupacion[k.id][o.fecha] = [];
    ocupacion[k.id][o.fecha].push(o.hora);
  }
}

const GENERAL = 'Medicina General';
```

```
const mismaEspecialidad = candidatos.filter(k => k.especialidad ===
medico.especialidad);
const generales = candidatos.filter(k => k.especialidad === GENERAL);

const grupos = medico.especialidad === GENERAL ? [generales] :
[mismaEspecialidad, generales];

function buscar(grupo, cita) {
  const m = minutos(cita.hora);
  let mejor = null;

  for (const k of grupo) {
    const franja = leer(k.franjas, []).find(f =>
      f.dia === cita.dia && minutos(f.inicio) <= m && m < minutos(f.fin));
    if (!franja) continue;

    const horas = ocupacion[k.id][cita.fecha] || [];

    if (horas.includes(cita.hora)) continue;

    const enFranja = horas.filter(h =>
      minutos(h) >= minutos(franja.inicio) && minutos(h) <
minutos(franja.fin)).length;
    if (enFranja >= franja.cupos) continue;

    const carga = horas.length;
    if (!mejor || carga < mejor.carga || (carga === mejor.carga && k.id <
mejor.id)) {
      mejor = { id: k.id, carga: carga };
    }
  }
  return mejor;
}

const reasignar = [];
const cancelar = [];

for (const cita of citas) {
  let elegido = null;
  for (const grupo of grupos) {
    elegido = buscar(grupo, cita);
    if (elegido) break;
  }

  if (elegido) {
```

```
    reasignar.push({ id_cita: cita.id, id_medico: elegido.id });
    if (!ocupacion[elegido.id][cita.fecha]) ocupacion[elegido.id][cita.fecha] =
[];
    ocupacion[elegido.id][cita.fecha].push(cita.hora);
  } else {
    cancelar.push(cita.id);
  }
}

return [{
  json: {
    resultado: 'ok',
    id_medico: medico.id,
    baja: datos.baja,
    reasignar: reasignar,
    cancelar: cancelar,
    reasignar_json: JSON.stringify(reasignar),
    cancelar_json: JSON.stringify(cancelar)
  }
}];
```

1.54.2 Código de test del nodo Reparte reemplazos

```
// sigac.wf09.repartereemplazos.test.js
//
// pruebas unitarias del nodo Reparte reemplazos de WF-09. decide, cita
// por cita, a que medico pasan las citas futuras del medico que se
// desactiva, o si se cancelan. no toca la base: trabaja sobre la
// fotografia que armo Lee situacion del medico.
//
// reglas: la cita conserva su fecha y hora; solo cambia el medico.
// primero se busca en la misma especialidad y, si no hay, en Medicina
// General. el candidato debe tener una franja ese dia que cubra la hora,
// no tener otra cita a esa hora y no haber llenado el cupo de la franja.
// entre varios, gana el de menor carga del dia; en empate, el id menor.
// cada reasignacion se anota en memoria para la siguiente cita.
//
// las fechas de prueba son martes (dia 2) y un sabado (dia 6).
//
// cada prueba lleva su ID de la tabla de pruebas: UT-RRM09-NN

'use strict';

const reparteReemplazos = require('../nodos/wf09_reparte_reemplazos.js');
const { ejecutar } = require('../simulacion.js');

const MARTES = '2026-11-10';
```

```
const SABADO = '2026-10-31';

// horario estandar de la clinica, como lo devuelve la consulta
function horarioEstandar(cuposManana = 8, cuposTarde = 7) {
  const franjas = [];
  for (let dia = 1; dia <= 5; dia++) {
    const reducido = dia === 3;
    franjas.push({ dia, inicio: '08:00', fin: '12:00', cupos: reducido ? 2 :
cuposManana });
    franjas.push({ dia, inicio: '13:30', fin: '17:00', cupos: reducido ? 2 :
cuposTarde });
  }
  return franjas;
}

const candidato = (id, especialidad, ocupadas = [], franjas = horarioEstandar())
=>
  ({ id, especialidad, franjas, ocupadas });
const ocupada = (hora, fecha = MARTES) => ({ fecha, hora });
const cita = (id, hora, fecha = MARTES, dia = 2) => ({ id, fecha, hora, dia });

const PEDIATRA_SALE = { id: 2, especialidad: 'Pediatria', activo: true, baja_en:
null };

function repartir({ medico = PEDIATRA_SALE, citas = [], candidatos = [], baja =
false }) {
  return ejecutar(reparteReemplazos, {
    entrada: [{ medico, citas, candidatos }],
    anteriores: { 'Valida datos': { id: medico ? medico.id : 99, baja } }
  });
}

const planDe = opciones => repartir(opciones).salida[0].json;

// ------------------------------------------------------------
// A. Medico inexistente o dado de baja
// ------------------------------------------------------------
describe('WF-09 · Reparte reemplazos · A. Estado del medico', () => {
  test('UT-RRM09-01 · medico nulo: no_existe', () => {
    expect(planDe({ medico: null })).toEqual({ resultado: 'no_existe' });
  });

  test('UT-RRM09-02 · medico con baja: ya_baja, sin plan', () => {
    expect(planDe({ medico: { ...PEDIATRA_SALE, baja_en: '2026-10-23T20:00:00Z' }
})).toEqual({ resultado: 'ya_baja' });
```

```
  });

  test('UT-RRM09-03 · medico activo sin citas: ok con listas vacias', () => {
    const plan = planDe({});
    expect(plan.resultado).toBe('ok');
    expect(plan.reasignar).toEqual([]);
    expect(plan.cancelar).toEqual([]);
  });

  test('UT-RRM09-04 · medico ya inactivo pero sin baja: se puede dar de baja
(ok)', () => {
    expect(planDe({ medico: { ...PEDIATRA_SALE, activo: false }
}).resultado).toBe('ok');
  });
});

// ------------------------------------------------------------
// B. A quien pasa cada cita
// ------------------------------------------------------------
describe('WF-09 · Reparte reemplazos · B. Reemplazo', () => {
  test('UT-RRM09-05 · un medico de la misma especialidad libre: la cita pasa a
el', () => {
    const plan = planDe({ citas: [cita(1182, '08:30')], candidatos: [candidato(6,
'Pediatria')] });
    expect(plan.reasignar).toEqual([{ id_cita: 1182, id_medico: 6 }]);
    expect(plan.cancelar).toEqual([]);
  });

  test('UT-RRM09-06 · sin medico de la especialidad: pasa a Medicina General', ()
=> {
    const plan = planDe({ citas: [cita(1182, '08:30')], candidatos: [candidato(1,
'Medicina General')] });
    expect(plan.reasignar).toEqual([{ id_cita: 1182, id_medico: 1 }]);
  });

  test('UT-RRM09-07 · la misma especialidad tiene prioridad aunque Medicina
General tenga menos carga', () => {
    const plan = planDe({
      citas: [cita(1182, '08:30')],
      candidatos: [candidato(1, 'Medicina General'), candidato(6, 'Pediatria',
[ocupada('09:00'), ocupada('10:00')])]
    });
    expect(plan.reasignar[0].id_medico).toBe(6);
  });
```

```
  test('UT-RRM09-08 · sin candidatos: la cita se cancela', () => {
    const plan = planDe({ citas: [cita(1182, '08:30')], candidatos: [] });
    expect(plan.reasignar).toEqual([]);
    expect(plan.cancelar).toEqual([1182]);
  });

  test('UT-RRM09-09 · cita un sabado: nadie tiene franja ese dia, se cancela', ()
=> {
    const plan = planDe({ citas: [cita(3744, '09:00', SABADO, 6)], candidatos:
[candidato(6, 'Pediatria')] });
    expect(plan.cancelar).toEqual([3744]);
  });

  test('UT-RRM09-10 · cita en el horario de almuerzo (12:30): se cancela', () =>
{
    expect(planDe({ citas: [cita(1, '12:30')], candidatos: [candidato(6,
'Pediatria')] }).cancelar).toEqual([1]);
  });

  test('UT-RRM09-11 · el fin de la franja no esta incluido: 12:00 se cancela,
08:00 se reasigna', () => {
    const plan = planDe({ citas: [cita(1, '12:00'), cita(2, '08:00')],
candidatos: [candidato(6, 'Pediatria')] });
    expect(plan.cancelar).toEqual([1]);
    expect(plan.reasignar).toEqual([{ id_cita: 2, id_medico: 6 }]);
  });
});

// ------------------------------------------------------------
// C. Disponibilidad del candidato
// ------------------------------------------------------------
describe('WF-09 · Reparte reemplazos · C. Disponibilidad', () => {
  test('UT-RRM09-12 · el candidato ya tiene una cita a esa hora: no puede
tomarla', () => {
    const plan = planDe({ citas: [cita(1, '08:30')], candidatos: [candidato(6,
'Pediatria', [ocupada('08:30')])] });
    expect(plan.cancelar).toEqual([1]);
  });

  test('UT-RRM09-13 · una cita del candidato en otra fecha no estorba', () => {
    const plan = planDe({ citas: [cita(1, '08:30')], candidatos: [candidato(6,
'Pediatria', [ocupada('08:30', '2026-11-11')])] });
    expect(plan.reasignar).toEqual([{ id_cita: 1, id_medico: 6 }]);
  });
```

```
  test('UT-RRM09-14 · la franja del candidato ya llego a su cupo: no puede
tomarla', () => {
    const lleno = candidato(6, 'Pediatria', [ocupada('08:00'), ocupada('08:30')],
horarioEstandar(2));
    expect(planDe({ citas: [cita(1, '10:00')], candidatos: [lleno]
}).cancelar).toEqual([1]);
  });

  test('UT-RRM09-15 · la franja de la manana llena no impide reasignar por la
tarde', () => {
    const llenoManana = candidato(6, 'Pediatria', [ocupada('08:00'),
ocupada('08:30')], horarioEstandar(2));
    expect(planDe({ citas: [cita(1, '15:00')], candidatos: [llenoManana]
}).reasignar).toEqual([{ id_cita: 1, id_medico: 6 }]);
  });

  test('UT-RRM09-16 · si el candidato de la especialidad no puede, se busca en
Medicina General', () => {
    const plan = planDe({
      citas: [cita(1, '08:30')],
      candidatos: [candidato(6, 'Pediatria', [ocupada('08:30')]), candidato(1,
'Medicina General')]
    });
    expect(plan.reasignar).toEqual([{ id_cita: 1, id_medico: 1 }]);
  });
});

// ------------------------------------------------------------
// D. Eleccion entre varios candidatos
// ------------------------------------------------------------
describe('WF-09 · Reparte reemplazos · D. Eleccion', () => {
  test('UT-RRM09-17 · gana el de menor carga ese dia', () => {
    const plan = planDe({
      citas: [cita(1, '10:00')],
      candidatos: [candidato(1, 'Medicina General', [ocupada('08:00'),
ocupada('08:30')]), candidato(4, 'Medicina General', [ocupada('08:00')])],
      medico: { id: 9, especialidad: 'Ginecologia', baja_en: null }
    });
    expect(plan.reasignar[0].id_medico).toBe(4);
  });

  test('UT-RRM09-18 · en empate de carga, gana el id menor', () => {
    const plan = planDe({
      citas: [cita(1, '10:00')],
```

```
      candidatos: [candidato(4, 'Medicina General'), candidato(1, 'Medicina
General')],
      medico: { id: 9, especialidad: 'Ginecologia', baja_en: null }
    });
    expect(plan.reasignar[0].id_medico).toBe(1);
  });

  test('UT-RRM09-19 · si el que sale es de Medicina General, no se busca en otra
especialidad', () => {
    const plan = planDe({
      medico: { id: 1, especialidad: 'Medicina General', baja_en: null },
      citas: [cita(1, '08:30')],
      candidatos: [candidato(4, 'Medicina General', [ocupada('08:30')]),
candidato(2, 'Pediatria')]
    });
    expect(plan.cancelar).toEqual([1]);
  });
});

// ------------------------------------------------------------
// E. Memoria entre citas del mismo plan
// ------------------------------------------------------------
describe('WF-09 · Reparte reemplazos · E. Memoria', () => {
  test('UT-RRM09-20 · siete citas del martes pasan al pediatra libre (caso real
del medico 2)', () => {
    const horas = ['08:30', '09:00', '09:30', '10:00', '10:30', '11:00',
'15:00'];
    const citas = horas.map((h, i) => cita(100 + i, h));
    const plan = planDe({ citas, candidatos: [candidato(6, 'Pediatria')] });
    expect(plan.reasignar).toHaveLength(7);
    expect(plan.reasignar.every(r => r.id_medico === 6)).toBe(true);
    expect(plan.cancelar).toEqual([]);
  });

  test('UT-RRM09-21 · el cupo se descuenta en memoria: la segunda cita va a otro
candidato', () => {
    const plan = planDe({
      citas: [cita(1, '08:00'), cita(2, '08:30')],
      candidatos: [candidato(6, 'Pediatria', [], horarioEstandar(1)),
candidato(1, 'Medicina General')]
    });
    expect(plan.reasignar).toEqual([{ id_cita: 1, id_medico: 6 }, { id_cita: 2,
id_medico: 1 }]);
  });
```

```
  test('UT-RRM09-22 · sin otro candidato, la cita que excede el cupo se cancela',
() => {
    const plan = planDe({
      citas: [cita(1, '08:00'), cita(2, '08:30')],
      candidatos: [candidato(6, 'Pediatria', [], horarioEstandar(1))]
    });
    expect(plan.reasignar).toEqual([{ id_cita: 1, id_medico: 6 }]);
    expect(plan.cancelar).toEqual([2]);
  });

  test('UT-RRM09-23 · la carga en memoria cambia la eleccion de la siguiente
cita', () => {
    const plan = planDe({
      medico: { id: 9, especialidad: 'Ginecologia', baja_en: null },
      citas: [cita(1, '08:00'), cita(2, '08:30')],
      candidatos: [candidato(1, 'Medicina General'), candidato(4, 'Medicina
General')]
    });
    expect(plan.reasignar).toEqual([{ id_cita: 1, id_medico: 1 }, { id_cita: 2,
id_medico: 4 }]);
  });
});

// ------------------------------------------------------------
// F. Formato de entrada y contrato
// ------------------------------------------------------------
describe('WF-09 · Reparte reemplazos · F. Formato y contrato', () => {
  test('UT-RRM09-24 · medico, citas y candidatos como texto json: se interpretan
igual', () => {
    const { salida } = ejecutar(reparteReemplazos, {
      entrada: [{
        medico: JSON.stringify(PEDIATRA_SALE),
        citas: JSON.stringify([cita(1, '08:30')]),
        candidatos: JSON.stringify([candidato(6, 'Pediatria')])
      }],
      anteriores: { 'Valida datos': { id: 2, baja: false } }
    });
    expect(salida[0].json.reasignar).toEqual([{ id_cita: 1, id_medico: 6 }]);
  });

  test('UT-RRM09-25 · franjas y ocupadas del candidato como texto json: se
interpretan igual', () => {
    const c = { id: 6, especialidad: 'Pediatria', franjas:
JSON.stringify(horarioEstandar()), ocupadas: JSON.stringify([ocupada('08:30')])
};
```

```
    expect(planDe({ citas: [cita(1, '08:30'), cita(2, '09:00')], candidatos: [c]
}))
      .toMatchObject({ cancelar: [1], reasignar: [{ id_cita: 2, id_medico: 6 }]
});
  });

  test('UT-RRM09-26 · las listas tambien viajan como texto json para la consulta
siguiente', () => {
    const plan = planDe({ citas: [cita(1, '08:30'), cita(3744, '09:00', SABADO,
6)], candidatos: [candidato(6, 'Pediatria')] });
    expect(plan.reasignar_json).toBe(JSON.stringify(plan.reasignar));
    expect(plan.cancelar_json).toBe('[3744]');
  });

  test.each([
    ['UT-RRM09-27', true],
    ['UT-RRM09-28', false]
  ])('%s · la bandera baja %p sale de Valida datos', (_, baja) => {
    expect(planDe({ baja }).baja).toBe(baja);
  });

  test('UT-RRM09-29 · el plan conserva el orden de las citas', () => {
    const plan = planDe({ citas: [cita(30, '10:00'), cita(10, '08:00'), cita(20,
'09:00')], candidatos: [candidato(6, 'Pediatria')] });
    expect(plan.reasignar.map(r => r.id_cita)).toEqual([30, 10, 20]);
  });

  test('UT-RRM09-30 · consulta Valida datos (verify)', () => {
    expect(repartir({}).$).toHaveBeenCalledWith('Valida datos');
  });

  test('UT-RRM09-31 · devuelve un item con los siete campos del plan', () => {
    const { salida } = repartir({});
    expect(salida).toHaveLength(1);
    expect(Object.keys(salida[0].json).sort()).toEqual(['baja', 'cancelar',
'cancelar_json', 'id_medico', 'reasignar', 'reasignar_json', 'resultado']);
  });
});

// ------------------------------------------------------------
// G. Traslapes (hipotesis C)
// ------------------------------------------------------------
// la misma regla que en Reparte cupos: cada cita dura 30 minutos, pero
// solo se compara la hora exacta.
describe('WF-09 · Reparte reemplazos · G. Traslapes', () => {
```

```
  test('UT-RRM09-32 · cita a las 08:15 y el candidato con una cita a las 08:00: no debe pasarle (hipotesis C)', () => {
    const plan = planDe({ citas: [cita(1, '08:15')], candidatos: [candidato(6, 'Pediatria', [ocupada('08:00')])] });
    expect(plan.reasignar).toEqual([]);
    expect(plan.cancelar).toEqual([1]);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf09.repartereemplazos.test.js
 FAIL  tests/sigac.wf09.repartereemplazos.test.js
  WF-09 · Reparte reemplazos · A. Estado del medico
    ✓ UT-RRM09-01 · medico nulo: no_existe (4 ms)
    ✓ UT-RRM09-02 · medico con baja: ya_baja, sin plan (1 ms)
    ✓ UT-RRM09-03 · medico activo sin citas: ok con listas vacias (1 ms)
    ✓ UT-RRM09-04 · medico ya inactivo pero sin baja: se puede dar de baja (ok)
  WF-09 · Reparte reemplazos · B. Reemplazo
    ✓ UT-RRM09-05 · un medico de la misma especialidad libre: la cita pasa a el (1 ms)
    ✓ UT-RRM09-06 · sin medico de la especialidad: pasa a Medicina General (1 ms)
    ✓ UT-RRM09-07 · la misma especialidad tiene prioridad aunque Medicina General tenga menos carga
    ✓ UT-RRM09-08 · sin candidatos: la cita se cancela (1 ms)
    ✓ UT-RRM09-09 · cita un sabado: nadie tiene franja ese dia, se cancela
    ✓ UT-RRM09-10 · cita en el horario de almuerzo (12:30): se cancela
    ✓ UT-RRM09-11 · el fin de la franja no esta incluido: 12:00 se cancela, 08:00 se reasigna (1 ms)
  WF-09 · Reparte reemplazos · C. Disponibilidad
    ✓ UT-RRM09-12 · el candidato ya tiene una cita a esa hora: no puede tomarla
    ✓ UT-RRM09-13 · una cita del candidato en otra fecha no estorba (1 ms)
    ✓ UT-RRM09-14 · la franja del candidato ya llego a su cupo: no puede tomarla
    ✓ UT-RRM09-15 · la franja de la manana llena no impide reasignar por la tarde
    ✓ UT-RRM09-16 · si el candidato de la especialidad no puede, se busca en Medicina General
  WF-09 · Reparte reemplazos · D. Eleccion
    ✓ UT-RRM09-17 · gana el de menor carga ese dia
    ✓ UT-RRM09-18 · en empate de carga, gana el id menor (1 ms)
    ✓ UT-RRM09-19 · si el que sale es de Medicina General, no se busca en otra especialidad
  WF-09 · Reparte reemplazos · E. Memoria
    ✓ UT-RRM09-20 · siete citas del martes pasan al pediatra libre (caso real del medico 2) (1 ms)
    ✓ UT-RRM09-21 · el cupo se descuenta en memoria: la segunda cita va a otro candidato (1 ms)
    ✓ UT-RRM09-22 · sin otro candidato, la cita que excede el cupo se cancela
    ✓ UT-RRM09-23 · la carga en memoria cambia la eleccion de la siguiente cita (1 ms)
  WF-09 · Reparte reemplazos · F. Formato y contrato
    ✓ UT-RRM09-24 · medico, citas y candidatos como texto json: se interpretan igual
    ✓ UT-RRM09-25 · franjas y ocupadas del candidato como texto json: se interpretan igual
    ✓ UT-RRM09-26 · las listas tambien viajan como texto json para la consulta siguiente (1 ms)
    ✓ UT-RRM09-27 · la bandera baja true sale de Valida datos
    ✓ UT-RRM09-28 · la bandera baja false sale de Valida datos
    ✓ UT-RRM09-29 · el plan conserva el orden de las citas (1 ms)
    ✓ UT-RRM09-30 · consulta Valida datos (verify)
    ✓ UT-RRM09-31 · devuelve un item con los siete campos del plan
  WF-09 · Reparte reemplazos · G. Traslapes
    ✕ UT-RRM09-32 · cita a las 08:15 y el candidato con una cita a las 08:00: no debe pasarle (hipotesis C) (3 ms)
```

```
  ● WF-09 · Reparte reemplazos · G. Traslapes › UT-RRM09-32 · cita a las 08:15 y el candidato con una cita a las 08:00: no debe pasarle (hipotesis C)

    expect(received).toEqual(expected) // deep equality

    - Expected  - 1
    + Received  + 6

    - Array []
    + Array [
    +   Object {
    +     "id_cita": 1,
    +     "id_medico": 6,
    +   },
    + ]

      285 |   test('UT-RRM09-32 · cita a las 08:15 y el candidato con una cita a las 08:00: no debe pasarle (hipotesis C)', () => {
      286 |     const plan = planDe({ citas: [cita(1, '08:15')], candidatos: [candidato(6, 'Pediatria', [ocupada('08:00')])] });
    > 287 |     expect(plan.reasignar).toEqual([]);
          |                            ^
      288 |     expect(plan.cancelar).toEqual([1]);
      289 |   });
      290 | });

      at Object.toEqual (tests/sigac.wf09.repartereemplazos.test.js:287:28)

  Test Suites: 1 failed, 1 total
  Tests:       1 failed, 31 passed, 32 total
  Snapshots:   0 total
  Time:        3.703 s
  Ran all test suites matching tests/sigac.wf09.repartereemplazos.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.54.2.1 Defecto detectado por las pruebas (12.25).

La prueba UT-RRM09-32 falló: cada cita dura 30 minutos, pero el nodo solo evita repetir la hora exacta, de modo que una cita a las 08:15 se cruza con otra a las 08:00 del mismo médico. Se corrige en n8n y, tras exportar el flujo y regenerar los nodos, la misma prueba confirma la corrección.

1.54.3 Nuevo código de Reparte Reemplazos.

```
const lectura = $input.first().json;
const datos = $('Valida datos').first().json;

// las columnas json normalmente llegan ya como objeto; si llegaran
// como texto se convierten aqui
function leer(valor, porDefecto) {
  if (valor === null || valor === undefined) return porDefecto;
  return typeof valor === 'string' ? JSON.parse(valor) : valor;
}

const medico = leer(lectura.medico, null);
const citas = leer(lectura.citas, []);
const candidatos = leer(lectura.candidatos, []);

if (!medico) {
  return [{ json: { resultado: 'no_existe' } }];
}
if (medico.baja_en) {
  return [{ json: { resultado: 'ya_baja' } }];
}

// "HH:MI" a minutos desde medianoche, para comparar horas como numeros
function minutos(hora) {
  return Number(hora.slice(0, 2)) * 60 + Number(hora.slice(3, 5));
}

// ocupacion en memoria: por candidato y por fecha, la lista de horas
const ocupacion = {};
for (const k of candidatos) {
  ocupacion[k.id] = {};
  for (const o of leer(k.ocupadas, [])) {
    if (!ocupacion[k.id][o.fecha]) ocupacion[k.id][o.fecha] = [];
    ocupacion[k.id][o.fecha].push(o.hora);
  }
}

const GENERAL = 'Medicina General';
```

```
const mismaEspecialidad = candidatos.filter(k => k.especialidad ===
medico.especialidad);
const generales = candidatos.filter(k => k.especialidad === GENERAL);

// si el que sale ya es de Medicina General, los dos grupos son el
// mismo y basta con revisar uno
const grupos = medico.especialidad === GENERAL ? [generales] :
[mismaEspecialidad, generales];

// devuelve el candidato con menor carga que puede atender la cita,
// o null si ninguno del grupo puede
function buscar(grupo, cita) {
  const m = minutos(cita.hora);
  let mejor = null;

  for (const k of grupo) {
    // la franja de ese dia de la semana que contiene la hora de la cita
    const franja = leer(k.franjas, []).find(f =>
      f.dia === cita.dia && minutos(f.inicio) <= m && m < minutos(f.fin));
    if (!franja) continue;

    const horas = ocupacion[k.id][cita.fecha] || [];

    // ya tiene una cita que se cruza con esta. defecto 12.25: cada cita
    // dura 30 minutos, pero antes solo se comparaba la hora exacta, y una
    // cita a las 08:15 se le pasaba a un medico que ya tenia otra a las
    // 08:00. ahora se descarta si alguna empieza a menos de 30 minutos,
    // igual que en Reparte cupos de WF-02.
    if (horas.some(h => Math.abs(minutos(h) - m) < 30)) continue;

    // la franja ya llego a su cupo
    const enFranja = horas.filter(h =>
      minutos(h) >= minutos(franja.inicio) && minutos(h) <
minutos(franja.fin)).length;
    if (enFranja >= franja.cupos) continue;

    // menor carga del dia; en empate, el id menor
    const carga = horas.length;
    if (!mejor || carga < mejor.carga || (carga === mejor.carga && k.id <
mejor.id)) {
      mejor = { id: k.id, carga: carga };
    }
  }
  return mejor;
}
```

```
const reasignar = [];
const cancelar = [];

for (const cita of citas) {
  let elegido = null;
  for (const grupo of grupos) {
    elegido = buscar(grupo, cita);
    if (elegido) break;
  }

  if (elegido) {
    reasignar.push({ id_cita: cita.id, id_medico: elegido.id });
    if (!ocupacion[elegido.id][cita.fecha]) ocupacion[elegido.id][cita.fecha] =
[];
    ocupacion[elegido.id][cita.fecha].push(cita.hora);
  } else {
    cancelar.push(cita.id);
  }
}

return [{
  json: {
    resultado: 'ok',
    id_medico: medico.id,
    baja: datos.baja,
    reasignar: reasignar,
    cancelar: cancelar,
    // las dos listas viajan como texto json para la consulta siguiente
    reasignar_json: JSON.stringify(reasignar),
    cancelar_json: JSON.stringify(cancelar)
  }
}];
```

1.54.4 Se corrió el mismo código test pero ya con la corrección del código base.

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf09.repartereemplazos.test.js
 PASS  tests/sigac.wf09.repartereemplazos.test.js
  WF-09 · Reparte reemplazos · A. Estado del medico
    ✓ UT-RRM09-01 · medico nulo: no_existe (5 ms)
    ✓ UT-RRM09-02 · medico con baja: ya_baja, sin plan (1 ms)
    ✓ UT-RRM09-03 · medico activo sin citas: ok con listas vacias (1 ms)
    ✓ UT-RRM09-04 · medico ya inactivo pero sin baja: se puede dar de baja (ok)
  WF-09 · Reparte reemplazos · B. Reemplazo
    ✓ UT-RRM09-05 · un medico de la misma especialidad libre: la cita pasa a el (2 ms)
    ✓ UT-RRM09-06 · sin medico de la especialidad: pasa a Medicina General (1 ms)
    ✓ UT-RRM09-07 · la misma especialidad tiene prioridad aunque Medicina General tenga menos carga (1 ms)
    ✓ UT-RRM09-08 · sin candidatos: la cita se cancela (1 ms)
    ✓ UT-RRM09-09 · cita un sabado: nadie tiene franja ese dia, se cancela
    ✓ UT-RRM09-10 · cita en el horario de almuerzo (12:30): se cancela (1 ms)
    ✓ UT-RRM09-11 · el fin de la franja no esta incluido: 12:00 se cancela, 08:00 se reasigna (7 ms)
  WF-09 · Reparte reemplazos · C. Disponibilidad
    ✓ UT-RRM09-12 · el candidato ya tiene una cita a esa hora: no puede tomarla
    ✓ UT-RRM09-13 · una cita del candidato en otra fecha no estorba (1 ms)
    ✓ UT-RRM09-14 · la franja del candidato ya llego a su cupo: no puede tomarla
    ✓ UT-RRM09-15 · la franja de la manana llena no impide reasignar por la tarde
    ✓ UT-RRM09-16 · si el candidato de la especialidad no puede, se busca en Medicina General (1 ms)
  WF-09 · Reparte reemplazos · D. Eleccion
    ✓ UT-RRM09-17 · gana el de menor carga ese dia
    ✓ UT-RRM09-18 · en empate de carga, gana el id menor
    ✓ UT-RRM09-19 · si el que sale es de Medicina General, no se busca en otra especialidad (1 ms)
  WF-09 · Reparte reemplazos · E. Memoria
    ✓ UT-RRM09-20 · siete citas del martes pasan al pediatra libre (caso real del medico 2) (1 ms)
    ✓ UT-RRM09-21 · el cupo se descuenta en memoria: la segunda cita va a otro candidato (1 ms)
    ✓ UT-RRM09-22 · sin otro candidato, la cita que excede el cupo se cancela (1 ms)
    ✓ UT-RRM09-23 · la carga en memoria cambia la eleccion de la siguiente cita
  WF-09 · Reparte reemplazos · F. Formato y contrato
    ✓ UT-RRM09-24 · medico, citas y candidatos como texto json: se interpretan igual (1 ms)
    ✓ UT-RRM09-25 · franjas y ocupadas del candidato como texto json: se interpretan igual (1 ms)
    ✓ UT-RRM09-26 · las listas tambien viajan como texto json para la consulta siguiente (1 ms)
    ✓ UT-RRM09-27 · la bandera baja true sale de Valida datos (1 ms)
    ✓ UT-RRM09-28 · la bandera baja false sale de Valida datos (1 ms)
    ✓ UT-RRM09-29 · el plan conserva el orden de las citas
    ✓ UT-RRM09-30 · consulta Valida datos (verify) (1 ms)
    ✓ UT-RRM09-31 · devuelve un item con los siete campos del plan
  WF-09 · Reparte reemplazos · G. Traslapes
    ✓ UT-RRM09-32 · cita a las 08:15 y el candidato con una cita a las 08:00: no debe pasarle (hipotesis C) (1 ms)

Test Suites: 1 passed, 1 total
Tests:       32 passed, 32 total
Snapshots:   0 total
Time:        3.8 s
Ran all test suites matching tests/sigac.wf09.repartereemplazos.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.55 Rechazo estado medico (WF-09)

El nodo Rechazo estado medico responde en los dos casos en que no se aplica nada al desactivar o dar de baja a un médico: no existe (404) o ya fue dado de baja (409). Lo alimenta la salida false del nodo ¿Se puede aplicar?.

1.55.1 Código base del nodo Rechazo estado medico

```
const r = $input.first().json;
const id = $('Valida datos').first().json.id;

if (r.resultado === 'no_existe') {
  return [{ json: { codigo: 404, estado: 'rechazado', mensaje: 'No existe el
medico ' + id } }];
}

return [{ json: { codigo: 409, estado: 'rechazado', mensaje: 'El medico ' + id +
' ya fue dado de baja; la baja es definitiva' } }];
```

1.55.2 Código de test del nodo Rechazo estado medico

```
// sigac.wf09.rechazoestadomedico.test.js
//
// pruebas unitarias del nodo Rechazo estado medico de WF-09. responde en
// los dos casos en que no se aplica nada al desactivar o dar de baja a un
// medico: no existe (404) o ya fue dado de baja (409). lo alimenta la
// salida false de ¿Se puede aplicar?
//
// cada prueba lleva su ID de la tabla de pruebas: UT-REM09-NN

'use strict';

const rechazoEstadoMedico = require('../nodos/wf09_rechazo_estado_medico.js');
const { ejecutar } = require('../simulacion.js');

const responder = (resultado, id = 6) =>
  ejecutar(rechazoEstadoMedico, { entrada: [resultado], anteriores: { 'Valida
datos': { id } } });
const salidaDe = (resultado, id) => responder(resultado, id).salida[0].json;

describe('WF-09 · Rechazo estado medico', () => {
  test('UT-REM09-01 · no_existe: 404 con el id', () => {
    expect(salidaDe({ resultado: 'no_existe' }, 99)).toEqual({ codigo: 404,
estado: 'rechazado', mensaje: 'No existe el medico 99' });
  });

  test('UT-REM09-02 · ya_baja: 409, la baja es definitiva', () => {
    expect(salidaDe({ resultado: 'ya_baja' }, 6)).toEqual({ codigo: 409, estado:
'rechazado', mensaje: 'El medico 6 ya fue dado de baja; la baja es definitiva'
});
```

```
  });

  test('UT-REM09-03 · cualquier otro resultado que llegara aqui se trata como
ya_baja', () => {
    expect(salidaDe({ resultado: 'desconocido' }).codigo).toBe(409);
  });

  test('UT-REM09-04 · solo tres campos: no expone citas ni tokens', () => {
    expect(Object.keys(salidaDe({ resultado: 'ya_baja', reasignar: [1], cancelar:
[2] })).sort()).toEqual(['codigo', 'estado', 'mensaje']);
  });

  test('UT-REM09-05 · el id del mensaje sale de Valida datos (verify)', () => {
    expect(responder({ resultado: 'no_existe' }).$).toHaveBeenCalledWith('Valida
datos');
  });

  test('UT-REM09-06 · devuelve una lista con un solo item', () => {
    expect(responder({ resultado: 'ya_baja' }).salida).toHaveLength(1);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf09.rechazoestadomedico.test.js
 PASS  tests/sigac.wf09.rechazoestadomedico.test.js
  WF-09 · Rechazo estado medico
    ✓ UT-REM09-01 · no_existe: 404 con el id (5 ms)
    ✓ UT-REM09-02 · ya_baja: 409, la baja es definitiva
    ✓ UT-REM09-03 · cualquier otro resultado que llegara aqui se trata como ya_baja (1 ms)
    ✓ UT-REM09-04 · solo tres campos: no expone citas ni tokens (1 ms)
    ✓ UT-REM09-05 · el id del mensaje sale de Valida datos (verify)
    ✓ UT-REM09-06 · devuelve una lista con un solo item

Test Suites: 1 passed, 1 total
Tests:       6 passed, 6 total
Snapshots:   0 total
Time:        3.691 s
Ran all test suites matching tests/sigac.wf09.rechazoestadomedico.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.56 Medico desactivado (WF-09)

El nodo Medico desactivado arma la respuesta 200 con el resumen de lo que pasó con las citas del médico: cuántas se reasignaron, cuántas se cancelaron, cuántas preferencias se liberaron y cuántos tokens se desactivaron. Los conteos llegan como texto desde PostgreSQL y se convierten a número. El detalle de los pacientes no va en la respuesta: va a los correos.

1.56.1 Código base del nodo Medico desactivado

```
const r = $input.first().json;
const id = $('Valida datos').first().json.id;

return [{
  json: {
    codigo: 200,
    estado: 'actualizado',
    mensaje: r.dado_de_baja
      ? 'Medico ' + id + ' dado de baja'
      : 'Medico ' + id + ' desactivado',
    citas_reasignadas: Number(r.reasignadas),
    citas_canceladas: Number(r.canceladas),
    preferencias_liberadas: Number(r.preferencias_liberadas),
    tokens_desactivados: Number(r.tokens_desactivados),
    hash_auditoria: r.hash_auditoria
  }
}];
```

1.56.2 Código de test del nodo Medico desactivado

```
// sigac.wf09.medicodesactivado.test.js
//
// pruebas unitarias del nodo Medico desactivado de WF-09. arma la
// respuesta 200 con el resumen de lo que paso con las citas del medico:
// reasignadas, canceladas, preferencias liberadas y tokens desactivados.
// los conteos llegan como texto desde postgres.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-MDE09-NN

'use strict';

const medicoDesactivado = require('../nodos/wf09_medico_desactivado.js');
const { ejecutar } = require('../simulacion.js');

const APLICADO = {
  dado_de_baja: false, reasignadas: '7', canceladas: '0', preferencias_liberadas:
'0',
  tokens_desactivados: '1', hash_auditoria: 'abc',
  detalle_reasignadas: [{ correo: 'a@correo.com', paciente: 'Ana Test' }],
detalle_canceladas: []
};
```

```
const responder = (aplicado = APLICADO, id = 2) =>
  ejecutar(medicoDesactivado, { entrada: [aplicado], anteriores: { 'Valida
datos': { id } } });
const salidaDe = (aplicado, id) => responder(aplicado, id).salida[0].json;

describe('WF-09 · Medico desactivado', () => {
  test('UT-MDE09-01 · desactivacion: respuesta completa', () => {
    expect(salidaDe()).toEqual({
      codigo: 200,
      estado: 'actualizado',
      mensaje: 'Medico 2 desactivado',
      citas_reasignadas: 7,
      citas_canceladas: 0,
      preferencias_liberadas: 0,
      tokens_desactivados: 1,
      hash_auditoria: 'abc'
    });
  });

  test('UT-MDE09-02 · baja: el mensaje dice dado de baja', () => {
    expect(salidaDe({ ...APLICADO, dado_de_baja: true }, 6).mensaje).toBe('Medico
6 dado de baja');
  });

  test('UT-MDE09-03 · los cuatro conteos llegan como texto: salen como numero',
() => {
    const salida = salidaDe({ ...APLICADO, reasignadas: '7', canceladas: '1',
preferencias_liberadas: '1', tokens_desactivados: '1' });
    expect([salida.citas_reasignadas, salida.citas_canceladas,
salida.preferencias_liberadas, salida.tokens_desactivados]).toEqual([7, 1, 1,
1]);
  });

  test('UT-MDE09-04 · medico sin citas: todo en cero', () => {
    const salida = salidaDe({ ...APLICADO, reasignadas: '0', canceladas: '0',
tokens_desactivados: '0' });
    expect([salida.citas_reasignadas, salida.citas_canceladas,
salida.tokens_desactivados]).toEqual([0, 0, 0]);
  });

  test('UT-MDE09-05 · no devuelve el detalle de los pacientes: ese va a los
correos', () => {
    const texto = JSON.stringify(responder().salida);
    expect(texto).not.toContain('a@correo.com');
```

```
    expect(texto).not.toContain('Ana Test');
  });

  test('UT-MDE09-06 · consulta Valida datos (verify)', () => {
    expect(responder().$).toHaveBeenCalledWith('Valida datos');
  });

  test('UT-MDE09-07 · devuelve una lista con un solo item de ocho campos', () =>
{
    const { salida } = responder();
    expect(salida).toHaveLength(1);
    expect(Object.keys(salida[0].json)).toHaveLength(8);
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf09.medicodesactivado.test.js
 PASS  tests/sigac.wf09.medicodesactivado.test.js
  WF-09 · Medico desactivado
    ✓ UT-MDE09-01 · desactivacion: respuesta completa (5 ms)
    ✓ UT-MDE09-02 · baja: el mensaje dice dado de baja (3 ms)
    ✓ UT-MDE09-03 · los cuatro conteos llegan como texto: salen como numero (1 ms)
    ✓ UT-MDE09-04 · medico sin citas: todo en cero
    ✓ UT-MDE09-05 · no devuelve el detalle de los pacientes: ese va a los correos
    ✓ UT-MDE09-06 · consulta Valida datos (verify)
    ✓ UT-MDE09-07 · devuelve una lista con un solo item de ocho campos (1 ms)

Test Suites: 1 passed, 1 total
Tests:       7 passed, 7 total
Snapshots:   0 total
Time:        3.661 s
Ran all test suites matching tests/sigac.wf09.medicodesactivado.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

1.57 Prepara correos (WF-09)

El nodo Prepara correos arma un item por cada paciente que debe recibir aviso: uno por cada cita reasignada, para decirle que su cita sigue en pie con otro médico, y uno por cada cita cancelada, para pedirle que llame a la clínica. Es el único nodo Code del sistema que devuelve varios items. Si el médico no tenía citas, devuelve una lista vacía y el nodo Envia correo no se ejecuta.

1.57.1 Código base del nodo Prepara correos

```
const r = $input.first().json;

function leer(valor) {
  if (!valor) return [];
  return typeof valor === 'string' ? JSON.parse(valor) : valor;
}

const correos = [];

for (const c of leer(r.detalle_reasignadas)) {
  correos.push({
    json: {
      para: c.correo,
      asunto: 'SIGAC - Cambio de medico en su cita del ' + c.fecha,
      texto:
        'Estimado(a) ' + c.paciente + ':\n\n' +
        'Le informamos que su cita del ' + c.fecha + ' a las ' + c.hora +
        ' sigue en pie, pero sera atendida por ' + c.medico_nuevo +
        ' (' + c.especialidad + '), porque el medico asignado anteriormente ya no
esta disponible.\n\n' +
        'No necesita hacer nada: la fecha y la hora se mantienen.\n\n' +
        'Clinica - SIGAC'
    }
  });
}

for (const c of leer(r.detalle_canceladas)) {
  correos.push({
    json: {
      para: c.correo,
      asunto: 'SIGAC - Su cita del ' + c.fecha + ' no esta disponible',
      texto:
        'Estimado(a) ' + c.paciente + ':\n\n' +
        'Lamentamos informarle que la cita que tenia programada para el ' +
c.fecha +
        ' a las ' + c.hora + ' ya no esta disponible, porque el medico asignado
no podra atenderla ' +
        'y no encontramos otro medico libre en ese horario.\n\n' +
```

```
        'Por favor, llame a la clinica para recibir mas informacion y reprogramar
su cita.\n\n' +
        'Clinica - SIGAC'
    }
  });
}

return correos;
```

1.57.2 Código de test del nodo Prepara correos

```
// sigac.wf09.preparacorreos.test.js
//
// pruebas unitarias del nodo Prepara correos de WF-09. arma un item por
// cada paciente que debe recibir aviso: uno por cita reasignada y uno
// por cita cancelada. es el unico nodo Code del sistema que devuelve
// varios items. si el medico no tenia citas, devuelve una lista vacia y
// Envia correo no se ejecuta.
//
// cada prueba lleva su ID de la tabla de pruebas: UT-PRC09-NN

'use strict';

const preparaCorreos = require('../nodos/wf09_prepara_correos.js');
const { ejecutar } = require('../simulacion.js');

const REASIGNADA = {
  id_cita: 1182, correo: 'a@correo.com', paciente: 'Ana Test', fecha:
'10/11/2026', hora: '08:30',
  medico_nuevo: 'Caal Tiul, Jorge Luis', especialidad: 'Pediatria'
};
const CANCELADA = { id_cita: 3744, correo: 'esp1@correo.com', paciente: 'Espera1
Prueba', fecha: '31/10/2026', hora: '09:00' };

const preparar = (detalle_reasignadas = [], detalle_canceladas = []) =>
  ejecutar(preparaCorreos, { entrada: [{ detalle_reasignadas, detalle_canceladas
}] });
const correos = (r, c) => preparar(r, c).salida.map(i => i.json);

// ------------------------------------------------------------
// A. Cantidad de correos
// ------------------------------------------------------------
describe('WF-09 · Prepara correos · A. Cantidad', () => {
  test('UT-PRC09-01 · una reasignada: un correo', () => {
    expect(correos([REASIGNADA])).toHaveLength(1);
  });
```

```
  test('UT-PRC09-02 · una cancelada: un correo', () => {
    expect(correos([], [CANCELADA])).toHaveLength(1);
  });

  test('UT-PRC09-03 · siete reasignadas y una cancelada: ocho correos', () => {
    const siete = Array.from({ length: 7 }, (_, i) => ({ ...REASIGNADA, id_cita:
i }));
    expect(correos(siete, [CANCELADA])).toHaveLength(8);
  });

  test('UT-PRC09-04 · sin citas: lista vacia, Envia correo no se ejecuta', () =>
{
    expect(preparar().salida).toEqual([]);
  });

  test.each([
    ['UT-PRC09-05', 'nulas', null, null],
    ['UT-PRC09-06', 'ausentes', undefined, undefined]
  ])('%s · listas %s: lista vacia, sin error', (_, __, r, c) => {
    expect(preparar(r, c).salida).toEqual([]);
  });

  test('UT-PRC09-07 · listas como texto json: se interpretan igual', () => {
    expect(correos(JSON.stringify([REASIGNADA]),
JSON.stringify([CANCELADA]))).toHaveLength(2);
  });

  test('UT-PRC09-08 · primero los avisos de reasignacion y despues los de
cancelacion', () => {
    const [a, b] = correos([REASIGNADA], [CANCELADA]);
    expect(a.para).toBe('a@correo.com');
    expect(b.para).toBe('esp1@correo.com');
  });
});

// ------------------------------------------------------------
// B. Correo de cambio de medico
// ------------------------------------------------------------
describe('WF-09 · Prepara correos · B. Cambio de medico', () => {
  const [correo] = correos([REASIGNADA]);

  test('UT-PRC09-09 · va al correo del paciente', () => {
    expect(correo.para).toBe('a@correo.com');
  });
```

```
  test('UT-PRC09-10 · el asunto nombra el cambio de medico y la fecha', () => {
    expect(correo.asunto).toBe('SIGAC - Cambio de medico en su cita del
10/11/2026');
  });

  test('UT-PRC09-11 · el texto saluda al paciente por su nombre', () => {
    expect(correo.texto.startsWith('Estimado(a) Ana Test:')).toBe(true);
  });

  test('UT-PRC09-12 · el texto dice que la cita sigue en pie, con la misma fecha
y hora', () => {
    expect(correo.texto).toContain('su cita del 10/11/2026 a las 08:30 sigue en
pie');
    expect(correo.texto).toContain('la fecha y la hora se mantienen');
  });

  test('UT-PRC09-13 · el texto nombra al medico nuevo y su especialidad, con la
coma del nombre', () => {
    expect(correo.texto).toContain('Caal Tiul, Jorge Luis (Pediatria)');
  });
});

// ------------------------------------------------------------
// C. Correo de cancelacion
// ------------------------------------------------------------
describe('WF-09 · Prepara correos · C. Cancelacion', () => {
  const [correo] = correos([], [CANCELADA]);

  test('UT-PRC09-14 · va al correo del paciente', () => {
    expect(correo.para).toBe('esp1@correo.com');
  });

  test('UT-PRC09-15 · el asunto dice que la cita no esta disponible', () => {
    expect(correo.asunto).toBe('SIGAC - Su cita del 31/10/2026 no esta
disponible');
  });

  test('UT-PRC09-16 · el texto pide llamar a la clinica para reprogramar', () =>
{
    expect(correo.texto).toContain('llame a la clinica');
    expect(correo.texto).toContain('reprogramar');
  });

  test('UT-PRC09-17 · el texto incluye la fecha y la hora de la cita perdida', ()
=> {
```

```
    expect(correo.texto).toContain('31/10/2026 a las 09:00');
  });
});

// ------------------------------------------------------------
// D. Contrato y datos
// ------------------------------------------------------------
describe('WF-09 · Prepara correos · D. Contrato', () => {
  test('UT-PRC09-18 · cada item tiene exactamente para, asunto y texto', () => {
    for (const correo of correos([REASIGNADA], [CANCELADA])) {
      expect(Object.keys(correo).sort()).toEqual(['asunto', 'para', 'texto']);
    }
  });

  test('UT-PRC09-19 · los correos no incluyen el id de la cita ni datos
clinicos', () => {
    const texto = JSON.stringify(correos([{ ...REASIGNADA, diagnostico:
'Faringitis' }], [CANCELADA]));
    expect(texto).not.toContain('1182');
    expect(texto).not.toContain('Faringitis');
  });

  test('UT-PRC09-20 · no consulta ningun nodo anterior (verify)', () => {
    expect(preparar([REASIGNADA]).$).not.toHaveBeenCalled();
  });
});
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npx jest tests/sigac.wf09.preparacorreos.test.js
 PASS  tests/sigac.wf09.preparacorreos.test.js
  WF-09 · Prepara correos · A. Cantidad
    ✓ UT-PRC09-01 · una reasignada: un correo (4 ms)
    ✓ UT-PRC09-02 · una cancelada: un correo
    ✓ UT-PRC09-03 · siete reasignadas y una cancelada: ocho correos
    ✓ UT-PRC09-04 · sin citas: lista vacia, Envia correo no se ejecuta (1 ms)
    ✓ UT-PRC09-05 · listas nulas: lista vacia, sin error (1 ms)
    ✓ UT-PRC09-06 · listas ausentes: lista vacia, sin error
    ✓ UT-PRC09-07 · listas como texto json: se interpretan igual (1 ms)
    ✓ UT-PRC09-08 · primero los avisos de reasignacion y despues los de cancelacion
  WF-09 · Prepara correos · B. Cambio de medico
    ✓ UT-PRC09-09 · va al correo del paciente
    ✓ UT-PRC09-10 · el asunto nombra el cambio de medico y la fecha (1 ms)
    ✓ UT-PRC09-11 · el texto saluda al paciente por su nombre
    ✓ UT-PRC09-12 · el texto dice que la cita sigue en pie, con la misma fecha y hora (1 ms)
    ✓ UT-PRC09-13 · el texto nombra al medico nuevo y su especialidad, con la coma del nombre
  WF-09 · Prepara correos · C. Cancelacion
    ✓ UT-PRC09-14 · va al correo del paciente
    ✓ UT-PRC09-15 · el asunto dice que la cita no esta disponible
    ✓ UT-PRC09-16 · el texto pide llamar a la clinica para reprogramar
    ✓ UT-PRC09-17 · el texto incluye la fecha y la hora de la cita perdida
  WF-09 · Prepara correos · D. Contrato
    ✓ UT-PRC09-18 · cada item tiene exactamente para, asunto y texto (1 ms)
    ✓ UT-PRC09-19 · los correos no incluyen el id de la cita ni datos clinicos
    ✓ UT-PRC09-20 · no consulta ningun nodo anterior (verify)

Test Suites: 1 passed, 1 total
Tests:       20 passed, 20 total
Snapshots:   0 total
Time:        3.847 s
Ran all test suites matching tests/sigac.wf09.preparacorreos.test.js.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

2 Ejecución de todos los test y todos pasaron y ya con todos los nodos corregidos.

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npm run test:unitarias 2>&1 | grep -E "✕|Tests:|Test Suites:"
Test Suites: 58 passed, 58 total
Tests:       909 passed, 909 total
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```

3 Pruebas con simulación (Mockito)

3.1 Organización

Las pruebas con simulación no están en archivos aparte: forman parte de los 57 archivos de prueba, junto con las demás pruebas de cada nodo. Cada archivo verifica su nodo de forma completa, y eso incluye cómo el nodo usa sus dependencias. Sin embargo, para que el documento sea más ordenado y la aplicación de Mockito se pueda revisar en un solo lugar, en esta sección se presentan separadas del resto.

En Java, Mockito simula las dependencias de una clase (un repositorio, un servicio externo) para probarla de forma aislada, y después verifica cómo las utilizó. En SIGAC, las dependencias de un nodo Code son los nodos anteriores que consulta con $('Nombre del nodo') y las funciones propias de n8n, como this.helpers. Jest incluye su propio mecanismo de simulación, jest.fn(), que cumple las mismas funciones que Mockito.

3.2 Pruebas con simulación (Mockito) por nodo Code

Cada fila muestra las dos partes de una prueba con Mockito. La columna Simulación preparada corresponde a @Mock + when(...).thenReturn(...): lo que la prueba le entrega al nodo en lugar de los nodos reales de n8n. La columna Verificación corresponde a verify(...): lo que se comprobó sobre cómo el nodo usó esas dependencias.

Los tipos de prueba son cuatro:

- verify: el nodo consultó exactamente los nodos de los que depende.
- verify never: el nodo no consultó ningún otro nodo, porque no debe depender de ninguno.
- verify never condicional: el nodo no consulta una dependencia en un caso concreto.
- times y ArgumentCaptor: cuántas veces se llamó a un servicio simulado y con qué argumentos.

3.3 ¿Por qué 52 nodos y no 57?

De los 57 nodos Code del sistema, 5 no tienen una prueba con simulación: el Hash del token de WF-01, WF-07, WF-08 y WF-09, y Cifra datos sensibles de WF-01. Estos cinco nodos trabajan únicamente con su propia entrada y, en el caso del cifrado, con la variable de entorno de la clave; no consultan ningún otro nodo del flujo. Al no tener dependencias que simular, sus archivos de prueba se concentraron en verificar el cálculo de la huella y el cifrado, y no incluyeron una prueba de tipo verify never que confirmara esa ausencia de dependencias. Sus demás pruebas sí se ejecutaron y fueron superadas.

3.4 ¿Por qué 53 pruebas en 52 nodos?

Cada nodo tiene una prueba de simulación, salvo dos que tienen dos, porque verifican aspectos distintos de sus dependencias:

Compone el reporte (WF-06) tiene dos dependencias de naturaleza diferente: los nodos anteriores que consulta y una función interna de n8n, prepareBinaryData. La prueba UT-REP06-16 verifica los nodos consultados, y la UT-REP06-13 verifica cuántas veces se llamó a la función y con qué argumentos.

Recepcionista reactivada (WF-09) consulta el token nuevo solo cuando la reactivación procede. La prueba UT-RRE09-06 verifica que lo consulta en el caso exitoso, y la UT-RRE09-07 verifica que no lo consulta cuando la reactivación se rechaza.

Flujo	Nodo Code	Prueba	Tipo	Simulación preparada	Verificación	Estado
WF-01	Valida datos	UT-VAL01-75	**verify**	Hash del token devuelve una solicitud válida; Lee medicos activos, un catálogo de tres médicos; Verifica token, el rol recepcion	Consultó los tres nodos	**Pass**
WF-01	Registro exitoso	UT-REG01-05	**verify**	Guarda solicitud devuelve la solicitud 71 pendiente; la bitácora llega con su hash	Consultó Guarda solicitud	**Pass**
WF-01	Rechazo 400	UT-R40001-11	**verify never**	Entrada con dos errores de validación; ningún nodo simulado	No consultó ningún nodo	**Pass**
WF-01	Rechazo 401	UT-R40101-08	**verify never**	Entrada vacía; ningún nodo simulado	No consultó ningún nodo	**Pass**
WF-01	Rechazo 403	UT-R40301-08	**verify never**	Entrada vacía; ningún nodo simulado	No consultó ningún nodo	**Pass**
WF-01	Rechazo 409	UT-R40901-07	**verify never**	Entrada vacía; ningún nodo simulado	No consultó ningún nodo	**Pass**
WF-02	Calcula prioridad	UT-PRI02-15	**verify never**	Entrada con una solicitud pendiente; ningún nodo simulado	No consultó ningún nodo: trabaja solo con su entrada	**Pass**
WF-02	Reparte cupos	UT-REP02-29	**verify**	Busca cupo devuelve seis franjas de tres médicos; Calcula prioridad, una solicitud	Consultó Busca cupo y Calcula prioridad	**Pass**
WF-04	Hash del token	UT-HASH04-22	**verify never**	Entrada con una petición vacía; ningún nodo simulado	No consultó ningún nodo	**Pass**
WF-04	Rechazo 401	UT-R40104-08	**verify never**	Entrada vacía; ningún nodo simulado	No consultó ningún nodo	**Pass**

WF-04	Rechazo 404	UT-R40404-08	**verify never**	Entrada vacía; ningún nodo simulado	No consultó ningún nodo	**Pass**
WF-04	Reasignación exitosa	UT-REA04-05	**verify**	Cancela cita devuelve la cita 53; Reasigna cupo llega con la cita nueva 2393 y la solicitud 13	Consultó Cancela cita	**Pass**
WF-04	Cancelado sin reasignar	UT-CAN04-04	**verify**	Cancela cita devuelve la cita 53	Consultó Cancela cita	**Pass**
WF-05	Hash del token	UT-HASH05-25	**verify never**	Entrada con una petición vacía; ningún nodo simulado	No consultó ningún nodo	**Pass**
WF-05	Rechazo 401	UT-R40105-08	**verify never**	Entrada vacía; ningún nodo simulado	No consultó ningún nodo	**Pass**
WF-05	Rechazo 404	UT-R40405-08	**verify never**	Entrada vacía; ningún nodo simulado	No consultó ningún nodo	**Pass**
WF-05	Cierre exitoso	UT-CIE05-06	**verify never**	Entrada con la fila de una cita cerrada como atendida; ningún nodo simulado	No consultó ningún nodo	**Pass**
WF-06	Compone el reporte	UT-REP06-13	**times y ArgumentCaptor**	Reune indicadores devuelve 16 indicadores; Carga por medico, dos médicos; la función de n8n prepareBinaryData, reemplazada por un jest.fn()	Se llamó **una sola vez** a prepareBinaryData; los argumentos capturados fueron un Buffer con el HTML, el nombre index.html y el tipo text/html	**Pass**
WF-06	Compone el reporte	UT-REP06-16	**verify**	Los mismos dos nodos y la misma función simulada	Consultó Reune indicadores y Carga por medico	**Pass**

WF-06	Nombra el archivo	UT-NOM06-07	**verify**	Compone el reporte devuelve el nombre semana_2026-43.pdf; la entrada es un PDF sin nombre útil	Consultó Compone el reporte	**Pass**
WF-07	Valida datos	UT-VAL07-30	**verify**	Hash del token devuelve una nota válida; Verifica token, el médico 2	Consultó Hash del token y Verifica token	**Pass**
WF-07	Cifra la nota	UT-CIF07-18	**verify**	La entrada es la cita encontrada; Valida datos devuelve la nota del médico 2 con diagnóstico e indicaciones	Consultó Valida datos	**Pass**
WF-07	Nota registrada	UT-NOT07-07	**verify**	Guarda nota devuelve la nota 3 de la cita 3432; la bitácora llega con su hash	Consultó Guarda nota	**Pass**
WF-07	Rechazo 400	UT-R40007-11	**verify never**	Entrada con dos errores de validación; ningún nodo simulado	No consultó ningún nodo	**Pass**
WF-07	Rechazo 401	UT-R40107-08	**verify never**	Entrada vacía; ningún nodo simulado	No consultó ningún nodo	**Pass**
WF-07	Rechazo 403	UT-R40307-08	**verify never**	Entrada vacía; ningún nodo simulado	No consultó ningún nodo	**Pass**
WF-07	Rechazo 404	UT-R40407-08	**verify never**	Entrada vacía; ningún nodo simulado	No consultó ningún nodo	**Pass**
WF-08	Valida datos	UT-VAL08-12	**verify**	Hash del token devuelve el id_cita 1; Verifica token, el médico 1	Consultó Hash del token y Verifica token	**Pass**
WF-08	Descifra	UT-DES08-19	**verify**	La entrada es una nota cifrada; Busca la cita devuelve al paciente 55 y la cita 3432; Valida datos, el médico 2	Consultó Busca la cita y Valida datos	**Pass**

WF-08	Historial listo	UT-HIS08-07	**verify**	Descifra devuelve un historial con una nota; la bitácora llega con su hash	Consultó Descifra	**Pass**
WF-08	Rechazo 400	UT-R40008-11	**verify never**	Entrada con dos errores de validación; ningún nodo simulado	No consultó ningún nodo	**Pass**
WF-08	Rechazo 401	UT-R40108-08	**verify never**	Entrada vacía; ningún nodo simulado	No consultó ningún nodo	**Pass**
WF-08	Rechazo 403	UT-R40308-08	**verify never**	Entrada vacía; ningún nodo simulado	No consultó ningún nodo	**Pass**
WF-08	Rechazo 404	UT-R40408-08	**verify never**	Entrada vacía; ningún nodo simulado	No consultó ningún nodo	**Pass**
WF-09	Valida datos	UT-VAL09-38	**verify**	Hash del token devuelve el alta de un médico; Verifica token, el rol admin	Consultó Hash del token y Verifica token	**Pass**
WF-09	Rechazo 400	UT-R40009-11	**verify never**	Entrada con dos errores de validación; ningún nodo simulado	No consultó ningún nodo	**Pass**
WF-09	Rechazo 401	UT-R40109-08	**verify never**	Entrada vacía; ningún nodo simulado	No consultó ningún nodo	**Pass**
WF-09	Rechazo 403	UT-R40309-08	**verify never**	Entrada vacía; ningún nodo simulado	No consultó ningún nodo	**Pass**
WF-09	Genera token	UT-GTK09-10	**verify never**	La entrada es la salida de Valida datos; ningún nodo simulado	No consultó ningún nodo: genera el token solo	**Pass**
WF-09	Recepcionista agregada	UT-RAG09-07	**verify**	La entrada es la recepcionista 3 recién agregada; Genera token devuelve su token	Consultó Genera token	**Pass**
WF-09	Estado recepcionista	UT-ERC09-08	**verify**	La entrada es el resultado no_existe; Valida datos devuelve el id 3	Consultó Valida datos	**Pass**

WF-09	Genera token nuevo	UT-GTN09-10	**verify never**	La entrada es la salida de Valida datos; ningún nodo simulado	No consultó ningún nodo	**Pass**
WF-09	Recepcionista reactivada	UT-RRE09-06	**verify**	La entrada es el resultado ok; Valida datos devuelve el id 3; Genera token nuevo, el token	Consultó Genera token nuevo	**Pass**
WF-09	Recepcionista reactivada	UT-RRE09-07	**verify never condicional**	La entrada es el resultado ya_activa; los mismos nodos simulados	**No** consultó Genera token nuevo: un token que no se guardó nunca se devuelve	**Pass**
WF-09	Estado paciente	UT-EPA09-09	**verify**	La entrada es el resultado ok; Valida datos devuelve el id 55	Consultó Valida datos	**Pass**
WF-09	Paciente reactivado	UT-PRE09-06	**verify**	La entrada es el resultado ok; Valida datos devuelve el id 55	Consultó Valida datos	**Pass**
WF-09	Genera token medico	UT-GTM09-10	**verify never**	La entrada es la salida de Valida datos; ningún nodo simulado	No consultó ningún nodo	**Pass**
WF-09	Medico agregado	UT-MAG09-06	**verify**	La entrada es el resultado ok con 10 franjas; Valida datos devuelve Pediatria y COL-90001; Genera token medico, el token	Consultó Genera token medico	**Pass**
WF-09	Genera token medico nuevo	UT-GMN09-10	**verify never**	La entrada es la salida de Valida datos; ningún nodo simulado	No consultó ningún nodo	**Pass**

WF-09	Medico reactivado	UT-MRE09-07	**verify**	La entrada es el resultado ok; Valida datos devuelve el id 6; Genera token medico nuevo, el token	Consultó Genera token medico nuevo	**Pass**
WF-09	Reparte reemplazos	UT-RRM09-30	**verify**	La entrada es la situación del médico 2 de Pediatría; Valida datos devuelve el id 2 y la bandera de baja	Consultó Valida datos	**Pass**
WF-09	Rechazo estado medico	UT-REM09-05	**verify**	La entrada es el resultado no_existe; Valida datos devuelve el id 6	Consultó Valida datos	**Pass**
WF-09	Medico desactivado	UT-MDE09-06	**verify**	La entrada son los conteos del cambio (7 citas reasignadas); Valida datos devuelve el id 2	Consultó Valida datos	**Pass**
WF-09	Prepara correos	UT-PRC09-20	**verify never**	La entrada es una cita reasignada; ningún nodo simulado	No consultó ningún nodo: arma los correos solo con su entrada	**Pass**

4 Qué hace JaCoCo en las pruebas

La cobertura de código es una métrica que indica qué proporción del código fuente fue ejecutada durante las pruebas. En el curso se trabajó con JaCoCo, que la mide en proyectos Java. En SIGAC, la misma función la cumple el informe de cobertura integrado en Jest, que se obtiene con la opción --coverage.

Mientras se ejecutan las pruebas, la herramienta registra cada sentencia, rama, función y línea de los 57 nodos que se ejecutó al menos una vez. Al terminar, genera un informe con el porcentaje alcanzado en cada métrica, por archivo y en total, y señala las líneas que ninguna prueba recorrió.

4.1 Las cuatro métricas son:

Métrica	Qué mide
Sentencias (*Statements*)	Las instrucciones del código que se ejecutaron
Ramas (*Branches*)	Los caminos de cada decisión (if, else, ? :, ||) que se recorrieron, en sus dos sentidos
Funciones (*Functions*)	Las funciones que fueron invocadas
Líneas (*Lines*)	Las líneas de código que se ejecutaron

Métrica	Cubiertas / Total	Resultado	Umbral
Sentencias	623 / 623	100 %	—
Ramas	407 / 408	99,75 %	80 %
Funciones	89 / 89	100 %	—
Líneas	592 / 592	100 %	80 %

Cincuenta y seis de los 57 nodos alcanzaron el 100 % en las cuatro métricas. La única rama no recorrida se encuentra en el nodo Valida datos de WF-01, dentro de la función que normaliza las especialidades: corresponde al valor por defecto que se usaría si un médico del catálogo no tuviera especialidad. Ese caso no puede ocurrir en el sistema, porque la columna especialidad de la tabla medico es obligatoria en la base de datos.

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npm run test:cobertura

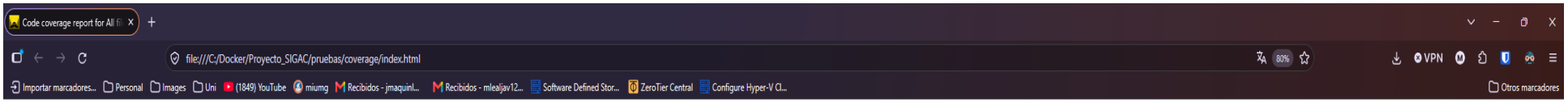
> pruebas-sigac@1.0.0 test:cobertura
> jest --testPathIgnorePatterns=integracion --coverage

 PASS  tests/sigac.wf07.notaregistrada.test.js (7.078 s)
 PASS  tests/sigac.wf04.canceladosinreasignar.test.js (6.569 s)
 PASS  tests/sigac.wf05.rechazo401.test.js (7.082 s)
 PASS  tests/sigac.wf02.calculaprioridad.test.js (7.095 s)
 PASS  tests/sigac.wf06.nombraelarchivo.test.js (7.096 s)
 PASS  tests/sigac.wf02.repartecupos.test.js (7.095 s)
 PASS  tests/sigac.wf08.descifra.test.js (7.207 s)
 PASS  tests/sigac.wf01.rechazo400.test.js
 PASS  tests/sigac.wf08.validadatos.test.js
 PASS  tests/sigac.wf04.rechazo404.test.js
 PASS  tests/sigac.wf01.hashdeltoken.test.js
 PASS  tests/sigac.wf01.cifradatossensibles.test.js
 PASS  tests/sigac.wf04.reasignacionexitosa.test.js
 PASS  tests/sigac.wf07.validadatos.test.js
 PASS  tests/sigac.wf09.hashdeltoken.test.js
 PASS  tests/sigac.wf06.componeelreporte.test.js
 PASS  tests/sigac.wf09.repartereemplazos.test.js
 PASS  tests/sigac.wf04.hashdeltoken.test.js
 PASS  tests/sigac.wf07.hashdeltoken.test.js
 PASS  tests/sigac.hashdeltoken.test.js
 PASS  tests/sigac.wf09.generatoken.test.js
 PASS  tests/sigac.wf09.preparacorreos.test.js
 PASS  tests/sigac.wf09.validadatos.test.js
 PASS  tests/sigac.wf09.pacientereactivado.test.js
 PASS  tests/sigac.wf01.validadatos.test.js
 PASS  tests/sigac.wf07.cifralanota.test.js
 PASS  tests/sigac.wf05.hashdeltoken.test.js
 PASS  tests/sigac.wf09.generatokenmediconuevo.test.js
 PASS  tests/sigac.wf08.rechazo403.test.js
 PASS  tests/sigac.wf09.rechazo403.test.js
 PASS  tests/sigac.wf09.estadorecepcionista.test.js
 PASS  tests/sigac.wf08.hashdeltoken.test.js
 PASS  tests/sigac.wf09.rechazo401.test.js
 PASS  tests/sigac.wf08.rechazo404.test.js
 PASS  tests/sigac.wf09.generatokenmedico.test.js
 PASS  tests/sigac.wf05.rechazo404.test.js
 PASS  tests/sigac.wf08.historiallisto.test.js
 PASS  tests/sigac.wf07.rechazo400.test.js
 PASS  tests/sigac.wf01.rechazo401.test.js
 PASS  tests/sigac.wf07.rechazo401.test.js
 PASS  tests/sigac.wf09.medicoagregado.test.js
 PASS  tests/sigac.wf08.rechazo401.test.js
 PASS  tests/sigac.wf01.rechazo403.test.js
 PASS  tests/sigac.wf09.medicoreactivado.test.js
 PASS  tests/sigac.wf09.recepcionistaagregada.test.js
 PASS  tests/sigac.wf07.rechazo404.test.js
 PASS  tests/sigac.wf01.rechazo409.test.js
 PASS  tests/sigac.wf09.rechazo400.test.js
 PASS  tests/sigac.wf04.rechazo401.test.js
 PASS  tests/sigac.wf09.recepcionistareactivada.test.js
 PASS  tests/sigac.wf05.cierreexitoso.test.js
 PASS  tests/sigac.wf01.registroexitoso.test.js
 PASS  tests/sigac.wf09.generatokennuevo.test.js
 PASS  tests/sigac.wf09.medicodesactivado.test.js
 PASS  tests/sigac.wf07.rechazo403.test.js
 PASS  tests/sigac.wf08.rechazo400.test.js
 PASS  tests/sigac.wf09.estadopaciente.test.js
 PASS  tests/sigac.wf09.rechazoestadomedico.test.js
```

```
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$ npm run test:cobertura
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	100	99.75	100	100	
wf01_cifra_datos_sensibles.js	100	100	100	100	
wf01_hash_del_token.js	100	100	100	100	
wf01_rechazo_400.js	100	100	100	100	
wf01_rechazo_401.js	100	100	100	100	
wf01_rechazo_403.js	100	100	100	100	
wf01_rechazo_409.js	100	100	100	100	
wf01_registro_exitoso.js	100	100	100	100	
wf01_valida_datos.js	100	99.17	100	100	29
wf02_calcula_prioridad.js	100	100	100	100	
wf02_reparte_cupos.js	100	100	100	100	
wf04_cancelado_sin_reasignar.js	100	100	100	100	
wf04_hash_del_token.js	100	100	100	100	
wf04_reasignacion_exitosa.js	100	100	100	100	
wf04_rechazo_401.js	100	100	100	100	
wf04_rechazo_404.js	100	100	100	100	
wf05_cierre_exitoso.js	100	100	100	100	
wf05_hash_del_token.js	100	100	100	100	
wf05_rechazo_401.js	100	100	100	100	
wf05_rechazo_404.js	100	100	100	100	
wf06_compone_el_reporte.js	100	100	100	100	
wf06_nombra_el_archivo.js	100	100	100	100	
wf07_cifra_la_nota.js	100	100	100	100	
wf07_hash_del_token.js	100	100	100	100	
wf07_nota_registrada.js	100	100	100	100	
wf07_rechazo_400.js	100	100	100	100	
wf07_rechazo_401.js	100	100	100	100	
wf07_rechazo_403.js	100	100	100	100	
wf07_rechazo_404.js	100	100	100	100	
wf07_valida_datos.js	100	100	100	100	
wf08_descifra.js	100	100	100	100	
wf08_hash_del_token.js	100	100	100	100	
wf08_historial_listo.js	100	100	100	100	
wf08_rechazo_400.js	100	100	100	100	
wf08_rechazo_401.js	100	100	100	100	
wf08_rechazo_403.js	100	100	100	100	
wf08_rechazo_404.js	100	100	100	100	
wf08_valida_datos.js	100	100	100	100	
wf09_estado_paciente.js	100	100	100	100	
wf09_estado_recepcionista.js	100	100	100	100	
wf09_genera_token.js	100	100	100	100	
wf09_genera_token_medico.js	100	100	100	100	
wf09_genera_token_medico_nuevo.js	100	100	100	100	
wf09_genera_token_nuevo.js	100	100	100	100	
wf09_hash_del_token.js	100	100	100	100	
wf09_medico_agregado.js	100	100	100	100	
wf09_medico_desactivado.js	100	100	100	100	
wf09_medico_reactivado.js	100	100	100	100	
wf09_paciente_reactivado.js	100	100	100	100	
wf09_prepara_correos.js	100	100	100	100	
wf09_recepcionista_agregada.js	100	100	100	100	
wf09_recepcionista_reactivada.js	100	100	100	100	
wf09_rechazo_400.js	100	100	100	100	
wf09_rechazo_401.js	100	100	100	100	
wf09_rechazo_403.js	100	100	100	100	
wf09_rechazo_estado_medico.js	100	100	100	100	
wf09_reparte_reemplazos.js	100	100	100	100	
wf09_valida_datos.js	100	100	100	100	

```
Test Suites: 58 passed, 58 total
Tests:       909 passed, 909 total
Snapshots:   0 total
Time:        19.75 s
Ran all test suites.
manuel12@MANUEL:/mnt/c/Docker/Proyecto_SIGAC/pruebas$
```



All files

100% Statements 623/623 **99.75%** Branches 487/488 **100%** Functions 89/89 **100%** Lines 592/592

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File		Statements		Branches		Functions		Lines	
wf01_cifra_datos_sensibles.js		100%	15/15	100%	4/4	100%	2/2	100%	14/14
wf01_hash_del_token.js		100%	8/8	100%	8/8	100%	1/1	100%	8/8
wf01_rechazo_400.js		100%	3/3	100%	0/0	100%	1/1	100%	3/3
wf01_rechazo_401.js		100%	3/3	100%	0/0	100%	1/1	100%	3/3
wf01_rechazo_403.js		100%	3/3	100%	0/0	100%	1/1	100%	3/3
wf01_rechazo_409.js		100%	3/3	100%	0/0	100%	1/1	100%	3/3
wf01_registro_exitoso.js		100%	3/3	100%	0/0	100%	1/1	100%	3/3
wf01_valida_datos.js		100%	93/93	99.17%	120/121	100%	5/5	100%	90/90
wf02_calcula_prioridad.js		100%	5/5	100%	2/2	100%	2/2	100%	4/4
wf02_reparte_cupos.js		100%	63/63	100%	31/31	100%	11/11	100%	54/54
wf04_cancelado_sin_reasignar.js		100%	3/3	100%	0/0	100%	1/1	100%	3/3
wf04_hash_del_token.js		100%	10/10	100%	14/14	100%	1/1	100%	10/10
wf04_reasignacion_exitosa.js		100%	3/3	100%	0/0	100%	1/1	100%	3/3
wf04_rechazo_401.js		100%	3/3	100%	0/0	100%	1/1	100%	3/3
wf04_rechazo_404.js		100%	3/3	100%	0/0	100%	1/1	100%	3/3
wf05_cierre_exitoso.js		100%	3/3	100%	0/0	100%	1/1	100%	3/3
wf05_hash_del_token.js		100%	11/11	100%	16/16	100%	1/1	100%	11/11
wf05_rechazo_401.js		100%	3/3	100%	0/0	100%	1/1	100%	3/3
wf05_rechazo_404.js		100%	3/3	100%	0/0	100%	1/1	100%	3/3
wf06_compone_el_reporte.js		100%	13/13	100%	0/0	100%	3/3	100%	11/11
wf06_nombra_el_archivo.js		100%	8/8	100%	0/0	100%	1/1	100%	8/8
wf07_cifra_la_nota.js		100%	16/16	100%	6/6	100%	2/2	100%	15/15
wf07_hash_del_token.js		100%	8/8	100%	8/8	100%	1/1	100%	8/8
wf07_nota_registrada.js		100%	5/5	100%	0/0	100%	1/1	100%	5/5
wf07_rechazo_400.js		100%	3/3	100%	0/0	100%	1/1	100%	3/3
wf07_rechazo_401.js		100%	3/3	100%	0/0	100%	1/1	100%	3/3
wf07_rechazo_403.js		100%	3/3	100%	0/0	100%	1/1	100%	3/3
wf07_rechazo_404.js		100%	3/3	100%	0/0	100%	1/1	100%	3/3
wf07_valida_datos.js		100%	23/23	100%	25/25	100%	1/1	100%	23/23
wf08_descifra.js		100%	23/23	100%	6/6	100%	5/5	100%	20/20
wf08_hash_del_token.js		100%	8/8	100%	8/8	100%	1/1	100%	8/8
wf08_historial_listo.js		100%	5/5	100%	2/2	100%	1/1	100%	5/5
wf08_rechazo_400.js		100%	3/3	100%	0/0	100%	1/1	100%	3/3
wf08_rechazo_401.js		100%	3/3	100%	0/0	100%	1/1	100%	3/3
wf08_rechazo_403.js		100%	3/3	100%	0/0	100%	1/1	100%	3/3
wf08_rechazo_404.js		100%	3/3	100%	0/0	100%	1/1	100%	3/3
wf08_valida_datos.js		100%	10/10	100%	10/10	100%	1/1	100%	10/10

wf09_estado_paciente.js		100%	13/13	100%	8/8	100%	1/1	100%	13/13
wf09_estado_recepcionista.js		100%	9/9	100%	6/6	100%	1/1	100%	9/9
wf09_genera_token.js		100%	6/6	100%	0/0	100%	1/1	100%	6/6
wf09_genera_token_medico.js		100%	6/6	100%	0/0	100%	1/1	100%	6/6
wf09_genera_token_medico_nuevo.js		100%	6/6	100%	0/0	100%	1/1	100%	6/6
wf09_genera_token_nuevo.js		100%	6/6	100%	0/0	100%	1/1	100%	6/6
wf09_hash_del_token.js		100%	8/8	100%	8/8	100%	1/1	100%	8/8
wf09_medico_agregado.js		100%	7/7	100%	2/2	100%	1/1	100%	7/7
wf09_medico_desactivado.js		100%	5/5	100%	2/2	100%	1/1	100%	5/5
wf09_medico_reactivado.js		100%	11/11	100%	6/6	100%	1/1	100%	11/11
wf09_paciente_reactivado.js		100%	11/11	100%	6/6	100%	1/1	100%	11/11
wf09_prepara_correos.js		100%	12/12	100%	4/4	100%	2/2	100%	11/11
wf09_recepcionista_agregada.js		100%	4/4	100%	0/0	100%	1/1	100%	4/4
wf09_recepcionista_reactivada.js		100%	11/11	100%	6/6	100%	1/1	100%	11/11
wf09_rechazo_400.js		100%	3/3	100%	0/0	100%	1/1	100%	3/3
wf09_rechazo_401.js		100%	3/3	100%	0/0	100%	1/1	100%	3/3
wf09_rechazo_403.js		100%	3/3	100%	0/0	100%	1/1	100%	3/3
wf09_rechazo_estado_medico.js		100%	7/7	100%	2/2	100%	1/1	100%	7/7
wf09_reparte_reemplazos.js		100%	62/62	100%	39/39	100%	9/9	100%	52/52
wf09_valida_datos.js		100%	46/46	100%	58/58	100%	1/1	100%	46/46



Figura Y. Detalle de cobertura del nodo Valida datos (WF-01). A la izquierda, el número de veces que las pruebas ejecutaron cada línea. En la línea 29, resaltado en amarillo, el valor por defecto de la función normalizar: la única rama no recorrida de las 408 del proyecto, que solo se ejecutaría con un médico sin especialidad, caso que la base de datos no permite.

Conclusiones

Las pruebas unitarias aplicadas a SIGAC cumplieron el objetivo general planteado: se verificó que los 57 nodos Code del sistema cumplen con las reglas de negocio, de validación y de seguridad para las que fueron diseñados, y se detectaron defectos que no se habían manifestado durante el desarrollo. Al cierre, las 857 pruebas fueron superadas.

En relación con los objetivos específicos, se obtuvieron los siguientes resultados:

- Diseño con técnicas formales. Los casos de cada nodo se seleccionaron mediante partición de equivalencia, valores límite, cobertura de ramas, manejo de errores y verificación del contrato de salida. Esta selección permitió recorrer cada condición del código en sus dos sentidos sin necesidad de probar todas las combinaciones posibles, que crecen como 2^n con el número de condiciones.
- Ejecución aislada con simulación. Cada nodo se ejecutó fuera de n8n, con sus dependencias simuladas a la manera de Mockito. Las 53 pruebas de verificación confirmaron que 52 de los 57 nodos consultan exactamente los nodos de los que dependen, ni más ni menos, y el mecanismo de dependencias estrictas actuó sobre la totalidad de las pruebas.
- Verificación de la seguridad. Las pruebas confirmaron que los tokens nunca se comparan ni se devuelven en claro, que el cifrado AES-256-GCM es reversible solo con la clave correcta y detecta cualquier alteración de los datos, que el cifrado de WF-01 y WF-07 es compatible con el descifrado de WF-08, y que las respuestas del sistema no exponen información confidencial.
- Detección y corrección de defectos. Las pruebas detectaron tres defectos reales en seis nodos. Dos de ellos permitían que un valor booleano fuera aceptado como un identificador válido, lo que en WF-09 habría podido dar de baja de forma definitiva a una persona sin que nadie lo solicitara. El tercero permitía asignar citas que se cruzaban en el horario de un mismo médico. Los tres se corrigieron en n8n, y las mismas pruebas que los detectaron confirmaron su corrección.
- Cobertura de código. Se alcanzó el 100 % de sentencias, funciones y líneas, y el 99,75 % de ramas, muy por encima del umbral mínimo del 80 %. De las 408 ramas del código, la única no recorrida corresponde a un caso que la propia base de datos impide.
- Reproducibilidad. El generador extraer_nodos.js mantiene el código de prueba idéntico al publicado en n8n, y cuatro scripts permiten regenerar los nodos, ejecutar las pruebas y medir la cobertura con un solo comando. Cualquier cambio futuro en los flujos puede verificarse de nuevo en segundos.

La principal lección del proceso es que las pruebas automatizadas encuentran lo que las pruebas manuales no buscan. Los tres defectos detectados solo aparecían con entradas que nadie habría enviado a propósito, como un valor booleano en lugar de un número o una hora fuera de los intervalos habituales de media hora. Las pruebas los encontraron porque fueron diseñadas para examinar sistemáticamente los límites de cada regla, y no solo los casos esperados.

Trabajo futuro. Como se estableció en el alcance, las pruebas unitarias no cubren las consultas SQL de los nodos PostgreSQL ni el funcionamiento de los flujos completos de extremo a extremo. El siguiente paso es complementar este trabajo con pruebas de integración que envíen peticiones reales a los webhooks del sistema en ejecución y verifiquen el estado resultante en la base de datos, de modo que la verificación abarque también las condiciones, inserciones y transacciones que hoy se ejecutan fuera del alcance de las pruebas unitarias.